DX Framework единая книга по движку и игровому AI

Полное учебное издание 10.0

Математика C++ ECS DirectX 11 Bullet Physics и игровые системы

Построчное чтение ключевых путей текущего исходного кода

NPC от восприятия и укрытий до оружия урона и расширяемых тактик

Алгоритмы инженерные практики задания и маршрут самостоятельной работы

Собрано по текущему снимку репозитория

1 ноября 2026 года

Маршрут по единой книге

Книга устроена слоями. Сначала изучи язык данных и математику. Затем пройди один кадр движка, физику и игровые системы. После этого переходи к NPC, алгоритмам и собственным модулям. Атлас в конце помогает быстро вернуться к нужному классу после перерыва.

Этап	Раздел	Результат
1	Основа	C++ ECS векторы матрицы лучи время
2	Движок	кадр сцена ресурсы ввод DirectX и шейдеры
3	Bullet	мир формы тела фильтры запросы контроллеры constraints
4	Gameplay	энергия атмосфера двери инвентарь ремонт эффекты
5	NPC	решение навигация укрытие оружие урон и тактики
6	Практика	алгоритмы качество тесты диагностика и roadmap
7	Атлас	карта файлов методов владельцев и точек расширения

Как проверять понимание

Перед запуском напиши прогноз. После запуска сохрани наблюдаемый факт. Если они различаются, ищи первую строку цепочки, где значение изменилось не так, как ожидалось. Именно это превращает чтение в инженерный навык.

Единая книга DX Framework

Издание 10.0 объединяет учебник, справочник, атлас исходников и практикум. Оно написано по снимку репозитория от 1 ноября 2026 года. Главный принцип книги: сначала понять данные и правило, затем пройти путь одного кадра по реальному коду, после этого сделать маленькое изменение и доказать результат отладкой.

Что входит в книгу

Часть	Что изучаем	Практический результат
Основа	C++ ECS время векторы матрицы лучи	читаешь выражение и можешь вручную предсказать результат
Ядро	кадр приложение ввод сцена ресурсы	понимаешь где начинается и заканчивается ответственность системы
Рендер	DirectX 11 камеры материалы шейдеры	прослеживаешь данные от Transform до пикселя
Bullet	мир тела формы фильтры запросы контроллеры constraints	умеешь создать проверить и отладить физический объект
Игровые системы	энергия атмосфера двери ремонт инвентарь эффекты	добавляешь механику полным вертикальным срезом
NPC	восприятие память решение навигация укрытие оружие урон	расширяешь AI отдельными модулями без монолитного Update
Инженерная практика	владение тесты профилирование сохранения диагностика	сохраняешь проект управляемым по мере роста
Учебный маршрут	задания от одного вечера до четырёх недель	учишься через собственные изменения а не через копирование

Что означает построчный разбор

Построчный разбор не является перепечаткой каждого include и каждой закрывающей скобки. Для каждой подсистемы книга выделяет её рабочий путь: вход, проверки, чтение компонентов, вычисление, запись результата и побочные эффекты. Каждая значимая строка объясняется по четырём вопросам.

1. Что строка читает.
2. Что строка изменяет.
3. Почему она стоит именно здесь.
4. Как проявится поломка её контракта.

Одинаковый служебный шаблон, например guard clause или `ForEach`, подробно объясняется один раз. В следующих системах книга указывает, какую роль этот шаблон играет в новом контексте. Так сохраняется построчность причин, но не создаётся сотня страниц одинаковых объяснений скобок.

Карта текущего проекта

Каталог или файл	Ответственность	Главный вопрос при чтении

Каталог или файл	Ответственность	Главный вопрос при чтении
src/Core	запуск кадр время фоновые задания физический мир	кто вызывает систему и на каком времени
src/ECS	сущности компоненты физика трансформы двери давление	где живут данные и кто имеет право их менять
src/Scene	создание удаление загрузка сохранение освещение	что является authoring данными а что runtime состоянием
src/Assets	асинхронная подготовка и загрузка ресурсов	на каком потоке разрешена операция
src/Input	сырые клавиши мышь фронты нажатий	требуется состояние или событие одного кадра
src/Render	DirectX ресурсы кандидаты материалы шейдеры	в каком пространстве и формате находятся данные
src/Animation	клипы поза контроллер IK оружия	кто выбирает действие и кто только строит позу
src/Physics	общие запросы к Bullet	какой объект игнорируется и какой фильтр используется
src/Gameplay/Player	ввод намерение движение камера параметры	где заканчивается желание игрока и начинается исполнение
src/Gameplay/Interaction	зоны события взаимодействия	какое событие одноразовое и кто его потребляет
src/Gameplay/Inventory	предметы слоты экипировка	какая Entity владеет предметом сейчас
src/Gameplay/Combat	оружие выстрел урон ближний бой	где подтверждён факт попадания и когда применяется урон
src/Gameplay/AI	задачи навигация дистанция сквадовые заготовки	это факт решение или исполнение
src/Gameplay/Power	генерация распределение потребление	единицы мощности и правило приоритета
src/Gameplay/Ship	комнаты среда тревога подлодка culling	какая величина переносится между отсеками
src/Gameplay/Repair	повреждение меша и ремонт	какая операция синхронная а какая фоновая
src/Gameplay/Effects	частицы взрывы импульсы	является эффект причиной или визуальным следствием
src/Editor	инспектор сцена debug UI drag and drop	меняется сохранённое поле или временный runtime объект
корневые Npc*.h и StateSystem.h	текущий модульный AI	кто наблюдает кто решает кто исполняет

Главная схема одного кадра

```
Windows input
    -> InputManager и GameInputFrame
    -> игровые намерения игрока и NPC
    -> решения AI и команды
    -> движение оружие взаимодействия среда
    -> Bullet fixed steps
    -> синхронизация Transform
    -> анимация и IK
    -> сбор render candidates
    -> DirectX 11 draw calls
    -> ImGui и отладочные представления
```

Стрелка означает передачу факта или намерения. Она не означает, что следующий слой должен заново принимать решение предыдущего слоя. Например, `NpcRangedCombatSystem` решает, разрешена ли попытка огня, а `WeaponSystem` не выбирает тактику: он проверяет механическую готовность оружия, строит фактический луч и создаёт событие урона.

Три вида данных

Вид	Примеры	Правило
Authoring	размер коллайдера урон оружия приоритет питания	сохраняется в сцене и редактируется человеком
Runtime	таймер перезарядки текущая скорость путь NPC	создаётся или изменяется во время симуляции
Derived	мировая матрица powerRatio видимость цели	вычисляется из других данных и не должна иметь двух независимых владельцев

Большинство трудноуловимых багов возникает, когда производное значение начинают редактировать как исходное или когда два слоя считают себя владельцами одного Transform.

Как заниматься по книге

Для каждой темы используй цикл из пяти шагов.

1. Запиши ожидаемое поведение простыми словами.
2. Нарисуй данные до и после системы.
3. Рассчитай один пример вручную.
4. Найди первую строку реального кода, которая может изменить результат.
5. Проверь в движке видимым индикатором, трассером, журналом или тестовой сценой.

Если после запуска результат неожиданен, не меняй сразу несколько коэффициентов. Сначала найди первый переход, где фактическое значение расходится с записанным ожиданием.

Обозначения

Обозначение	Значение
Инвариант	условие которое обязано оставаться истинным
Guard clause	ранний выход при недопустимом входе

Обозначение	Значение
Владелец	код который отвечает за создание изменение и уничтожение ресурса
Источник истины	единственное место откуда берётся окончательное значение
Intent	желание выполнить действие ещё не гарантированное физикой
Fact	уже подтверждённое событие например попадание луча
Fixed step	шаг симуляции постоянной длительности
Query	чтение мира без изменения его состояния
Command	запрос изменить состояние позже или в определённом слое

Учебное обещание издания

После полного прохождения книги ты должен уметь открыть незнакомую систему проекта и ответить на семь вопросов: кто её вызывает, какие компоненты она читает, какие компоненты пишет, на каком времени работает, какие ресурсы не владеет, что обязано быть истинно на входе и каким наблюдаемым признаком доказать её работу.

Математика движка с нуля

Эта глава не является списком формул. Каждая тема проходит один и тот же путь: смысл, ручной расчёт, запись на C++, применение в DX Framework и проверка результата. Если ручной расчёт не совпал с кодом, не переходи дальше.

1 Числа единицы измерения и погрешность

`float` хранит приближённое число. Поэтому координаты, длины и таймеры почти никогда не сравнивают с нулём через `==`.

```
constexpr float epsilon = 0.0001f;
const bool almostZero = std::fabs(value) <= epsilon;
```

Всегда подписывай единицы мысленно или в имени:

Величина	Единица	Пример
позиция	метры	`position = { 4, 1, -2 }`
скорость	метры в секунду	`runSpeed = 5.5f`
время кадра	секунды	`deltaTime = 0.016f`
угол DirectXMath	радианы	`XMConvertToRadians(45.0f)`
здоровье	условные единицы	`health = 100.0f`

Если скорость равна 5 м/с, а кадр длится 0.02 с, перемещение за кадр равно:

$$\text{deltaPosition} = \text{speed} \cdot \text{deltaTime} = 5 \cdot 0.02 = 0.1$$

Ошибка новичка: прибавлять `speed` каждый кадр. Тогда результат зависит от FPS.

2 Точка и вектор

Точка отвечает на вопрос «где». Вектор отвечает на вопрос «куда и насколько».

Пусть NPC находится в точке A = (2, 1, 3), а цель в B = (8, 1, 7). Направление от NPC к цели получается вычитанием начала из конца:

$$\mathrm{d} = \mathrm{B} - \mathrm{A} = (8-2, 1-1, 7-3) = (6, 0, 4)$$

```
XMFLOAT3 direction{
    target.x - npc.x,
    target.y - npc.y,
    target.z - npc.z
};
```

Перепутанный порядок даст вектор от цели к NPC. Это не «почти тот же результат»: знак определяет направление.

Сложение векторов

Складываем соответствующие координаты:

$$(a_x, a_y, a_z) + (b_x, b_y, b_z) = (a_x + b_x, a_y + b_y, a_z + b_z)$$

Пример: движение вперёд (0, 0, 3) и вправо (2, 0, 0) даёт (2, 0, 3).

Умножение на число

Каждая координата умножается на одно число:

$$2 \cdot (3,0,-1) = (6,0,-2)$$

Так из единичного направления получают скорость, а из скорости — смещение за кадр.

3 Длина нормализация и расстояние

Длина вектора вычисляется по теореме Пифагора:

$$|v| = \sqrt{(x^2 + y^2 + z^2)}$$

Для d = (6, 0, 4):

$$|d| = \sqrt{(6^2 + 0^2 + 4^2)} = \sqrt{(52)} \approx 7.211$$

Нормализация оставляет направление, но делает длину равной единице:

$$\text{unit d} = (d)/(|d|) \approx (0.832,0,0.555)$$

```
XMVECTOR raw = XMLoadFloat3(&direction);
const float length = XMVectorGetX(XMVector3Length(raw));

if (length > 0.0001f)
{
    XMVECTOR unit = XMVector3Normalize(raw);
}
```

Нулевой вектор нормализовать нельзя: деление на ноль породит `NaN`. Поэтому сначала проверяется длина.

Расстояние без квадратного корня

Если нужно только сравнить расстояние с радиусом, корень не нужен:

$$\text{distanceSquared} = dx^2 + dy^2 + dz^2$$

```
const float distanceSquared = dx * dx + dy * dy + dz * dz;
if (distanceSquared < attackRange * attackRange)
{
    // Цель внутри дальности.
}
```

Это дешевле и хорошо подходит для массового поиска кандидатов. Настоящую длину вычисляй только для финального кандидата или интерфейса.

4 Скалярное произведение и поле зрения

Скалярное произведение двух нормализованных векторов показывает, насколько одинаково они направлены:

$$a \cdot b = a_xb_x + a_yb_y + a_zb_z$$

Значение	Геометрический смысл
1	направления совпадают
0	угол 90 градусов
-1	направления противоположны

Связь с углом:

$$a \cdot b = |a||b|\cos(\theta)$$

Если оба вектора единичные, остаётся `cos(theta)`. Для поля зрения шириной 90 градусов половинный угол равен 45 градусов:

$$\text{threshold} = \cos(45°) \approx 0.707$$

```
const XMVECTOR forward = XMVector3Normalize(forwardVector);
const XMVECTOR toTarget = XMVector3Normalize(targetVector);
const float dot = XMVectorGetX(XMVector3Dot(forward, toTarget));
const float threshold = std::cos(XMConvertToRadians(45.0f));
const bool insideFov = dot >= threshold;
```

Важно: поле зрения ещё не доказывает видимость. После него нужен raycast, иначе NPC увидит цель сквозь стену.

5 Векторное произведение

Векторное произведение создаёт вектор, перпендикулярный двум исходным:

$$a \times b = (a_yb_z - a_zb_y, a_zb_x - a_xb_z, a_xb_y - a_yb_x)$$

В движке оно полезно, чтобы получить правое направление относительно `forward` и мирового `up`:

```
XMVECTOR right = XMVector3Normalize(XMVector3Cross(up, forward));
```

Порядок важен: `Cross(a, b)` и `Cross(b, a)` смотрят в противоположные стороны. Если правая точка выглядывания оказалась слева, первым делом проверь порядок аргументов и систему координат.

6 Что такое матрица

Матрица — прямоугольная таблица чисел. В 3D часто используется матрица 4 на 4: она помещает поворот, масштаб и перенос в одну операцию.

$$M = [\ m_(11), m_(12), m_(13), m_(14);\ m_(21), m_(22), m_(23), m_(24);\ m_(31), m_(32), m_(33), m_(34);\ m_(41), m_(42), m_(43), m_(44)\ ]$$

Индекс `m23` означает строку 2, столбец 3. Прежде чем программировать матрицы, научись уверенно находить ячейку по двум индексам.

Единичная матрица

Единичная матрица ничего не меняет:

$$I = [\ 1, 0, 0, 0;\ 0, 1, 0, 0;\ 0, 0, 1, 0;\ 0, 0, 0, 1\ ]$$

Она играет ту же роль, что число 1 при обычном умножении.

7 Как складывать матрицы

Матрицы можно складывать, только если у них одинаковое число строк и столбцов. Складывается каждая пара соответствующих ячеек.

$$A = [\ 1, 2;\ 3, 4\ ], B = [\ 5, 6;\ 7, 8\ ]$$

$$A + B = [1 + 5, 2 + 6;\ 3 + 7, 4 + 8] = [6, 8;\ 10, 12]$$

Псевдокод показывает главный принцип:

```
for (int row = 0; row < rowCount; ++row)
    for (int column = 0; column < columnCount; ++column)
        result[row][column] = a[row][column] + b[row][column];
```

Сложение матриц редко используется для построения `world`-матрицы. Трансформации обычно соединяются умножением. Но сложение важно знать для накопления поправок, смешивания данных и понимания линейной алгебры.

Умножение матрицы на число

Число умножается на каждую ячейку:

$$2A = [2, 4;\ 6, 8]$$

8 Как умножать матрицы

Чтобы вычислить одну ячейку результата, берём строку левой матрицы и столбец правой, перемножаем пары и складываем.

$$C_(row, column) = \Sigma_k\ A_(row, k) B_(k, column)$$

Рассмотрим полный расчёт:

$$A = [1, 2;\ 3, 4], B = [5, 6;\ 7, 8]$$

$$C_(11) = 1 \cdot 5 + 2 \cdot 7 = 19$$

$$C_(12) = 1 \cdot 6 + 2 \cdot 8 = 22$$

$$C_(21) = 3 \cdot 5 + 4 \cdot 7 = 43$$

$$C_(22) = 3 \cdot 6 + 4 \cdot 8 = 50$$

$$AB = [19, 22;\ 43, 50]$$

Матрицы не перестановочны: обычно `A * B != B * A`. Поэтому порядок `scale * rotation * translation` является частью поведения, а не косметикой.

```
for (int row = 0; row < 2; ++row)
{
    for (int column = 0; column < 2; ++column)
    {
        result[row][column] = 0.0f;
        for (int k = 0; k < 2; ++k)
            result[row][column] += a[row][k] * b[k][column];
    }
}
```

9 Масштаб поворот и перенос

В `TransformComponent::GetWorldMatrix` текущего проекта операции создаются отдельно:

```
XMMATRIX scaleMatrix = XMMatrixScaling(scale.x, scale.y, scale.z);
XMMATRIX rotX = XMMatrixRotationX(rotation.x);
XMMATRIX rotY = XMMatrixRotationY(rotation.y);
XMMATRIX rotZ = XMMatrixRotationZ(rotation.z);
XMMATRIX rotationMatrix = rotX * rotY * rotZ;
XMMATRIX translationMatrix = XMMatrixTranslation(position.x, position.y, position.z);
return scaleMatrix * rotationMatrix * translationMatrix;
```

Читаем не как «набор непонятных матриц», а как конвейер:

1. локальная вершина получает масштаб;
2. результат поворачивается по X, затем Y, затем Z в принятой проектом конвенции;
3. результат переносится в позицию сущности;
4. получается мировая координата.

Если сначала перенести объект, а затем повернуть, перенос тоже может повернуться вокруг начала координат. Поэтому объект начнёт двигаться по дуге вместо поворота на месте.

Координата и направление преобразуются по разному

`XMVector3TransformCoord` применяет перенос. Для точки дула это правильно. Направление не должно получать перенос; для него используют `XMVector3TransformNormal` или только поворотную часть.

```
XMVECTOR worldPoint = XMVector3TransformCoord(localPoint, worldMatrix);
XMVECTOR worldDirection = XMVector3TransformNormal(localDirection, worldMatrix);
```

10 Обратная матрица

Обратная матрица отменяет преобразование:

$$M^{(-1)}M = I$$

Если `world` переводит локальную точку в мир, `inverseWorld` возвращает мировую точку в локальное пространство.

```
XMMATRIX world = transform.GetWorldMatrix();
XMMATRIX inverseWorld = XMMatrixInverse(nullptr, world);
XMVECTOR localPoint = XMVector3TransformCoord(worldPoint, inverseWorld);
```

Практическое применение: определить, где попадание находится относительно отсека или корпуса, а не относительно всей сцены.

Не всякая матрица обратима. Нулевой масштаб по оси делает преобразование вырожденным. Поэтому масштаб `(0, 1, 1)` опасен для кода, которому нужна обратная матрица.

11 Пространства координат

Одна и та же точка проходит несколько пространств:

Пространство	Вопрос
локальное	где точка относительно модели
мировое	где точка на уровне
вид камеры	где точка относительно камеры
clip	как точка прошла проекцию
NDC	где точка в нормализованном экране
экранное	какой это пиксель

Типовой конвейер:

$$p_{(clip)} = p_{(local)} \cdot World \cdot View \cdot Projection$$

Если модель видна, но raycast идёт не туда, часто смешаны локальная и мировая координаты. Всегда спрашивай: «в каком пространстве находится эта переменная?»

12 Что такое луч

Луч задаётся началом O и нормализованным направлением D:

$$P(t) = O + tD, t \geq 0$$

`t` — расстояние вдоль луча, если D единичный. Для конечного отрезка вводится `0 <= t <= maxDistance`.

Пример: O = (1, 2, 3), D = (0, 0, 1). При t = 5 получаем P = (1, 2, 8).

```
XMFLOAT3 origin = muzzleWorldPosition;
XMFLOAT3 rawDirection{
    aimPoint.x - origin.x,
    aimPoint.y - origin.y,
    aimPoint.z - origin.z
};

XMVECTOR direction = XMVector3Normalize(XMLoadFloat3(&rawDirection));
```

Создание луча всегда состоит минимум из трёх вопросов:

1. Откуда начинается луч?
2. Куда он направлен?
3. Какова максимальная длина и кого он должен игнорировать?

Для винтовки начало должно быть у дула, а не у центра NPC или камеры. Иначе визуальный ствол и физический выстрел разойдутся.

13 Луч из курсора камеры

Для выбора объекта мышью нужно построить две мировые точки: на ближней и дальней плоскости камеры.

1. Перевести пиксель мыши в NDC.
2. Создать точки near и far в clip-пространстве.
3. Умножить их на обратную `ViewProjection`.
4. Разделить на `w`.
5. Началом сделать near, направлением — `normalize(far - near)`.

$$x_(ndc) = 2(x_(screen)/width) - 1$$

$$y_(ndc) = 1 - 2(y_(screen)/height)$$

Y переворачивается, потому что экранные координаты обычно растут вниз, а NDC — вверх.

14 Пересечение луча с плоскостью

Плоскость задаётся точкой Q и нормалью N. Подставляем луч в уравнение плоскости:

$$t = ((Q - O) \cdot N)/(D \cdot N)$$

Алгоритм:

1. вычислить denominator = dot(D, N);
2. если модуль denominator почти нулевой, луч параллелен плоскости;
3. вычислить t;
4. если t < 0, пересечение позади начала луча;
5. вернуть O + tD.

```
const float denominator = Dot(rayDirection, planeNormal);
if (std::fabs(denominator) <= 0.0001f)
    return false;

const float t = Dot(planePoint - rayOrigin, planeNormal) / denominator;
if (t < 0.0f)
    return false;

hitPoint = rayOrigin + rayDirection * t;
```

Применение: поставить маркер на полу, найти точку приказа, пересечь прицел с условной тактической плоскостью.

15 Пересечение луча со сферой

Сфера задаётся центром C и радиусом r. Ищем точку луча, расстояние которой до C равно r:

$$|O + tD - C|^2 = r^2$$

После раскрытия получается квадратное уравнение. Удобная реализация для нормализованного D:

```
const Vec3 oc = origin - center;
const float b = Dot(oc, direction);
const float c = Dot(oc, oc) - radius * radius;
const float discriminant = b * b - c;

if (discriminant < 0.0f)
    return false;

const float root = std::sqrt(discriminant);
float t = -b - root;
if (t < 0.0f)
    t = -b + root;
return t >= 0.0f;
```

Отрицательный дискриминант означает промах. Два корня — вход и выход из сферы. Обычно нужен ближайший неотрицательный.

16 Пересечение луча с AABB

AABB — коробка, выровненная по мировым осям. Slab-алгоритм рассматривает диапазон допустимых `t` для каждой оси и пересекает диапазоны.

```
float tMin = 0.0f;
float tMax = maxDistance;

for (int axis = 0; axis < 3; ++axis)
{
    if (std::fabs(direction[axis]) < epsilon)
    {
        if (origin[axis] < boxMin[axis] || origin[axis] > boxMax[axis])
            return false;
        continue;
    }

    float t1 = (boxMin[axis] - origin[axis]) / direction[axis];
    float t2 = (boxMax[axis] - origin[axis]) / direction[axis];
    if (t1 > t2)
        std::swap(t1, t2);

    tMin = (std::max)(tMin, t1);
    tMax = (std::min)(tMax, t2);
    if (tMin > tMax)
        return false;
}
```

Ключевая мысль: луч должен одновременно находиться внутри допустимого интервала по X, Y и Z.

17 Raycast в физическом мире

Математический луч только описывает линию. Raycast задаёт миру вопрос: «какой физический объект первым встретится на этом отрезке?»

В проекте при выстреле передаются:

Аргумент	Назначение
`origin`	мировая позиция дула
`direction`	направление выстрела
`world`	Bullet physics world
`range`	конечная дальность
`shooterEntity`	сущность, которую нельзя принять за цель
`shooterObject`	физическое тело стрелка для исключения
`rayHit`	структура результата

Не используй заранее выбранного `targetEntity` как доказательство попадания. Урон должен получить первый объект луча: стена, союзник или цель.

18 Угол поворота без скачка через 360 градусов

Обычное вычитание углов ломается рядом с границей `-pi` и `pi`. Кратчайшая разница вычисляется так:

$$\text{error} = \operatorname{atan2}(\sin(\text{target} - \text{current}), \cos(\text{target} - \text{current}))$$

```
const float yawError = std::atan2(
    std::sin(targetYaw - currentYaw),
    std::cos(targetYaw - currentYaw));
```

Результат всегда лежит в диапазоне от `-pi` до `pi`. Знак говорит, в какую сторону вращаться.

19 Время таймеры и плавное изменение

Таймер обратного отсчёта:

```
timer = (std::max)(0.0f, timer - deltaTime);
if (timer <= 0.0f)
{
    // Событие разрешено.
}
```

Линейная интерполяция:

$$\mathrm{lerp}(a, b, t) = a + (b - a)t$$

```
const float value = a + (b - a) * t;
```

Ограничивай `t` диапазоном 0..1, если не нужна экстраполяция. Для плавности, одинаковой при разном FPS, полезно экспоненциальное приближение:

$$\mathrm{alpha} = 1 - e\text{^}(-\mathrm{speed} \cdot \ \mathrm{deltaTime})$$

```
const float alpha = 1.0f - std::exp(-responseSpeed * deltaTime);
current += (target - current) * alpha;
```

20 Как математика связана с текущим NPC

Кодовая задача	Математика	Проверка в отладчике
найти ближайшего врага	квадрат расстояния	линия до выбранной цели
проверить поле зрения	нормализация и dot	конус или граничные лучи
подтвердить видимость	raycast	зелёный или красный луч
направить ствол	матрицы кости и мира	точка muzzle
запретить выстрел из стены	короткий ray chest to muzzle	луч упирается в укрытие
проверить линию огня	конечный raycast	первый пересечённый entity
оценить укрытие	несколько лучей на высотах тела	трассеры к кандидату
повернуться к угрозе	atan2 и кратчайшая разница	yaw error

21 Практика по математике

Задание 1 направление

NPC находится в `(3, 0, 2)`, цель в `(11, 0, -4)`. Вручную вычисли направление, длину и единичный вектор. Затем сравни с DirectXMath.

Задание 2 поле зрения

Направление NPC `(0, 0, 1)`. Проверь цели в направлениях `(0, 0, 1)`, `(1, 0, 1)` и `(1, 0, 0)` для FOV 90 градусов. Сначала предскажи результат.

Задание 3 матрицы

Вручную сложи и умножь матрицы:

$$A = [2,1;\ 0,3], B = [4,-1;\ 2,5]$$

Для умножения выпиши все четыре ячейки отдельно.

Задание 4 порядок трансформаций

Возьми точку `(1, 0, 0)`. Сравни два случая: сначала перенос `(5, 0, 0)`, потом поворот на 90 градусов; и сначала поворот, потом перенос. Нарисуй результат сверху.

Задание 5 собственный луч

Сделай маленькую функцию, которая принимает начало и конечную точку, возвращает нормализованное направление и длину. Она обязана отклонять почти нулевой отрезок.

Задание 6 визуальная проверка

Нарисуй в отладке три разных цвета:

- жёлтый от глаз NPC к кандидату;
- красный от груди к дулу;
- зелёный от дула до первого попадания.

По одному кадру должно быть понятно, какой именно тест запретил или разрешил выстрел.

22 Ответы для самопроверки

Для задания 1 направление равно `(8, 0, -6)`, длина равна `10`, нормализованный вектор равен `(0.8, 0, -0.6)`.

Для задания 2 первая цель точно внутри. Вторая лежит на границе 45 градусов и должна считаться внутри с небольшим допуском. Третья находится под 90 градусов к forward и снаружи.

Для задания 3:

$$A + B = [6,0;\ 2,8]$$

$$AB = [10,3;\ 6,15]$$

23 Контрольный список

Перед следующей частью ты должен уметь без подсказки:

- объяснить разницу между точкой и вектором;
- получить направление как `end - start`;
- безопасно нормализовать вектор;
- сравнить расстояния без квадратного корня;
- объяснить dot через угол;
- вручную сложить две матрицы;
- вручную вычислить каждую ячейку произведения матриц;
- объяснить, почему порядок матриц меняет результат;
- построить конечный луч от дула к точке;
- объяснить разницу между математическим лучом и физическим raycast;
- найти кратчайшую ошибку угла;
- обновить таймер независимо от FPS.

Как читать и менять DX Framework с нуля

Этот раздел рассчитан на человека, который умеет открыть проект, но хочет заново понять собственный C++ код. Здесь не требуется предварительно помнить названия паттернов. Мы будем идти от одного значения в памяти к компоненту, от компонента к системе, затем к целому кадру. Если встретилось незнакомое слово, сначала выясни, какие данные оно обозначает и кто ими владеет. Только после этого переходи к следующему методу. Файлы и имена ниже описывают состояние проекта на 29 октября 2026 года; строки могут сдвинуться после изменения исходников, поэтому ищи указанное имя метода.

1. Как разобрать строку C++ и не потерять смысл

1.1. Тип, имя, значение и время жизни

Возьми простую строку `float deltaTime = 0.016f;`. `float` задаёт тип числа с плавающей точкой, `deltaTime` — имя, `0.016f` — начальное значение. Суффикс `f` означает литерал типа float, иначе дробный литерал в C++ обычно имеет тип double. Но тип не сообщает единицы измерения. В игровом кадре `deltaTime` — секунды, а `position` обычно измеряется в метрах. Если прибавить `deltaTime` к позиции без скорости, объект будет двигаться на численно похожее расстояние, но игровое правило окажется неявным. При чтении каждой арифметической строки подпиши единицы аргументов и результата.

Локальная переменная создаётся при входе в область видимости и уничтожается при выходе из неё. Поле объекта живёт столько же, сколько объект. Динамически созданный ресурс живёт до освобождения владельцем. Это различие важно в `ApplicationClass::Frame`: локальный `registry` как ссылка на `m_sceneManager.GetECSRegistry()` не создаёт второй мир. Когда функция заканчивается, сама ссылка исчезает, но Registry остаётся полем SceneManager. Нельзя вернуть указатель на локальную временную переменную из функции и продолжать пользоваться им в следующем кадре.

1.2. Значение, ссылка и указатель

`Entity e` передаёт числовой идентификатор по значению: изменение локальной копии не меняет ID у вызывающего кода. `ECSRegistry& registry` означает ссылку на существующий Registry: система работает с тем же объектом, а не копирует все компоненты. `const RepairableComponent* repairable` — указатель, который может быть `nullptr`, и через который нельзя менять компонент, пока действует `const`. Звёздочка в объявлении указывает тип указателя; `*repairable` в выражении разыменовывает его. `repairable->progress` — доступ к полю через указатель. Разыменовать `nullptr` нельзя.

Если `GetComponent<T>(entity)` возвращает `T*`, следующий безопасный шаг — проверить результат. Сам `Entity` может быть живым, но компонента T у него нет. Для примера, у декоративного ящика есть Transform, но нет Repairable. Правильное условие `if (auto* repairable = registry.GetComponent<RepairableComponent>(e))` связывает проверку и использование. Однако даже ненулевой указатель не обещает вечной жизни: после `DestroyEntity`, `RemoveComponent` или структурной операции с pool проверь его заново. Особо опасно хранить `T&` между вызовами систем, которые могут удалить цель.

1.3. `const`, `auto` и `enum`

`const` не означает, что весь игровой мир заморожен; оно ограничивает изменение через конкретный доступ. `const auto* brain = registry.GetComponent<NpcBrainComponent>(e)` позволяет читать brain, но не записывать state через этот указатель. `auto` выводит тип выражения, а не превращает язык в

динамический: если справа `T*`, тип остаётся указателем. При чтении сложного кода при необходимости выпиши настоящий тип `auto`, иначе легко перепутать копию с ссылкой. `auto&` создаёт ссылку, `auto` в range loop может копировать элемент.

`enum class` задаёт именованные варианты состояния: `NpcState::Idle`, `NpcState::Attacking` и так далее. Он помогает компилятору отличать состояние NPC от произвольного числа. Но enum сам не определяет переходы: их задают условия систем. Если персонаж одновременно хранит `state=Attacking` и `target=INVALID_ENTITY`, это логически противоречивое состояние, даже если оба поля по отдельности имеют допустимые типы. Составь инварианты: при Attacking цель жива и враждебна; при Repairing есть действительная цель и достижимая рабочая точка. Claim нужен для автоматического назначения; для оборудования может использоваться WorkZone, а пробоина в текущем коде обрабатывается без него.

1.4. `std::vector`, `unordered_map` и итерация

`std::vector<T>` хранит последовательность. Добавление элемента иногда перераспределяет память, поэтому старые указатели и ссылки на элементы могут стать недействительными. `std::unordered_map<Key,Value>` хранит значения по ключу; изменение структуры контейнера влияет на итераторы по правилам конкретной операции. В ECS Registry каждый тип компонента имеет pool, а `ForEach` итерирует его. Поэтому не удаляй текущий компонент из того же pool прямо во время `ForEach`, если контракт `ComponentManager` этого явно не гарантирует. Безопасный подход: во время обхода складывай Entity ID в `pendingDestroy`, после обхода применяй удаления. Это не просто стиль — так сохраняется корректность итератора и ссылок.

1.5. Лямбда и захват

`[&](Entity e, auto& component) { ... }` — функция без имени, которую можно передать `ForEach`. `[&]` разрешает использовать внешние локальные переменные по ссылке. Это удобно для накопления результатов, но опасно для фоновой задачи: после выхода из вызывающей функции захваченная ссылка может стать висячей. В синхронном `ForEach`, который вызывает лямбду до возврата, захват по ссылке обычно соответствует времени жизни. При `JobSystem::SubmitJob` код может выполниться позже; копируй нужные входные данные и применяй итог в главном потоке после проверки живости Entity и версии сцены.

1.6. RAII и владение ресурсом

RAII означает, что освобождение ресурса связано с уничтожением объекта-владельца. `std::unique_ptr<IComponentPool>` внутри `ECSRegistry` ясно выражает единоличное владение pool. Если Registry уничтожается, unique_ptr освободит pool. Bullet body, GPU buffer и job queue живут по другим правилам: удалить C++ объект недостаточно, если он всё ещё зарегистрирован в physics world или привязан к GPU command. Для каждого такого ресурса составь пару «создание — удаление» и момент отписки от внешнего мира. Если scene reload оставил collider без Entity, это нарушение жизненного цикла, а не визуальный баг.

Проверочное упражнение. Прочитай сигнатуру `void StateSystem::Update(ECSRegistry& registry, float deltaTime, JobSystem& jobs)`. Назови, какие аргументы копируются, какие ссылаются на существующий объект, где ожидаются единицы, какие изменения возможны. Затем найди один вызов функции из `ApplicationClass::Frame`. Проверь, передан ли `simulationDeltaTime`, когда Play поставлен на паузу. Не переходи к телу метода, пока сигнатура не понятна.

2. Разбираем `ECSRegistry.h` буквально по шагам

2.1. `CreateEntity`: от ID до живой сущности

Файл `src/ECS/ECSRegistry.h`, метод `CreateEntity` (в текущем файле около строк 15–22). Строка с `if (nextEntity == INVALID_ENTITY)` защищает от выдачи зарезервированного значения. `throw std::overflow_error` останавливает операцию явной ошибкой; без этой проверки переполнение ID могло бы дать новое значение, совпадающее с «нет сущности». `const Entity entity = nextEntity++;` сначала берёт прежний номер, затем увеличивает счётчик. `m_liveEntities.insert(entity)` фиксирует, что ID существует. Возврат отдаёт номер вызывающему коду. В этот момент у сущности ещё нет Transform, RenderComponent или NPCBrain: CreateEntity создаёт идентичность, а не готовый игровой объект.

Проверь на бумаге: если `nextEntity=10`, первый вызов возвращает 10 и оставляет 11; второй возвращает 11. `IsAlive(10)` после первого вызова истинно, но `HasComponent<TransformComponent>(10)` может быть ложно. Это объясняет, почему одно лишь создание Entity не рисует объект. Если игра удалит 10, текущая реализация не переиспользует этот номер немедленно; generation handle здесь не реализован. При добавлении повторного использования в будущем понадобятся поколения, чтобы старый сохранённый ID не указывал на новую сущность.

2.2. `DestroyEntity`: удаление из набора и pools

Около строк 24–28 `DestroyEntity` сначала делает `m_liveEntities.erase(e)`, затем проходит `for (auto& [ti, pool] : m_pools)` и вызывает `pool->Remove(e)` у каждого типа компонента. Это универсальная очистка ECS данных: Registry не обязан знать заранее, какие компоненты были у Entity. Но у внешних систем есть дополнительные ресурсы. Если компонент содержал `btRigidBody*`, перед уничтожением нужно убедиться, что тело исключено из Bullet world и освобождено по договору владельца. Если async job хранит ссылку на компонент, удаление до завершения job опасно. Поэтому в gameplay коде зачастую нужен более высокий `DespawnEntity` путь, который согласует ECS, physics, renderer, jobs и временные reservations.

На проверочной сцене создай Entity с двумя компонентами, вызови Destroy, затем проверь `IsAlive=false`, `GetComponent<T>=nullptr` для обоих. Потом проверь физический мир: число тел тоже должно уменьшиться, если Entity владело телом. Если не уменьшилось, Registry выполнил свой контракт, а вызывающий lifecycle путь — нет. Важно различать эти уровни ответственности.

2.3. `AddComponent` и две перегрузки

Около строк 33–43 есть две формы. `AddComponent(Entity e, const T& comp)` копирует объект и разрешена только для copy-constructible T. `AddComponent(Entity e, T&& comp)` пересылает временное значение или объект, который можно переместить; это нужно для некопируемых компонентов, например тех, что владеют физическим ресурсом. `GetOrCreatePool<T>()` находит pool этого типа либо создаёт его. Обрати внимание: в этих строках не видно проверки `IsAlive(e)`. Следовательно, вызывающий код должен соблюдать контракт живой Entity или нужно усилить Registry; не предполагай невидимую защиту. Если компонент добавлен на несуществующий ID, pool может хранить мусорную запись, которая не соответствует `m_liveEntities`.

Для новичка полезна аналогия: Entity ID — номер папки, component pool — отдельная таблица одного вида данных. `AddComponent` добавляет строку в таблицу по номеру папки. Если номер папки неверен, таблица не обязательно сама поймёт это. Поэтому фабрика объекта должна создавать Entity один раз, затем последовательно добавлять обязательные компоненты и валидировать готовую конфигурацию.

2.4. `GetComponent`, `HasComponent`, `RemoveComponent`

`GetComponent<T>` находит pool типа T. Если pool отсутствует, возвращает `nullptr`; иначе вызывает `pool->Get(e)`. `HasComponent<T>` проверяет наличие pool и Entity в нём. `RemoveComponent<T>` удаляет только этот тип у выбранной Entity. Ни один из этих методов не меняет `m_liveEntities`. Следовательно, NPC может оставаться живым, но временно потерять оружие, и системы должны обрабатывать это как допустимое или ошибочное состояние по своему правилу. Нельзя писать `*registry.GetComponent<T>(e)` без проверки, если отсутствие T возможно.

`GetComponents<T>()` в отличие от `GetComponent` создаёт pool, если его ещё не было. Это полезно для системного обхода, но имеет побочный эффект создания пустого pool. При диагностике «почему появился pool» учитывай это. Для read-only запроса к одному объекту предпочитай `GetComponent`/`HasComponent`, чтобы чтение не изменяло структуру Registry без необходимости.

2.5. `ForEach<T1,T2>` построчно

Около строк 45–59 берутся два указателя на pools: `FindPool<T1>()` и `FindPool<T2>()`. Если один отсутствует, `return` заканчивает обход без вызова callback. Затем цикл проходит каждый элемент первого pool. Для текущего Entity проверяется `pool2->Has(entity)`. Если второй компонент найден, берётся ссылка `comp2` и вызывается `fn(entity, comp1, comp2)`. Это значит, что порядок типов влияет на количество итераций: `ForEach<Редкий,Частый>` обычно делает меньше проверок, чем `ForEach<Частый,Редкий>`. Но менять порядок нужно только если callback аргументы и логика соответственно переставлены.

Ключевое ограничение: callback получает ссылки на элементы pools. Если внутри callback удалить текущий компонент или Entity, цикл может потерять валидность итератора. Даже если конкретный контейнер иногда переживает удаление другого элемента, не строй на этом неявную гарантию. Собирай команды на структурные изменения отдельно. Вопрос для самопроверки: `ForEach<NpcBrainComponent,TransformComponent>` найдёт ли Entity только с Transform? Нет: она отсутствует в первом pool и не участвует в цикле. Найдёт ли Entity только с Brain? Цикл её увидит, но проверка второго pool не пропустит callback.

2.6. `type_index` и закрытая часть класса

В `private` находятся `nextEntity`, набор живых ID и `unordered_map<std::type_index,std::unique_ptr<IComponentPool>>`. `typeid(T)` превращается в ключ вида компонента. `FindPool<T>` ищет ключ и приводит базовый `IComponentPool*` к конкретному `ComponentManager<T>*`; корректность зависит от того, что под данным ключом всегда лежит pool именно T. `GetOrCreatePool<T>` создаёт его при первом обращении. Это простая реализация ECS, которую можно понять без магии: каждый тип имеет собственную таблицу, а Registry связывает таблицы общим Entity ID.

Упражнение: вручную нарисуй три pools — Transform у Entity 1 и 2, Repairable у Entity 2 и 3, NPCBrain у Entity 1. Что вызовет `ForEach<TransformComponent,RepairableComponent>`? Только Entity 2. Что произойдёт после `DestroyEntity(2)`? ID 2 удалён из набора живых и обоих pools. После этого попытка `GetComponent<RepairableComponent>(2)` должна вернуть null. Проверь тем же набором в маленьком тестовом запуске.

3. Главный кадр: читаем порядок, а не названия систем

3.1. Вход `Frame(float dt)` и режимы времени

`ApplicationClass::Frame` начинается около строки 1023 `src/Core/ApplicationClass.cpp`. Параметр `dt` — длительность кадра. В отдельном режиме проверки оружия код принудительно ставит `dt=1/60`, поэтому измерение FPS в этой ветке не отражает реальное время. Затем обновляется DebugHud, input manager и очищаются события текущего кадра. Уже здесь читатель должен спросить: `m_events.Clear()` удаляет события прошлого кадра до или после того, как их потребители обработали? Ответ ищи в местах публикации и чтения событий, а не предполагай. Для нового события выбери конкретную фазу кадра, чтобы оно не исчезало сразу после создания.

Переключение F5 проверяет не только `IsKeyDown`, но и `!m_wasF5Down`: так одно долгое удержание клавиши не переключает режим каждый кадр. При входе в Play `EnterPlayMode` проверяет сцену, затем сбрасываются timer, event log, trigger state, открытый UI и режим камеры. В AI-only сцене без игрока код отдельно не создаёт фиктивный инвентарь для `INVALID_ENTITY`. Это пример защитной проверки на границе режимов. При выходе в Editor нужно очистить накопленное намерение движения character controller; иначе физика продолжает шагать, используя последнюю команду Play. Этот комментарий в коде показывает, что режим Editor не является полной остановкой всех подсистем.

3.2. От ввода игрока до намерения движения

В Play при работающей симуляции `m_gameplayController.Update` получает Registry, `simulationDeltaTime`, input, режим захвата UI и yaw камеры. Он не обязан непосредственно перемещать mesh. Его задача — превратить действие игрока и направление камеры в намерение. Далее системы движения и Bullet определят фактическое положение. Если игрок нажал W, а персонаж стоит, проверь по цепи: получен ли GameAction; заблокировал ли его UI; записана ли желаемая скорость; вызван ли character controller; есть ли контакт с collider; изменился ли Transform после physics. Если первый разрыв на входе, физику менять рано.

3.3. Почему NPC pipeline стоит именно так

В текущем кадре перед выбором и выстрелами уменьшаются таймеры оружия (`WeaponSystem::UpdateTimers`). Затем `NpcUtilitySystem` рассчитывает автоматическое действие; `CrewTaskCoordinator` закрепляет работу за конкретным членом экипажа; `NpcCombatSystem` обновляет боевую видимость и память; `NpcCoverSystem` выбирает индивидуальное укрытие; `StateSystem` исполняет итоговое состояние, строит путь и заполняет движение; `CharacterFacingSystem` поворачивает персонажа. Групповой `NpcSquadCombatSystem::Update` в этой ревизии закомментирован. Позднее, после обновления позы и оружейных данных, `NpcRangedCombatSystem` может фактически стрелять. Не говори «NpcCombatSystem стреляет» только потому, что он выбирает Attacking; выстрел имеет отдельные условия и место кадра.

На бумаге проследи одного инженера. До кадра у него Idle и нет цели. Utility видит пробоину и рассчитывает полезность; Coordinator ставит claim; StateSystem в этом же кадре может увидеть назначение и запросить путь; Facing поворачивает модель. Фактическая капсула может сдвинуться позже на physics step. Если поменять местами Coordinator и StateSystem, первое движение задержится до следующего кадра. Это не обязательно катастрофа, но меняет наблюдаемое поведение. Любая перестановка вызовов должна сопровождаться таблицей «кто пишет поле → кто читает».

3.4. Кто владеет состоянием и что значит «на один кадр старое»

В одном кадре разные поля могут отражать разные моменты. У NPC `brain.state` уже новый, `desiredVelocity` вычислена по новому маршруту, а `Transform` ещё хранит позицию до physics step. Renderer после physics может увидеть обновлённое положение, а HUD, если был заполнен раньше, покажет старую дистанцию. Это обычный вопрос фаз, пока задержка определена и мала. Для диагностики показывай timestamp/fixed tick рядом с данными. Если поле используется через несколько кадров без инвалидации, это уже ошибка контракта, особенно для raycast muzzle и lastKnownPosition.

4. Как добавлять систему так, чтобы она заработала во всей игре

4.1. Сперва сформулируй правило без C++

Выберем пожар. Правило для игрока: повреждённый прибор может загореться; огонь расходует ресурс и нагревает комнату; инженер с огнетушителем может потушить его; закрытая дверь влияет на доступность; HUD сообщает причину, если работа невозможна. Запиши, как игрок видит начало, развитие и конец события. Затем выпиши состояния `Igniting`, `Burning`, `Dying`, `Extinguished`; для каждого — кто меняет его, что видно, можно ли сохранять. Если эту схему нельзя изложить словами, реализация тоже будет противоречивой.

Данные огня принадлежат `FireComponent` на Entity очага: интенсивность, топливо, room ID, рабочая зона и временной параметр. Температура комнаты принадлежит комнате, заряд огнетушителя — инвентарю NPC. Не дублируй одно и то же число в нескольких местах без явного правила синхронизации. Например, если и FireComponent, и RoomComponent хранит «текущую температуру пожара», один из них неизбежно станет устаревшим. Выбери источник правды и производные показатели.

4.2. Реализация минимального тика

Создай систему, которая работает с одним очагом без NPC и рендера. Пример ниже — учебный псевдокод, потому что конкретных `FireComponent` и `RoomTemperatureComponent` в текущем проекте ещё может не быть. Он показывает контракт обновления, а не готовую вставку:

```
void FireSystem::Update(ECSRegistry& ecs, float dt) {
    ecs.ForEach<FireComponent>([&](Entity fireEntity, FireComponent& fire) {
        if (fire.state != FireState::Burning) return;
        const float burn = std::min(fire.fuel, fire.burnRate * dt);
        fire.fuel -= burn;
        fire.intensity = std::clamp(fire.intensity + fire.growthRate * dt,
                                    0.0f, fire.maxIntensity);
        if (fire.fuel <= 0.0f) fire.state = FireState::Dying;
    });
}
```

Разбор. Проверка state предотвращает рост потушенного очага. `burnRate*dt` переводит расход «в секунду» в расход за шаг. `std::min` не позволяет забрать больше топлива, чем есть. Clamp удерживает интенсивность в допустимых границах. Переход в Dying выполняется при исчерпании топлива. Но этот код ещё не нагревает комнату, не наносит урон и не создаёт задачу NPC. Добавляй по одному звену и каждый раз проверяй инвариант, чтобы при сбое знать, на каком этапе он появился.

4.3. Место в кадре и зависимости

Если Coordinator должен выбирать пожар по текущей интенсивности, `FireSystem::Update` должен выполняться до него либо решение сознательно использует предыдущий tick. Если тушение снижает интенсивность после работы NPC, порядок должен не удваивать изменение за один кадр. Отдельно реши, когда атмосфера реагирует на кислородный расход: до или после FireSystem, и нужна ли

задержка. Напиши таблицу входов и выходов каждой функции. Не вставляй новую систему «куда удобно» между случайными вызовами `ApplicationClass::Frame`.

4.4. Сцена, Inspector, сохранение, UI

После работающего тика нужно дать автору игры создать очаг в Editor. Добавь сериализацию только authoring полей, например начальная интенсивность, запас топлива и room ID. Runtime указатель на комнату, временные claim, path cache, Bullet pointer и GPU handle не записывай как числа в JSON. Для продолжаемой игры отдельно реши, какие runtime поля нужно сохранять: оставшееся топливо и текущий state обычно важны. После LoadScene восстанови Entity references и runtime ресурсы. В Inspector показывай и редактируемые authoring поля, и read-only live данные, чтобы пользователь не путал сохранённый параметр со значением текущего кадра.

4.5. Связь с NPC и контроль отказов

Новая работа требует обнаружения очага, оценки полезности, claim, доступной WorkZone, маршрута, расхода огнетушителя, прогресса тушения и события завершения. Каждое из этих звеньев имеет отдельную причину отказа: нет очага, он уже занят, путь закрыт, нет ресурса, комната опасна, WorkZone внутри collider. Покажи причину в debug UI. Начни с одного NPC и одного очага, затем два NPC и две цели. Смерть исполнителя должна освободить claim. Ручная команда должна вытеснить automatic работу по ясному правилу. После тушения удалённый Entity не должен оставаться в чужом `repairTarget`.

4.6. Проверка результата в игре

Построй маленькую сцену с двумя комнатами, дверью и одним очагом. Запиши прогноз на 10 секунд: сколько топлива сгорит, до какой интенсивности вырастет пожар, когда инженер войдёт в WorkZone, сколько заряда потратит и когда огонь перейдёт в Extinguished. Выполни прогон при 30, 60 и 120 FPS на одинаковое simulation time. Если числа кратно различаются, проверь `dt`. Сохрани игру посередине, загрузи и убедись, что огонь не создался второй раз, ресурс не вернулся, claim не дублировался. Это минимальный стандарт новой системы до включения её в большую миссию.

Системы DX Framework: чтение реального кода от кадра до пикселя

Эта часть учебника учит читать существующий движок как цепь причин и следствий. Если объект переместился, надо выяснить, кто записал `TransformComponent`; если погас свет, пройти от генератора через сеть питания к `LightComponent`; если пропал предмет после загрузки, проверить перечисление сущностей в `SaveScene`, поле JSON, обратное чтение и remap ссылок. Здесь описано **состояние кода**, проверенное по файлам проекта, а изменения, которых в нём пока нет, помечены **«Проектное предложение»**. Пути вроде `src/Core/ApplicationClass.cpp` отсчитываются от `DX_FrameWork/` внутри репозитория. Номера строк помогают найти блок в текущей версии, но после правок будут сдвигаться.

Ключевой принцип для переучивания: у каждого значения есть владелец, единицы измерения, время жизни и момент записи. `Entity` — числовой ключ, `TransformComponent` — авторское или итоговое положение, Bullet хранит физическое тело, `PowerConsumerComponent::hasPower` — вычисленное за кадр состояние, JSON — снимок части компонентов, а видеопамять и потоковые задания — временные ресурсы. Когда эти категории смешиваются, появляются ошибки, которые выглядят как «физика дрожит», «шейдер не работает» или «сохранение иногда ломается».

1. Один кадр целиком: как читать `ApplicationClass::Frame`

1.1. Входные данные, режим и два времени

Точка входа кадра — `ApplicationClass::Frame(float dt)` (`src/Core/ApplicationClass.cpp:1023`). `dt` выражает прошедшее время кадра в секундах. В начале есть специальная ветка проверки оружия: при `m_weaponQA` значение принудительно становится `1/60`. Затем `DebugHud` обновляется даже при паузе игрового времени. Вызовы `m_inputManager.Update(m_input)` и `m_events.Clear()` (`:1033–1034`) означают соответственно новый снимок ввода и пустой список событий на новый кадр. Событие, оставшееся только в прошлой очереди, здесь исчезнет; производитель и потребитель события должны работать в ожидаемом порядке в одном кадре либо хранить долгоживущее состояние отдельно.

F5 обрабатывается по фронту нажатия (`f5Down && !m_wasF5Down`, `:1078`). При входе в Play вызывается `m_gameplayController.EnterPlayMode`, затем сбрасываются таймер, журнал событий мира, trigger-состояние, взаимодействие и камера. Для игрока создаётся инвентарь и тестовая винтовка; сцена для наблюдения за AI без игрока отдельно защищена проверкой `player != INVALID_ENTITY`. При выходе из Play очищаются команды движения у `CharacterMovement` и контроллеров. Важная привычка: читать **условие вызова** вместе с вызываемым методом. Комментарий возле выхода из Play о том, что физика продолжает шагать в Editor, устарел относительно фактического вызова `UpdatePhysics(isPlayMode ? simulationDeltaTime : 0.0f)` и раннего возврата при `dt <= 0` (`:1570`, `:2276`). По текущему вызову Editor не шагает Bullet и не выполняет затопление/давление.

На строках `1185–1186` вводятся `simulationDeltaTime` и `simulationRunning`. В обычном Play это время симуляции, в NPC Studio его может изменить пауза или одиночный шаг (`:1279–1283`); `simulationRunning = simulationDeltaTime > 0` (`:1352`). Поэтому `dt` не всегда равно времени игрового мира. UI, камера и диагностические сообщения могут обновляться по кадру, а оружие, AI и физика — по симуляционному времени. Когда добавляешь таймер перезарядки, выбери время осознанно: при паузе Studio перезарядка должна остановиться, значит ей нужен `simulationDeltaTime`. Когда обновляешь анимацию воды в окне редактора, используется `dt`, чтобы визуал оставался живым (`:1571`). Если перепутать их, ошибка обнаружится лишь в паузе или при нестандартной частоте кадров.

1.2. Порядок стадий и зависимость от предыдущей записи

После ввода главный поток выполняет `m_jobSystem.ExecuteMainThreadJobs()`, `m_assetLoadSystem.Update(device)` и `UpdateAsyncShipImport(...)` (`:1228–1234`). Первый вызов переносит готовые фоновые результаты на главный поток; второй может создать буферы Direct3D из подготовленных CPU-данных. Затем идут редакторский ввод, взаимодействие, подбор оружия, двери и ремонт. Камера до движения считывает новый yaw мыши (`:1405–1442`), чтобы направление намерения игрока и взгляда было согласовано. Блок `isPlayMode && simulationRunning` (`:1448`) формирует намерения управления и последовательность NPC. Таймеры оружия обновляются **до** попыток стрелять (`:1467`); AI принимает решение, движение и контроллеры получают команду, анимация выбирает клипы; `DamageSystem` потребляет события урона (`:1549–1551`). Отдельная глава книги разбирает NPC, здесь важно лишь место их записи в общей цепи.

Далее идут `SyncCompartmentPositions`, `UpdatePhysics`, `UpdateWaterAnimation` (`:1566–1571`). В `UpdatePhysics` строго задана последовательность: затопление, создание соединений, обновление корня подлодки, физика Bullet, проверка сломанных соединений, давление (`:2274–2302`). Это не косметический порядок. Затопление изменяет объём воды до расчёта выталкивания; соединение создаётся до физического шага; разрушение соединения проверяется после нагрузки; давление читает уже новую позицию тела. Значит новый ущерб от давления станет виден следующей фазе, которая его читает, а не «одновременно» со старым состоянием.

После физики камера следует итоговому `TransformComponent` игрока (`:1583–1613`). Далее `SceneManager::UpdateAll` получает ноль игрового времени в Editor (`:1628–1631`), обновляет анимационные и корабельные системы, а `UpdateLights` формирует положение светильников. Лишь затем камера строит view-матрицу (`:1634–1635`). Отдельный блок позднее вычисляет луч оружия: прицел берётся из камеры, но фактический выстрел начинается от дула. `WeaponSystem::Update` и `NpcRangedCombatSystem::Update` идут после позы и физики (`:1757–1769`), поэтому положение дула соответствует уже актуальной руке. Наконец `m_renderer->RenderFrame` (`:2010`) читает подготовленную сцену. Если новую систему помещают до той, чьи данные она читает, она увидит вчерашнее состояние; если после рендера, эффект станет заметен лишь через кадр.

Практика чтения кадра: выпиши для новой системы четыре пункта — вход, источник времени, изменяемые компоненты, первый читатель результата. Например «электроснабжение»: вход — мощности генераторов и потребностей, время — `deltaTime` из `SceneManager::UpdateAll`, запись — `hasPower`, читатели — свет, кислородный генератор, насос. Затем найди все вызовы этих читателей, а не только название класса. Так выявляется реальный pipeline.

1.3. Почему один кадр не равен одному шагу физики

`PhysicsWorld::Update` (`src/Core/PhysicsWorld.h:86–113`) складывает `deltaTime` в аккумулятор, ограничивает его `0.25` секунды и, пока хватает времени, шагает Bullet фиксированным `1/60` секунды. В кадре длительностью `1/120` секунды физический шаг произойдёт через два визуальных кадра; при резком лаге может произойти несколько шагов подряд, но не бесконечная «догонялка». `m_interpolationAlpha = accumulator / timestep` предназначен для сглаживания визуала между фиксированными состояниями. Это причина, почему нельзя рассчитывать игровую скорость как «два метра за вызов Update»: нужно умножать скорость в м/с на секунды. И наоборот, нельзя в коде силы умножить на `dt`, если затем эту величину как силу передать в `applyCentralForce` и Bullet уже интегрирует её по шагу: тогда время фактически учитывается дважды. Для конкретной силы проверяй единицы и API тела.

2. ECS: сущность, компонент, система и владение состоянием

2.1. `Entity` — ключ, а не объект

В `src/ECS/ECSRegistry.h:15-21` метод `CreateEntity` проверяет исчерпание пространства ID, выдаёт `nextEntity++` и помещает ID в `m_liveEntities`. У сущности нет встроенной позиции, здоровья или меша. Это числовая связь между отдельными хранилищами компонентов. `DestroyEntity` (`:24-28`) убирает ключ из множества живых и вызывает `Remove` во всех пулах. `IsAlive` проверяет именно множество живых (`:30`). В этой реализации не видно generation counter: после полного `Clear` реестр создаётся заново и ID опять начинаются с нуля. Поэтому долгоживущую ссылку на `Entity` нельзя переносить через `LoadScene` или `Clear`, даже если после загрузки цифра случайно совпала. До использования сохраняемой ссылки обычно нужны `IsAlive`, наличие нужного компонента и, если это сериализованная ссылка, корректный remap.

`AddComponent<T>` делегирует вставку пулу (`:33-43`). У метода нет проверки `IsAlive(e)`: вызывающая сторона обязана создать сущность и не добавлять компонент к уничтоженному ID. Более того, `ComponentManager<T>::Add` сначала удаляет прежнюю запись, затем делает `emplace` (`src/ECS/Components/ComponentManager.h:29-42`). Это **замена всего компонента**, а не изменение одного поля. Если код сохранил `T*` и затем повторно вызвал `AddComponent<T>` для того же Entity, старый указатель уже недействителен. Для обычного изменения состояния бери существующий компонент через `GetComponent` и меняй конкретное поле; `AddComponent` используй при создании или осознанной полной замене.

`GetComponent<T>` возвращает указатель или `nullptr` (`ECSRegistry.h:61-65`), `HasComponent<T>` даёт булево (`:67-71`), `RemoveComponent<T>` удаляет запись при наличии пула (`:73-77`). `GetComponents<T>` вызывает `GetOrCreatePool<T>` даже при намерении только прочитать пул (`:79-82`). `ForEach<T>` не создаёт пустое хранилище, если тип ещё нигде не использовался (`:83-92`). Двухкомпонентный `ForEach<T1,T2>` (`:45-59`) перебирает **первый** пул и ищет такой же Entity во втором. Поэтому выбери более редкий тип первым: если `PressureHullComponent` имеют 20 сущностей, а `TransformComponent` — 3000, `ForEach<PressureHullComponent,TransformComponent>` делает существенно меньше итераций. Это вывод из структуры обхода, а не гарантия порядка: внутри `unordered_map` порядок сущностей не задан.

`ActiveComponent` хранится особым образом как множество ID (`ComponentManager.h:69-96`). У него нет индивидуальных полей на каждую сущность; `Get` возвращает указатель на один `m_dummy`. Для проверки активности используй `HasComponent<ActiveComponent>`. Нельзя записывать в найденный через `Get` экземпляр и ожидать уникального состояния объекта. И важное отличие: живость и активность независимы. Неактивный объект остаётся сущностью, может сохраняться в сцену и иметь все компоненты; системы сами решают, пропустить его или нет. Например физика отключает контактную реакцию и движение неактивного RigidBody (`PhysicsSystem.cpp:1007-1027`), а `PressureSystem::Update` явной проверки `ActiveComponent` не делает. Если правило нового эффекта требует паузы для неактивных объектов, добавь проверку в этом эффекте.

2.2. Где живёт положение объекта

`TransformComponent` содержит `position`, `rotation`, `scale` и вычисляет мировую матрицу как `scaleMatrix * rotationMatrix * translationMatrix` (`src/ECS/Components/Components.h:27-40`). Углы вращения применяются в порядке X, затем Y, затем Z. Для соглашения `mul(position, matrix)` в шейдере позиция вершины сначала масштабируется, потом вращается, потом переносится. `SceneManager::UpdateAll` прямо не поддерживает отдельную копию трансформации в старом `GameObject`; `RenderAllECS` читает `transform->GetWorldMatrix()` (`SceneManager.cpp:2457`). Значит обычный путь изменения позиции для отображения — компонент, а не

ручной сдвиг матрицы только перед одним draw call. Исключение — объекты с Bullet: там надо понимать, кто хозяин на данном этапе.

Кинематическое тело получает положение из ECS каждый кадр; динамическое обычно получает положение из Bullet, а внешнее редактирование `TransformComponent` распознаётся и отправляется обратно в Bullet. Статическое тело не интегрируется динамикой, но может быть явно пересинхронизировано при изменении трансформации. В `PhysicsSystem.cpp:965–982` сравниваются `lastSyncedPosition`, `lastSyncedRotation`, `lastSyncedScale` с текущим `Transform`. Это не «почти равно», а прямое сравнение записанных значений. Для внешней телепортации динамического тела (`:1080–1113`) обновляются world transform, motion state, interpolation transform и AABB, затем сбрасываются скорости и силы. Поэтому нельзя использовать прямую запись `Transform.position` как способ плавного толчка: это будет воспринято как телепорт. Для толчка применяй силу/импульс к Bullet-телу в подходящей фазе. После шага динамическое тело пишет позицию и вращение обратно в ECS (`:1353–1394`). Именно это означает «владелец Transform» в этой системе: постоянное хранилище отображения — ECS, но авторитет движения динамического объекта во время симуляции — Bullet.

2.3. Как безопасно итерировать и удалять

Обычные пулы — `std::unordered_map<Entity,T>` (`ComponentManager.h:15–56`). `ForEach` держит итератор пула, пока выполняет callback. Удаление текущей записи внутри callback инвалидирует текущий итератор; добавление того же типа может вызвать rehash и сломать обход. Поэтому для массового удаления сначала собери ID в отдельный `std::vector<Entity>`, закончи обход, затем удали. Если создаёшь компоненты другого типа, проверь, не держит ли параллельный обход указатели в этом пуле. Здесь не требуется универсальный запрет на любую запись, но структурную модификацию **перебираемого** пула нужно считать потенциально опасной.

Удаление сущности сцены — больше чем `registry.DestroyEntity`. `SceneManager::RemoveObject` (`src/Scene/SceneManager.cpp:2092–2122`) сначала убирает соединения Bullet для обоих видов joint, удаляет `ActiveComponent`, освобождает уникальный материал, уничтожает runtime character controller, затем удаляет компоненты из реестра. Если обойти `RemoveObject` и удалить только ECS ID, constraints могут сохранить указатель на уже уничтоженное тело. Для новой системы заранее реши, есть ли у компонента внешний ресурс: Bullet body, GPU buffer, открытый дескриптор, очередь job или кэш NavMesh. Если есть, у удаления нужен явный этап освобождения. `Clear` (`:2128–2161`) сбрасывает асинхронный импорт, инвалидирует NavMesh, вызывает `ReleasePhysicsRuntime`, удаляет материалы и кэш моделей, после чего целиком заменяет `m_registry = ECSRegistry{}`. `ReleasePhysicsRuntime` (`:2163–2178`) сначала отсоединяет constraints, затем контроллеры персонажей — порядок обусловлен зависимостями Bullet. Это полезный образец для любого ресурса, который ссылается на другой.

3. Сцена и сохранение: путь от Entity к JSON и обратно

3.1. Что именно сохраняет `SaveScene`

`SceneManager::SaveScene` (`src/Scene/SceneManager.cpp:3270`) создаёт JSON с `scene_name`, `version = 2` и массивом `objects`. Лямбда `saveEntity` проверяет `IsAlive`, пропускает `NpcWeaponVisualComponent` (визуал экипировки восстанавливается позже) и `ShipPartComponent` (импортированные части восстанавливаются из корня GLB), а множество `savedEntities` защищает от дублирования одной сущности, найденной в нескольких пулах (`:3277–3289`). Затем метод перечисляет корни кораблей, объекты с мешем, отсеки, комнаты, поверхности NavMesh, физические тела, пробоины, NPC без меша, оружие, области мира, взаимодействия, триггеры, инвентарь, предметы и именованные логические сущности (`:3292–3415`). Это список **владельцев включения** в файл. Если ты создашь новый «чисто логический» компонент без

меша, имени или любого уже перечисляемого признака, он может не попасть в JSON, даже если `SerializeObject` умеет записать его поля. Нужно проверить и перечисление, и сериализацию.

На `:3417–3420` файл открывается и записывается `scene.dump(4)`. Возврат `true` означает, что `ofstream` удалось открыть и запись дошла до конца функции; явной проверки ошибки после `file <<` здесь нет. `SerializeObject(entity,id)` (`:3428–3433`) пишет два разных идентификатора: `id` — последовательный индекс объекта в документе, `entity_id` — реальный runtime ID на момент сохранения. Ссылки других компонентов ремапятся по второму полю. Не путай индекс в массиве с ключом ECS. Хранить в ручном JSON ссылку на `id` вместо `entity_id` — значит потерять цель при загрузке.

В коде чтения есть блок `scene["world"]` для fog и skybox (`LoadScene:4314–4326`), но в текущем `SaveScene` нет присваивания `scene["world"]`. Следствие: новые файлы, записанные этим методом, не несут мировые настройки этим путём, хотя старые или внешне подготовленные файлы могут их восстановить. Это факт конкретной реализации, а не обещание, что какой-то другой редакторский механизм обязательно исправляет пробел. **Проектное предложение:** включить `WorldSettings` в `SaveScene` симметрично `LoadScene`, затем проверить round-trip тумана и пути skybox на новой сцене.

3.2. Почему загрузка не может просто восстановить старые числа ID

`LoadScene` (`:4301–4455`) сначала открывает файл и парсит JSON, после этого очищает старую сцену (`Clear`). Затем создаёт таблицу `old entity_id → new Entity` и набор `restoredEntities`. Для корня корабля с `has_ship_root` вызывается `ImportShipFromGlb`, после чего возвращённому корню применяются позиция, вращение, масштаб, состояние аккумулятора и сохранённые данные генераторов; части корабля создаются импортом заново. Обычный объект идёт через `DeserializeObject`. На `:4476–4490` десериализатор вызывает `CreateEntity`, создаёт `TransformComponent` и по полям JSON добавляет `ActiveComponent`, `InvulnerableComponent`, `NameComponent`. Далее большой метод восстанавливает остальные поддержанные типы. Важно: даже если прежний ID был `41`, новый может оказаться `76`: импорт корня GLB и его частей расходует ID в другом порядке. Поэтому ссылка «дверь ведёт в комнату 41» не готова к употреблению до remap.

Когда все объекты созданы, `RemapEntityReferences(remap,restoredEntities)` (`:4433–4437`, определение `:5890`) проходит по поддержанным ссылочным полям. Обрабатываются только объекты из сериализованного набора, чтобы случайно не переназначить новые ID импортированных деталей корабля, которые уже указывают друг на друга правильно. Затем физические тела, сохранённые лишь как данные компонента, получают новые Bullet runtime bodies (`:4439–4448`), разрешаются зоны ремонта и соединения комнат. Это двухфазный протокол: сначала создание, потом связи и ресурсы. Если добавить компонент, хранящий `Entity target`, но забыть `RemapEntityReferences`, после загрузки цель может стать другой сущностью или не существовать. Для новых ссылок удобно требовать `INVALID_ENTITY` при отсутствующей цели.

В `RemapEntityReferences` есть два варианта: `remapEntity` при отсутствии записи печатает диагностику и возвращает старый ID (`:5894–5905`), `remapEntityOrInvalid` возвращает `INVALID_ENTITY` (`:5906–5912`). Первый вариант допускает, что старое число случайно совпадёт с новым живым Entity; для связи, которая должна быть гарантированно сериализована, это риск. **Проектное предложение:** для каждой ссылки явно определить политику «цель отсутствует» и использовать её, а не оставлять старый ID по умолчанию; проверить на сцене, где целевой объект специально удалён из JSON. Парсер `file >> scene` не обёрнут локальной обработкой исключений и `LoadScene` не проверяет схему до `Clear`. Битый JSON бросит исключение; неверное поле может привести к частично восстановленной сцене. Это не повод дописывать «магический» catch вслепую: сначала нужны тестовые файлы для отсутствующего `objects`,

неверного типа координат и отсутствующего asset, затем явная политика ошибки и безопасная точка замены сцены.

3.3. Упражнение: добавляем сохраняемый «маяк тревоги»

Проектное предложение, не существующая функция. Допустим, `AlarmBeaconComponent` содержит `float radiusMeters` и `Entity owner`. Путь реализации: создать компонент; решить, кто создаёт его Entity (редактор, импорт корабля или gameplay); обеспечить перечисление сущности в `SaveScene`, если она может быть без меша и имени; записать `has_alarm_beacon`, радиус и старый `owner` в `SerializeObject`; прочитать поля в `DeserializeObject`; добавить `owner` в remap; после загрузки пересоздать любые runtime-ресурсы (например, свет) и проверить `IsAlive(owner)`. На тестовой сцене разместить два маяка, один выключенный, сохранить, очистить и загрузить. Сравни не только количество компонентов, но и радиус, активность, связь с правильным кораблём и видимость в рендере. Это цепочка проверки, а не один тест функции сериализации.

4. Bullet и ECS: два представления одного физического объекта

4.1. Создание мира и тела

`PhysicsWorld::Initialize` (`src/Core/PhysicsWorld.h:20–84`) создаёт collision configuration, dispatcher, broadphase, constraint solver и `btDiscreteDynamicsWorld`; включает гравитацию `(0,-9.81,0)`, ghost callback для персонажей и GImpact для динамических вогнутых мешей. Настроены 50 итераций солвера, split impulse и ERP `0.2`. Эти значения — параметры текущей реализации, а не универсальные константы физики. Они влияют на устойчивость соединений тяжёлых отсеков и стоимость шага. Метод сначала создаёт зависимости во временных `unique_ptr` и переносит их в поля лишь после успешной настройки. Это уменьшает риск мира, который наполовину инициализирован. Повторная инициализация при уже существующем мире возвращает успех без пересоздания: нельзя ожидать, что вызов `Initialize` «обнулит» тела.

`PhysicsSystem::CreateRigidBody` (`src/ECS/System/PhysicsSystem.cpp:554–960`) требует `RigidBodyComponent` и `TransformComponent`; дальше выбирает форму по `shapeType`, строит Bullet shape, local inertia для массы, motion state, `btRigidBody`, collision group/mask и добавляет тело в мир. Внутри есть ветки Sphere, Box, Cylinder, Capsule, Plane, ConvexHull, TriangleMesh, GImpactMesh, составные элементы и особые внутренние коллизии корабля. При чтении этого длинного метода не пытайся запомнить каждую строку. Сначала выдели инвариант: на входе у сущности есть авторские параметры формы/массы и world transform; на выходе `rb.body` существует в Bullet и `lastSynced...` соответствует `Transform`. Если редактируется форма, старое тело нужно снять из мира и создать заново; в `RefreshRigidBody` (`:491–554`) сохранён старый transform, удаляются body и motion state, затем вызывается создание. Поэтому изменение масштаба нельзя считать обычным сдвигом матрицы: collision shape часто зависит от масштаба и требует перестройки.

В `PhysicsWorld` fixed step `1/60` секунд, но `PhysicsSystem::Update` получает кадровый `deltaTime` и до физического шага применяет силы плавучести. Если рендер идёт быстрее или медленнее 60 FPS, здесь стоит отдельно проверять, сколько раз силы ставятся относительно реальных Bullet шагов. Код уже ограничивает накопление времени, но сила задаётся на вызов `PhysicsSystem::Update`, а шаги выполняются внутри `PhysicsWorld::Update`. **Проектное предложение:** для воспроизводимой симуляции перенести расчёт сил, зависящих от физического шага, в цикл fixed step или явно проверить текущую схему на 30/60/144 FPS. На тесте одинаковый корабль должен иметь близкую траекторию через десять секунд игрового времени при разных частотах отображения.

4.2. Разбор `PhysicsSystem::Update` по фазам

На `PhysicsSystem.cpp:984–988` берутся пулы плавучести, тел и трансформаций. Первая фаза (`:1003–1145`) идёт по `RigidBodyComponent`. Если `rb.body == nullptr`, объект не участвует в Bullet и пропускается. Если нет `ActiveComponent`, тело получает флаг `CF_NO_CONTACT_RESPONSE`, обнуляет скорости и силы, переводится в `DISABLE_SIMULATION`; прежние collision flags и activation state сохранены для восстановления при включении (`:1007–1040`). Это runtime-остановка, не удаление авторского компонента. При повторной активации флаги и состояние возвращаются, тело будится.

Далее требуется `TransformComponent`. Для кинематического тела (`:1045–1076`) ECS всегда авторитетен: позиция и Euler-вращение преобразуются в `btTransform`, записываются в body и motion state, обновляются interpolation transform и AABB. Если изменился scale, вызывается `RefreshRigidBody` и текущая итерация заканчивается, поскольку форма зависит от размера. Для динамического тела (`mass > 0`, `:1077–1116`) обычное движение живёт в Bullet. Только если `TransformChangedExternally` обнаружил отличия от последней обратной синхронизации, ECS принудительно отправляется в Bullet: это телепорт/редакторское изменение, со сбросом скоростей и сил. Для тела с нулевой массой есть отдельная ветка внешнего изменения (`:1117–1144`) без динамической интеграции. Следовательно, при добавлении новой системы движения сначала реши: объект физически движется из-за сил либо следует программному пути? Для первого варианта не перезаписывай `Transform` каждый кадр; для второго явно выбирай кинематику.

Вторая фаза (`:1149–1344`) рассчитывает плавучесть по `BuoyancyComponent`. Третья синхронизирует ghost-персонажей из ECS до шага, вызывает `PhysicsWorld::Get().Update(deltaTime)` и читает ghost-позиции после шага (`:1345–1348`). `SyncCharacter` (`:32–217`) учитывает смещение центра капсулы относительно визуальных ног, отслеживает внешнюю правку позиции, защищает от NaN и сглаживает визуальную позицию при простое. Комментарий `:87–93` объясняет, почему визуальный поворот персонажа не следует каждый кадр вталкивать в симметричную Y-up капсулу: лишний warp может потянуть контроллер назад из-за интерполяции. Это пример разделения «где смотрит меш» и «как столкновения видят капсулу».

Четвёртая фаза (`:1353–1395`) берёт динамические активные тела, пропускает кинематические и статические, получает Bullet world transform и записывает позицию/вращение в ECS. Перед записью есть защита от NaN координат и от `abs(y)>10000`, в проблемных случаях тело останавливается. После успешной записи обновляются `lastSynced...`: на следующем кадре `TransformChangedExternally` не примет собственный результат Bullet за внешнюю телепортацию. Проверка NaN здесь касается только позиции, а не всех полей quaternion или масштабов; это граница защиты, о которой важно помнить при поиске неисправной физики.

4.3. Пример: импульс, телепорт и статическая преграда

Представь коробку массой 10 кг с `RigidBodyComponent::Box` на координатах `(0,2,0)`. Если нужно «толкнуть» её взрывом, игровая система получает `rb.body` и применяет Bullet импульс в выбранном направлении. Затем шаг Bullet меняет скорость и координаты, а обратная синхронизация обновляет ECS. Если вместо этого каждый кадр писать `transform.position.x += speed*dt`, `TransformChangedExternally` будет запускать ветку телепорта и обнулять скорости; объект может проходить сквозь динамические ожидания и разрушать естественную реакцию столкновений. Телепорт же при респавне как раз должен обновить world transform, motion state, AABB и сбросить старую скорость: в таком сценарии ветка текущего кода соответствует смыслу действия. Статическая преграда с массой ноль не падает под гравитацией; если дизайнер подвинул её в редакторе, Bullet должен получить новый transform, иначе луч попадания и визуальная стена разойдутся.

Для отладки ставь рядом три наблюдения: `TransformComponent::position`, `rb.body->getWorldTransform().getOrigin()` и `rb.lastSyncedPosition`. Если первое меняется до шага, а второе нет, ищи проверку активности, отсутствие body или неверную ветку sync. Если второе меняется, а первое нет, ищи пропуск обратной фазы, статическое/кинематическое тело или защиту NaN. Если визуал меняется, а луч Bullet попадает в старое место, проверь AABB и перерождение формы после scale. Такое сравнение локализует проблему лучше, чем перестановка `Update` наугад.

5. Вода, глубина, пробоины и атмосферное давление: не смешивать среды

5.1. Текущая физическая среда проекта

Хотя проект развивается в сторону космического симулятора, конкретные `PressureSystem`, `FloodingSystem`, `BuoyancyComponent`, `SubmarineComponent` и часть renderer описывают **подводную** среду. У них есть горизонтальный `waterLevel`, глубина `waterLevel - worldY`, плотность воды и выталкивающая сила. В вакууме формула давления от глубины неприменима: наружное давление близко к нулю, для повреждения от разгерметизации важна разность между внутренним давлением отсека и внешним. Эти модели можно использовать как отдельный «водный режим» корабля, но учебник не должен выдавать их за уже реализованную космическую разгерметизацию. Внутренние комнаты `RoomEnvironmentSystem` и кислород — другая подсистема (`SceneManager.cpp:2287–2291`), её состояние нужно соединять с пробоинами отдельным контрактом.

`PressureSystem::Update` (`src/ECS/System/PressureSystem.cpp:5–79`) объявляет `P0=101325 Pa`, `ρ=1000 kg/m³`, `g=9.81 m/s²`. Перебирает `PressureHullComponent`. Неуязвимость убирает флаги течи и пропускает урон, но наличие/отсутствие `ActiveComponent` в этом методе не проверяется. Объект без `RigidBodyComponent` или `rb.body` пропускается; глубина берётся из **Bullet**, а не из ECS. `depth=max(0,waterLevel-bodyY)`. Давление записывается в `hull.lastPressure` и считается как `P=P0+ρgh`. При превышении `hull.maxPressure` отношение `pressure/maxPressure` формирует `damagePerSecond=(ratio-1)*20`, здоровье убывает на `dps*deltaTime`, ниже нуля не идёт, утечка включается при здоровье меньше 100. Иначе здоровье медленно растёт на `repairRate*deltaTime*0.1` до 100. Это игровая кривая повреждения, а не формула прочности реального корпуса. `maxPressure` по умолчанию `202000 Pa` (`Components.h:778–786`); при 10 м глубины давление `101325+1000·9.81·10=199425 Pa`, ещё ниже порога, а около 10.3 м порог будет превышен. Если `maxPressure<=0`, код делит на него без валидации — проектная задача для конфигурационного контроля.

5.2. Почти построчный путь воды через отсек

`FloodingSystem::Update` (`src/ECS/System/FloodingSystem.cpp:5–143`) берёт отсеки, transform и связи с подлодкой. Отсек без `SubmarineComponent`, без валидного корня, сам корень или без `TransformComponent` пропускается. Это важно при создании нового корабля: одной пробоины в списке `CompartmentComponent::breaches` недостаточно; нужна правильная привязка к корню. Для каждой незапечатанной `BreachData` позиция хранится локально в отсеке. `XMVector3TransformCoord(localPos, compartmentWorldMatrix)` превращает её в world position (`:29–38`). Затем глубина пробоины `globalWaterLevel - breachWorldPos.y`. Над поверхностью расход ноль, под поверхностью вычисляется `pressure=breachDepth*9.81f`, затем `CalculateFlowRate`. В `CalculateFlowRate` (`:227–237`) площадь считается как `π·size²`, скорость как `sqrt(2·waterPressure/1000)`, расход как `area·velocity`. Именно здесь видно несогласование единиц: если `waterPressure` должен быть в паскалях, для глубины `h` нужно `ρgh`, а сейчас передаётся лишь `gh`. Получаемый расход примерно в `sqrt(1000)` раз меньше физически согласованного с такой интерпретацией. Возможно, это намеренно ослабленная игровая скорость, но тогда переменная `pressure` и комментарий о Па вводят в заблуждение; коэффициент баланса надо выделить явно. Не меняй это число автоматически в готовой сцене: течение может резко ускориться.

Например, пробоина радиуса `0.1 м` на глубине `10 м`: площадь `π·0.1²≈0.0314 м²`. Текущий код передаст `98.1`, даст `v≈0.443 м/с` и `Q≈0.0139 м³/с`. При гидростатическом перепаде `98100 Па` и той же формуле `v≈14.0 м/с`, `Q≈0.439 м³/с` до коэффициента расхода и иных ограничений. Для игры может быть разумным расход между ними, но его надо выбирать как параметр дизайна в единицах, а не получать из пропущенной плотности. **Проектное предложение:** ввести `pressureDifferencePa`, `dischargeCoefficient`, ограничение по доступному объёму/размеру шага и тесты на глубинах 0, 1, 10, 100 м.

Насос на `:73–77` откачивает `comp.pumpRate`, только если `comp.hasPower` и воды больше нуля. Чистый объём за кадр равен `(totalInflow-totalOutflow)*deltaTime`; `waterVolume` ограничивается `[0,comp.volume]` (`:84–86`). Высота воды выводится как доля заполнения, умноженная на геометрическую высоту отсека (`:91–99`). Для этого нужны `comp.volume>0` и положительная высота; в коде проверена высота, но в ветке `waterVolume/comp.volume` нет отдельной проверки ненулевого объёма. Потом создаётся `DamageDecalComponent`, если его не было, и синхронизируется с пробоинами (`:101–106`). Это перевод логического повреждения в визуальные данные для шейдера. `CreateBreachAtWorldPosition` (`:175–193`) использует обратную мировую матрицу: координата попадания мира превращается в координату отсека и сохраняется локально, поэтому маркер движется вместе с корпусом. `SealBreach` (`:198–204`) только выставляет `isSealed`; накопленная вода не исчезает и должна быть откачана отдельно.

Текущая связь насоса с сетью питания неполная. `CompartmentComponent::hasPower` по умолчанию `true` (`Components.h:692`) и сохраняется/читается как `compartment_has_power` (`SceneManager.cpp:4221`, `:4963`). По поиску в текущем `src/` нет записи из `PowerConsumerComponent::hasPower` обратно в `CompartmentComponent::hasPower`; `ShipPowerGridSystem` меняет свои consumer-поля, а не это поле. Поэтому нельзя утверждать, что отключение реактора автоматически остановит откачку каждого отсека. **Проектное предложение:** назначить насос отдельным consumer, определить поведение при частичном `powerRatio`, а вычисленное питание использовать до `FloodingSystem`. Сейчас `SceneManager::UpdateAll` с расчётом сети вызывается **после** физической стадии `FloodingSystem`; при исправлении зависимости надо осознанно перестроить порядок, иначе насос увидит состояние прошлого кадра.

5.3. Плавучесть и два противоречащих действия кода

`PhysicsSystem::Update` (`:1149–1344`) вычисляет погружение по вертикальной области тела `[objectY-height/2, objectY+height/2]` относительно поверхности. Если объект внутри отсека, вместо внешней поверхности берётся уровень воды отсека (`:1175–1202`). Для корня подлодки суммируется вода во всех отсеках. Сглаженный коэффициент погружения — смесь старого значения и нового с коэффициентом `0.3`; это уменьшает скачок силы, но вводит временную задержку и зависит от количества кадров. Выталкивающая сила считается как `fluidDensity * g * buoyancy.volume * submersionRatio * buoyancyMultiplier` (`:1271–1272`), добавочный вес воды — `totalWaterInCompartments * 1000 * g`, итоговая вертикальная сила — разность (`:1274–1286`). Затем есть сопротивление по скорости, ослабление вертикального drag при затоплении, пружинная стабилизация при малом затоплении и угловое демпфирование. Всё это смешивает физическое приближение с игровыми стабилизаторами: формулу можно объяснять, но коэффициенты `50`, `5`, `0.3` нельзя называть законом природы.

Важная несогласованность: комментарий и формула в `PhysicsSystem.cpp:1253–1279` говорят, что внешний вытесняемый объём корпуса при заливании воды **не уменьшается**, а вода добавляет вес. Но `FloodingSystem.cpp:116–140` при `lastCalculatedVolume>0` прямо записывает `buoy->volume = max(0,lastCalculatedVolume-totalWaterVolume)`. Следовательно, в таком состоянии вода одновременно уменьшает выталкивающую силу и добавляет вес — двойной штраф. Пока `lastCalculatedVolume<=0`, уменьшения объёма не будет, поэтому проявление зависит от того, кто и когда вычислил это поле. **Проектное предложение:** выбрать единую модель: внешний объём остаётся постоянным, а масса

внутренней воды добавляется один раз; либо явно описать и проверить альтернативу для открытого корпуса. Создать диагностическую сцену с известной массой и объёмом, по шагам прибавлять 0, 1, 2 м^3 воды, записывать `buoy.volume`, `waterWeight`, `netUpForce` и высоту через 10 секунд при разных FPS. Только после этого править формулы.

Ещё одна деталь: `FloodingSystem::CreateBreach` (`:148–169`) проверяет, выходит ли локальная точка за границы `localMin/localMax`, но тело ветки пустое — фактического запрета или clamp нет. Если учебный пример «стреляем в стену» создаёт дырку вне отсека, функция всё равно добавит её в `breaches`; мировой уровень воды будет вычисляться из этой точки. Следует либо явно разрешать внешнюю точку с объяснением, либо возвращать ошибку/подрезать координату. Такое исправление относится к проектному предложению, а не к текущему поведению.

5.4. Космический режим как отдельный контракт

Проектное предложение. Для космического симулятора добавь тип среды `EnvironmentMode` или отдельную конфигурацию мира. В режиме Space наружное давление задаётся в Па, гравитация берётся из локального источника, нет глобальной `waterLevel`, `FloodingSystem` и плавучесть не запускаются. Комната хранит объём в м^3, количество газа/давление, температуру, соединения дверей и пробоины в вакуум. Поток газа определяется разностью давлений и геометрией отверстия, а здоровье экипажа — парциальным давлением кислорода и временем воздействия. `PressureHullComponent` в таком режиме сравнивает **разность** внутреннего и наружного давления с прочностью, а не глубину. Один и тот же `HullBreachComponent` может быть входом двум моделям, но алгоритмы и единицы должны быть разделены. Начни с простого теста: отсек 100 м^3 при 101325 Па, закрытые двери, пробоина в вакуум — давление должно монотонно падать, масса газа не возрастать, а без пробоины оставаться постоянной. Реалистичные скорость звука и истечение можно добавить после проверки сохранения массы; пока это проект, не заявляй, что он уже реализован.

6. Электросеть, генератор и зависимые системы корабля

6.1. Генератор производит мощность, сеть её распределяет

В `SceneManager::UpdateAll` (`src/Scene/SceneManager.cpp:2237–2250`) сначала выключаются генераторы у уничтоженных объектов, затем вызывается `GeneratorSystem::Update` и только после него `ShipPowerGridSystem::Update`. `GeneratorSystem` (`src/Gameplay/Power/GeneratorSystem.h:13–113`) пропускает неактивную сущность, обнулив текущую выходную мощность и расход топлива, но сохранив настройки. При `deltaTime<=0` не расходует топливо и не меняет тепловое состояние. Для активного генератора он ограничивает `maxOutputMw`, `targetOutputMw` и остатки дейтерия/трития допустимыми диапазонами; проверяет наличие обоих видов топлива; при превышении `scramTemperature` ставит `isScrammed=true`, `isOnline=false`, а повторный запуск разрешается только после охлаждения до `restartTemperature`. `currentOutputMw` приближается к эффективной цели максимум на `outputRampMwPerSecond*deltaTime`, то есть не прыгает мгновенно. Затем из отношения текущей мощности к максимуму выводятся расход топлива в кг/с и изменение температуры. Формула температурного сглаживания `1-exp(-response*dt)` даёт более устойчивый результат при разном `dt`, чем постоянное «добавь 10% каждый кадр».

`ShipPowerGridSystem::Update` (`src/Gameplay/Power/ShipPowerGridSystem.h:18–62`) собирает временные `WorkingGrid` отдельно по корню корабля. `FindGridRoot` (`:74–84`) возвращает собственный Entity, если на нём `PowerGridComponent`, иначе `ShipPartComponent::root`, иначе `INVALID_ENTITY` для локальной сети вне корабля. Это защищает от питания одного корабля генератором другого, но все потребители без корня попадают в один условный `INVALID_ENTITY` grid: эту семантику стоит учесть при добавлении независимых

станций. Активный генератор добавляет `max(0,currentOutputMw)` только если `isOnline && !isScrammed`. Каждый consumer перед сбором сбрасывает `suppliedMw`, `powerRatio`, `hasPower`; неактивный остаётся обесточенным. Его `requiredMw` добавляется к demand без отрицательных значений. Если ошибочно считать `hasPower` сохранённым авторским переключателем, сброс каждый кадр будет выглядеть багом. Это вычисленная телеметрия, которую нужно заново получить от сети.

`UpdateGrid` (`:86-154`) начинает с мощности генераторов. Если `demandMw` больше `availableMw` и есть `PowerGridComponent`, вызывается разряд аварийной батареи на дефицит. Затем приоритеты `0..3` обходятся по порядку; в каждом собираются потребители с положительной потребностью и вызывается `SupplyEvenly`. Значит низшее число получит ресурс раньше. `SupplyEvenly` (`:156-189`) делит имеющееся поровну между потребителями данного приоритета; потребитель с малой нуждой насыщается, остаток перераспределяется на следующем круге. После распределения `powerRatio=clamp(supplied/required,0,1)`, `hasPower` становится true только от `0.999`. У потребителя с `requiredMw<=0` считается полное питание (`:119-123`). Телеметрия grid записывает generation, demand, supplied, output батареи, резерв, load ratio и brownout. Система электричества не знает о насосе или лампе по имени — она выставляет общий контракт consumer.

Пример. Генератор дал 8 MW, батарея недоступна. Приоритет 0: устройства требуют 3 и 2 MW — оба получают полностью, остаётся 3 MW. Приоритет 1: устройства требуют 4 и 1 MW. Первая равная доля 1.5 MW; маленькое устройство берёт только 1 MW, большое 1.5 MW, остаётся 0.5 MW. Во втором круге оно идёт большому: итог 2 MW из 4, `powerRatio=0.5`, `hasPower=false`. `suppliedMw` сети равна 8 MW, `demandMw=10 MW`, `brownout=true`. Если добавить аккумулятор, который выдаёт 2 MW, дефицит может исчезнуть на этот шаг — пока хватает энергии и не превышен предел мощности. Именно поэтому для части устройств полезно использовать `powerRatio`, а для бинарных устройств — `hasPower`. Текущий код лампы пользуется бинарным признаком (`SceneManager.cpp:2253-2267`), так что лампа погаснет и при 99% номинала.

6.2. MW, MWh и правильное время батареи

`EmergencyBatterySystem::Discharge` (`src/Gameplay/Power/EmergencyBatterySystem.h:9-23`) возвращает минимум из требуемой мощности, аппаратного лимита разряда и ограничения по запасённой энергии `batteryChargeMwh*3600/deltaTime`. Затем вычитает `suppliedMw*deltaTime/3600` MWh. Это размерно корректно: MW — скорость передачи энергии, MWh — запас; час содержит 3600 секунд. Например, батарея 1 MWh при отдаче 2 MW теоретически выдержит 0.5 часа, то есть 1800 секунд, если нет других ограничений. При `dt=1 c` расход `2/3600≈0.000556 MWh`. `Charge` симметрично ограничивает мощность surplus, лимитом зарядки и свободной ёмкостью (`:25-46`). Если в новом коде вычесть `MW*dt` прямо из MWh, ресурс сгорит в 3600 раз быстрее. Если применить этот же расход при `deltaTime=0`, батарея не должна измениться — методы уже имеют ранние проверки.

После сети `SceneManager::UpdateAll` связывает питание с отдельными приборами: свет с `PowerConsumerComponent` включается лишь при живом объекте и полном питании (`:2253-2267`); кислородному генератору выставляются `hasPower` и `broken`, затем обновляются `RoomEnvironmentSystem`, `OxygenGeneratorSystem`, `PlayerVitalsSystem`, `ShipAlertSystem` (`:2269-2292`). Обрати внимание на временную причинность: физическое затопление сейчас уже прошло до `UpdateAll`, а питание сети рассчитывается здесь. Если насос станет настоящим consumer, последовательность надо перестроить. Также `UpdateAll(0)` в Editor вызывает `ShipPowerGridSystem::Update`: её телеметрия может измениться без расхода топлива, поскольку сеть пересчитывает текущие мощности, а генератор при нулевом времени просто возвращается. Это допустимо как preview, но при расширении системы следи, чтобы новый consumer не расходовал ресурсы в Editor.

6.3. Добавление новой системы жизнеобеспечения

Проектное предложение. Пусть нужен фильтр CO_2. Компонент содержит заданный расход воздуха и энергоёмкость; `PowerConsumerComponent.requiredMw` показывает запрос. `FilterSystem::Update` читает `powerRatio`, состояние повреждения, текущую концентрацию CO_2 и `deltaTime`, рассчитывает удалённую массу и записывает её в комнату. Помести фильтр после `ShipPowerGridSystem`, но до расчёта витальных эффектов игрока; иначе здоровье получит концентрацию до очистки. Питание фильтра сделай либо непрерывным (`effectiveFlow = nominalFlow * powerRatio`), либо пороговым с документированным порогом; для двоичного `hasPower` при небольшом brownout фильтр полностью выключится. При нулевом времени функция не должна менять массу газа. На интеграционном тесте сравни комнату с фильтром при 100%, 50% и 0% питания, затем сохранение/загрузку значения авторского расхода и отдельно вычисляемого runtime-состояния. Всё это решение дизайна; текущего `FilterSystem` в коде нет.

7. Renderer и шейдеры: от Transform до цвета пикселя

7.1. Что делает CPU перед GPU

`Renderer::RenderFrame` (`src/Render/Renderer.cpp:203–514`) получает Direct3D context, view и projection, выбранный Entity и параметры отладки. В начале обновляется контекст вида от первого лица, создаётся snapshot кандидатов видимости для фоновой frustum-проверки (`:213–222`), собирается статистика активных мешей и обнаруживаются материалы, которым нужны reflection/refraction. Метод `InspectSceneRenderFeatures` (`:34–59`) считает reflection нужным для Water/Reflection, refraction — для Glass/Water. Это означает, что дорогие дополнительные проходы запускаются по **реально активным материалам**, а не всегда. Однако сама проверка проходит по пулу мешей, значит при большом числе объектов это тоже измеряемая работа на CPU.

Далее `RenderFrame` обновляет направленный свет и environment map, рисует shadow map, при нужде reflection и refraction в отдельные render textures, при наличии editor callback — preview, потом основную сцену в backbuffer (`:263–448`). Shadow pass использует выбранные видимые комнаты. Reflection pass строит отражённую view-матрицу для плоскости `y=0`; это конкретное соглашение текущей водной сцены, поэтому для произвольной наклонной отражающей поверхности его не хватает. Refraction pass пропускает Glass/Water, чтобы текстура преломления содержала фон за ними (`:316–382`). После каждого render texture вызываются восстановление backbuffer/viewport и `ClearSRVSlots`: в D3D11 один ресурс нельзя бездумно держать одновременно как вход SRV и выход RTV. Main pass очищает кадр цветом fog или чёрным, рисует skybox с view-матрицей без переноса камеры (`:415–445`), обычные 3D объекты, sprites, частицы, отладочные объёмы и в конце first-person view model (`:447–510`). Для оружия от первого лица очищается depth buffer и задаётся отдельная near/far projection (`:516–550`), чтобы руки не исчезали за мировой стеной из-за глубины. Это визуальный приём, а не изменение попаданий: луч выстрела отдельно вычисляется из камеры/дула в мире.

`Render3DObjects` (`:556–606`) включает Z-buffer, выключает alpha blending, передаёт fog в шейдеры, привязывает shadow map, затем просит `SceneManager::RenderAllECS`. Тот (`src/Scene/SceneManager.cpp:2393–2550`) собирает свет, строит frustum, список видимых комнат и проходит по `MeshRendererComponent`. Для непрозрачного прохода он пропускает неактивные объекты, исключённые из мирового вида, неподходящие кандидаты, editor gizmo, отсутствие Transform/model/material, невидимые комнаты и объекты вне frustum (`:2434–2462`). Лишь после этих фильтров `GetWorldMatrix()` становится матрицей конкретного draw. Далее `PreparePbrDrawState` привязывает decal-данные и вызывается renderer меша. Если объект не виден, искать причину нужно последовательно: `ActiveComponent`, видовой слой,

готовность модели, комната, frustum, затем шейдер. Замена текстуры не поможет, если draw вообще не дошёл до GPU.

7.2. Три пространства координат и вершинный этап

Для одного треугольника CPU берёт локальные вершины меша, `TransformComponent` даёт world matrix, камера даёт view и projection. В `PBRShaderClass::Render` (`src/Render/Shader/PBRShaderClass.cpp:143–176`) C++ транспонирует `world`, `view`, `projection` перед отправкой в VS constant buffer `b0`; `SetMatrixBuffers` делает то же (`:127–140`). Это контракт C++/HLSL о памяти матриц, а не лишнее вращение объекта. Если убрать transpose только в одном месте, модель может исчезнуть или вести себя зависимо от камеры. В `pbr.vs` (`src/Render/Shader/Vs/pbr.vs:7–12`) тот же `b0` объявляет три `matrix` в том же порядке. Затем код использует row-vector умножение:

```
float4 worldPos = mul(finalPos, worldMatrix);
float4 viewPos  = mul(worldPos, viewMatrix);
output.position = mul(viewPos, projectionMatrix);
```

Здесь `finalPos` — координата в пространстве модели, `worldPos` — в мире, `viewPos` — относительно камеры, `SV_POSITION` — координата для растеризации. Позиция `(1,0,0)` в меше сначала масштабируется и поворачивается вокруг локального начала, затем переносится в мир. Если свет имеет позицию в мире, расстояние до него надо вычитать из `worldPos`, а не из `SV_POSITION`: последняя уже зависит от камеры и перспективы. Pixel shader получает `worldPos` отдельно через `TEXCOORD5` (`pbr.vs:50–64`, `pbr.ps:80–94`). Между VS и PS rasterizer интерполирует атрибуты по поверхности треугольника; это не новый вызов C++ для каждого пикселя.

У анимированной модели до world/view/projection выполняется skinning (`pbr.vs:71–116`). Каждый входной vertex хранит до четырёх индексов костей и весов. Шейдер пропускает индекс вне `[0,128)`, неположительный вес, суммирует преобразованные позиции и направления, затем делит на общую сумму весов. При отсутствии валидных весов он использует исходную вершину, а не случайную `boneTransforms[0]`, которая могла остаться после предыдущего skinned draw (`:98–114`). Если модель «разъехалась», проверяй `boneIDs`, веса, bind pose и тот факт, что статический меш действительно пришёл без валидных весов. Нельзя тестировать skinning только на одной позе: ошибки иногда скрываются в bind pose и проявляются при повороте руки.

Нормаль — направление поверхности, не точка. При неравномерном scale простое `mul(normal,world)` даёт неверную ориентацию. `normal_transform.hlsli` строит normal matrix через кофакторы, соответствующую обратной транспонированной линейной части, нормализует нормаль и переортогонализует tangent/binormal. Вершинный шейдер вызывает `BuildWorldTangentBasis` (`pbr.vs:127–136`). На CPU тот же принцип используется в `PBRShaderClass::SetDamageDecals` для нормали трещины (`:327–335`): позиция `XMVector3TransformCoord` учитывает перенос, нормаль `XMVector3TransformNormal` — нет. Тест: поверхность с scale `(2,1,0.5)` и нормалью не по оси. Если освещение сохраняется при вращении и scale, преобразование согласовано; если блик внезапно перескакивает, сначала проверь normal/TBN, а не roughness.

7.3. Пиксельный этап: текстуры, материал и свет

`pbr.ps` (`src/Render/Shader/Ps/pbr.ps`) объявляет albedo `t0`, normal `t1`, metalness `t2`, roughness `t3`, crack `t6`, shadow `t7`, environment `t8`. C++ в `PBRShaderClass::Render` выставляет `t0..t3`, crack и environment, вызывает `DrawIndexed`, затем отвязывает `t0..t8` (`PBRShaderClass.cpp:210–245`). Это точное соответствие слотов: если HLSL читает `t8`, а C++ привязал новый ресурс к `t5`, пиксель не «догадается» о другом месте. `m_defaultNormalSRV`, `m_defaultMetalnessSRV`, `m_defaultRoughnessSRV`, `m_defaultCrackSRV` создаются как 1×1 fallback (`:91–95`) для отсутствующих карт. Albedo при `nullptr` получает предупреждение после draw (`:247–`

249); это не полноценная гарантия корректного цвета, поэтому путь создания материала надо проверять отдельно.

PS берёт albedo, metalness из синего канала, roughness из зелёного (pbr.ps:220–255); это соответствует metallic-roughness упаковке glTF и отдельным grayscale картам, если все каналы одинаковы. Нормаль из карты переводится из [0,1] в [-1,1], затем через TBN — в мир (:257–268). Вектор к наблюдателю V=normalize(cameraPos-worldPos) и нормаль N должны быть в одном пространстве. F0 смешивает 0.04 для диэлектрика с albedo для металла (:270–274). Направленный свет и до четырёх локальных источников используют GGX distribution, Smith geometry и Schlick Fresnel (:188–218, :282–342). Физический смысл: NDF отвечает за распределение микрофасеток, geometry term — за взаимное затенение, Fresnel — за долю отражения при данном угле. Диффузная часть уменьшается для металла через (1-metalness), чтобы поверхность не получала одновременно весь металлический блик и полный диффузный цвет. Точечный свет затухает с нормализованным расстоянием до радиуса, spot дополнительно проверяет угол конуса (:304–340). Это художественная модель затухания проекта; её нельзя автоматически считать точной обратноквадратичной физикой.

Ambient и отражение окружения добавляются на :344–381. Sky visibility приближённо выводится из shadowFactor, так что освещённость закрытых комнат уменьшается без полноценной системы light probes. Затем идут tone mapping и gamma (:383–386), alpha стекла (:391–408), трещины (:416–473), подсветка выделенного объекта и fog (:476–489). Последовательность имеет последствия: трещина в текущем коде затемняет **уже посчитанный color** после освещения. Комментарий о «повышении roughness» есть, но вычисленная ранее переменная roughness после трещины не меняется и BRDF повторно не считается (:466–472 прямо это признают). **Проектное предложение:** если нужен физически изменённый блик на повреждении, перенеси смешивание карты трещин до расчёта света, отдельно проверь влияние на metalness/roughness и сохранение визуального совпадения с пробоиной.

7.4. Как менять контракт CPU/GPU без случайной поломки

Для нового параметра сначала выбери, где он хранится: авторская настройка материала, вычисленное состояние компонента или глобальное состояние кадра. Затем выбери HLSL register(bN/tN/sN) и зеркальную привязку в C++. В PBRShaderClass.h:60–93 структуры C++ соответствуют MatrixBuffer, PBRParamsBuffer, DamageBuffer HLSL; порядок float3 + float или массивы float4 нужен для 16-байтной упаковки constant buffer. Проверяй sizeof и offsets, особенно при добавлении bool или структуры с другим выравниванием. Если параметр per-object, обновляй буфер перед **каждым** draw; иначе второй объект унаследует состояние первого. PreparePbrDrawState (SceneManager.cpp:2295–2391) именно перед draw отправляет трещины выбранного объекта, в том числе переводит локальные decal координаты панелей в координаты batch-меша. Тестируй сперва постоянным ярким цветом без освещения, потом одной текстурой, затем комбинацией света, тени и анимации. Наконец проверь выключенный объект и следующий draw: старый ресурс в слоте может создать «призрачный» эффект даже при правильном шейдере.

8. Выстрел, Bullet raycast и `DamageEvent`: что происходит с «пулей»

8.1. Текущая модель — мгновенный луч

Слово «Bullet» в проекте имеет два возможных значения: библиотека физики Bullet и пуля оружия. Текущая ветка WeaponSystem::TryFire (src/Gameplay/Combat/WeaponSystem.h:75–155) реализует **hitscan**: в момент выстрела бросает луч в btDynamicsWorld, находит первое столкновение и создаёт событие урона. Отдельной ECS-сущности летящего снаряда, его позиции/скорости и физического времени полёта в этой ветке нет. Видимый muzzle flash — частица, а отладочная линия — диагностическое изображение

луча; ни одно из них не определяет попадание. Это критично для будущего космического оружия: лазер или сверхскоростной выстрел может оставаться hitscan, ракета и медленный плазменный шар потребуют отдельной реализации.

Игрок и NPC используют общий механический метод `TryFire`. Владелец и оружие — разные сущности: `GetEquippedWeaponEntity` читает `EquippedWeaponComponent`, с совместимостью для старой сцены, где `WeaponComponent` лежит на самом персонаже (`WeaponSystem.h:20–29`). `UpdateTimers` один раз за симуляционный кадр сбрасывает `triggerHeld` и продвигает cooldown/reload у всех оружий (`:63–71`), поэтому нельзя дополнительно вызывать `AdvanceTimers` в каждом NPC: ускорится стрельба и перезарядка. `WeaponFiringRules::AdvanceTimers` (`src/Gameplay/Combat/WeaponFiringRules.h:9–33`) уменьшает cooldown, recoil и reload timer на `dt`; по окончании перезарядки переносит из резерва только недостающие патроны. `CanFire` (`:59–66`) проверяет конечные положительные fireRate/range, готовый cooldown, наличие патрона и отсутствие перезарядки. `CommitShot` (`:80–90`) выставляет `1/fireRate` секунд до следующего выстрела, вычитает патрон, увеличивает счётчик, flash timer и recoil. При `fireRate=4` это `0.25 с` между разрешёнными выстрелами, не четыре вызова кадра.

8.2. Проверка стрелка, разброс и первый контакт

Вход `TryFire` — registry, стрелок, world-координата начала, направление, указатель на Bullet world, DamageSystem, debug, device и jobs. Ранние проверки `WeaponSystem.h:82–93` отклоняют отсутствующий мир, мёртвого/неактивного стрелка, некорректный луч, отсутствующее оружие, пустой магазин и неготовое оружие. Пустой магазин пытается начать механическую перезарядку. Затем `direction` нормализуется, читается `CharacterMovement::isMoving`, вычисляется множитель разброса. Две случайные выборки дают направление внутри конуса через `WeaponFiringRules::ConeDirection` (`:93–110`): радиальная выборка задаёт cos угла между центральным лучом и выстрелом, азимут — поворот вокруг оси. Код строит ортонормированный базис `right/up/forward` в мировых координатах и возвращает нормализованный результат. Подвижность и накопленная отдача увеличивают полуугол конуса. Случайность берётся **один раз на состоявшийся выстрел**, и этот же луч используется для debug, урона и разрезания корпуса (`WeaponSystem.h:102–110`); иначе линия трассера могла бы показывать одно, а пробоина появлялась бы в другом месте.

`CommitShot` вызывается **до** raycast (`:110`), поэтому и промах тратит патрон и запускает cooldown; это ожидаемая игровая семантика. После необязательной вспышки `PhysicsRaycast::CastRay` проверяет мир до `weapon->range` (`:119–131`). Сам `CastRay` (`src/Physics/PhysicsRaycast.h:72–123`) обнуляет `outHit`, требует положительную дистанцию, нормализует направление, проверяет конечность его компонент, строит точки начала и конца `origin + normalized*distance`, вызывает `world->rayTest`. Специальный callback (`:20–53`) игнорирует собственный ghost/body стрелка, объекты с тем же `userIndex` Entity и тела в `DISABLE_SIMULATION`. Результат содержит первый collision object, Entity из Bullet `userIndex`, мировую точку и нормаль попадания. Возврат false при объекте с `INVALID_ENTITY` означает: геометрический контакт мог быть, но игровой Entity не разрешён; debug при этом может показывать контакт, а урона не будет. При диагностике такого случая проверяй установку `setUserIndex` при создании RigidBody, а не сразу множитель damage.

Стена перед NPC остановит луч: `ClosestRayResultCallback` отдаёт ближайший допустимый контакт, `TryFire` не переносит урон на выбранную AI цель за стеной. Если `CastRay` вернул false, `TryFire` всё равно возвращает true — выстрел состоялся, только не было игровой цели. Если есть hit, создаётся `DamageEvent`: source = Entity оружия, target = hit Entity, damageAmount, physical type, world-space hit position/normal и breachRadius (`WeaponSystem.h:133–154`). Для панели корпуса `CanDamageHull` true только у `WeaponType::ShipCannon` (`src/Gameplay/Combat/WeaponComponent.h:79–82`). `CalculateHullDamage` оценивает, дойдёт ли здоровье панели до порога; при разрешённом D3D device и возможности пробоины

вызывается `MeshCuttingTest::CutByRayAsync`. Его возвращаемое bool означает принятую/запущенную операцию разрезания, не то, что новый меш уже готов. `meshCutOperationId` связывает runtime-операцию с будущей ремонтной логикой. Обычный ручной выстрел по броне может показать искру, но не снимать structural HP.

8.3. Когда урон реально применяется

`DamageSystem::AddDamageEvent` (`src/Gameplay/Combat/DamageSystem.h:15–18`) лишь добавляет копию в `std::deque`. `DamageSystem::Update` (`src/Gameplay/Combat/DamageSystem.cpp:104–371`) проходит очередь, проверяет валидную цель, положительный урон, активность. Тактическая реакция NPC на попадание может зарегистрироваться до проверки `InvulnerableComponent`; затем неуязвимость отменяет уменьшение здоровья и пробоины (`:123–194`). Для игрока меняется `PlayerComponent::health`, для NPC без общего `DamageableComponent` — `NPCComponent::health`, для панели корпуса — её собственное здоровье/пробоины, для остальных — `DamageableComponent`. По окончании очередь очищается (`:371`). Если событие поставлено **до** вызова `m_damageSystem.Update` в `ApplicationClass::Frame:1549`, оно будет применено в этом кадре. Выстрел игрока и NPC через `WeaponSystem::Update` вызывается позже, на `:1757–1769`, значит его событие обычно дождётся обработки следующего кадра. Это небольшой, но реальный сдвиг: UI, который читает здоровье сразу после `TryFire`, увидит прежнее число. Если нужна обработка в тот же кадр, меняй порядок или добавляй вторую строго определённую фазу обработки, а не вызывай `DamageSystem::Update` случайно из оружия.

Для `HullPanelComponent` (`DamageSystem.cpp:215–328`) capability «может повредить броню» и факт созданной пробоины различны. Если capability false, рождается только metal impact. Иначе здоровье уменьшается на эффективный урон. Если `canBreachHull` true, создаётся новый `HullBreachComponent` с привязкой к панели и комнате, мировым hit, радиусом, локальной точкой и нормалью; добавляются `DecompressionSourceComponent`, `RepairableComponent`, ссылка на mesh cut operation, `ActiveComponent`; пишется `WorldEventType::BreachOpened`, создаются decal и частицы. Если порог пробоины ещё не достигнут, система только отмечает визуальные стадии повреждения при пересечении 20%/50%. Это не тот же список `CompartmentComponent::breaches`, который использует водный `FloodingSystem`; для подводной течи нужен явный мост между типами повреждений. Иначе можно увидеть красивую дырку в панели без роста `waterVolume`. В космической модели `DecompressionSourceComponent` — подходящий вход для атмосферы, но проверяй конкретный код потребителя перед утверждением о результате.

8.4. Как добавить настоящий летящий снаряд

Проектное предложение. Создай `ProjectileComponent` с owner/weapon Entity, скоростью в м/с, направлением, временем жизни, радиусом столкновения, типом урона и уникальным shot ID. При выстреле выполняй всё то же `CanFire`/`CommitShot`, но вместо немедленного raycast создавай projectile Entity. На каждом **фиксированном физическом шаге** перемещай снаряд от `previousPosition` к `nextPosition=previousPosition+velocity*fixedDt`; делай swept ray/sphere cast вдоль **всего сегмента**, чтобы быстрая пуля не проскочила тонкую стену между кадрами. Игнорируй owner и собственный collider, как делает `IgnoreEntityRayCallback`. При первом контакте сформируй тот же `DamageEvent` и уничтожь/деактивируй снаряд после обработки. Жизненный цикл должен учитывать попадание, максимальную дальность, истёкшее время, выгрузку сцены и save/load политику: сохранять ли пули на лету или стирать их при загрузке. Для ракеты добавь поворот к цели и взрывной радиус; для энергетического шарика — расход энергии и визуальный след. Тестируй одну стену между стрелком и целью при 30/60/144 FPS и скорость, за шаг проходящую больше толщины стены. Только после такого теста можно говорить о реальных «снарядах» в коде.

9. Фоновые задачи и ассеты: когда результат готов к использованию

9.1. Две очереди `JobSystem`

`JobSystem` (`src/Core/JobSystem.h`) содержит очередь заданий worker threads и отдельную очередь завершений для главного потока. `Initialize(workerCount)` (`:31–59`) требует положительное число работников, останавливает прежние потоки, очищает флаг остановки и создаёт worker threads. В `ApplicationClass` вызывается `m_jobSystem.Initialize(4)` (`src/Core/ApplicationClass.cpp:558`). `SubmitJob` (`JobSystem.h:61–72`) под mutex отклоняет работу, если система остановлена, иначе копирует `std::function` в очередь и будит один поток. Начальное значение `stop=true` (`:158–161`) не даёт тихо поставить задачу до инициализации, когда некому её исполнить.

`WorkerLoop` (`:124–150`) ждёт задание или остановку, извлекает job, увеличивает `activeJobs`, отпускает mutex и исполняет callback. После выполнения уменьшает `activeJobs` и, если очередь пуста, будит ожидающий `Wait`. Важно: `job()` не обёрнут в `try/catch`; `try/catch` в `Initialize` защищает создание потоков, а не любое пользовательское задание. Необработанное исключение внутри worker не превращается в `AssetState::Error` автоматически и может завершить процесс. **Проектное предложение:** ловить исключение на границе worker, передавать диагностическое состояние на главный поток и гарантировать уменьшение счётчика активных заданий. Но при проектировании конкретной задачи лучше возвращать структурированный результат с `success/error`, как делает загрузчик ассетов, и не бросать из обычной ошибки файла.

`SubmitMainThread` (`:74–78`) складывает callback в другую очередь под отдельным mutex. `ExecuteMainThreadJobs` (`:80–94`) быстро меняет очередь на локальную, отпускает блокировку и выполняет jobs уже в вызывающем потоке; `ApplicationClass::Frame` делает это на `:1228`. Поэтому worker может читать и готовить CPU-данные, а создание D3D buffers и изменение ECS, зависящее от текущей сцены, нужно делать в главном потоке после передачи результата. `Wait` (`:96–102`) ждёт только worker-очередь и `activeJobs==0`; он **не выполняет** `ExecuteMainThreadJobs`. Если worker завершился и положил callback в главную очередь, `Wait()` может вернуться до видимого результата. Это частый источник ошибки «загрузка закончилась, но меша нет».

`Shutdown` (`:104–118`) выставляет `stop=true`, будит всех, присоединяет workers. Worker выходит лишь когда `stop && jobs.empty`, поэтому уже поставленные задания будут доработаны. В `ApplicationClass::Shutdown` вызывается `m_jobSystem.Shutdown()`, затем `ExecuteMainThreadJobs()`, а позже `m_assetLoadSystem.Shutdown()` (`ApplicationClass.cpp:598–639`): готовые callback успевают примениться, пока asset system жива. Если поменять порядок удаления объектов, lambda, захватившая `this` загрузчика, может обратиться к уже уничтоженному экземпляру. Для нового async-сервиса заранее запиши, кто владеет объектом, сколько живёт callback и как отменяется работа при `SceneManager::Clear`. `Clear` отдельно сбрасывает `m_asyncShipImport` и использует shared state, чтобы поздний результат не вошёл в новую сцену (`SceneManager.cpp:2128–2132`).

9.2. `AssetLoadSystem`: четыре перехода состояния

`AssetState` (`src/Assets/AssetsLoadSystem.h:15–22`) задаёт `Unloaded`, `Loading`, `CpuLoaded`, `Ready`, `Error`. `RequestMeshLoad(path)` (`:103–157`) первым делом ищет запись в `m_meshes`. Если она есть и не в Error, повторный запрос ничего не запускает; если была Error, старые GPU-буферы освобождаются, запись стирается и тот же путь можно попробовать снова после исправления файла. Новая запись получает `Loading` и tracker прогресса. Worker вызывает `LoadMeshFromFile`, а его результат передаёт в main-thread callback: при ошибке ставятся `Error` и текст, при успехе CPU-меш переносится в asset и состояние становится `CpuLoaded` (`:125–147`). Если `SubmitJob` отклонён, метод сразу ставит Error (`:150–155`). Такой

автомат состояний лучше не сокращать до одного bool `loaded`: различаются чтение файла, подготовка геометрии, создание GPU и ошибка.

`LoadMeshFromFile` (`:411-...`) использует Assimp с triangulation, generated normals, tangent calculation, объединением одинаковых вершин и `aiProcess_PreTransformVertices` (`:420-429`). Это CPU-этап: он читает формат модели и формирует массивы, которые можно безопасно подготовить вне D3D context. Флаг pretransform означает, что данный быстрый путь превращает узловые преобразования в вершины; для skinned иерархий нужен отдельный проверенный путь импорта, не следует переносить сюда анимацию без анализа костей. `AssetLoadSystem::Update(device)` (`:158-178`) на главном потоке ищет `CpuLoaded` и вызывает `CreateGpuMesh`. Тот требует device и непустые vertices/indices, делает D3D vertex и index buffers, а после успеха очищает CPU-массивы (`:354-409`). Если index buffer не удалось создать, освобождает уже созданный vertex buffer (`:396-400`), чтобы не оставить половину GPU-ресурса. Статус становится `Ready` или `Error`.

`BindMeshAsset` (`:255-...`) отказывается от ресурса, пока нет `Ready` и обоих GPU buffers. Это граница между «файл прочитан» и «можно рисовать». При медленной загрузке renderer не должен пытаться читать `cpuData.vertices` как D3D buffer. В `Renderer.cpp:705` этот путь привязывает готовый async mesh для shadow pass; у `SceneManager::RenderAllECS` есть проверка особого пути асинхронного меша, которая при наличии spot light возвращается к полному renderer, поскольку быстрый путь передаёт лишь point-light данные (`SceneManager.cpp:2467-2470`). Значит «готовый GPU buffer» ещё не означает «полностью функциональный материал во всех проходах»; проверяй каждый render pass.

Практический тест загрузчика: запросить существующий меш дважды и убедиться, что загрузка не дублируется; запросить отсутствующий путь, увидеть Error и текст; создать файл на том же пути и повторить запрос; получить Loading → CpuLoaded → Ready, где `Ready` наступает только после main-thread pump и GPU Update. Затем закрыть приложение при активной загрузке: порядок Shutdown должен завершить work и освободить ресурсы без позднего обращения к уничтоженным объектам. Поскольку результат асинхронен, проверка `Ready` сразу после `RequestMeshLoad` не является тестом ошибки — это нормальная задержка.

10. Как добавлять систему, чтобы она стала частью игры

10.1. Контракт системы до первой строки C++

Для новой механики составь короткую карточку: «какая сущность и компонент владеют настройкой», «какие данные вычисляются каждый кадр», «в каких единицах поля», «кто обновляет и когда», «кто читает результат», «что сохраняется», «что уничтожается», «что происходит при паузе и загрузке». Например для аварийной сирены авторские `volume` и `enabled` можно хранить в компоненте, а вычисляемое `isAudible` зависит от питания, повреждения, расстояния до игрока и активного режима тревоги. Если просто добавить `SirenSystem::Update` в конец `Frame`, она может прочесть старое питание, а звук будет обновляться после UI или после освобождения сцены. Правильное место определяется входами и потребителями, а не алфавитным названием функции.

Сначала найди ближайший существующий путь. Для оборудования это цепочка `GeneratorSystem` → `ShipPowerGridSystem` → конкретный consumer → `RoomEnvironmentSystem`/HUD. Для повреждения — `WeaponSystem::TryFire` → `DamageEvent` → `DamageSystem` → `HullBreachComponent`/decal/particle → ремонт. Для визуала — ECS `Transform` + mesh + material → culling → shader buffers → VS/PS. Для ассета — запрос → worker CPU → main thread → GPU Ready → render pass. Использование готового потока данных сокращает число новых соглашений. Но нельзя насильно использовать `PressureSystem` для вакуума: среда и единицы там другие.

10.2. Минимальный полный вертикальный срез

Проектное предложение. Возьми небольшую функцию «аварийная лампа у пробоины». На сцене должны быть корабль, панель, пробоина, лампа и источник питания. `DamageSystem` открывает пробоину; система тревоги читает активные `HullBreachComponent` и ставит сигнал; consumer лампы получает мощность; визуальный компонент меняет цвет/эмиссию; HUD показывает аварийную комнату; сохранение/загрузка возвращает авторское расположение и состояние пробоины, а runtime телеметрия пересчитывается заново. Тест из одного события проверяет всю цепь. Если лампа не горит, измерь на каждом звене: создан ли breach Entity, какой у него `room`, есть ли `ActiveComponent`, какой `requiredMw`, `suppliedMw`, `hasPower`, дошёл ли объект до `RenderAllECS`, что лежит в emissive constant buffer. Так можно объяснить конкретную причину, а не переписать систему целиком.

При добавлении компонента с `Entity`-ссылками обнови SaveScene и remap, при добавлении GPU ресурса — инициализацию, освобождение и состояние после потери/пересоздания устройства, при добавлении Bullet тела — порядок удаления constraints/body, при добавлении фоновой задачи — lifetime capture и main-thread commit. Если поле влияет на геймплей и рендер, не позволяй шейдеру быть единственным хранилищем факта: он должен показывать состояние, которое есть в ECS. Если поле выводится из других каждый кадр, обычно не надо сохранять его как авторскую истину; при загрузке сначала восстанови входы, затем пересчитай выход. Если порядок зависит от предыдущего результата, зафиксируй его рядом с вызовом в `Frame` или `UpdateAll`, чтобы будущий рефакторинг не переставил его «для красоты».

10.3. Набор проверок перед большим рефакторингом

Перед переписыванием системы сделай маленькую сцену с известными числами. Для матриц — одна вершина и один объект с scale/rotation/translation; для физики — одно тело с заданной массой и импульсом; для давления — глубина 0/10/20 м; для сети — две ступени приоритета и батарея с известными MW/MWh; для рендера — один меш без normal map, один с ней и один с неравномерным scale; для сохранения — объект без меша с ссылкой на другой Entity. На каждом опыте запиши вход и ожидаемый выход в единицах. Это не «тест ради теста», а способ зафиксировать смысл кода, который ты собираешься переучить. Особенно полезно сравнивать один и тот же сценарий при 30/60/144 FPS, Editor/Play и до/после Save/Load: эти три оси обнаруживают неверное время, скрытую зависимость от порядка и недосериализованные данные.

Для работы в перерывах достаточно читать один поток за раз: `Frame` показывает **когда**, компонент — **где лежит состояние**, система — **как меняется**, serializer — **что переживает загрузку**, renderer — **как это видно**. Пройдя этот путь для одного объекта от создания до удаления, ты узнаешь больше о движке, чем после заучивания названий всех классов. В следующем шаге выбери конкретную фичу космического режима и заполни эту же цепочку без пропусков.

11. Атмосфера комнат: уже работающая игровая модель и её границы

`RoomEnvironmentSystem::Update` (`src/Gameplay/Ship/RoomEnvironmentSystem.h:15–45`) уже реализует часть эффекта разгерметизации. Это важно отделить от проектного предложения о более физической модели газа. При `deltaTime<=0` метод ничего не меняет; положительное время ограничивает `0.10 c` за вызов. Для каждой активной `RoomComponent` он сбрасывает текущую `leakRate`, считает персонажей по `objectsInRoom` с `CharacterController` и уменьшает `room.oxygen` на `count*0.01*dt`, ограничивая результат `[0,1]`. После этого вызываются `ApplyBreaches` и `EqualizeConnectedRooms`. `RoomComponent::pressure` и `oxygen` здесь — **нормализованные доли 0..1**, а `temperature` по умолчанию `20` (`src/ECS/Components/Components.h:1012–1021`); не подставляй `room.pressure=1` как один паскаль в формулу `pV=nRT`.

`ApplyBreaches` (`RoomEnvironmentSystem.h:57–105`) перебирает `HullBreachComponent`, пропускает запечатанную пробоину или пробоину без комнаты. Для активной комнаты обеспечивает наличие `DecompressionSourceComponent`, в том числе у старого сохранения. Отношение радиуса к `0.25` ограничивается `[0.25,4]` и умножает `breach.leakRate`; затем `lostFraction=clamp(effectiveLeak*dt,0,0.95)`. Давление и кислород умножаются на `(1-lostFraction)`, температура приближается к `outsideTemperature`, а `room.leakRate` хранит текущую интенсивность. По прежнему давлению, радиусу действия и расстоянию `ApplyPlayerSuction` (`:107–160`) добавляет игроку внешнюю скорость в направлении пробоины через `CharacterExternalVelocitySystem`. В текущем коде этот suction применяется только к `PlayerGameTag` из `objectsInRoom`, предметы с `RigidBodyComponent` пока не затрагиваются; этот ограниченный эффект не следует описывать как физическое истечение газа, воздействующее на любые тела.

`EqualizeConnectedRooms` (`:162–255`) проходит пары из `connectedRooms` только один раз (`roomEntity >= connectedRoomEntity` пропускается), ищет дверь между комнатами и берёт её `GetEffectiveFlowRate`. При отсутствии двери для уже объявленной связи множитель равен 1. Для кислорода, давления и температуры используются собственные скорости `0.35`, `0.55`, `0.14`; `LimitedFlow` берёт разность значений, умножает на `rate*doorFlow*dt` и ограничивает половиной текущей разности. Сначала собираются `RoomTransfer`, потом изменения применяются к обеим комнатам симметрично. Это устраняет зависимость расчёта потока от порядка записей в `unordered_map`, хотя последующий clamp может влиять на точное сохранение суммы в крайних случаях. Объём комнаты в этих формулах не учитывается: маленькая и большая комнаты с одной и той же разностью `pressure` обмениваются одинаковой долей. Поэтому это игровая модель выравнивания нормализованных показателей, не сохранение массы газа по м^3. **Проектное предложение:** при переходе к физической атмосфере хранить количество газа/массу, объём и температуру, выводить давление из состояния, переносить массу между комнатами равными и противоположными приращениями, а наружный отток считать отдельно. На первой итерации сохраняй существующий интерфейс HUD и NPC как нормализованные производные величины, но источник правды сделай размерным.

Связь с выстрелом такова: `DamageSystem` создаёт `HullBreachComponent` и `DecompressionSourceComponent` (`src/Gameplay/Combat/DamageSystem.cpp:241–276`), затем в `SceneManager::UpdateAll` атмосфера видит пробоину и уменьшает показатели. Водный `FloodingSystem` читает другой тип — `CompartmentComponent::breaches` — и потому не получает её автоматически. Ремонт, который выставит `breach.sealed`, останавливает атмосферную утечку на следующем `ApplyBreaches`, но воду в отсеке откачивает отдельный насос. Это две разные цепочки, и именно на их границе удобнее всего строить осознанное переключение «космос / вода».

12. Секции корабля, масса и разрушаемые соединения

`SubmarineSystem::AddCompartmentToSubmarine` (`src/Gameplay/Ship/SubmarineSystem.h:13–79`) проверяет у корня `TransformComponent` и `SubmarineComponent`, создаёт Entity отсека, задаёт его Transform, геометрические `localMin/localMax`, `CompartmentComponent`, ссылку `SubmarineComponent::submarineRoot`, `ActiveComponent`, добавляет ID в `rootSub->allCompartments` и выставляет `needsPhysicsUpdate`. Здесь список отсеков — часть владения корня: если компонент есть, но ID не добавлен, `FloodingSystem` ещё может найти отсек через его связь, но суммарная масса/объём и сила корня будут неполными. У метода есть параметры `localOffset` и `compartmentMass`, но текущий код ставит `Transform.position` равной позиции корня, `localOffset` кладёт в `comp.localCenter`, а `compartmentMass` не использует. Не предполагай, что переданная масса и смещение сами превращаются в отдельное физическое тело отсека: это следует проверять визуально и в Bullet для конкретной сцены. **Проектное предложение:** если API обещает смещение и

массу, либо задействовать их последовательно, либо переименовать/убрать неработающие параметры и покрыть сценой с двумя отсеками на разных позициях.

`RecalculateSubmarineMass` (`:82–133`) суммирует `comp.volume` по `allCompartments`, добавляет `15 м³` условного корпуса, затем считает нейтральную массу как `fluidDensity*totalVolume` и выбирает `0.95` от неё для небольшого запаса плавучести. Результат пишет в `rootRb->mass`, `rootBuoy->volume`, `rootBuoy->lastCalculatedVolume` и кэши `rootSub`. Это **водная** модель и один из источников `lastCalculatedVolume`, из-за которого описанное выше двойное снижение плавучести при затоплении реально может активироваться. `RefreshSubmarinePhysics` (`:136–175`) сохраняет линейную и угловую скорости, удаляет прежнее тело из Bullet, пересчитывает массу, создаёт новое через `PhysicsSystem::CreateRigidBody`, возвращает скорости. Для динамического корабля восстановление скорости избегает остановки после редакторского изменения отсека, но момент такого пересоздания всё равно может влиять на constraints и контакты — проверяй их после перестройки.

`UpdateSubmarine` (`:201–223`) не пересчитывает всё каждый кадр. `NeedsPhysicsUpdate` проверяет явный флаг, изменение количества отсеков или их размеров через кэш (`:177–199`). После refresh флаг снимается, число отсеков и размеры кэшируются. Если новая система меняет физическую геометрию, она обязана выставить `needsPhysicsUpdate`; если меняется только уровень воды, геометрию пересоздавать каждый кадр не нужно. В `ApplicationClass::UpdatePhysics` метод вызывается только для корневых `SubmarineComponent` до `PhysicsSystem::Update` (`src/Core/ApplicationClass.cpp:2286–2295`). Такая позиция позволяет Bullet увидеть новую форму и массу при следующем шаге.

Есть два вида соединений. `JointConnectionSystem::Update` (`src/ECS/System/JointConnectionSystem.cpp:5–40`) находит `JointConnectionComponent`: если target выбран, joint отсутствует и он не сломан, создаёт связь; если target сброшен — удаляет; если Bullet отключил существующий joint — отмечает `isBroken` и снимает его. `CreateJoint` (`:42–82`) требует оба `RigidBodyComponent` и живые Bullet bodies, создаёт `btFixedConstraint` в локальных pivot A/B, задаёт порог импульса `breakForce`, 100 итераций solver, регистрирует constraint в мире с выбранной настройкой столкновений между связанными телами. Если world отсутствует, выделенный joint удаляется. Это пример полного пути «авторская ссылка → runtime constraint»; только сериализовать указатель `joint` нельзя.

`SubmarineJointSystem::CreateJoint` (`src/ECS/System/SubmarineJointSystem.cpp:5–85`) похож, но хранит constraint на отдельной ECS-сущности `SubmarineJointComponent`, передаёт владение компоненту до регистрации в Bullet, чтобы исключение при создании Entity не оставило неуправляемый constraint. Его `Update` вызывается **после** Bullet шага (`ApplicationClass.cpp:2297–2298`) и читает `joint->getAppliedImpulse()` как `currentStress`; если `!joint->isEnabled()` или импульс больше `breakForce`, вызывает `ResetJoint`, отмечает `isBroken` и `isDetached` у обоих отсеков (`SubmarineJointSystem.cpp:87–123`). `RemoveJointsForEntity` сначала собирает ID связей в vector, затем удаляет их вне обхода (`:135–148`) — ровно тот безопасный способ изменения ECS-пула, который нужен при удалении сцены. Когда пробоина или взрыв должны разрывать секцию, не удаляй тело прямо из callback контакта: создай событие, дай физическому шагу закончиться, затем удали constraint и только после этого тело.

Дополнительные системы DX Framework: от нажатия клавиши до события и эффекта

Предыдущие главы подробно разобрали кадр, ECS, NPC, физику и основные сети корабля. Здесь мы пройдём ещё шесть подсистем текущего репозитория. Для каждой важно одно и то же: где хранятся постоянные данные, где живёт временное состояние, в какой момент кадра оно меняется и какой результат увидит игрок. Путь к методу указан для поиска в исходнике; точный номер строки после правок может сдвинуться.

1. Ввод игрока: от физической клавиши к игровому действию

1.1. Почему ввод разделён на два слоя

`InputClass` знает про клавиши и кнопки мыши. `GameplayInputSystem` в `src/Gameplay/Player/GameplayInputSystem.cpp` знает, что клавиша W означает `MoveForward`, R — `Reload`, E — `Interact`, а V — `ToggleView`. Это различие даёт возможность поменять привязку без изменения кода движения. Движение не должно спрашивать `IsKeyDown('W')` напрямую: иначе новая настройка клавиш подействует только на часть механик. В конструкторе `GameplayInputSystem` вызывается `SetBindings` для каждой команды. У `Fire` сейчас две привязки: левая кнопка мыши и H; у `PrimaryMouse` тоже левая кнопка. Это не ошибка само по себе, но контекст Editor/Play должен решить, кому достанется нажатие.

`Sample(input, playMode, gameplayEnabled)` проходит все `GameAction`. Для каждого действия вычисляет `enabled = playMode && (gameplayEnabled || IsSessionAction(action))`. Значит обычные действия заблокированы вне Play и при отключённом gameplay, а команды сессии вроде паузы и вида могут продолжать работать. Затем цикл перебирает привязки и складывает их логическим ИЛИ: если нажата хотя бы одна, `down=true`. Когда добавишь переназначение клавиш, не забудь проверять конфликт двух действий на одной физической кнопке в одном контексте. Сейчас такая общая кнопка существует намеренно, поэтому простое «запретить любой дубль» было бы неверной валидацией.

1.2. `pressed`, `down`, `released` на трёх кадрах

Ключевые строки `state.pressed = down && !state.down; state.released = !down && state.down; state.down = down` сравнивают новое наблюдение со старым. На первом кадре без клавиши получаем `(false,false,false)`. При нажатии: новое `down=true`, старое `state.down=false`, значит `pressed=true`, `released=false`. Пока клавиша удерживается, старое `down=true`, новое тоже true: `pressed=false`, `down=true`. В кадре отпускания новое false и старое true: `released=true`. Это маленький конечный автомат, а не три независимых сигнала.

Для движения нужна непрерывная проверка `Down(MoveForward)`: персонаж идёт всё время удержания. Для переключения камеры нужен `Pressed(ToggleView)`: иначе камера будет менять режим каждый кадр и почти сразу вернётся назад. `UpdateSession` именно так обрабатывает `ToggleView`, `TogglePause`, ускорение и замедление времени. `UpdateInventory` по `Pressed(Slot1..Slot3)` выбирает слот и по `Pressed(DropWeapon)` передаёт позицию и направление камеры в `WeaponPickupSystem`. `static_assert` там связан с тремя слотами инвентаря: если расширишь его до четырёх, компилятор напомнит обновить список команд. Это пример полезной проверки архитектурной связи во время сборки.

Упражнение. Нарисуй таблицу для клавиши R на четырёх последовательных кадрах: не нажата, нажата, удерживается, отпущена. Затем проследи, почему `RequestReload` должен запускаться на `Pressed`, а автоматический огонь — использовать `Down` либо отдельный флаг спуска. При потере фокуса окна

проверь, что `InputClass` сбрасывает состояния: иначе старое `down=true` может оставить персонажа движущимся без руки на клавиатуре.

2. Триггерные зоны: геометрия, память пересечения и действие

2.1. Зона не является Bullet телом

`TriggerZoneSystem.h` прямо описывает point-based вычисление. `Update(registry, deltaTime, events)` сначала накапливает собственное время, затем собирает `TriggerZoneComponent + TransformComponent` в список зон. Вторым проходом собирает кандидатов актёров по Transform и классификации типа. Оба списка сортируются по `Entity`. Это даёт устойчивый порядок событий даже если внутренний порядок pools меняется. Проверка пересечения идёт по координате актёра и форме зоны, а не по контакту Bullet ghost object. Поэтому видимый меш, физический collider и зона могут не совпадать; debug отображение формы зоны обязательно для авторской сцены.

На бумаге возьми круг радиуса 2 м в точке `(5;0;5)` и игрока `(6;0;5)`. Разница `(1;0;0)`, квадрат расстояния 1, квадрат радиуса 4: игрок внутри. Для игрока `(8;0;5)` квадрат расстояния 9, он снаружи. Проверка квадрата избавляет от `sqrt` и сохраняет тот же ответ при положительном радиусе. Текущий `ContainsPoint` поддерживает условия формы из компонента; при изменении способа расчёта сравни граничные точки ровно на поверхности. Флаг «внутри» относится к одному актёру и одной зоне, а не ко всей сцене.

`TriggerZoneSystem` хранит прошлые пересечения, cooldown и one-shot состояние в своём `m_runtime`, а не в авторском компоненте. Поэтому `Enter` означает переход «в прошлом кадре снаружи, сейчас внутри», `Stay` — «внутри снова», `Exit` — «был внутри, теперь снаружи». `Reset` очищает эту память при начале нового эпизода. Если удалить зону, система удалит её временную запись без выдуманного Exit: исходной зоны уже нет. Память состояния важна: без неё каждое нахождение внутри выглядело бы новым входом, и дверь либо HUD срабатывали бы десятки раз за секунду.

2.2. От события к команде

`TriggerActionSystem.h` получает `EventQueue`, ищет в `GameEvent` вариант `TriggerZoneEvent` через `std::get_if` и читает `TriggerActionComponent` у зоны. Флаги `executeOnEnter`, `executeOnStay`, `executeOnExit` решают, на какой переход реагировать. Затем `Execute` выбирает цель: явный `action.target`, иначе `event.linkedEntity`. При выходе и `invertOnExit` логическое значение меняется на противоположное. Например, зона входа активирует свет, зона выхода деактивирует тот же свет без второго компонента.

Доступные действия в текущем коде: сообщение HUD, включение/выключение Entity через `ActiveComponent`, переключение активности и установка `DoorComponent::isOpen`. При `SetDoorOpen` эта система меняет желаемое состояние двери, но физический проход зависит от дальнейшего обновления DoorSystem, а питание и анимация могут препятствовать фактическому открытию. Событие не должно обещать, что collider уже освобождён. `TriggerActionFeedback` держит текст и оставшееся время для HUD; `Update` уменьшает таймер по `deltaTime`, а `GetActivationsThisFrame` помогает отладке.

Практика: сделай зону перед дверью, `SetDoorOpen=true` на Enter и `invertOnExit=true` на Exit. Выведи по кадрам положение игрока, состояние inside, тип события, `door.isOpen`, питание двери и положение collider. Пройди границу медленно, затем телепортируй игрока через неё, поставь паузу и перезагрузи сцену. Так обнаруживаются повторный Enter, неверный cooldown и оставшаяся после reload runtime память.

3. Инвентарь: одна Entity оружия и три слота

`InventorySystem.h` отделяет данные `InventoryComponent` от операций `PickupWeapon`, `SelectSlot`, `DropSlot`, `SwapSlots`, `Sanitize` и `SyncEquippedWeapon`. Слот хранит ссылку на Entity предмета; ссылка должна указывать на живой объект и на допустимого владельца. `EnsureInventory` создаёт компонент, если его ещё нет. `PickupWeapon` не должен просто нарисовать винтовку в руке: он должен занять слот, назначить владельца, убрать или изменить представление предмета в мире и при необходимости экипировать его. Если операция неуспешна из-за полного инвентаря, состояние владельца и world pickup должно остаться прежним.

`WeaponPickupSystem.h` связывает взаимодействие и инвентарь: `Update` обрабатывает запросы контакта/подбора, `DropActive` возвращает выбранное оружие в мир, при настройке может создать физику для брошенного предмета. `ConsumeInventoryChanged` возвращает и сбрасывает флаг изменения — это сигнал для HUD или другого потребителя, чтобы он не перечитывал всё без причины. `InventoryOperation` и статусы нужны не только программе: игроку полезно объяснить «слоты заняты» или «предмет уже у другого владельца», а тесту — сравнить конкретный отказ.

Инвариант: одна Entity оружия не должна одновременно быть предметом двух инвентарей. На сцене с игроком A и NPC B помести оружие W в слот A, попытайся поднять его B, затем выбрось A и подними B. На каждом шаге запиши владельца W, слот, активный предмет и world представление. Если после reload W встречается дважды, ищи ошибку сериализации ссылок или восстановления производного визуала. `SanitizeSlots` проверяет живость и валидность ссылок — полезная защита после удаления Entity, но она не заменяет правильный lifecycle в операции подбора.

4. Анимация: игровое время, поза и вид от первого лица

`AnimationSystem::Update` в `src/Animation/AnimationSystem.h` делает два прохода по `AnimationComponent`. Сначала обновляет мировых персонажей. Синхронизируемые first-person копии пропускаются, чтобы их не опередил произвольный порядок Entity в pool. Если `isPlaying=false`, нет меша, модели или клипов, текущая Entity пропускается. Для персонажа `AssimpModelClass::UpdateAnimation(animComp, deltaTime)` является единственным местом накопления времени клипа; двойное прибавление `dt` в системе ускорило бы анимацию. В редакторском socket preview пауза даёт `UpdateAnimation(...,0)`: поза сэмплируется в установленное время без хода часов.

Вторым проходом first-person представление получает `owner`, читает его анимацию и копирует нужные настройки скорости и фаз. Так руки в камере и мировой персонаж используют согласованную позу. Когда мир уже обновлён, fp-копия может брать его текущее время; если сделать наоборот, оружие в руках и ствол на мировой модели разойдутся на один кадр. Для NPC это критично: `NpcRangedCombatSystem` строит мировую позицию дула из кости и Transform. Если вызвать стрельбу до обновления костей, математически правильный `muzzlePosition` будет принадлежать прошлой позе. Случай «дуло у ног» часто является ошибкой пространства или порядка матриц, а не логики AI.

Анимация показывает действие, но не обязана быть владельцем игрового результата. `RepairSystem` хранит прогресс работы, а не кадр клипа ремонта. WeaponSystem списывает патрон и создаёт луч, а не цвет пикселя muzzle flash. Если добавить animation event, определи, что именно оно запускает и как исключить повтор события при зацикливании или пересэмплировании клипа в редакторе. Проверь: пауза игры останавливает gameplay действие; пауза preview меняет только редакторскую позу; изменение FPS не меняет длительность шага атаки.

5. Атмосфера комнат: простая игровая модель и физические пределы

Текущий `RoomEnvironmentSystem::Update` в `src/Gameplay/Ship/RoomEnvironmentSystem.h` уже уменьшает кислород, давление и температуру при пробоине. Это важно назвать точно: `PressureSystem` из другого каталога описывает внешнюю водную глубину, а `RoomEnvironmentSystem` — упрощённую атмосферу комнат. В начале update неположительный `dt` прекращает работу, положительный ограничивается 0,10 с. Затем `CharacterExternalVelocitySystem::BeginAtmosphereFrame` начинает новый набор внешних скоростей. Для активной комнаты считается число `CharacterController` среди `objectsInRoom`, после чего `oxygen -= count*0.01*dt` и ограничивается [0,1]. Это игровая доля, не масса кислорода в килограммах.

`ApplyBreaches` обходит незапечатанные `HullBreachComponent`, проверяет комнату и добавляет недостающий `DecompressionSourceComponent` для старых сохранений. `effectiveLeak = max(0,leakRate)*clamp(radius/0.25,0.25,4)`. За шаг `lostFraction=clamp(effectiveLeak*dt,0,0.95)`; давление и кислород умножаются на `1-lostFraction`. Если `effectiveLeak=0.2/c` и `dt=0.1c`, то доля потери 0.02, давление 1.0 станет 0.98; на следующем таком шаге 0.98 станет 0.9604, а не 0.96. Поэтому кривая похожа на экспоненциальный спад. За 10 шагов получится `0.98^10≈0.817`, если другие системы ничего не добавляют. Это численное правило текущей игры, не уравнение газодинамики через сечение отверстия в паскалях.

Температура сдвигается к `outsideTemperature` по `thermalStep`; давление, кислород и температура ограничиваются. `ApplyPlayerSuction` находит игрока в комнате, нормирует направление к пробоине, вычисляет спад `(1-distance/radius)^2` и передаёт внешнюю скорость через `CharacterExternalVelocitySystem`. При нулевой длине направления деление запрещено. Тяга сейчас применяется к игроку, не ко всем предметам. После пробоин `EqualizeConnectedRooms` считает обмен между связанными помещениями. Текст учебника может использовать закон сохранения массы как цель дальнейшей физической модели, но эти поля [0,1] и коэффициенты обмена сейчас являются игровым приближением. Для добавления реалистичной модели пришлось бы хранить объём комнаты, количество газа и температуру в единицах, затем проверять баланс вещества при каждой связи.

Проверка на одной сцене: две одинаковые комнаты, дверь закрыта, в первой давление 1, во второй 0.5. Зафиксируй показатели до и после 1 с, затем открой дверь и повтори. С пробоиной отдельно сравни 30/60/120 FPS по одинаковому игровому времени. Если разница растёт, проверь ограничение `dt=0.10`, порядок открытия двери и вызовы EnvironmentSystem. Не смешивай этот тест с `FloodingSystem`: тот считает воду и плавучесть.

6. Частицы: рождение, жизнь и отрисовка

`ParticleSystem.cpp` хранит emitters, частицы и render items в заранее зарезервированных векторах. `CreateEmitter` ограничивает число источников `MAX_EMITTERS`, выдаёт ненулевой ID, сохраняет описание и может сразу подготовить первое рождение через `spawnAccumulator`. Обычно скорость эмиссии выражается в частицах/с: если она 12/с и `dt=0.25c`, аккумулятор увеличится на 3 и можно создать три частицы. При 60 FPS каждый шаг даст 0.2, но после пяти шагов сумма снова равна 1. Без дробного аккумулятора эмиттер с малой скоростью на высоком FPS вообще ничего бы не порождал.

В `Render` есть защита от двойного обновления: Play уже вызывает update из `ApplicationClass`, а Editor может сделать запасной шаг для preview. Флаг `m_updatedSinceLastRender` решает, нужен ли этот шаг; после рендера он сбрасывается. `ParticleRenderer` создаётся при необходимости и только один раз пробует `Initialize(device)`; это граница CPU симуляции и GPU ресурсов. Если GPU инициализация не удалась, gameplay событие взрыва не должно превращаться в отсутствие урона. Текстуры и shader particles — способ показать событие, но его игровая причина хранится вне render items.

Для одной частицы с начальной позицией `(0;0;0)`, скоростью `(2;4;0)` м/с и временем жизни 1 с простое движение без сил даёт через 0.25 с позицию `(0.5;1;0)`. Если подключить гравитацию, положение зависит от выбранного интегратора. Цвет можно считать `lerp(startColor,endColor,t/lifetime)`; при половине жизни результат находится между цветами по каждой компоненте. `Lerp` и `LerpColor` есть в этом файле. Следи, чтобы визуальная альфа и физический урон не управляли друг другом: исчезновение дыма не должно «отменять» уже полученное повреждение.

7. Взрыв: видимый эффект и импульс для физических тел

`ExplosionSystem::Emit` в `src/Gameplay/Effects/ExplosionSystem.cpp` принимает позицию и `ExplosionSettings`. Сначала ограничивает radius снизу 0.05 м, impulse — нулём, и вызывает `particleSystem.EmitExplosion`. Это визуальная часть. Затем проходит `RigidBodyComponent`. Тела без Bullet pointer, статические и кинематические исключаются; масса через `getInvMass()` должна быть положительной. Это важно: от взрыва не должен двигаться неподвижный интерьер лишь потому, что у него есть видимый меш.

Для оставшегося тела система берёт центр масс и вектор от эпицентра. Если расстояние больше радиуса, ничего не происходит. Если почти равно нулю, вместо деления на ноль выбирается направление вверх. После добавления `upwardBias` направление нормируется. Сила толчка задаётся `impulseStrength*(1-distance/radius)^2`. При radius=10 м, impulseStrength=100 Н·с и distance=5 м получаем долю 0.5, квадрат 0.25, импульс 25 Н·с. Для тела массой 5 кг это могло бы дать изменение скорости 5 м/с в направлении импульса до влияния других контактов. У границы radius импульс ноль; рядом с центром почти полный. Применение `applyCentralImpulse` будит тело, а небольшой `applyTorqueImpulse` добавляет вращение. Возвращаемое число — количество затронутых физических тел, а не число получивших урон персонажей.

У взрыва здесь нет обещания прямого уменьшения здоровья: этот метод занимается частицами и физическим импульсом. Если нужен игровой урон по радиусу, создай отдельный путь через `DamageEvent` и `DamageSystem`, с проверкой стен и фракций по выбранным правилам. Не используй число затронутых Bullet тел как суррогат количества раненых: статический игрок или персонаж с отдельным character controller может быть опасно близко, но не попасть в этот список.

8. Тревога корабля: выбор самого серьёзного сообщения

`ShipAlertSystem::Update` в `src/Gameplay/Ship/ShipAlertSystem.h` каждый раз очищает `s_current`, находит комнату игрока по `objectsInRoom` и затем последовательно проверяет кислород, пробоину, power grid и генераторы. При кислороде ≤0.20 выставляется Critical, при ≤0.45 — Warning. Открытая пробоина именно в комнате игрока также даёт Critical. У энергосети brownout — Critical; работа от батареи без генерации — Warning; запас батареи ≤15% — Advisory. У генератора `isScrammed` даёт Critical, высокая температура — Warning, нехватка топлива — Advisory.

`SetAlert` заменяет сообщение только если новый `level` строго выше текущего. Если два события имеют уровень Critical, остаётся первое, встреченное в порядке обхода. Для UI это простой приоритет, но он скрывает остальные аварии. Например, пробоина и brownout одновременно покажут лишь одну надпись. Если игра должна показывать очередь тревог, потребуется список событий с устойчивым порядком и сроком действия, а не одно поле `s_current`. Обрати внимание и на область видимости: найденный `shipRoot` ограничивает проверку генераторов, если корень известен. Если комната игрока не найдена, логика работает с fallback `INVALID_ENTITY`; этот случай стоит испытать отдельно в сцене без разметки комнат.

Для проверки создай комнату с кислородом 0.44 и генератором без топлива: ожидается Warning, потому что оно выше Advisory. Добавь пробоину: ожидается Critical. Закрой пробоину и подними oxygen до 0.8 — останется Advisory, если именно низкий запас топлива всё ещё виден системе. Это не «переключение текста» в HUD: оно показывает результат чтения нескольких ECS компонентов в определённом порядке.

9. Room culling: какие комнаты видит камера

`RoomCullingSystem.cpp` уменьшает число объектов, передаваемых рендеру. `FindRoomContainingPoint` проходит комнаты, берёт Transform и переводит мировую точку камеры в локальное пространство комнаты через обратную world matrix. Затем сравнивает её с `localMin/localMax` с небольшими допусками границ. Почему нужен inverse? Если отсек переместили или повернули, сравнение мировой координаты камеры с локальными min/max ошибочно. Обратная матрица возвращает точку в систему координат отсека, где прямоугольные границы снова имеют простой смысл.

Кэш `s_roomTransformCache` хранит position, rotation, scale и worldToLocal. При неизменном Transform повторно вычислять inverse не нужно. Это экономит работу при множестве объектов, но требует корректного сброса при загрузке сцены и изменении geometry bounds. `BuildVisibleRooms` начинает с комнаты камеры и использует связи дверей, чтобы сформировать список видимых помещений. Дверь может быть видна из любой из двух соседних комнат. Это уровень отсечения кандидатов, не доказательство фактической видимости каждого треугольника: от камеры до предмета ещё может стоять колонна.

Сложность. Если в корабле 100 комнат и 10 тысяч объектов, отправлять все объекты в каждый draw pass дорого. Отсечение по комнатам сокращает число кандидатов, но сама проверка принадлежности комнаты тоже имеет цену. Поэтому в коде есть кэши Entity→Room и debug статистика: число запросов, проверок и текущая комната камеры. При подозрении на исчезающий объект сначала сравни найденную комнату камеры, комнату объекта, состояние двери и итоговый visibleRooms. Не лечи culling отключением depth или изменением PBR shader — объект, который не попал в список draw candidates, GPU никогда не увидит.

10. Показатели игрока: перевод комнаты в здоровье и выносливость

`PlayerVitalsSystem::Update` в `src/Gameplay/Player/PlayerVitalsSystem.h` ищет `PlayerComponent` и комнату, содержащую Entity игрока. Доля `room.oxygen` в [0,1] превращается в целевой процент игрока умножением на 100. Текущее `player.oxygen` приближается к цели со скоростью 25 процентных пунктов в секунду. Если комната внезапно упала с 1.0 до 0.5, а у игрока пока 100, при `dt=0.1` показатель уменьшится до 97.5, а не мгновенно до 50. Такой лаг является правилом текущей системы; он может представлять запас дыхания/снаряжения, но код прямо не моделирует объём лёгких.

При нуле кислорода здоровье убывает на 10 единиц/с, если нет god mode или `InvulnerableComponent`. Значения ограничиваются [0,100]. `StaminaUpdate` уменьшает stamina на 20/с при `isRunning`, восстанавливает на 10/с, если `wantsRun=false`. Различие двух флагов существенно: игрок может хотеть бег, но фактически не бежать из-за иных условий; тогда обычное восстановление по этой ветке не происходит. Проверь границы: нулевой `dt`, потерю комнаты, паузу, смерть, переход между комнатами и смену флага бега. Система считывает атмосферу; она не должна самостоятельно менять давление комнаты, иначе двое игроков или NPC нарушат единый источник состояния среды.

Шейдеры с нуля: от одного треугольника до материала DX Framework

Шейдер — небольшая программа, которую GPU выполняет для элементов рисования. Начинающему полезно представить фабрику: CPU подготавливает геометрию, матрицы и ресурсы; vertex shader получает одну вершину и сообщает, где она окажется на экране и какие данные передать дальше; rasterizer определяет покрытые треугольником экранные фрагменты и интерполирует данные; pixel shader рассчитывает выходной цвет. Это не отдельный мир, который сам знает про Entity, здоровье NPC или двери. Он видит только переданные числа, buffers и textures. Официальное описание стадий и семантик есть в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources) и справочнике HLSL semantics (https://learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl-semantics).

1. Один треугольник до первого HLSL слова

Пусть в локальной плоскости есть вершины A=`(-1;-1;0)`, B=`(1;-1;0)`, C=`(0;1;0)`. Vertex buffer содержит их координаты, возможно цвет и UV; index buffer говорит, что треугольник составлен из индексов 0,1,2. Input layout сообщает D3D, в каких байтах каждой вершины находятся `POSITION`, `COLOR`, `TEXCOORD`. Ошибка смещения в layout означает, что HLSL прочтёт цвет как позицию или мусор, даже если текст шейдера безупречен. Для первого опыта используй один треугольник без освещения и текстуры: меньше стадий могут сломаться одновременно.

После vertex shader вершины проходят через world, view, projection. Для принятой в проекте конвенции с вектором-строкой: `clip = local * world * view * projection`. `world` ставит объект в мир, `view` описывает мир относительно камеры, `projection` создаёт перспективу. Результат содержит четыре координаты `(x;y;z;w)`. После деления на w получается нормализованная экранная позиция; если w некорректен или вершина вне видимого объёма, часть геометрии отсечётся. Не путай переменную `SV_POSITION` в pixel shader с мировой позицией: там уже экранная информация. Если нужен свет в мировом пространстве, передай отдельный `worldPos` и нормаль из VS.

2. Читаем простейший vertex shader почти по строкам

Файл `src/Render/Shader/Vs/color.vs` объявляет `cbuffer MatrixBuffer : register(b0)` с тремя матрицами. `b0` — слот буфера констант vertex stage, который CPU должен заполнить и привязать. `VertexInputType` получает `position : POSITION` и `color : COLOR`; `PixelInputType` отдаёт `position : SV_POSITION` и цвет. В `ColorVertexShader` сначала `input.position.w=1.0f`: это точка, на которую должен действовать перенос. Затем идут три `mul` в порядке world, view, projection. Последняя строка `output.color=input.color` переносит значение к следующей стадии. Если убрать эту запись, pixel shader, который ожидает COLOR, получит неопределённые для твоего алгоритма данные; если текущий pixel shader их не читает, картинка может случайно остаться прежней.

Минимальная учебная версия того же смысла:

```
cbuffer MatrixBuffer : register(b0) {
    float4x4 worldMatrix;
    float4x4 viewMatrix;
    float4x4 projectionMatrix;
};
struct VSIn { float4 position : POSITION; float4 color : COLOR; };
struct VSOut { float4 position : SV_POSITION; float4 color : COLOR; };
VSOut MainVS(VSIn input) {
    VSOut output;
    input.position.w = 1.0f;
    output.position = mul(input.position, worldMatrix);
    output.position = mul(output.position, viewMatrix);
    output.position = mul(output.position, projectionMatrix);
    output.color = input.color;
    return output;
}
```

Это учебный фрагмент для объяснения. Реальные entry point и buffer layout бери из файла проекта и соответствующего C++ класса. На CPU `ColorShaderClass::SetShaderParameters` в `src/Render/Shader/colorshaderclass.cpp` получает матрицы, делает `Map` dynamic buffer, записывает значения, `Unmap` и привязывает буфер через `VSSetConstantBuffers`. `InitializeShader` компилирует VS и PS через `D3DCompileFromFile`, создаёт shader objects и input layout. Если текст HLSL правильный, но CPU не привязал `b0`, шейдер не получит нужные матрицы. Если размер/упаковка C++ структуры не совпали с HLSL, одна из матриц или параметров будет считана неверно. Сначала сравни поля и их порядок, затем слоты и стадию привязки.

3. Что делает rasterizer между VS и PS

Vertex shader выполняется для вершин, но экран состоит из пикселей. Rasterizer превращает треугольник в покрытые им фрагменты. Для внутренней точки он вычисляет веса относительно трёх вершин. Если веса равны `(0.2;0.3;0.5)` и вершинные цвета красный `(1;0;0)`, зелёный `(0;1;0)`, синий `(0;0;1)`, интерполированный цвет равен `(0.2;0.3;0.5)`. Веса суммируются до 1. Для точек внутри треугольника они неотрицательны; если один вес отрицателен, точка лежит вне него в этой простой геометрической проверке. Для UV применяется схожая идея, но при перспективе GPU учитывает глубину, чтобы текстура не выглядела растянутой неправильно.

Rasterizer также учитывает лицевую сторону треугольника, режим culling, viewport и depth. Если порядок индексов поменять с (0,1,2) на (0,2,1), треугольник может стать обратной стороной и исчезнуть при back-face culling. Если он за другой поверхностью, depth test не позволит ему перекрасить ближний пиксель. Поэтому «pixel shader не работает» сначала проверяют одноцветным материалом, culling и depth, затем текстуру и свет. В документации Microsoft описаны pipeline stages и rasterization (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources); конкретные states выбирает renderer твоего движка.

4. Простейший pixel shader и ловушка текущего файла

`src/Render/Shader/Ps/color.ps` объявляет `ColorBuffer : register(b0)` и `ColorPixelShader(...):SV_TARGET`. `SV_TARGET` означает цвет, записываемый в текущий render target. Хотя структура входа содержит `color : COLOR`, текущая реализация возвращает `uniformColor`, полученный из constant buffer. Следовательно, разноцветные вершины из `color.vs` **не создадут градиент** с этим pixel shader. Это важный урок чтения пары VS/PS: надо проследить, что PS действительно использует из вывода VS. Для учебного эксперимента можно заменить возврат на `input.color`, но тогда потребуется согласовать C++

параметры и ожидаемое назначение `ColorShaderClass`; это изменение поведения проекта, а не описание текущего результата.

`register(b0)` в VS и `register(b0)` в PS не означают один общий буфер: это слоты двух разных стадий. C++ привязывает matrix buffer к VS и color buffer к PS. Если ошибочно привязать оба через `VSSetConstantBuffers`, PS не увидит цвет. Шейдер ничего не «ищет» по имени C++ переменной — совпадают слот, стадия и байтовая форма данных.

5. Текстура: UV, sampler и прозрачный вырез

В `src/Render/Shader/Vs/texture.vs` к позиции добавляется `float2 tex : TEXCOORD0`. Vertex shader вычисляет clip position, затем копирует UV в выход. `texture.ps` объявляет `Texture2D shaderTexture : register(t0)` и `SamplerState SampleType : register(s0)`. Выражение `shaderTexture.Sample(SampleType,input.tex)` возвращает цвет текстуры в интерполированной UV точке. Texture resource и sampler — разные вещи: первый хранит данные, второй задаёт способ выборки за границами, фильтрацию и другие правила. `TextureShaderClass::SetShaderParameters` передаёт texture SRV в pixel stage через `PSSetShaderResources`; без этой привязки `t0` не содержит нужную картинку.

Текущий `texture.ps` выполняет `clip(textureColor.a-0.01f)`. Если alpha меньше 0.01, выражение отрицательно и фрагмент отбрасывается. Это пороговый вырез, полезный для решётки или листвы: там нет полупрозрачного смешения у края. Если нужен дым или стекло с alpha 0.5, одного `clip` недостаточно; нужны соответствующие blend и depth правила. Пиксель с RGBA `(0.8;0.2;0.1;0.005)` отбрасывается, с alpha `0.02` остаётся. Не путай alpha cutout с «прозрачностью всего объекта».

Для проверки UV создай квадрат из двух треугольников и текстуру 2×2: красный, зелёный, синий и белый углы. Если цвета переставлены, проверь U/V и ориентацию изображения. Если виден один размазанный цвет, проверь UV в vertex buffer, input layout, передачу TEXCOORD из VS, sampler и привязку t0. Последовательная проверка каждого интерфейса быстрее случайной замены файла изображения.

6. Нормаль, Lambert и первая формула освещения

Нормаль N — единичное направление, перпендикулярное поверхности. Направление L от поверхности к источнику тоже нормируем. Простой диффузный свет пропорционален `max(dot(N,L),0)`. Если N и L совпадают, dot=1 и поверхность освещена сильнее. Если угол 60°, dot=cos60°=0.5. Если свет за поверхностью, dot отрицателен, `max` даёт 0. Зачем нормировать? При N длиной 2 и L длиной 3 dot может быть 6, хотя косинус никогда не больше 1. Тогда яркость ошибочно зависит от длины вектора.

Нормаль не является точкой: перенос world matrix не должен на неё влиять. При неравномерном масштабе простое умножение на world 3×3 может испортить перпендикулярность; нужна матрица, обратная транспонированной линейной части, затем нормировка. В проекте есть `normal_transform.hlsli`; сравни его с `pbr.vs` при диагностике растянутого меша. Проверка на кубе с scale `(2;1;1)`: выведи normal как RGB `(N*0.5+0.5)` и вращай камеру. Если цвет меняется от камеры, нормаль, вероятно, оказалась в другом пространстве, чем ожидалось. Если нормаль верна, переходи к light direction и shadow map.

7. От обычного света к PBR без заклинаний

`pbr.vs` и `pbr.ps` уже сложнее учебной пары. Вершинный этап может учитывать кости персонажа, преобразует позицию world/view/projection и передаёт сведения для освещения. Пиксельный этап

читает material textures и вычисляет цвет на основе normal, направления света и камеры, roughness и metallic. Сначала покажи на debug material каждую входную величину отдельно: albedo; normal; roughness; metallic; shadow factor. Затем объединяй их. Если итоговая картинка чёрная, один из входов может быть неверным; общая формула не объяснит, какой.

Albedo обычно представляет цвет и проходит через sRGB преобразование при чтении/выводе; normal, roughness и metallic — числовые данные, которые нельзя бездумно декодировать как цвет. Roughness влияет на ширину блика, metallic меняет баланс отражения и диффузной составляющей. PBR микрофасеточная модель использует распределение микронормалей, маскирование и Fresnel. Для новичка сначала достаточно понять энергетический вопрос: сколько света приходит, в какую сторону отражается и почему отражение меняется с углом взгляда. Параметры материала — входы модели, не независимые «эффекты красоты».

Тень обычно начинается отдельным проходом. `ShadowMap` хранит глубину от источника; основной pixel shader сравнивает текущую глубину с записанной и ослабляет прямой свет, если на пути была другая поверхность. Если тень оторвана от объекта, проверь bias, матрицу света и единицы глубины. Если поверхность исчезает лишь в теневом проходе, проверь culling и skinning обоих проходов. Световое PBR и теневая карта должны смотреть на одну и ту же геометрию текущего кадра.

8. Как добавить свой маленький шейдерный эффект в движок

Поставь конкретную задачу: аварийная панель плавно мигает, когда уровень опасности комнаты больше нуля. Gameplay система вычисляет `severity` в [0,1]. C++ передаёт её и время визуального эффекта в pixel constant buffer; PS рассчитывает `pulse=0.5+0.5*sin(time*frequency)` и смешивает базовый цвет с красным по `severity*pulse`. При `severity=0` панель остаётся исходной; при `severity=1` и `pulse=1` становится красной. Время можно брать из render clock, если мигание декоративное. Причина тревоги живёт в ECS, чтобы AI, сохранение и HUD видели то же состояние без GPU.

Сделай этот эффект по шагам. Сначала верни постоянный красный `float4(1,0,0,1)` и убедись, что нужный объект получает новый PS. Потом передай один `severity` через buffer, проверь значения 0, 0.5, 1. Затем добавь time и синус. После этого проверь обновление constant buffer (`Map`, запись, `Unmap`, `PSSetConstantBuffers`) и отсутствие конфликта слотов с другим материалом. Наконец проверь, что renderer сбрасывает ресурсы между draw calls: иначе следующая обычная панель может наследовать аварийный buffer.

Выполни четыре диагностические сцены: один треугольник с uniformColor; треугольник с vertex colors после явного изменения PS; квадрат с текстурой 2×2; куб с normal visualizer и одним источником Lambert. Каждый шаг добавляет только одну неизвестную. Когда они работают, читай `PBRShaderClass.cpp` как перечень CPU→GPU договоров: какие constant buffers созданы, в какие слоты привязаны, какие SRV поступают в PS, какой input layout ожидается и когда вызывается DrawIndexed. Эта цепь делает сложный шейдер проверяемым.

9. Буфер констант — точное соглашение о байтах

HLSL `cbuffer` не является C++ объектом, который GPU «находит» по имени. Это область байтов с правилами упаковки и привязкой к стадии и номеру регистра. Если в HLSL есть `float4 color; float severity;`, на стороне CPU нельзя без проверки положить подряд только 20 байт и предположить, что следующий buffer начнётся сразу после `severity`. Для D3D11 размер constant buffer должен соответствовать требованиям выравнивания, а поля внутри упаковываются по правилам HLSL. Проще начинать с группы из `float4` и матриц, сверять `sizeof` C++ структуры и отладочным шейдером выводить

каждое поле как цвет. Официальное объяснение constant buffers и их привязки есть в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources).

Путь обновления выглядит так: `Map(buffer,WRITE_DISCARD)` даёт CPU доступ к новой памяти, код копирует структуру, `Unmap` завершает запись, `VSSetConstantBuffers` или `PSSetConstantBuffers` привязывает buffer к нужному stage. Для карты, которая меняется каждый кадр, такой путь должен соответствовать флагам её создания. Если `Map` не удался, не вызывай Draw со случайным старым содержимым, если игра ожидает новую матрицу. Если изменился HLSL layout, но C++ структура осталась прежней, ошибка может выглядеть как «всё тёмное» или «свет прыгает», хотя компиляция обеих сторон успешна: C++ компилятор не проверяет контракт с HLSL автоматически.

Матрицы требуют ещё одной осторожности. Транспонирование перед записью в shader buffer связано с соглашением размещения и выбранной формой `mul`, а не с желанием поменять геометрический порядок. В конкретном `PBRShaderClass.cpp` CPU использует `XMMatrixTranspose`; в `pbr.vs` HLSL умножает позицию слева на world, затем view, projection. При отладке запиши одну тестовую точку на бумаге: локальная `(1;0;0;1)`, перенос по X на 5, камера без поворота. После world ожидается X=6. Если шейдер выдаёт другое число, сначала проверь upload и соглашение вектора/матрицы, а не подбирай camera FOV.

10. Какие ещё бывают шейдеры и нужны ли они здесь

В этом курсе подробно рассматриваются vertex и pixel shader, потому что они составляют основной путь видимого mesh в текущем DirectX 11 рендере. В графическом pipeline могут быть и другие стадии: geometry shader работает с примитивами, hull/domain используются для tessellation, compute shader запускает общее параллельное вычисление вне обычного draw треугольника. Наличие этих стадий в API не означает, что проект уже применяет их в главном рендере. Добавляй стадию только под конкретную задачу с измерением стоимости и данных. Для обычной подсветки аварийной панели новый compute pass не нужен: достаточно параметра pixel shader.

Один и тот же pixel shader может запускаться для очень многих фрагментов. Если экран 1920×1080, это около 2.07 млн пикселей до учёта перекрытия объектов и нескольких render passes. Дополнительная операция на каждом пикселе может стоить гораздо больше одной операции на каждом из нескольких тысяч объектов CPU. Но нельзя оценивать производительность только числом строк HLSL: выборки текстур, ветвления, память, overdraw и кэш GPU имеют разные цены. Измеряй GPU время конкретного прохода, а при диагностике сначала сделай одноцветную версию, затем по одному возвращай normal map, shadow, IBL и эффекты повреждения.

11. Цвет, яркость и тональная коррекция

Цвет текстуры, число света и цвет монитора относятся к разным пространствам значений. Большинство привычных цветных изображений предназначены для просмотра в sRGB; расчёт освещения удобнее делать в линейном пространстве, где удвоение числа примерно соответствует удвоению светового вклада. Если просто сложить два закодированных sRGB цвета и ограничить каждый канал единицей, результат может быть неверно ярким или тёмным. Карты roughness, normal и metallic хранят данные; применять к ним такое же цветовое преобразование нельзя. В отладке отдельно показывай сырое значение sample и значение после необходимого декодирования.

HDR позволяет временно хранить значения яркости больше 1.0. Затем tone mapping переводит диапазон сцены к диапазону экрана, а output conversion готовит цвет для отображения. Если

светильник в HDR buffer даёт `(4;2;1)`, простой clamp сделал бы `(1;1;1)` и потерял цвет яркого источника; tone mapping может сохранить различие каналов. Но выбранная кривая — художественное и техническое решение, а не физический закон для всей игры. Не пытайся исправить отсутствующую normal map увеличением exposure: сначала отладь входы материала.

12. Задачи для самостоятельной проверки шейдеров

Задача A. VS получил локальную точку `(2;0;0;1)`, world переносит по X на 3, view и projection условно единичные. Какой X уйдёт из VS? Ответ: 5. Если `w=0`, перенос не должен подействовать; это уже направление, а не точка. Найди в `color.vs` строку, которая принудительно ставит `w=1`.

Задача B. В вершинах треугольника цвета `(1;0;0)`, `(0;1;0)`, `(0;0;1)` и веса внутренней точки `(0.25;0.25;0.5)`. Интерполированный цвет `(0.25;0.25;0.5)`. Почему текущая пара `color.vs` + `color.ps` не покажет этот цвет? Pixel shader возвращает `uniformColor`, а не `input.color`.

Задача C. Normal N=`(0;1;0)`, направление света L=`(0.6;0.8;0)`, оба единичные. `dot=0.8`, Lambert diffuse factor 0.8. Если L=`(0;-1;0)`, фактор `max(-1,0)=0`. Проверь, что N и L находятся в одном пространстве: world normal и view light direction нельзя складывать без преобразования.

Задача D. Texture sample вернул alpha 0.009. `texture.ps` считает `clip(0.009-0.01)=clip(-0.001)`, поэтому фрагмент отбрасывается. При 0.011 выражение положительно, фрагмент остаётся. Почему это не равнозначно полупрозрачности 1%? Потому что `clip` принимает двоичное решение оставить/отбросить, а смешение с фоном выполняется другой стадией при подходящем blend state.

Задача E. Добавь свой параметр `severity` в учебный pixel shader. Сначала выведи `float4(severity,severity,severity,1)` для значений 0, 0.5, 1. Если экран остаётся одинаковым, проверь C++ buffer, `PSSetConstantBuffers`, номер `register(b#)` и то, что Draw действительно использует этот PS. Только после прохода такого простого теста добавляй цветовое смешение и синусоидальный pulse.

Bullet Physics в DX Framework

Эта часть объясняет Bullet не как набор загадочных классов, а как отдельную симуляцию, которую движок связывает с ECS. В проекте Bullet отвечает за столкновения, rigid body, character controller, raycast, convex sweep, импульсы взрывов и constraints подлодки. ECS хранит игровые данные, DirectX рисует результат, а Bullet вычисляет физическое состояние.

1 Ментальная модель Bullet

У физического объекта есть несколько разных представлений.

Представление	Пример	Ответственность
Игровая сущность	`Entity 42`	связывает компоненты и правила игры
Положение игры	`TransformComponent`	позиция для сцены редактора и рендера
Геометрия столкновения	`btCollisionShape`	отвечает где объект физически занимает место
Физическое состояние	`btRigidBody`	масса скорость силы трение сон
Регистрация в мире	`btDynamicsWorld`	делает объект участником запросов и симуляции
Визуальный меш	`MeshRendererComponent`	рисует форму но сам не сталкивается

Главный инвариант: render mesh, collision shape и `TransformComponent` могут описывать один предмет, но это не один объект памяти. Изменение масштаба модели не перестраивает коллайдер автоматически. Перенос `TransformComponent` не обязан переносить dynamic body, если система синхронизации не подтверждает внешний teleport.

2 Слои вычисления столкновения

Bullet не сравнивает каждый треугольник со всеми остальными треугольниками.

1. Broadphase хранит грубые AABB и находит потенциальные пары.
2. Narrowphase проверяет реальные формы потенциальной пары.
3. Dispatcher выбирает алгоритм столкновения.
4. Solver вычисляет импульсы, которые исправляют скорость и проникновение.
5. Dynamics world интегрирует движение и обслуживает constraints.

В текущем мире используется `btDbvtBroadphase`. DBVT означает динамическое дерево ограничивающих объёмов. Когда объект переносится вручную, его AABB должен быть обновлён, иначе broadphase некоторое время знает старое место.

3 Создание физического мира построчно

Файл `src/Core/PhysicsWorld.h` создаёт Bullet зависимости во временных `unique_ptr` и только после успеха передаёт их полям singleton.

```
auto collisionConfig = std::make_unique<btDefaultCollisionConfiguration>();
auto dispatcher = std::make_unique<btCollisionDispatcher>(collisionConfig.get());
auto broadphase = std::make_unique<btDbvtBroadphase>();
auto solver = std::make_unique<btSequentialImpulseConstraintSolver>();
auto ghostCallback = std::make_unique<btGhostPairCallback>();
auto dynamicsWorld = std::make_unique<btDiscreteDynamicsWorld>(
    dispatcher.get(), broadphase.get(), solver.get(), collisionConfig.get());
```

Строка	Что создаётся	Почему порядок важен
`collisionConfig`	настройки и фабрики narrowphase	dispatcher хранит на неё не владеющий указатель
`dispatcher`	выбор алгоритма для пары форм	мир вызывает его при обработке потенциальных пар
`broadphase`	DBVT для грубого отсева	без него мир не знает какие пары проверять
`solver`	последовательный импульсный решатель	исправляет скорости контактов и constraints
`ghostCallback`	поддержка overlap пар ghost object	нужен character controller
`dynamicsWorld`	объект который соединяет подсистемы	не владеет всеми переданными зависимостями

Временные `unique_ptr` защищают от частично созданного мира. Если конструктор одного следующего объекта бросит исключение, уже созданные зависимости будут уничтожены автоматически. После успешной настройки они перемещаются в поля. Адрес самих Bullet объектов при перемещении `unique_ptr` не меняется.

4 Гравитация ghost callback и GImpact

```
dynamicsWorld->setGravity(btVector3(0, -9.81f, 0));
dynamicsWorld->getPairCache()->setInternalGhostPairCallback(ghostCallback.get());
btGImpactCollisionAlgorithm::registerAlgorithm(dispatcher.get());
```

Первая строка задаёт ускорение мира в метрах на секунду в квадрате. Вторая разрешает pair cache отслеживать пересечения `btPairCachingGhostObject`. Третья регистрирует narrowphase алгоритм для GImpact. Создать `btGImpactMeshShape` недостаточно: dispatcher должен знать, каким алгоритмом обрабатывать такую пару.

5 Настройка solver

В текущем проекте число итераций увеличено до 50, включён split impulse и настроен ERP.

Параметр	Что делает	Симптом слишком малого или большого значения
`m_numIterations`	сколько раз solver уточняет ограничения	мало даёт мягкие дрожащие joints много повышает CPU стоимость
`m_splitImpulse`	отделяет коррекцию проникновения от обычной скорости	без него глубокое проникновение может подбросить тело
`m_splitImpulsePenetrationThreshold`	глубина для split correction	неверный порог меняет характер исправления контакта

Параметр	Что делает	Симптом слишком малого или большого значения
`m_erp`	доля позиционной ошибки исправляемая за шаг	слишком высоко даёт жёсткие рывки слишком низко оставляет ошибку

Эти параметры не заменяют правильную начальную расстановку тел. Если два тяжёлых отсека создаются глубоко друг в друге, solver всё равно вынужден бороться с большой ошибкой.

6 Fixed timestep и аккумулятор

`PhysicsWorld::Update` не передаёт произвольный кадровый `dt` напрямую в физику.

```
constexpr float timestep = 1.0f / 60.0f;
m_accumulator += (std::max)(0.0f, deltaTime);
if (m_accumulator > 0.25f)
    m_accumulator = 0.25f;
while (m_accumulator >= timestep)
{
    m_dynamicsWorld->stepSimulation(timestep, 1, timestep);
    m_accumulator -= timestep;
    ++m_stepIndex;
}
m_interpolationAlpha = std::clamp(m_accumulator / timestep, 0.0f, 1.0f);
```

Строка	Смысл
`1 / 60`	один физический шаг всегда длится одну шестидесятую секунды
прибавление `deltaTime`	кадр отдаёт накопителю прошедшее реальное время
clamp до `0.25`	предотвращает spiral of death после долгого зависания
`while`	выполняет столько полных шагов сколько накоплено
`stepIndex`	сообщает синхронизации что появился новый физический результат
`interpolationAlpha`	показывает долю пути от прошлого fixed state к текущему

Если рендер работает на 144 Hz, а Bullet на 60 Hz, несколько кадров могут видеть одинаковый физический state. Интерполяция делает визуальное движение плавным, но не должна возвращаться в Bullet как новая физическая позиция.

7 Static dynamic и kinematic

Тип	Масса	Кто задаёт Transform	Реагирует на силы	Типичный объект
Static	`0`	создаётся один раз	нет	пол стена
Dynamic	больше `0`	Bullet после шага	да	ящик обломок отсек
Kinematic	обычно `0` и флаг	ECS перед шагом	не как обычное dynamic body	управляемая дверь платформа

Нельзя определять тип только по массе. Kinematic body имеет специальный collision flag. В `PhysicsSystem::Update` проект сначала проверяет этот флаг, затем массу.

8 Владелец Transform и направление синхронизации

Текущий `PhysicsSystem::Update` разделяет синхронизацию на две стороны.

```
до fixed step
    kinematic        ECS -> Bullet каждый кадр
    dynamic teleport ECS -> Bullet только при внешнем изменении
    static           обычно не трогаем

после fixed step
    dynamic          Bullet -> ECS
    character        ghost object -> визуальный Transform с интерполяцией
```

Это один из главных контрактов физики. Если каждый кадр писать Transform dynamic body обратно в Bullet, физика не владеет движением: скорость и solver результат постоянно перезаписываются. Если никогда не отправлять внешний teleport, перемещение в редакторе останется только визуальным.

9 Проверка внешнего изменения

`TransformChangedExternally` сравнивает текущий Transform с `lastSyncedPosition`, `lastSyncedRotation` и `lastSyncedScale`. Равенство допустимо здесь потому, что обе стороны сравнения были записаны самим синхронизатором. Это не общий рецепт сравнения любых вычисленных float.

При внешнем изменении dynamic body проект:

1. строит новый `btTransform`;
2. записывает world transform тела и motion state;
3. обновляет interpolation transform;
4. обнуляет скорости и силы;
5. вызывает `updateSingleAabb`;
6. активирует тело;
7. сохраняет последние синхронизированные значения.

Обнуление скорости превращает операцию в teleport, а не в продолжение прежнего полёта. Для игровой механики броска с переносом понадобится отдельный контракт, который сохраняет или назначает скорость явно.

10 Деактивация ECS сущности

Неактивный объект не удаляется из Bullet, но временно переводится в состояние без контактной реакции и симуляции.

```
rb.body->setCollisionFlags(
    rb.collisionFlagsBeforeSuspend |
    btCollisionObject::CF_NO_CONTACT_RESPONSE);
rb.body->setLinearVelocity(btVector3(0, 0, 0));
rb.body->setAngularVelocity(btVector3(0, 0, 0));
rb.body->clearForces();
rb.body->forceActivationState(DISABLE_SIMULATION);
```

Перед этим сохраняются прежние flags и activation state. При возвращении `ActiveComponent` они восстанавливаются. Иначе временная деактивация навсегда превращала бы body в trigger или спящий объект.

11 Collision shape и масштаб

Bullet формы получают размеры при создании. Текущий проект поддерживает box, sphere, cylinder, capsule, convex hull, triangle mesh, GImpact и compound shape.

Shape	Использование	Ограничение
`btBoxShape`	стены ящики простые предметы	передаются полуразмеры
`btSphereShape`	снаряд область простая динамика	не зависит от вращения
`btCylinderShape`	цилиндрические объекты	ось формы должна совпасть с моделью
`btCapsuleShape`	персонаж	height относится к цилиндрической части Bullet API
`btConvexHullShape`	динамический сложный объект	должен оставаться выпуклым
`btBvhTriangleMeshShape`	точная статическая геометрия	не предназначен для обычного dynamic body
`btGImpactMeshShape`	динамический вогнутый меш	дороже простых и convex форм
`btCompoundShape`	несколько простых частей	нужно отдельно уничтожать child shapes

Нулевой и отрицательный scale недопустимы. В проекте они заменяются на единицу. Когда scale меняется после создания, `RefreshRigidBody` полностью перестраивает shape, потому что одного изменения визуального Transform недостаточно.

12 Почему box принимает полуразмеры

Для полного размера `(2, 4, 6)` в Bullet передаётся `(1, 2, 3)`. Эти числа означают расстояние от центра формы до каждой грани. Ошибка в два раза создаёт коллайдер, который выглядит смещённым или чрезмерно большим, хотя его центр математически правильный.

13 Triangle mesh и владение памятью

Текущий `CreateTriangleMeshShape` создаёт два связанных объекта: `btTriangleMesh` хранит треугольники, а `btBvhTriangleMeshShape` использует эту геометрию для collision query. Поэтому `RigidBodyComponent` хранит указатель `triangleMesh` отдельно. Удалить backing mesh раньше shape нельзя.

Путь каждой тройки индексов:

```
const auto& p0 = vertices[ia].position;
const auto& p1 = vertices[ib].position;
const auto& p2 = vertices[ic].position;
triangleMesh->addTriangle(
    btVector3(p0.x * scale.x, p0.y * scale.y, p0.z * scale.z),
    btVector3(p1.x * scale.x, p1.y * scale.y, p1.z * scale.z),
    btVector3(p2.x * scale.x, p2.y * scale.y, p2.z * scale.z),
    true);
```

Индексы проверяются до обращения к vertices. Масштаб применяется к каждой координате. Флаг `true` разрешает удаление дублирующихся вершин внутри `btTriangleMesh` по его правилам.

14 GImpact и цена точности

GImpact позволяет dynamic concave mesh, но его нельзя делать формой по умолчанию для каждого предмета. Простые формы и convex hull дешевле, устойчивее и дают solver более понятные контакты.

GImpact оправдан, когда игровая геометрия действительно вогнута и должна двигаться как единое тело.

После создания текущий код вызывает `updateBound`. Без обновлённой границы broadphase не имеет корректного AABB формы.

15 Compound shape

Compound shape хранит дочерние формы и локальный transform каждой части. Это позволяет собрать коллайдер корабля из нескольких box и capsule без одного тяжёлого triangle mesh.

```
world transform body
    * local transform child 0 -> box
    * local transform child 1 -> cylinder
    * local transform child 2 -> convex hull
```

При удалении compound текущий код проходит child shapes, удаляет их из compound и освобождает память. Одного `delete compound` недостаточно, если проект сам владеет созданными через `new` детьми.

16 Создание rigid body

Минимальная последовательность Bullet выглядит так.

```
btVector3 inertia(0, 0, 0);
if (mass > 0.0f)
    shape->calculateLocalInertia(mass, inertia);
auto* motionState = new btDefaultMotionState(startTransform);
btRigidBody::btRigidBodyConstructionInfo info(
    mass, motionState, shape, inertia);
auto* body = new btRigidBody(info);
world->addRigidBody(body, group, mask);
```

Строка	Контракт
нулевая inertia	static body не нуждается в динамическом моменте инерции
`calculateLocalInertia`	shape и mass определяют сопротивление вращению
motion state	канал между body и внешним представлением transform
construction info	собирает массу форму motion state и inertia
`addRigidBody`	только здесь body начинает участвовать в мире

Созданный объект до `addRigidBody` существует в памяти, но raycast мира его не увидит.

17 Collision groups и masks

Group отвечает на вопрос кто я. Mask отвечает на вопрос кого я хочу проверять. Пара рассматривается, когда оба битовых условия разрешают друг друга.

```
(groupA & maskB) != 0
and
(groupB & maskA) != 0
```

В `PhysicsSystem.h` группы оформлены как биты: default, dynamic, submarine exterior, submarine interior, player, projectile, trigger. Такое разделение позволяет игроку не сталкиваться со служебной внутренней формой подлодки, а внешнему корпусу не получать контакты с персонажами внутри.

При добавлении новой группы выпиши таблицу взаимодействий до кода. Ошибка маски проявляется как объект, который виден, но не сталкивается, либо как trigger, который никогда не получает overlap.

18 Entity внутри Bullet объекта

Проект связывает collision object с ECS через `setUserIndex` и читает его через `getUserIndex`. Это удобно для raycast, но имеет контракт: `INVALID_ENTITY` и диапазон типа должны обрабатываться одинаково во всех местах. После уничтожения Entity зарегистрированный Bullet object не должен остаться в мире с устаревшим index.

19 Raycast как запрос мира

Луч задаётся двумя точками, а не только направлением.

```
start = origin
end = origin + normalize(direction) * distance
```

`PhysicsRaycast::CastRay` сначала очищает `outHit`, проверяет мир и дистанцию, нормализует направление, отвергает нечисловой результат и только после этого вызывает `world->rayTest`.

```
IgnoreEntityRayCallback callback(
    outHit.rayStart, outHit.rayEnd,
    ignoredObject, ignoredEntity);
world->rayTest(outHit.rayStart, outHit.rayEnd, callback);
```

Raycast не двигает тела и не создаёт урон. Он только отвечает на геометрический вопрос о первом разрешённом контакте.

20 Пользовательский callback луча

`IgnoreEntityRayCallback` наследует `ClosestRayResultCallback` и переопределяет `addSingleResult`.

```
if (hitObject == m_ignoredObject ||
    static_cast<Entity>(hitObject->getUserIndex()) == m_ignoredEntity ||
    hitObject->getActivationState() == DISABLE_SIMULATION)
{
    return 1.0f;
}
return ClosestRayResultCallback::addSingleResult(
    rayResult, normalInWorldSpace);
```

Возврат `1.0f` сообщает callback не принимать этот кандидат как ближайшее попадание. Сравнение делается и по указателю объекта, и по Entity, потому что у одной сущности могут быть дополнительные collision objects. Неактивный объект также пропускается.

21 Closest и all hits

Callback	Результат	Пример
`ClosestRayResultCallback`	ближайший разрешённый hit	оружие взгляд взаимодействие
`AllHitsRayResultCallback`	массив всех hit	прострел нескольких слоёв анализ материалов
custom closest	ближайший после собственных фильтров	игнорирование стрелка и disabled objects

Для обычного оружия нужен closest hit. Иначе можно случайно нанести урон выбранной AI цели за стеной, проигнорировав реальный первый контакт.

22 От raycast к выстрелу

Текущее оружие является hitscan, а не летящим физическим снарядом.

```
нажатие или решение NPC
    -> WeaponSystem TryFire
    -> проверка магазина кулдауна и луча
    -> разброс и recoil
    -> PhysicsRaycast ближайший контакт
    -> DamageEvent
    -> DamageSystem в своей стадии кадра
```

Bullet здесь отвечает только за геометрическое подтверждение. Патроны, перезарядка, урон и фракции являются gameplay правилами.

23 Convex sweep

Raycast проверяет бесконечно тонкий отрезок. Персонаж имеет объём, поэтому для движения и step-up нужен sweep выпуклой формы.

```
world.convexSweepTest(
    &shape,
    fromTransform,
    toTransform,
    callback);
```

Sweep отвечает: как далеко форма может пройти по траектории до контакта. `m_closestHitFraction` находится в диапазоне от нуля до единицы. Позиция контакта вдоль перемещения вычисляется как `from + delta * fraction`.

24 Нормаль поверхности и допустимый склон

Character callbacks фильтруют поверхность по скалярному произведению нормали с мировой осью вверх.

```
upDot = normal dot (0, 1, 0)
```

Горизонтальный пол имеет normal примерно `(0, 1, 0)` и `upDot` около единицы. Вертикальная стена имеет значение около нуля. Порог можно получить как `cos(maxSlopeAngle)`. Чем больше допустимый угол склона, тем меньше порог.

25 Ghost object и character controller

`CharacterSpawnSystem` создаёт capsule shape, `btPairCachingGhostObject` и `btKinematicCharacterController`. Ghost object хранит форму и overlap пары, но сам не решает динамический контакт как rigid body. Character controller выполняет специальную логику ходьбы, step up, step down и penetration recovery.

Порядок регистрации:

1. создать capsule;
2. создать ghost object;
3. назначить transform и collision shape;
4. добавить `CF_CHARACTER_OBJECT`;
5. создать controller;
6. настроить gravity, slope и step height;
7. добавить collision object с group и mask;
8. добавить controller как action мира.

При уничтожении порядок обратный: удалить action, удалить collision object, затем освободить controller, ghost object и shape.

26 Почему центр капсулы выше ног

`TransformComponent` персонажа хранит игровую позицию у основания, а Bullet transform ghost object относится к центру формы. Поэтому используется `capsuleCenterOffsetY`.

```
bulletCenterY = feetY + capsuleCenterOffsetY
feetY = bulletCenterY - capsuleCenterOffsetY
```

Если вычесть offset дважды, модель провалится. Если не вычесть после чтения Bullet, она будет висеть над поверхностью.

27 Ground probe и step up

Ground detection не должен полагаться только на один флаг controller. Текущий код использует ray и convex sweep с фильтром поверхности. Для step up обычно проверяются три этапа.

1. Горизонтальное движение встречает препятствие.
2. Поднятая капсула имеет свободное место.
3. Из поднятой и сдвинутой позиции можно опуститься на допустимую поверхность.

Каждый этап требует отдельного наблюдаемого debug луча или sweep. Иначе баг выглядит как общее застревание, хотя причина может быть в потолке, слишком высокой ступени или неверной нормали посадки.

28 Контакты и manifolds

После шага dispatcher хранит contact manifolds. Каждый manifold относится к паре collision objects и содержит contact points. У точки доступны глубина, normal, позиции на A и B, а также применённый импульс.

Схема чтения:

```
const int count = dispatcher->getNumManifolds();
for (int i = 0; i < count; ++i)
{
    btPersistentManifold* manifold = dispatcher->getManifoldByIndexInternal(i);
    for (int j = 0; j < manifold->getNumContacts(); ++j)
    {
        const btManifoldPoint& point = manifold->getContactPoint(j);
        if (point.getDistance() <= 0.0f)
        {
            // подтверждённый контакт
        }
    }
}
```

Не создавай gameplay событие каждый fixed step без памяти пары. У устойчивого контакта manifold живёт много шагов, поэтому звук удара или damage on enter требует отслеживания перехода пары из отсутствующей в активную.

29 Trigger и no contact response

Trigger использует collision object с `CF_NO_CONTACT_RESPONSE`. Broadphase и overlap продолжают работать, но solver не выталкивает тела. Это отличается от mask, которая полностью запрещает образование пары.

Требование	Механизм
видеть вход без физического толчка	trigger и overlap pairs
полностью игнорировать категорию	group и mask
временно выключить объект	удалить из мира или согласованно отключить simulation

30 Force impulse velocity и teleport

Операция	Единица и смысл	Когда использовать
`applyCentralForce`	ньютон действует в течение шага	двигатель плавучесть постоянная тяга
`applyCentralImpulse`	изменение импульса мгновенно	взрыв удар прыжок dynamic body
`setLinearVelocity`	прямая замена скорости	специальный контролируемый режим
`setWorldTransform`	мгновенная смена положения	teleport редактор восстановление save

Сила должна учитывать время через интегратор Bullet. Импульс уже описывает мгновенное изменение количества движения и не умножается вручную на `deltaTime`.

31 Импульс взрыва построчно

`ExplosionSystem::Emit` перебирает rigid bodies, исключает static и kinematic, считает направление от центра взрыва и применяет квадратичный falloff.

```
const float normalizedDistance =
    (std::min)(distance / radius, 1.0f);
const float falloff = 1.0f - normalizedDistance;
const float finalImpulse = impulseStrength * falloff * falloff;
body->activate(true);
body->applyCentralImpulse(direction * finalImpulse);
```

Строка	Результат
`distance / radius`	перевод расстояния в диапазон области взрыва
`1 - normalized`	максимум в центре и ноль на границе
квадрат	быстро ослабляет дальнюю часть импульса
`activate`	будит спящее тело до применения
central impulse	меняет линейную скорость без точки приложения

Дополнительный torque impulse создаёт вращение. Для физически более точной модели можно применять обычный impulse в точке `applyImpulse`, тогда плечо само создаст момент.

32 Масса и обратная масса

Bullet часто хранит inverse mass. Для dynamic body `invMass = 1 / mass`. Для static и kinematic inverse mass равна нулю. Поэтому проверка `getInvMass() <= 0` быстро исключает тела, которые нельзя разогнать обычным импульсом.

Одинаковый impulse сильнее ускоряет лёгкое тело, потому что изменение скорости пропорционально inverse mass.

33 Трение restitution и damping

Свойство	Что регулирует	Ошибка настройки
friction	сопротивление скольжению в контакте	предмет едет как по льду или цепляется
restitution	доля упругого отскока	дребезг при слишком большом значении
linear damping	потеря линейной скорости без контакта	объект зависает или движется бесконечно
angular damping	потеря вращения	обломок никогда не перестаёт крутиться

Damping не заменяет сопротивление среды, когда нужна зависимость от скорости, глубины и направления. В физике подлодки проект дополнительно прикладывает drag force.

34 Плавучесть текущего проекта

Плавучесть следует идее Архимеда.

```
F buoyancy = fluidDensity * gravity * displacedVolume * submersionRatio
```

Вода внутри не уменьшает внешний объём корпуса. Она добавляет вес. Поэтому текущий код оставляет buoyancy volume прежним и вычитает дополнительный `waterWeight` из направленной вверх силы. Затем ограничивает экстремальное значение, добавляет drag и angular damping.

Этот раздел относится к gameplay approximation. Для точной плавучести нужны несколько sample points или погружённый объём сложной формы, центр приложения силы и torque от несовпадения центра плавучести с центром масс.

35 Constraints и joints

`btFixedConstraint` связывает два rigid body через два локальных frame. Frame A задан в локальном пространстве тела A, frame B в локальном пространстве тела B. Они описывают точки и ориентации, которые solver старается совместить.

В `SubmarineJointSystem::CreateJoint` сначала проверяются оба body, создаются локальные frames, затем constraint помещается в ECS компонент и только после этого регистрируется в Bullet.

```
jointComp.joint = new btFixedConstraint(
    *rbA->body, *rbB->body, frameA, frameB);
jointComp.joint->setBreakingImpulseThreshold(breakForce);
jointComp.joint->setOverrideNumSolverIterations(100);
```

Порог относится к solver impulse, а не непосредственно к силе в ньютонах. Название `breakForce` удобно для gameplay, но при балансировке важно помнить фактическую величину Bullet.

36 Владение constraint

Проект сначала передаёт constraint в ECS компонент, потом вызывает `world->addConstraint`. Это exception safe порядок: если добавление компонента не удалось, локальный владелец может освободить ресурс, и в мире не останется не владеющий dangling pointer.

При удалении constraint сначала отсоединяется от мира, затем уничтожается. `PhysicsWorld::Shutdown` также проходит все constraints и collision objects и снимает регистрации до разрушения зависимостей мира.

37 Сон и активация

Bullet усыпляет спокойные dynamic bodies для экономии CPU. Сила, impulse, значимое изменение скорости или `activate(true)` будит тело. `DISABLE_DEACTIVATION` запрещает обычный сон и нужен только для объектов, которые действительно должны постоянно обновляться, например некоторых управляемых механизмов.

Слишком широкое использование `DISABLE_DEACTIVATION` повышает стоимость мира и может скрыть ошибку: объект постоянно получает микросилу или дрожит в контакте.

38 CCD и быстрые тела

Discrete collision detection проверяет состояния по шагам. Очень быстрый маленький объект может оказаться по разные стороны тонкой стены на соседних шагах. Это tunneling.

Для dynamic projectile можно настроить CCD motion threshold и swept sphere radius. Но текущее стрелковое оружие hitscan и использует raycast, поэтому CCD ему не нужен. CCD понадобится реальному летящему снаряду, обломку высокой скорости или торпеде с маленьким коллайдером.

39 Настоящий физический снаряд

Чтобы заменить hitscan летящим projectile, нужны отдельные данные и жизненный цикл.

```
WeaponSystem
    -> создаёт projectile Entity
    -> Transform и ProjectileComponent
    -> sphere или capsule rigid body
    -> initial velocity

ProjectileSystem
    -> lifetime
    -> contact enter
    -> DamageEvent
    -> удаление body из Bullet
    -> DestroyEntity
```

Такой снаряд не должен оставлять body в мире после уничтожения Entity. Также нужен запрет попадания в стрелка на первых шагах через group mask, ignored owner или отдельный collision rule.

40 Потокобезопасность

Текущий Bullet world следует считать ресурсом главного потока. Фоновая задача может готовить CPU геометрию, но не должна одновременно добавлять body, менять shape или выполнять запрос мира, пока основной поток делает `stepSimulation`.

Безопасная схема асинхронной подготовки:

1. worker вычисляет чистые массивы вершин или decomposition;
2. результат помещается в main thread queue;
3. главный поток создаёт Bullet shape и регистрирует body;
4. ECS получает готовый runtime resource.

41 Отладочная визуализация

Для физического бага полезно одновременно видеть:

- collision shapes;
- AABB;
- contact points и normals;
- velocity vectors;
- raycast и sweep paths;
- center of mass;
- group и mask выбранного объекта;
- active state и collision flags;
- источник последней синхронизации Transform.

Один цвет должен иметь одно значение. Например зелёный query дошёл без hit, красный hit, жёлтый отфильтрованный объект, синий нормаль поверхности.

42 Диагностика по симптомам

Симптом	Первая проверка	Вероятная причина
видимый объект не блокирует луч	зарегистрирован ли body и mask луча	нет shape body не в мире неверная группа
dynamic body замер	mass invMass activation state	тело static или спит
объект дрожит	кто пишет Transform до и после шага	два владельца или проникновение
коллайдер остался на старом месте	AABB и rebuild shape	scale или teleport без updateSingleAabb
персонаж застрял на ступени	три sweep этапа и slope normal	нет места сверху плохой offset высокий step
raycast попадает в стрелка	ignored object и userIndex	не тот collision object или Entity
constraint рвётся сразу	начальные frames и penetration	большая стартовая ошибка
объект проходит через стену	скорость размер fixed step CCD	tunneling или неправильный mask
после load физика отличается	порядок создания и runtime поля	сохранён указатель или не пересобран body
утечка после смены сцены	remove из world до delete	нарушен жизненный цикл

43 Чек лист создания физического объекта

1. Определи источник истины Transform.
2. Выбери самый простой подходящий shape.
3. Запиши единицы размера и массы.
4. Реши static dynamic или kinematic.
5. Рассчитай inertia только для dynamic body.
6. Назначь group и mask по таблице взаимодействий.
7. Запиши Entity в userIndex.
8. Зарегистрируй body в мире только после готовности владельца.
9. Добавь debug draw формы и AABB.
10. Определи обратный порядок удаления.
11. Проверь save и повторную загрузку.
12. Проверь объект при 30, 60 и 144 кадрах рендера.

44 Практические задания по Bullet

Задание 1 Луч

Нарисуй start, end, нормаль и Entity первого hit. Добавь объект между оружием и мишенью и предскажи результат до запуска.

Задание 2 Фильтры

Составь таблицу для player, world, trigger и projectile. Для каждой пары вычисли оба битовых условия вручную.

Задание 3 Тип тела

Создай одинаковую коробку как static, dynamic и kinematic. Запиши кто меняет её Transform и как она реагирует на impulse.

Задание 4 Sweep

Покажи debug линии трёх этапов step up. Сделай препятствие немного ниже и немного выше порога.

Задание 5 Constraint

Соедини два простых dynamic box, задай небольшой threshold и выведи applied impulse. Затем измени только начальное проникновение и сравни результат.

Задание 6 Жизненный цикл

Десять раз создай и удали тестовый body. После каждого цикла проверь число collision objects мира. Оно должно возвращаться к исходному значению.

45 Контрольные вопросы

1. Почему render mesh не является collision shape.
2. Кто владеет Transform dynamic body после fixed step.
3. Зачем нужен broadphase и когда обновлять AABB.
4. Чем raycast отличается от convex sweep.
5. Почему trigger использует no contact response, а не пустую mask.
6. Зачем ghost pair callback character controller.
7. Чем force отличается от impulse.
8. Почему triangle mesh backing storage живёт не меньше shape.
9. Почему hitscan оружию не нужен CCD.
10. В каком порядке удалять constraint, body, motion state и shape.

Если на каждый вопрос можешь ответить через конкретный путь текущего кода, Bullet перестал быть внешней чёрной коробкой и стал понятным слоем движка.

NPC в DX Framework: от данных ECS до ремонта, поиска и выстрела

Эта глава написана по исходникам проекта на 29 октября 2026 года. Её цель — дать возможность восстановить в голове причинно следственную цепочку: **какой проход читает компонент, какое решение принимает, что записывает и какой следующий проход использует эту запись**. Здесь нет абстрактного «обычного игрового AI», выданного за уже реализованный код. Формулировка «сейчас» означает поведение проверенных исходников. Формулировка «предложение» означает изменение, которое ещё нужно реализовать.

Соглашения для чтения: `Entity` — идентификатор, а не адрес объекта; `INVALID_ENTITY` — отсутствие выбранной сущности; `dt` или `deltaTime` — секунды одного шага симуляции; `TransformComponent::position` — мировая позиция; координаты `x,z` задают горизонтальную палубу, `y` — высоту. Ссылки вида StateSystem.h:1629 (../StateSystem.h#L1629) ведут к проверяемым строкам. После изменения кода номера строк могут сдвинуться, поэтому ищите также названия функций.

1. Сначала поймём один игровой кадр

NPC не живёт внутри одного класса `Npc`. Его поведение составлено из ECS компонентов и систем, проходящих по ним. Это ключ к переучиванию кода: **компонент хранит факт или намерение; система превращает входы в новые факты или намерения**. Например, `NpcUtilitySystem` может записать `brain.repairTarget`, но не передвигает капсулу и не чинит объект. `StateSystem` превращает эту цель в направление движения или прогресс ремонта. `CharacterControllerSystem` передаёт направление в Bullet. `RepairSystem` завершает изменение игрового мира.

В ApplicationClass.cpp:1448–1551 (../src/Core/ApplicationClass.cpp#L1448) при `isPlayMode && simulationRunning` цепочка начинается с ввода игрока. Дальше порядок таков:

1. `WeaponSystem::UpdateTimers` один раз уменьшает таймеры всех `WeaponComponent` и сбрасывает `triggerHeld`.
2. `m_npcUtilitySystem.Update` считает локальную пользу ремонта для каждого подходящего мирного NPC.
3. `CrewTaskCoordinator::Update` очищает старые заявки, назначает ремонтные цели и ставит `CrewTaskClaimComponent` на саму цель.
4. `NpcCombatSystem::Update` выбирает возможного врага, проверяет сектор и физический луч, обновляет личную память наблюдения.
5. `NpcCoverSystem::Update` выбирает или проверяет укрытие, выставляет тактический `moveTarget` и состояние `TakingCover`, когда это требуется.
6. `StateSystem::Update` исполняет ручную команду, ремонт, погоню, поиск или тактическое движение. Он пишет намерение в `CharacterMovement`.
7. `CharacterFacingSystem`, `CharacterControllerSystem`, `AnimationController` обрабатывают разворот, физическое движение и позу. `DamageSystem::Update` в районе 1549 (../src/Core/ApplicationClass.cpp#L1549) применяет **накопленную до этого момента** очередь событий урона.
8. После обновления позы и физики ApplicationClass.cpp:1757–1769 (../src/Core/ApplicationClass.cpp#L1757) выполняются выстрел игрока и `NpcRangedCombatSystem::Update`. Их `WeaponSystem::TryFire` добавляет событие в очередь `DamageSystem`.

Следствие пункта 8: событие от выстрела в конце данного кадра обычно будет применено следующим вызовом `DamageSystem::Update`, то есть на следующем шаге симуляции. Это **очередь событий**, а не немедленное `target.health -= damage`. В том же конце кадра уже виден результат трассировки луча и оружейный кулдаун. Реакция здоровья и `underFireTime` наступает при обработке очереди. WeaponSystem.h:119–155 (../src/Gameplay/Combat/WeaponSystem.h#L119), DamageSystem.cpp:104–113 (../src/Gameplay/Combat/DamageSystem.cpp#L104).

В CrewTaskCoordinator.h:107–109 (../src/Gameplay/AI/CrewTaskCoordinator.h#L107) сохранился комментарий «вызывать до Utility/State». Реальный вызов в `ApplicationClass` ставит координатор **после Utility и до State**. Читая код, доверяйте месту вызова, а комментарий считайте устаревшей памяткой. Важная практическая причина: `Utility` даёт предварительный лучший `repairTarget`, но координатор может заменить его окончательным назначением, и `StateSystem` в том же кадре пойдёт уже к нему.

В ApplicationClass.cpp:1494–1504 (../src/Core/ApplicationClass.cpp#L1494) вызов `NpcSquadCombatSystem::Update` закомментирован. Поля `squadId`, `NpcSquadRole` и связанные типы существуют, но текущий запущенный боевой цикл **не назначает роли прикрывающего и обходящего через этот проход**. Индивидуальные укрытия, короткие очереди и поиск работают через другие системы. Не приписывайте им координацию отряда только потому, что поля отряда есть в компоненте.

1.1. Как читать ECS вызов `ForEach`

В ECSRegistry.h:45–59 (../src/ECS/ECSRegistry.h#L45) `ForEach<T1,T2>` итерирует пул первого типа и для каждой сущности проверяет наличие второго. Он не гарантирует порядок ID. `GetComponent<T>(entity)` возвращает указатель либо `nullptr`; `HasComponent` отвечает только на наличие; `IsAlive` проверяет, что ID ещё зарегистрирован как живой. `ActiveComponent` в этом проекте служит отдельным игровым разрешением участвовать в симуляции. Живая `Entity` может быть неактивной. Поэтому типичная

охрана имеет несколько проверок: ID существует, компоненты есть, Entity активна и здоровье больше нуля.

Ссылки из `GetComponent` и параметров `ForEach` привязаны к хранилищу. После `RemoveComponent<T>` или `DestroyEntity` соответствующая ссылка недействительна. Добавление компонента в **тот же** пул во время его обхода тоже опасно: `ComponentManager` может перераспределить контейнер. Координатор сначала собирает ID экипажа в `vector`, и уже затем в отдельном цикле добавляет отсутствующий `CrewMemberComponent`: это правильная граница между чтением и структурной мутацией. CrewTaskCoordinator.h:130–151 (../src/Gameplay/AI/CrewTaskCoordinator.h#L130). В ремонте пробоина уничтожается при завершении, поэтому `StateSystem` после `ApplyRepairProgress` не продолжает читать старый `RepairableComponent*`, а обрабатывает результат и очищает свои ID. StateSystem.h:1909–1932 (../StateSystem.h#L1909).

1.2. Два вида времени

`deltaTime` участвует в движении, прицеливании, таймерах и прогрессе ремонта. Скорость 2 единицы/с при `dt=0.1` означает желаемый путь порядка 0.2 единицы за кадр до коллизий. `worldTime` в `CrewTaskCoordinator` используется как временная метка назначения, а не как множитель скорости. Поэтому `assignedAtSeconds` не надо увеличивать каждый кадр. CrewTaskCoordinator.h:104–110 (../src/Gameplay/AI/CrewTaskCoordinator.h#L104). `StateSystem::Update` сразу возвращается при `deltaTime <= 0`, так что пауза не увеличивает ремонт. StateSystem.h:212–217 (../StateSystem.h#L212).

2. Карта данных: кому принадлежит какое состояние

2.1. Сущность, здоровье, фракция

`NPCComponent` хранит имя, описание, `health`, `maxHealth` и старый флаг `isHostile`. `NpcBrainComponent` хранит решение и память маршрута. Разделение видно в NPCComponents.h:601–650 (../NPCComponents.h#L601): здоровье — свойство персонажа, а состояние маршрута — рабочее состояние AI. Если заменить `NpcBrainComponent`, не следует терять здоровье. Если NPC получил урон, не нужно менять маршрут там же, если только тактическая система не получила специальный сигнал.

Боевой выбор фракций находится в NPCComponents.h:51–89 (../NPCComponents.h#L51). `GetFactionRelation(a,b)` даёт `Friendly` для одинаковых `FactionId`, `Neutral`, если хотя бы одна сторона `Neutral`, и `Hostile` для прочих разных фракций. `AreEnemies` — короткая проверка именно `Hostile`. Старый `isHostile` ещё исключает NPC из мирной repair utility и координатора, но боевой поиск и стрельба проверяют `FactionComponent` пары стрелок–цель. Поэтому для нового противника нужно настроить фракцию, а не надеяться только на `isHostile=true`.

Есть две схемы здоровья цели. Игрок имеет `PlayerComponent::health`, обычный NPC — `NPCComponent::health`; устройство — `DamageableComponent`. `DamageSystem` отдельно обрабатывает игрока, NPC без `DamageableComponent`, затем устройство. Если на NPC одновременно есть `NPCComponent` и `DamageableComponent`, в конце обработка общего `Damageable` синхронизирует `npc.health` с ним. DamageSystem.cpp:196–211 (../src/Gameplay/Combat/DamageSystem.cpp#L196), DamageSystem.cpp:331–339 (../src/Gameplay/Combat/DamageSystem.cpp#L331). При добавлении нового типа персонажа это место придётся осознанно расширять: иначе восприятие не сочтёт его живой целью, даже если на нём есть фракция и Transform.

2.2. `NpcState` — итог текущего шага; `NpcCommand` — внешний приоритет

Перечисление NPCComponents.h:92–109 (../NPCComponents.h#L92) содержит `Idle`, `Wandering`, `MoveToRepair`, `Repairing`, `WaitingForDoor`, `Chasing`, `Searching`, `Attacking`, `TakingCover`, `InCover` и другие состояния. Это не очередь работ. `state` означает, какую ветку `StateSystem::UpdateNpc` выполнит сейчас; несколько систем вправе переписать его за один кадр по описанному порядку. Например, `Utility` пишет `MoveToRepair`, но `NpcCombatSystem` при подтверждённом контакте может записать `Chasing`. Для задач экипажа важно понимать, что координатор успевает поставить `repairTarget` до восприятия боя, но текущий боевой NPC обычно вообще исключён из мирной utility по фракционному признаку.

`NpcCommand::type` — другой слой. Значение `None` означает автономию; `Stop`, `Wander`, `MoveToPosition`, `MoveToRoom`, `FollowEntity`, `RepairEntity` задают ручной приказ. NPCComponents.h:117–126 (../NPCComponents.h#L117). `IsActive()` проверяет `type != None`; `Clear()` стирает целевые ID и повышает `revision`. NPCComponents.h:167–200 (../NPCComponents.h#L167). `revision` передаётся в NavMesh кеш как часть ключа пути. Если меняется приказ, старый маршрут не обязан оставаться пригодным.

Почти все автономные системы проверяют активную команду. `StateSystem::UpdateNpc` делает это первым условием и немедленно `return` после `UpdateManualCommand`. StateSystem.h:395–413 (../StateSystem.h#L395). Это правило сильнее текущего `state`: даже если другой проход случайно поставил `Repairing`, активная команда `Stop` всё равно остановит движение. `Stop`, `Wander`, `FollowEntity` остаются до отмены; `MoveToPosition`, `MoveToRoom`, `RepairEntity` сами очищаются после выполнения. Если цель Follow потеряла Transform, система показывает `Blocked`, но команду не удаляет: сцену можно исправить и продолжить. StateSystem.h:1345–1458 (../StateSystem.h#L1345).

2.3. Движение, двери, рабочая зона

`NPCMovementComponent` задаёт авторские настройки ходьбы 2, бега 4.5 единицы/с, порог бега 8, обычный допуск прибытия 0.45, дистанции открытия и ожидания двери 2.25 и 1.25, глубину выхода за дверь 1.5. NPCComponents.h:616–641 (../NPCComponents.h#L616). `StateSystem` переносит эти настройки в `CharacterMovement` и пишет `walkDirection`, `currentSpeed`, `isMoving`, `isRunning`, `locomotionState`. Поле `canFly` сейчас не влияет на `MoveTowards`: вертикальная компонента направления обнуляется.

В `NpcBrainComponent` лежат `currentRoom`, `navigationTargetRoom`, `doorRoute`, `doorRouteIndex`, `activeDoor`, `routeDirty`, `repathTimer`, параметры ремонта и диагностические строки. Это не просто «большой компонент»; эти поля отвечают на конкретные вопросы: где я сейчас, куда хочу, какие двери запланированы, какую прохожу и нужно ли перестроить путь. NPCComponents.h:646 и далее (../NPCComponents.h#L646). `WorkZoneComponent` на отдельной Entity указывает `owner` оборудования. `StateSystem` ищет её и подбирает точку на поверхности, достижимую по NavMesh, если он доступен. RepairComponent.h:17–20 (../src/Gameplay/Repair/RepairComponent.h#L17), StateSystem.h:135–203 (../StateSystem.h#L135).

2.4. Бой, память, тактика и оружие

`NPCCombatComponent` в NPCComponents.h:475–599 (../NPCComponents.h#L475) хранит `targetEntity`, `targetVisible`, `visibleAimPoint`, радиус зрения и атаки, таймеры, последнее **лично увиденное** положение, наблюдавшуюся скорость и план поиска. `visibleAimPoint` актуальна только при `targetVisible=true`; не используйте старую точку без проверки. `NpcTacticalComponent` хранит авторский и активный стиль, позицию укрытия, позицию выглядывания, таймеры фаз, короткую очередь стрельбы на ходу, счётчик неудачных выглядываний и диагностический статус. NPCComponents.h:305–395 (../NPCComponents.h#L305). `WeaponComponent` находится на Entity оружия; `WeaponSystem::GetEquippedWeaponEntity(registry, owner)` разрешает владельца в оружие. `WeaponOwnerComponent` в событии урона позволяет обратное разрешение оружия в стрелка.

Не смешивайте память наблюдения и текущий Transform цели. Сам `NpcCombatSystem` смотрит на текущую геометрию кандидата, чтобы **проверить**, может ли NPC его увидеть. Но после потери прямой видимости `StateSystem` идёт к `lastKnownPosition`, а `NpcCoverSystem` оценивает укрытие относительно неё. Новую координату скрытой цели эти слои не записывают. NpcCombatSystem.h:373–379 (../NpcCombatSystem.h#L373), NpcCoverSystem.h:199–229 (../NpcCoverSystem.h#L199).

3. Мирный NPC: как задача ремонта превращается в назначение

3.1. `NpcUtilitySystem::Update`: допуск к автоматике

Сигнатура `void Update(ECSRegistry& registry, float deltaTime)` в NpcUtilitySystem.h:54–104 (../NpcUtilitySystem.h#L54) показывает два входа, но `deltaTime` пока намеренно не используется: score строится из текущего мира. `ForEach<NPCComponent,NpcBrainComponent>` получает каждую пару данных. Внутри дополнительно читаются `CrewMemberComponent`, `NPCCombatComponent`, `FactionComponent`. Комбинация боевого компонента с ненейтральной фракцией считается боевым NPC и выводится из старой crew utility. Неактивная сущность, `isHostile` и член экипажа `onDuty=false` также не получают автоматический ремонт. Для них система очищает диагностический score и при отсутствии ручного приказа приводит автоматическое ремонтное состояние к бездействию. Это защита от случайного «дёргания» боевого персонажа между погоней и пробоиной.

Вызов `EvaluateNpc(registry,npc,brain)` происходит только для допущенного NPC. Его выход — изменения `brain.idleScore`, `brain.repairScore`, `brain.repairTarget`, `brain.routeDirty`, `brain.state`, `brain.status`. Он **не добавляет заявку** и не изменяет `RepairableComponent`. Это важно для трассировки: увидев `brain.repairScore=0.8`, нельзя заключать, что цель уже принадлежит этому NPC.

3.2. `EvaluateNpc`: два прохода и арбитраж

В NpcUtilitySystem.h:115–301 (../NpcUtilitySystem.h#L115) базовый `idleScore` принудительно ставится в 0.10. Затем код перебирает все `HullBreachComponent`, считает оценку, запоминает максимум; отдельно перебирает `DamageableComponent`, пропуская объекты, которые одновременно являются пробоинами. Перед оценкой обоих видов проверяется `CrewTaskCoordinator::IsClaimedByOther`, чтобы предварительная utility не отправляла второго NPC на чужую заявку. Внутренний цикл не создаёт маршрут и не вызывает Bullet. Его результат — ID одного лучшего кандидата и численная оценка.

`CalculateBreachRepairScore` в NpcUtilitySystem.h:406–523 (../NpcUtilitySystem.h#L406) сначала проверяет наличие пробоины, `RepairableComponent`, Transform NPC и Brain; запечатанная или помеченная `repaired` пробоина получает 0. Далее считается трёхмерное расстояние `sqrt(dx²+dy²+dz²)`. Если `detectionRange>0` и дистанция больше диапазона, оценка 0. В пределах диапазона `distanceScore = 1 - clamp(distance/detectionRange,0,1)`, `severity = clamp(leakRate/0.15,0.15,1)`, итог `clamp(0.75*severity + 0.25*distanceScore,0,1)`. Коэффициенты выражают политику: интенсивность утечки весит втрое больше близости. При расстоянии 6 из 30 и `leakRate=0.12` получаются `distanceScore=0.8`, `severity=0.8`, итог `0.75*0.8+0.25*0.8=0.8`.

`CalculateEquipmentRepairScore` в NpcUtilitySystem.h:532–702 (../NpcUtilitySystem.h#L532) требует `Damageable`, `Repairable`, оба Transform и Brain; полное здоровье либо `repaired=true` даёт 0. `damageScore` — доля потерянного здоровья `1 - clamp(health/maxHealth,0,1)`. Тип устройства задаёт `importance`: обычное 0.40, генератор 1.0, кислородный генератор 0.90, свет 0.15. Итог `clamp(0.45*damageScore + 0.40*importance + 0.15*distanceScore,0,1)`. Для генератора с 50/100 HP на расстоянии 6 из 30 получаем `0.45*0.5+0.40*1+0.15*0.8=0.745`. Эта таблица зашита в utility; если добавить новый критичный тип

оборудования, он получит 0.40, пока вы не расширите правило. Формула координатора другая, поэтому debug `brain.repairScore` не обязан равняться `CrewTaskClaimComponent::priority`.

После сканирования `brain.repairScore` сохраняется для диагностики. Активная ручная команда останавливает дальнейшую запись состояния: score всё равно виден в UI, но команду нельзя перехватить. Если NPC уже `Repairing` и цель ещё пригодна, цель сохраняется. Иначе при новом лучшем `repairTarget` ставится `routeDirty=true`; score больше `idleScore` переводит в `MoveToRepair`, иначе `Idle`. NpcUtilitySystem.h:219–301 (../NpcUtilitySystem.h#L219). Нулевой score не даёт автоматическую работу. Важная особенность: utility рассматривает местную выгодность, но окончательное распределение происходит следующим проходом.

3.3. `CrewTaskCoordinator::Update`: одна цель — один исполнитель

Вход `Update(ECSRegistry& registry,double worldTime)` не получает `dt`. В начале CrewTaskCoordinator.h:110–138 (../src/Gameplay/AI/CrewTaskCoordinator.h#L110) сбрасывается снимок телеметрии, удаляются негодные заявки, собираются незавершённые цели и уже занятые ID. Вектор экипажа строится через `ForEach<NPCComponent,NpcBrainComponent>` и сортируется по Entity ID. Сортировка делает назначение повторяемым при одинаковых входах, несмотря на порядок unordered хранилища. Внутри цикла при отсутствии `CrewMemberComponent` он добавляется с настройками по умолчанию. Это значит, что созданный в редакторе мирный NPC без явной роли всё равно может быть назначен на ремонт.

В CrewTaskCoordinator.h:139–170 (../src/Gameplay/AI/CrewTaskCoordinator.h#L139) неактивный, `isHostile`, `onDuty=false` или управляемый вручную персонаж теряет старую заявку. При ручной команде автоматическое состояние не перезаписывается. Если назначение уже существует и валидно, `KeepExistingAssignment` оставляет его. Эта «липкость» важна: без неё два ремонтника могли бы каждый кадр менять цели при малом изменении расстояния. При смене ID `brain.repairTarget` ставится `routeDirty`, а не перестраивается путь прямо в координаторе.

Здесь есть точная граница между двумя проходами, которую легко пропустить: `NpcUtilitySystem` исключает также сочетание `NPCCombatComponent` с ненейтральной фракцией, а `CrewTaskCoordinator` **не повторяет этот `factionCombatant` фильтр**; он проверяет старый `isHostile`. Значит, боевой персонаж с ненейтральной фракцией, `NPCCombatComponent`, но `isHostile=false` может не участвовать в utility и всё же получить ремонтную заявку от координатора, если он активен и на дежурстве. Для нового вражеского NPC явно задавайте согласованную конфигурацию; для развития кода лучше вынести общий предикат «может работать в экипаже» в одну функцию и вызывать из обоих проходов. Это предложение, а не уже существующая единая политика.

Для свободного NPC цикл по незанятым кандидатам вычисляет `CalculateAssignmentScore` и выбирает только результат выше `brain.idleScore`. При равном score используется меньший ID цели. На выбранную цель ставится `CrewTaskClaimComponent{assignee,type,priority,claimedAtSeconds}`; на NPC пишутся `member.assignedTask`, время, `brain.repairTarget`, `routeDirty`, `MoveToRepair` и статус. `claimedTargets.insert` сразу не даёт следующему NPC этого же кадра занять цель. CrewTaskCoordinator.h:172–220 (../src/Gameplay/AI/CrewTaskCoordinator.h#L172). Инвариант в нормальном проходе: для одной ремонтной цели один `CrewTaskClaimComponent`, и назначенный NPC указывает на неё через `assignedTask`.

Формула координатора CrewTaskCoordinator.h:382–446 (../src/Gameplay/AI/CrewTaskCoordinator.h#L382) **отличается** от формулы utility. Для пробоины серьёзность `clamp(leakRate/0.15,0.2,1)`, для оборудования — доля утраченного HP. Дистанция снова трёхмерная; за `detectionRange` оценка 0. Профессия меняет множитель: инженер 1.25, Operations 1.05,

Security 0.72, Medic 0.62, Generalist 0.90. Operations получает ещё 0.15 для генератора или кислородного генератора. `repairSkill` ограничен диапазоном 0.25–2. Итог:

```
score = clamp((0.65 * severity + 0.35 * distanceScore)
              * roleAffinity * repairSkill, 0, 2)
```

Например, инженер с `repairSkill=1.5`, в 6 единицах от пробоины `leakRate=0.12`, с `detectionRange=30`: `severity=0.8`, `distanceScore=0.8`, базовый score 0.8, итог `0.8*1.25*1.5=1.5`. Он превосходит `idleScore=0.10`. Второй такой же NPC уже увидит цель в `claimedTargets` и должен искать другую. Обратите внимание: оценка использует **геометрическое расстояние**, не длину реального маршрута через двери. Поэтому близкая через стену цель может оказаться назначенной раньше доступной, но более далёкой. Это текущий компромисс алгоритма, а не доказательство достижимости.

`CleanupInvalidClaims` в CrewTaskCoordinator.h:281–308 (../src/Gameplay/AI/CrewTaskCoordinator.h#L281) удаляет заявку, если цель больше не нуждается в ремонте либо исполнитель исчез, выключен, снят с дежурства или получил ручную команду. Удаление делается во втором цикле после сбора ID, чтобы не менять пул во время его перебора. `completedTasks` здесь увеличивается при невалидной цели с совпадающим `assignedTask`; отдельно `RepairSystem` увеличивает его **до уничтожения** успешно закрытой пробоины. Поэтому при аналитике не следует считать `completedTasks` безусловно точным числом только успешных окончаний для всех способов удаления целей. RepairSystem.h:660–690 (../src/Gameplay/Repair/RepairSystem.h#L660).

4. Исполнение ремонта почти по строкам

4.1. `StateSystem::Update`: подготовка физического исполнителя

`StateSystem::Update(ECSRegistry&,float,JobSystem&)` в StateSystem.h:212–357 (../StateSystem.h#L212) проходит по `NpcBrainComponent` и `TransformComponent`. Это шире, чем список `NPCComponent`: любая Entity с мозгом и Transform может дойти до этого кода. Поэтому при добавлении компонентов в редакторе следите за целостностью сущности, а не только за именем класса.

Первые строки защищают паузу и неактивность. Для неактивного NPC система обнуляет `CharacterMovement` и, если есть готовый Bullet контроллер, его walk direction и velocity interval. Неактивность сама по себе не удаляет Entity. Далее проверяется конфликт `RigidBodyComponent` с `btKinematicCharacterController`: оба владельца физики не должны одновременно переписывать один Transform. Код удаляет `RigidBodyComponent` и пишет диагностику. Это изменение состава Entity происходит до получения нижележащих указателей на движение и контроллер.

`EnsureComponent<CharacterMovement>`, `EnsureComponent<CharacterController>`, `EnsureComponent<NPCMovementComponent>` добавляют недостающие runtime компоненты. Контроллер бывает полностью готовым (`controller`, `ghostObj`, `capsuleShape` все есть), полностью пустым либо частично построенным. Пустой спавнится `CharacterSpawnSystem::Spawn`, частичный не пересоздаётся поверх старых Bullet объектов. После этого настройки NPC нормализуются: скорость ходьбы не ниже нуля, бег не ниже ходьбы. Если модель умеет анимации, недостающий `AnimationComponent` добавляется автоматически. Эти шаги важны при странном поведении созданного через редактор NPC: «мозг» сам по себе не создаёт модель и корректный скелет, но часть инфраструктуры движения система действительно создаёт.

Текущая комната определяется каждый кадр `FindContainingRoom` с предпочтением прежней комнаты в перекрывающихся bounds двери. Затем `UpdateNpc` заполняет намерение. После него отдельное разведение капсул добавляет локальное steering направление, если два живых активных NPC слишком близко. Оно не телепортирует Transform, а меняет `walkDirection`. Если в текущем кадре не требовался

временный обход коллизии дверного портала, он отключается. Последним вызывается `UpdateStuckStatus`, который сравнивает физический сдвиг Transform после прошлых шагов, а не просто факт желаемого движения. StateSystem.h:297–355 (../StateSystem.h#L297).

4.2. `UpdateNpc`: диспетчер состояний

Сигнатура StateSystem.h:395–403 (../StateSystem.h#L395) принимает Registry, ID, ссылки на Brain, Transform и `CharacterMovement`, константные настройки NPC движения, `deltaTime` и `JobSystem`. Она **не возвращает значение**: выходом являются изменённые компоненты и, для ремонта, вызов `RepairSystem`. Первым проверяется ручная команда. Ниже `switch(brain.state)` направляет `MoveToRepair`, `Repairing` и `WaitingForDoor` в одну функцию `UpdateRepairTask`; это позволяет после открытия двери продолжить ту же работу без нового назначения. `TakingCover` идёт к `tactical->moveTarget`; `Chasing` идёт к последней увиденной точке; `Searching` исполняет план поиска; `Attacking` останавливает нижнюю часть тела, разворачивает корпус и готовит атаку. StateSystem.h:453–538 (../StateSystem.h#L453).

Почему `WaitingForDoor` не самостоятельная «задача»? Потому что дверь — промежуточная фаза **любого** маршрута, а не новая цель экипажа. При каждом кадре `UpdateRepairTask` сначала вновь проверит, что цель всё ещё нужна, и затем снова вызовет `NavigateTo`. Если дверь теперь открыта, маршрут продолжится. Так ремонтник не может бесконечно «ждать старую дверь», когда утечку уже закрыли другим способом.

4.3. `UpdateRepairTask`: вход и две формы цели

Сигнатура StateSystem.h:1629–1639 (../StateSystem.h#L1629) включает `Entity target` и `bool manualCommand`. Один и тот же механизм обслуживает автоматическое назначение и ручной `RepairEntity`; различается только способ завершить работу. На входе по `target` берётся `RepairableComponent*`. Отсутствие компонента либо `repaired=true` немедленно завершает ручную команду через `CompleteManualCommand` или очищает автоматический `brain.repairTarget`, переводит в `Idle`, помечает маршрут грязным и останавливает ходьбу. Никакой прогресс в таком кадре не начисляется.

Дальше функция локально создаёт `repairPosition`, `repairRoom`, `validTarget`, `targetType`. Для пробоины она требует `HullBreachComponent` и `!sealed`, берёт `worldPosition` и `room`. Для оборудования берёт `DamageableComponent`, требует `health < maxHealth`, затем `TransformComponent`. В качестве запасной точки служит Transform самого устройства. `RoomMembershipComponent` даёт комнату, а `FindRepairZoneForOwner` может найти отдельный `WorkZoneComponent`; тогда `TryPickReachableWorkZonePoint` выберет точку на рабочей зоне. StateSystem.h:1671–1776 (../StateSystem.h#L1671). Порядок `if breach ... else if damageable ...` означает, что сущность с обоими компонентами трактуется как пробоина. Это согласовано с пропуском двойного учёта в utility.

`TryPickReachableWorkZonePoint` в StateSystem.h:135–203 (../StateSystem.h#L135) сначала пробует ближайшую точку в зоне, центр и восемь локальных образцов, переведённых в мир. Если известна комната, каждый кандидат должен спроецироваться через `NpcNavMeshSystem::FindReachablePoint` на связный остров сетки. Из успешных берётся ближайший к NPC по горизонтали. Если комната неизвестна, допускаются исходные точки. Если ни один кандидат не подходит при известной комнате, функция возвращает `false`, и остаётся fallback к Transform оборудования. Поэтому отсутствие рабочей зоны не блокирует ремонт, но может привести NPC слишком близко к коллайдеру оборудования. Это место нужно проверить при авторинге сложных аппаратов.

После распознавания типа `!validTarget` снова приводит к ручному завершению или автоматическому `Idle`. Затем StateSystem.h:1813–1862 (../StateSystem.h#L1813) вычисляет **горизонтальное** расстояние до рабочей позиции. Здесь Y специально исключён: ремонтник ходит по палубе, а точка пробоины

может быть на стене выше центра капсулы. Это отличается от трёхмерной дистанции в оценке utility и координатора. Если `distance > brain.repairDistance`, ставится `MoveToRepair` и вызывается `NavigateTo` с комнатой, допуском и `movingState`. Результат `Blocked` даёт конкретную строку для пробоины или оборудования; в любом случае функция возвращается до начисления прогресса. `Waiting` тоже означает отсутствие прогресса. Если расстояние уже в допуске, маршрут в этом кадре не вызывается: ходьба останавливается, состояние становится `Repairing`.

4.4. От секунд к прогрессу и завершению

`brain.repairDuration` — время ремонта с нуля при `repairSkill=1`, нижний предел 0.05 с. `CrewMemberComponent::repairSkill` ограничивается диапазоном 0.25–2. В StateSystem.h:1875–1915 (../StateSystem.h#L1875) вычисляется:

```
progressDelta = requiredProgress * dt * repairSkill / max(repairDuration, 0.05)
```

В исходнике эта формула записана так:

```
const float progressDelta =
    repairable->requiredProgress *
    deltaTime *
    repairSkill /
    duration;
```

Здесь множитель `requiredProgress` переводит долю работы за кадр в единицы шкалы компонента, `deltaTime` делает начисление пропорциональным прошедшему времени, а деление на `duration` задаёт полное время при навыке 1. Нельзя заменить `deltaTime` константой «за кадр»: частота 30 и 120 кадров/с тогда дала бы разные скорости ремонта.

Если `requiredProgress=1`, `repairDuration=3`, `dt=0.1`, skill=1.5, то `progressDelta=0.05`. Из нуля потребуется примерно 20 таких шагов, то есть 2 секунды симуляции. Причина деления на duration проста: за duration секунд при skill=1 сумма `requiredProgress * dt / duration` достигает `requiredProgress`. При skill=1.5 скорость в полтора раза выше, полное время — duration/1.5. Если `requiredProgress=5`, за тот же шаг добавится 0.25, а нормированная доля всё равно 0.05. Это делает авторскую шкалу прогресса независимой от скорости труда.

Вызов `RepairSystem::ApplyRepairProgress(registry,target,progressDelta,&jobs)` — общий путь ремонта, не отдельная NPC симуляция. RepairSystem.h:189–234 (../src/Gameplay/Repair/RepairSystem.h#L189) он отвергает отсутствующий, уже законченный компонент и `progressDelta<=0`, увеличивает `progress` с ограничением `requiredProgress`, а до порога возвращает `false`. Достижение численного порога запускает `CompleteRepair`. Для пробоины с `HullBreachRepairLinkComponent` там сначала восстанавливается runtime mesh и collision; если асинхронная операция ещё не готова, результат `false`, хотя численный прогресс уже 100%. На следующих кадрах `ApplyRepairProgress` снова вызывает завершение, пока меш не готов. Поэтому статус `Repair: waiting for runtime operation` означает **не** недостаточную скорость NPC, а ожидание структурного восстановления корпуса. RepairSystem.h:526–556 (../src/Gameplay/Repair/RepairSystem.h#L526).

Если `completed=false`, `StateSystem` заново получает указатель `RepairableComponent*` и лишь затем читает `Progress01()`, ставит `brain.repairProgress` и статус `Repairing hull breach`, `Repairing equipment` либо ожидание runtime операции. StateSystem.h:1921–1967 (../StateSystem.h#L1921). Повторное получение важно даже если сегодняшний конкретный путь при `false` обычно сохраняет Entity: это безопасная привычка после функции, которая может структурно менять ECS. При `completed=true` код **не разыменовывает прежний `repairable`**. Он выставляет debug прогресс 1, очищает `repairTarget`, `doorRoute`, `doorDestinationRooms`, индекс и активную дверь; затем либо снимает ручную команду, либо переводит автоматический NPC в `Idle`. StateSystem.h:1970–2024 (../StateSystem.h#L1970).

При реальном закрытии пробоины `RepairSystem::CompleteRepair` помечает `sealed`, обнуляет `leakRate`, корректирует связанную панель, помечает `repairable.repaired`, учитывает завершённую заявку экипажа, затем вызывает `registry.DestroyEntity(repairEntity)` и возвращает `true`. RepairSystem.h:570–690 (../src/Gameplay/Repair/RepairSystem.h#L570). Для оборудования Entity остаётся: `Damageable.health` становится `maxHealth`, `isDestroyed=false`, генератор включается, у кислородного генератора снимается `broken`, `repairable.repaired=true`. RepairSystem.h:698–765 (../src/Gameplay/Repair/RepairSystem.h#L698). В текущем NPC вызове не переданы `eventLog`, `worldTime`, `repairSource`; необязательные параметры остаются значениями по умолчанию. Поэтому не обещайте, что именно автоматический ремонт запишет `BreachSealed` или `EquipmentRepaired` в `WorldEventLog`, пока не проведёте эти аргументы через вызов. Это конкретный пример полезного улучшения для аналитики и миссий.

4.5. Ручной ремонт и отмена

`UpdateManualCommand` в StateSystem.h:1345–1459 (../StateSystem.h#L1345) при `RepairEntity` передаёт `brain.command.targetEntity` в ту же `UpdateRepairTask` с `manualCommand=true`. Он не обязан владеть `CrewTaskClaimComponent`: ручной приказ имеет другой источник приоритета. При завершении `CompleteManualCommand` вызывает `brain.command.Clear()`, возвращает `Idle`, помечает маршрут грязным, очищает дверной путь и останавливает движение. StateSystem.h:3071–3085 (../StateSystem.h#L3071). На следующем кадре utility снова имеет право назначить автоматическую работу. Для ручного `Stop` команда намеренно не очищается; восстановление автономии требует внешнего снятия команды.

Отказы в ремонте нельзя сводить к одному «не работает». Если NPC вообще не получает цель — смотрите `Active`, `CrewMember.onDuty`, фракцию, `isHostile`, `Repairable`, `Damageable`/`HullBreach`, `repaired`, диапазон и score. Если цель есть, но NPC не идёт — смотрите `brain.state`, `routeDirty`, `currentRoom`, дверной граф, NavMesh и `CharacterController`. Если стоит рядом, но не ремонтирует — сравните **горизонтальное** расстояние с `repairDistance` и проверьте, не выбрана ли недоступная fallback позиция устройства. Если `progress=100%`, но цель не исчезла — смотрите завершение async mesh операции.

5. Навигация: граф комнат, локальная сетка и Bullet капсула

5.1. `NavigateTo` — общий контракт для разных задач

Сигнатура StateSystem.h:2184–2196 (../StateSystem.h#L2184) принимает `finalTarget`, `targetRoom`, `arrivalDistance`, `movingState` и `dt`. Возвращаемый внутренний `NavigationResult` может означать `Moving`, `Arrived`, `Waiting` или `Blocked`; вызвавшая ветка решает, что это значит для своей задачи. На входе стоит **мировая** цель. На выходе `NavigateTo` не меняет Transform напрямую: он заполняет `CharacterMovement`. Поэтому если при отладке `walkDirection` ненулевой, а позиция стоит, проблема уже ниже AI — в физике, коллизии или контроллере.

В начале `repathTimer` уменьшается на `dt`, debug поля дверного шага сбрасываются. Если `currentRoom` и `targetRoom` известны и различаются, нужен маршрут по комнатам. Если переход через дверь уже начат, он сохраняется, даже если `currentRoom` на пороге временно сменился: иначе путь обрывался бы посередине проёма. Новый target room либо `routeDirty` после истечения `repathTimer` вызывает `BuildDoorRoute`. Неудача ставит `Blocked` с `No traversable door route to target room`; ручная команда при этом сохраняется. Если маршрут грязный, но кулдаун перестроения ещё не истёк, NPC безопасно стоит. Если обе точки уже в одной комнате, промежуточный дверной путь очищается. StateSystem.h:2201–2270 (../StateSystem.h#L2201).

`BuildDoorRoute` в StateSystem.h:2503–2582 (../StateSystem.h#L2503) строит граф: вершина — `RoomComponent` Entity, ребро — `DoorComponent(roomA,roomB)`. BFS выбирает путь с минимальным **числом дверей**, так как

каждая дверь имеет одинаковую стоимость. Дверь с требованием питания исключается, если она не открыта, не сломана и питания нет; сломанная дверь считается проходимой по контракту `DoorSystem`. `visited` хранит предыдущую комнату и дверь, затем путь разворачивается в порядок прохождения. `doorRoute` и `doorDestinationRooms` — параллельные векторы одинаковых шагов. Если позже появится «опасная дверь», «запертая дверь для определённой роли» или стоимость перехода по времени, BFS уже не выразит это корректно; понадобится весовой алгоритм и политика доступа.

`FindContainingRoom` в StateSystem.h:2584–2632 (../StateSystem.h#L2584) преобразует мировую позицию обратной мировой матрицей комнаты в локальное пространство и сравнивает с локальными bounds. Это принципиально для повернутой или масштабированной комнаты: сравнивать мировую точку прямо с `localMin/max` нельзя. Предпочтение прежней комнаты предотвращает прыжки ID в перекрывающихся объёмах у двери. Возврат `INVALID_ENTITY` допустим; тогда верхний граф не строится и остаётся локальное прямое направление. На сложной геометрии это может довести NPC до стены, поэтому пустой `currentRoom` в диагностике — значимый симптом, а не всегда безобидная мелочь.

5.2. Дверной переход как последовательность фаз

В StateSystem.h:2272–2499 (../StateSystem.h#L2272) цикл берёт очередную дверь, её `DoorComponent` и мировую позицию. Отсутствие двери или Transform помечает маршрут грязным и возвращает `Blocked` без чтения несуществующих данных. Затем вычисляется точка выхода **внутри следующей комнаты** на `doorExitDistance`; точка подхода смещена назад от оси выхода. Эти две точки предотвращают типичную ошибку: кратчайший NavMesh путь к центру двери приводит капсулу по диагонали в косяк.

Когда дистанция до двери не больше `doorOpenDistance`, вызывается `DoorComponent::TryOpen(hasPower)`. NPC не анимирует створки; он задаёт намерение, а `DoorSystem` продвигает `openProgress`. Если открыть нельзя, `Blocked` показывает причину. Когда NPC подошёл ближе `doorWaitDistance`, а `openProgress < 0.65`, он останавливается с `WaitingForDoor`. После открытия сначала выравнивается к approach point; затем короткий участок через портал проходит напрямую к exit point, потому что NavMesh исходной комнаты не может проектировать точку в соседней. В этой фазе код временно игнорирует коллизию **конкретной открытой двери** для капсулы и восстанавливает её вне перехода. Дверной шаг считается законченным после достаточного смещения за плоскость двери или близости к exit point. Только после этого начинается движение к следующей двери или финальной цели.

Для понимания проекции у двери: `progressBeyondDoor` — скалярное произведение вектора от двери к NPC с осью выхода, делённое на длину оси. Положительное значение означает продвижение в сторону следующей комнаты. Сравнивать только `currentRoom` недостаточно, потому что bounds комнат могут пересекаться прямо в проёме. Поэтому дополнительно требуется `progressBeyondDoor >= max(0.35,0.75*doorExitDistance)` или фактическая близость к точке выхода. Это локальный пример того, зачем в игре нужна векторная математика, а не только «расстояние до цели».

5.3. `MoveTowardsNavigated`, NavMesh и движение

`MoveTowardsNavigated` в StateSystem.h:2950–3020 (../StateSystem.h#L2950) сначала запрашивает `NpcNavMeshSystem::FindNextWaypoint`. Если запрос успешен, текущей целью становится ближайший waypoint; иначе используется исходная цель. Поверх него `FindDynamicAvoidanceWaypoint` может выдать короткий объезд подвижного Bullet AABB. Для подхода к двери `requireForwardProgress` дополнительно проверяет положительное скалярное произведение направления кандидата с направлением к двери: боковой обход не должен отправить NPC назад от единственного прохода. Это не решение всей задачи планирования пути; это локальная защита конкретного портального участка.

В конфигурации проекта DX_FrameWork.vcxproj:336–338 (../DX_FrameWork.vcxproj#L336) свойство `NpcNavMeshEnabled` по умолчанию 1, а `NpcNavMeshSystem.cpp` включён в компиляцию. Заголовок NpcNavMeshSystem.h:85–180 (../src/Gameplay/AI/NpcNavMeshSystem.h#L85) имеет условные заглушки при `NPC_NAVMESH_ENABLED=0`. Тогда запросы возвращают `false`, `StateSystem` идёт напрямую, а не падает. При включённом слое NpcNavMeshSystem.cpp:1495–1541 (../src/Gameplay/AI/NpcNavMeshSystem.cpp#L1495) кеширует путь по NPC, комнате, `command.revision` и изменению цели больше 0.60. Неудачный запрос не повторяется каждый кадр до изменения ключа или явной инвалидизации. `FindReachablePoint` проектирует желаемую точку на связный остров. Значит, в отладке «NavMesh не нашёл путь» и «StateSystem не пытался идти» — разные случаи: при первом может действовать прямой fallback.

`MoveTowards` в StateSystem.h:3024–3056 (../StateSystem.h#L3024) берёт `to-from`, обнуляет Y, вычисляет длину, делит X и Z на неё и записывает единичное направление. Без нормализации NPC ускорялся бы с расстоянием до цели: вектор `(10,0,0)` сильнее `(1,0,0)` при одной и той же настройке скорости. Нулевая длина вызывает `StopMovement`, чтобы не делить на ноль. Выбор Walk/Run идёт по `allowRunning && distance >= runWhenFartherThan`; это решение по локальной текущей цели движения. Физический сдвиг происходит позднее через `CharacterControllerSystem` и Bullet, а анимация читает `locomotionState`. ApplicationClass.cpp:1537–1548 (../src/Core/ApplicationClass.cpp#L1537).

`UpdateStuckStatus` в StateSystem.h:3108–3180 (../StateSystem.h#L3108) при неподвижном намерении обнуляет таймер; при движении сравнивает новую позицию с `lastProgressPosition`. Порог в коде `0.08` единицы, несмотря на комментарий «3 см» рядом с ним; ориентируйтесь на число в условии. Если прогресса нет в течение `stuckTimeout` (в Brain по умолчанию 2 с), собирается подробный snapshot двери, позиции, фаз и направления. Не пытайтесь чинить это простым бесконечным `repath` каждый кадр: если геометрия запирает капсулу, новые одинаковые пути не сдвинут физический объект.

6. Противник: выбор, зрение, память и состояния боя

6.1. `NpcCombatSystem::Update`: выбор кандидата

Сигнатура NpcCombatSystem.h:42–47 (../NpcCombatSystem.h#L42) принимает Registry, `deltaTime`, Bullet `dynamicsWorld` и настройки отладочных лучей. Главный `ForEach<NPCCombatComponent>` означает: участие в **этом** восприятии определяется наличием боевого компонента. Внутри получаются `NPCComponent`, `NpcBrainComponent`, `TransformComponent`; без одного из них NPC пропускается. Активная ручная команда сбрасывает время стационарного прицеливания и не даёт автоматическому бою принять решение. Отсутствие `FactionComponent` сбрасывает `targetVisible` и прицеливание. NpcCombatSystem.h:57–97 (../NpcCombatSystem.h#L57).

Далее внутренний `ForEach<FactionComponent,TransformComponent>` перебирает возможные цели. Кандидат не может быть самим наблюдателем, должен быть активным, иметь `NPCComponent` или `PlayerComponent`, иметь здоровье больше нуля и быть враждебным по `AreEnemies`. Сравнивается `candidateDistanceSquared = dx²+dy²+dz²`, выбирается ближайшая по трёхмерной дистанции Entity. Квадратный корень не нужен, так как для неотрицательных чисел `a<b` тогда и только тогда, когда `sqrt(a)<sqrt(b)`. Это небольшая, но правильная экономия внутри вложенного цикла. NpcCombatSystem.h:101–169 (../NpcCombatSystem.h#L101).

Важное ограничение текущей реализации: **сначала** выбирается ближайший живой враг, и **только затем** проверяются FOV и заслонение. Если близкий враг находится за стеной, а другой дальше, но виден, второй не проверяется как запасная визуальная цель в этом проходе. Это не сознательная система приоритета угроз, а следствие порядка фильтров. Для улучшения позже выбирайте

ближайшего среди **видимых** кандидатов либо отдельно храните ближайшего и лучшего видимого; при этом не ломайте личную память цели. При полном отсутствии живых врагов `targetEntity` очищается, `targetVisible=false`, `rangedAimTime=0`.

Ещё одно точное различие guard условий: внутри `NpcCombatSystem::Update` для **наблюдателя** проверяется наличие `NPCComponent`, Brain, Transform, команды и фракции, но нет отдельного раннего `ActiveComponent` или `npcData.health>0` guard. Поздний `NpcRangedCombatSystem` и `NpcCoverSystem` такие ограничения ставят. Поэтому выключенный или мёртвый NPC теоретически может ещё обновить свои поля восприятия, хотя стрелять и выполнять движение не должен. При рефакторинге добавьте единый предикат жизнеспособного наблюдателя и проверьте обратную совместимость с отключением Entity. Это предложение; сегодняшнюю трассу выстраивайте по существующим ранним выходам.

Указатель `candidateTransform` из внутренней лямбды не сохраняется наружу. После выбора ID код заново получает `targetTransform` — правильный способ избежать ссылки на временный контекст выбора. `stanceTimeRemaining` уменьшается на `dt`. Прежде чем считать дистанцию атаки, система отдельно готовит геометрию наблюдения. NpcCombatSystem.h:171–205 (../NpcCombatSystem.h#L171).

6.2. Сектор зрения и три луча

Глаза NPC по умолчанию находятся на `position.y+1.6`. При доступной анимации и кости `mixamorig:Head` точка берётся из актуальной матрицы головы, умноженной на мировую матрицу персонажа. NpcCombatSystem.h:208–221 (../NpcCombatSystem.h#L208). Это объясняет, почему один и тот же NPC может видеть поверх низкой стены при подходящей позе, а константная точка на высоте корпуса дала бы ложное заслонение. Для цели пробуются высоты груди 1.0, головы игрока по `PlayCameraComponent::firstPersonEyeOffset.y` или 1.6 для NPC, затем нижней части корпуса 0.55.

До дорогих лучей вычисляются расстояние от глаз к груди и горизонтальный сектор. Текущий forward для yaw модели:

```
XMFLOAT3 forward = { -sin(rotation.y), 0.0f, -cos(rotation.y) };
bool inCone = dot >= cos(radians(fieldOfViewDegrees * 0.5f));
```

При `fieldOfViewDegrees=120` половина сектора 60°, `cos(60°)=0.5`. Цель точно впереди даёт dot около 1 и проходит, под углом 75° даёт примерно 0.259 и отбрасывается. Вектор к цели нормализуется **по XZ**, потому что сектор должен быть горизонтальным; высота цели сама по себе не меняет yaw обзора. Если горизонтальные координаты почти совпали, сектор считается пройденным, чтобы избежать деления на почти ноль. NpcCombatSystem.h:258–298 (../NpcCombatSystem.h#L258).

После теста диапазона и сектора цикл пробует три высоты. Для каждой требуется ненулевой `dynamicsWorld`, длина луча больше 0.001 и не больше `seeRange`. `PhysicsRaycast::CastRay` идёт от глаз к точке, исключая самого NPC; цель видна, если луч никого не задел или первым задел именно целевую Entity. При первой успешной высоте `visibleAimPoint` запоминает **эту** открытую точку, debug renderer может нарисовать луч. NpcCombatSystem.h:300–352 (../NpcCombatSystem.h#L300). Отсутствие Bullet мира здесь не означает «видно всё»: цикл продолжает без успешного результата. Это полезная безопасная семантика при загрузке/остановке мира.

6.3. Память не равна всеведению

После лучей `previouslyVisible` истинно, только если в прошлом кадре была видна **эта же** цель. Затем записывается новый `targetVisible`. Если видимость подтверждена, `combat.targetEntity=bestTargetEntity`, и `ObserveVisibleTarget(targetTransform->position,dt,previouslyVisible)` обновляет факт наблюдения. NpcCombatSystem.h:354–379 (../NpcCombatSystem.h#L354). В NPCComponents.h:535–551 (../NPCComponents.h#L535) `ObserveVisibleTarget` сначала обнуляет `lastSeenVelocity`, а при непрерывной видимости вычисляет `Δx/dt` и `Δz/dt` по двум наблюдениям; Y не предсказывается. После повторного

выхода из за стены скорость начинается заново, чтобы прыжок скрытой цели не превратился в «измеренную» скорость. Функция обнуляет таймер забывания и старый поисковый план.

При потере видимости новая текущая позиция цели не попадает в память. `EstimateSearchPosition()` в NPCComponents.h:555–570 (../NPCComponents.h#L555) берёт `lastKnownPosition + lastSeenVelocity*predictionSeconds` по горизонтали, ограничивая длину прогноза `maxPredictionDistance` (по умолчанию 3). Например, последняя лично увиденная точка `(10,0,5)`, скорость `(4,0,0)` единиц/с и горизонт 0.8 с дают сырое упреждение 3.2; ограничение до 3 даёт план `(13,0,5)`. Это гипотеза, не новое наблюдение. Пока цель скрыта, её реальный Transform не должен «таскать» план вслед за ней.

Когда цель видна, вооружённый NPC входит в `Attacking` при расстоянии не больше `min(combat.rangedAttackRange,weapon.range)`; иначе `Chasing`. Без оружия используется `attackRange` и гистерезис `ShouldHoldMeleeStance`. При входе в стойку `EnterMeleeStance` устанавливает минимальную паузу до удара, не сокращая уже более длинный cooldown. Система не сбрасывает активный путь к укрытию, если NPC сейчас исполняет неоткрытую cover фазу: `executingCoverOrder` удерживает прежнее состояние. NpcCombatSystem.h:407–435 (../NpcCombatSystem.h#L407). Это пример арбитража между сенсором и тактическим исполнителем: видимость — факт, но не любой факт немедленно отменяет уже выданную фазу движения.

Если ранее выбранная цель пропала из зрения, но Entity остаётся, NPC продолжает поиск. `timeSinceTargetSeen` растёт только в `Searching`, когда план уже готов и NPC больше не идёт к предполагаемой точке. `loseTargetDelay` по умолчанию 3 с; до забывания завершается круговой осмотр `searchTurnRemaining` и при соответствующем стиле назначенные сектора. Только затем `targetEntity` и личная память очищаются, `state=Idle`. NpcCombatSystem.h:436–467 (../NpcCombatSystem.h#L436). Не путайте этот таймер с «прошло 3 секунды с первого кадра невидимости»: на пути к последней позиции время ожидания не расходуется.

6.4. `StateSystem` в погоне, поиске и атаке

В `Chasing` StateSystem.h:642–817 (../StateSystem.h#L642) проверяется цель и память. Для маршрута используется `combat.lastKnownPosition`, не скрытый текущий Transform, хотя Transform цели первоначально читается для валидации существования. У безоружного NPC включается бег уже на коротком расстоянии; вооружённый учитывает стиль, входящий огонь и желаемую дистанцию. `Defensive` удерживает позицию, прекращает наступательные поисковые приказы и при потере контакта переходит в `Searching`, где не бежит вслед за предположением. `Assault` и `Balanced` могут исследовать продолжение наблюдаемой скорости и ограниченное число секторов. При достижении последней точки без видимой цели состояние становится `Searching`, а не автоматически `Attacking`.

В `Searching` StateSystem.h:819–915 (../StateSystem.h#L819) сначала останавливается ходьба и проверяется существование наблюдения. План строится один раз на эпизод потери цели: `searchPlanReady=true`, `BeginSearchTurn()` задаёт полный оборот `2π`, таймеры сбрасываются. `AdvanceSearchTurn(yaw,rotationSpeed,dt)` ограничивает шаг остатком угла и возвращает нормализованный угол через `atan2(sin,cos)`. При скорости 90°/с и `dt=0.1` один шаг 9°, полный поворот требует примерно 40 шагов. Если скорость настройки равна нулю, используется безопасный запас 90°/с. NPCComponents.h:511–530 (../NPCComponents.h#L511). Важна конечность этого действия: угловой остаток уменьшается, и NPC не вращается вечно, если дальше должен забыть цель.

В `Attacking` StateSystem.h:1156–1335 (../StateSystem.h#L1156) сначала проверяются живой ID, личная видимость, Transform цели и дистанция. При выходе за дальность NPC снова `Chasing`; при потере прямого зрения стоящий стрелок останавливается и тоже переходит к погоне/поиску. В состоянии атаки `StopMovement` сразу делает нижнюю анимацию Idle, корпус разворачивается к цели ограниченным

угловым шагом `rotationSpeed*dt`. Допуск 8° проверяется **после** поворота. Для вооружённого NPC этот проход только накапливает `rangedAimTime`; фактический луч из дула будет построен после обновления скелета. Для безоружного проход отсчитывает `attackTimer`, уважает паузу стойки и активный Punch клип, выбирает взвешенный вариант анимации и запрашивает `AnimationState::Punch`. Сейчас эта ветка, согласно коду и его собственному комментарию, **создаёт анимацию, но сама не наносит melee урон**. Не объявляйте рукопашный бой готовым для релиза без отдельной проверки hit window, дальности и события урона.

7. Укрытие, оружие и очередь урона

7.1. `NpcCoverSystem::Update`: тактика одного NPC

Сигнатура NpcCoverSystem.h:36–46 (../NpcCoverSystem.h#L36) требует Bullet мир и положительное конечное `dt`. `ForEach<NPCCombatComponent>` дополнительно требует `NpcBrainComponent`, `NpcTacticalComponent`, `FactionComponent`, `TransformComponent`, `NPCComponent`. Нет одного — нет решения укрытия. Система уменьшает `underFireTime`, `decisionTimer`, `phaseTimer`, таймер отклонённой точки и другие индивидуальные интервалы. `damageReactionPending` будит новое решение, но сам сигнал не сообщает точную позицию стрелявшего: для геометрии используется лично наблюдавшаяся `lastKnownPosition`. Неактивный, мёртвый, управляемый вручную NPC или персонаж без подтверждённой вражеской цели и памяти выходит до поиска позиции. NpcCoverSystem.h:90–229 (../NpcCoverSystem.h#L90).

Выбранный `reservedCover` — авторская Entity с `NpcCoverPointComponent`; динамическое укрытие Entity не имеет и отмечено `dynamicCoverActive`. При существующем укрытии `NpcCoverQuery::IsProtectedFromTarget` проверяет защиту относительно последней наблюдавшейся позиции. Один неудачный raycast увеличивает `coverValidationFailureTime`, но позиция сохраняется до `coverValidationGraceDuration`; это гасит однокадровое дрожание у края коллайдера. NpcCoverSystem.h:230–285 (../NpcCoverSystem.h#L230). Если NPC открыт в peek фазе, истёк таймер или он получил попадание, цель движения возвращается к `coverPosition`, состояние становится `TakingCover`. Попадания во время peek увеличивают `failedPeekCount`; после порога запрашивается смена укрытия, но она ждёт окончания `coverHoldTimer`. Когда скрытая фаза заканчивается, `moveTarget=peekPosition`, NPC идёт выглянуть. NpcCoverSystem.h:287–375 (../NpcCoverSystem.h#L287).

Когда укрытие нужно выбрать, `NpcTacticalDecisionSystem::EvaluateCoverRequest` учитывает стиль и угрозу. `NpcCoverQuery::FindBest` ищет авторскую или динамическую точку по текущей комнате, позиции NPC, памяти о цели и параметрам тактики. Если не найдено — ставится `noSuitableCover` и cooldown, чтобы не делать дорогой поиск каждый кадр. Если найдено — сохраняются `coverPosition`, `peekPosition`, `moveTarget`, тип укрытия, таймер удержания и `brain.state=TakingCover`; старый NavMesh путь инвалидируется. NpcCoverSystem.h:390–479 (../NpcCoverSystem.h#L390). `NpcCoverSystem` не перемещает NPC. На следующем проходе **того же кадра** `StateSystem` увидит `TakingCover` и построит движение. При прибытии к скрытой точке он ставит `InCover` и таймер ожидания, при прибытии к peek — `exposed=true`, `Attacking` и окно огня не короче `rangedAimDelay+1.2`. StateSystem.h:539–600 (../StateSystem.h#L539).

`ReleaseCurrentCover` NpcCoverSystem.h:490–506 (../NpcCoverSystem.h#L490) временно запрещает немедленное повторное избрание старой авторской точки на 4 с и очищает runtime флаги. Это лучше, чем бесконечный цикл «эта же точка снова лучшая — она же снова не защищает». Параметры `hideDuration`, `exposeDuration`, `minimumCoverHoldDuration`, `failedPeeksBeforeRelocate` из `NpcTacticalComponent` дают четыре разных рычага. Не подменяйте их одним общим «агрессивность»: один управляет ритмом выглядывания, другой устойчивостью к смене позиции.

7.2. `NpcRangedCombatSystem::Update`: серия обязательных ворот

Сигнатура NpcRangedCombatSystem.h:30–37 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L30) получает Registry, Bullet мир, `DamageSystem`, D3D устройство, `JobSystem` и debug параметры. `ForEach<NPCComponent,NPCCombatComponent>` начинает с `rangedStatus="Not shooting"`. Потом последовательно отсекаются отсутствующий Brain, мир, `Active`, `PlayerGameTag`, фракция, нулевое здоровье, активная команда, отсутствие экипированного оружия. NpcRangedCombatSystem.h:39–118 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L39). Это полезная отладочная лестница: если ранний guard вернул управление без специальной строки, новый статус останется `Not shooting`.

При `weapon.isReloading` стрельба заканчивается статусом `Reloading`. При пустом магазине запрашивается `WeaponSystem::RequestReload`; статус сообщает, началась ли перезарядка или боеприпасы исчерпаны. Затем определяется `movingFire`: требуется тактический компонент, подходящее состояние `Chasing` или `TakingCover` и личная видимость. На ходу используются собственные `movingAimTime`, размер и пауза короткой очереди. Стационарный NPC обязан быть в `Attacking`, и **в обоих режимах** нужен `combat.targetVisible` именно в текущем кадре. Пока соответствующий таймер меньше `rangedAimDelay`, выстрел не создаётся. NpcRangedCombatSystem.h:120–189 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L120).

После этого цель вновь проверяется: живая Entity, активна, не мертва, имеет фракцию и остаётся враждебной. Повторная фракционная проверка непосредственно перед огнём защищает от устаревшего `targetEntity`, если отношения изменились между выбором и выстрелом. NpcRangedCombatSystem.h:192–245 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L192). Для вычисления мирового дула нужны Transform, `CharacterMovement`, модель и анимация. Кость руки берётся по `weapon.rightHandBone`; если она отсутствует, статус `Missing right-hand bone`. `muzzlePosition` — локальная точка оружия; цепочка `GripMatrix(worldGrip) * hand * transform.GetWorldMatrix()` переводит её через хват, кость и мир. Порядок умножения связан с принятой в DirectXMath конвенцией проекта. Если заменить порядок на `world * hand * grip`, дуло окажется не там, где рука. NpcRangedCombatSystem.h:247–307 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L247).

Направление выстрела — `visibleAimPoint - origin`. `WeaponFiringRules::ValidRay` отбрасывает некорректный или нулевой вектор. Дальность сверяется с `weapon.range`, затем корпус должен смотреть на цель с ошибкой не более 8°. Ошибка yaw нормализуется через `atan2(sin(Δ),cos(Δ))`, иначе при переходе через ±π обычное вычитание выбрало бы длинный оборот. NpcRangedCombatSystem.h:309–363 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L309). Наконец короткий луч от груди к дулу проверяет, не оказался ли длинный ствол за укрытием, а отдельный луч от дула к видимой точке блокирует огонь, если первым на линии стоит живой NPC дружественной фракции. NpcRangedCombatSystem.h:365–451 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L365).

Только после всех ворот ставится `weapon.triggerHeld=true` и вызывается `WeaponSystem::TryFire`. Результат записывается как `Fired`, `Reloading`, `Weapon cooling down` либо `Weapon unavailable`. Если это успешный выстрел на ходу при свободной линии, `firedOnRoute` и счётчик очереди обновляются; по исчерпании очереди выставляется `movingBurstPauseTimer`. NpcRangedCombatSystem.h:453–500 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L453). **Подтверждённый дефект текущего кода:** `NpcRangedCombatSystem` проверяет положительный `movingBurstPauseTimer` перед следующей очередью, но ни он, ни другой проход его не уменьшает. Пока `movingFire` остаётся активным, стрельба на ходу может зависнуть после первой очереди со статусом `Moving: pause between bursts`; переход к обычной стрельбе сбрасывает таймер. NpcRangedCombatSystem.h:155–174 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L155). `triggerHeld` — признак попытки спуска в

этом шаге, а не доказательство попадания. Доказательство состоявшегося выстрела — возврат `TryFire=true`; доказательство попадания — результат физического луча внутри оружейной системы.

Короткая цитата из NpcRangedCombatSystem.h:176–181 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L176) фиксирует главный запрет:

```
if ((!movingFire && brain.state != NpcState::Attacking) || !combat.targetVisible)
{
    combat.rangedStatus = "No visible target in firing stance";
    return;
}
```

Внешние скобки важны: при разрешённом `movingFire` стационарная стойка не нужна, но `!combat.targetVisible` запрещает **оба** режима. Это место стоит перечитать перед добавлением подавляющего огня: простой новый state не превратится в возможность стрельбы по невидимой цели.

7.3. `WeaponSystem::TryFire` и `DamageSystem::Update`

`WeaponSystem::TryFire` в WeaponSystem.h:75–155 (../src/Gameplay/Combat/WeaponSystem.h#L75) — общая точка игрока и NPC. Он проверяет мир, живого и активного стрелка, корректный луч, оружие, патроны и `WeaponFiringRules::CanFire`. При пустом магазине пытается начать механическую перезарядку. Затем нормализует направление и рассчитывает множитель разброса от движения. Две случайные выборки дают направление внутри конуса с учётом `spreadDegrees` и текущей отдачи; `CommitShot` расходует патрон и запускает cooldown. После этого Bullet луч длиной `weapon.range` останавливается на **первом** объекте. Даже промах считается состоявшимся выстрелом и возвращает `true`. Если луч задел стену перед выбранным AI врагом, урон цели за стеной не переносится: `DamageEvent.target` — Entity физического первого попадания.

Две точные строки из WeaponSystem.h:121–131 (../src/Gameplay/Combat/WeaponSystem.h#L121) показывают различие между фактом выстрела и фактом попадания:

```
const bool hit = PhysicsRaycast::CastRay(origin, normalized, world,
    weapon->range, shooterEntity, shooterObject, rayHit);
```

После необязательной отладочной отрисовки стоит отдельная строка:

```
if (!hit) return true;
```

Здесь `true` у `TryFire` означает, что патрон уже израсходован и луч создан. Логический `hit=false` означает отсутствие первого пересечения, поэтому события урона нет. В этом движке «пуля» обычного оружия — **hitscan луч**, не отдельная летящая Entity с траекторией и временем полёта. `Bullet` в названиях физического мира — библиотека коллизий; она проверяет пересечение луча и управляет капсулой NPC. Это два разных смысла слова «bullet». Если позже понадобится медленный плазменный снаряд, ему потребуется собственный компонент движения, время жизни и обработка столкновения; текущий `TryFire` сам такой снаряд не создаёт.

`DamageEvent` содержит Entity оружия как `source`, Entity попадания как `target`, число урона, тип, точку, нормаль и параметры возможной пробоины. Для корпуса дополнительно вычисляются разрешение и порог резки меша. Затем событие добавляется в очередь `DamageSystem`. WeaponSystem.h:133–155 (../src/Gameplay/Combat/WeaponSystem.h#L133). `DamageSystem::Update` позже обрабатывает очередь: проверяет валидную активную цель и положительный урон; ставит `NpcTacticalComponent::underFireTime=3`, считает попадания за окно 4 с и может поставить `damageReactionPending`. Событие оружия разрешается через `WeaponOwnerComponent` во владельца, но боевой `targetEntity` жертвы не заполняется магически — направление входящего огня лишь короткая ориентировка. DamageSystem.cpp:104–189 (../src/Gameplay/Combat/DamageSystem.cpp#L104). `InvulnerableComponent` и god mode не уменьшают здоровье; обычная тактическая реакция на попадание при этом может всё же появиться, кроме специально тихого события `silentKill`.

Для NPC без общего `Damageable` здоровье уменьшается с ограничением от 0 до max; при нуле снимается `ActiveComponent`. Для устройства `Damageable.health` уменьшается, `Normalize` приводит состояние к допустимому диапазону, при переходе в уничтоженное состояние создаётся взрыв и событие уничтожения, а `Repairable.progress` сбрасывается в 0, `repaired=false`. DamageSystem.cpp:204–211 (../src/Gameplay/Combat/DamageSystem.cpp#L204), DamageSystem.cpp:331–369 (../src/Gameplay/Combat/DamageSystem.cpp#L331). Последнее связывает бой с работой экипажа: после попадания повреждённое оборудование снова становится кандидатом ремонта на следующем AI проходе.

8. Прогоним данные руками: два проверяемых сценария

8.1. Один кадр назначения и один кадр фактического ремонта

Зададим **учебную сцену** с активным мирным NPC `Entity=10` в `(0,0,0)` и активной незапечатанной пробоиной `Entity=40` в `(6,0,0)`. У NPC есть `NPCComponent{isHostile=false}`, `NpcBrainComponent` с `detectionRange=30`, `idleScore` после utility 0.10, `repairDistance=1`, `repairDuration=3`, `CrewMemberComponent{role=Engineer,onDuty=true,repairSkill=1.5}`. У пробоины есть `HullBreachComponent{leakRate=0.12,room=5}`, `RepairableComponent{progress=0,requiredProgress=1,repaired=false}`. Путь по комнатам и Bullet капсула считаются корректными. Пусть `dt=0.1` с. Это выбранные числа для расчёта, а не обещание о конкретной сохранённой сцене.

На первом проходе `NpcUtilitySystem` видит `Entity=40`, заявок другого NPC нет. `distance=6`, `distanceScore=1-6/30=0.8`, `severity=clamp(0.12/0.15,0.15,1)=0.8`, `repairScore=0.8`. Так как `0.8>0.10`, он пишет `brain.repairTarget=40`, `state=MoveToRepair`, `routeDirty=true`. Если рядом есть второй мирный NPC, на этом предварительном этапе **оба** могут посчитать одну пробоину лучшей; это пока только заявка в голове каждого.

Затем координатор заново собирает кандидатов. Для инженера базовая смесь `0.65*0.8+0.35*0.8=0.8`, роль 1.25, skill 1.5, итог `1.5`. Он создаёт `CrewTaskClaimComponent` на пробоине: `assignee=10`, `type=RepairBreach`, `priority=1.5`, время текущего мира. У NPC `assignedTask=40`. Если следующий NPC в том же проходе увидит `claimedTargets` с 40, он уже не возьмёт эту пробоину. Обратите внимание: `brain.repairScore=0.8` и `claim.priority=1.5` одновременно корректны, потому что это разные формулы и разные задачи проходов.

В том же кадре `StateSystem` до физики видит расстояние 6>1. `UpdateRepairTask` ставит `MoveToRepair`, вызывает `NavigateTo` к `(6,0,0)`. Допустим комната та же и первый waypoint совпадает с финальной точкой. `MoveTowards` пишет `walkDirection=(1,0,0)`, `currentSpeed=2` при обычной ходьбе; `Repairable.progress` всё ещё 0. Позднее `CharacterControllerSystem` передаст движение Bullet. Если нет коллизий, ожидаемый шаг порядка `2*0.1=0.2` единицы; фактическая позиция может отличаться из за физики. Нельзя требовать увеличения прогресса уже в кадре назначения при дистанции 6.

Предположим, после нескольких шагов Transform NPC на **начале** следующего AI кадра находится в `(5.1,0,0)`, то есть горизонтальная дистанция до пробоины 0.9. `Utility` сохраняет или повторно выбирает цель, `KeepExistingAssignment` сохраняет заявку без скачка к другой аварии. `StateSystem::UpdateRepairTask` теперь не вызывает `NavigateTo`, а останавливает ходьбу, ставит `Repairing`. За `dt=0.1` вычисляется `progressDelta=1*0.1*1.5/3=0.05`; `RepairSystem` записывает `progress=0.05` и возвращает `false`, потому что до `requiredProgress=1` ещё далеко. `brain.repairProgress=0.05`. Через 20 таких рабочих шагов сумма достигнет 1.0, если авария не была повторно повреждена и dt не менялся. При пробоине с async mesh восстановлением число уже может показывать 1.0, а `completed=false` продлится до готовности

операции. После фактического `DestroyEntity(40)` нельзя читать прежний `repairable*`; следующий координатор увидит отсутствие цели и не должен сохранять старую заявку.

Этот прогон проверяет два инварианта. Во первых, оценка полезности не равна физическому ремонту; между ними есть арбитраж и маршрут. Во вторых, сумма `progressDelta` зависит от **симуляционного** времени, поэтому пауза и `dt=0` не чинят корпус. Если в реальной сцене прогресс идёт у NPC за стеной, измерьте `repairPosition`, `currentRoom`, выбранный work zone point и горизонтальный `repairDistance`: формула сама по себе не проверяет физическую видимость рабочей поверхности.

8.2. Один кадр подтверждённого выстрела и следующий кадр урона

Зададим другой учебный снимок. Вооружённый активный NPC в `(0,0,0)`, yaw=0, его модель смотрит в направлении `(0,0,-1)` согласно формуле forward. Единственный живой вражеский игрок в `(0,0,-8)`, оба имеют разные ненейтральные `FactionId`. У NPC `seeRange=10`, `fieldOfViewDegrees=120`, `rangedAttackRange=12`, `rangedAimDelay=0.35`. Экипированное оружие имеет `range=20`, 5 патронов, `currentCooldown=0`, `spreadDegrees=0` и нулевую текущую отдачу; анимация содержит нужную кость руки. Предположим реальный Bullet мир содержит коллайдер игрока, но не содержит стены или союзника на луче. `dt=0.1` с; перед кадром `rangedAimTime=0.30` от уже видимой цели.

`WeaponSystem::UpdateTimers` сначала сбрасывает `triggerHeld=false` и обновляет оружейный таймер. `NpcCombatSystem` в поиске кандидатов вычисляет `distanceSquared=64`; игрок единственный вражеский персонаж, поэтому выбирается его ID. Вектор до груди примерно `(0,-0.6,-8)` от фиксированной точки глаз `(0,1.6,0)`; горизонтальное направление `(0,0,-1)`, скалярное произведение с forward равно 1, порог `cos(60°)=0.5`, сектор пройден. Луч к груди (или к следующей открытой высоте, если грудь закрыта) подтверждает видимость. Система записывает `targetVisible=true`, `visibleAimPoint` и наблюдаемую позицию `(0,0,-8)`; дальность центр–центр 8 меньше `min(12,20)=12`, значит `brain.state=Attacking`.

Пусть `NpcCoverSystem` в этом кадре не выдал приказа уйти в укрытие. `StateSystem` останавливает ходьбу, проверяет дальность и оставшуюся ошибку yaw. Поскольку корпус уже повернут на цель, ошибка 0<8°, `rangedAimTime` становится 0.40. Далее контроллер, анимация и физика обновляют позу. `NpcRangedCombatSystem` в конце кадра видит готовое оружие, `Attacking`, личную видимость и 0.40≥0.35. Из матрицы хвата, руки и мира он получает **фактический** origin дула; для примера пусть он равен `(0,1.4,-0.4)`. Точка груди игрока `(0,1.0,-8)` даёт ненормализованный луч `(0,-0.4,-7.6)` с длиной около 7.61, меньше 20. Короткий луч грудь NPC–дуло свободен, полный луч до цели не встречает союзника.

`WeaponSystem::TryFire` нормализует луч; при нулевом разбросе он остаётся направлен к выбранной точке. `CommitShot` расходует один патрон и устанавливает кулдаун согласно параметрам оружия. Bullet луч от дула первым пересекает коллайдер игрока. `TryFire` создаёт `DamageEvent{source=weaponEntity,target=playerEntity,damageAmount=weapon.damage,...}`, добавляет его в очередь и возвращает `true`; `combat.rangedStatus="Fired"`. Если бы физический луч промахнулся из за разброса или движения цели после проверки зрения, возврат всё равно был бы `true`, но `DamageEvent` бы не появился. Если бы первым попал в стену, событие адресовалось бы стене, не игроку.

В этом же кадре `DamageSystem::Update` уже был вызван **раньше**, перед поздним выстрелом. При обычном цикле новый `DamageEvent` ждёт следующего шага. Допустим `weapon.damage=20`, здоровье игрока перед очередью 100 и нет god mode. Следующий `DamageSystem::Update` уменьшит его до 80. Если целью вместо игрока был NPC, его `NpcTacticalComponent` получит `underFireTime=3` и сигнал реакции; при нуле HP обычный NPC потеряет `ActiveComponent`. Эта задержка на границе очереди важна при тестах: проверка HP сразу после `NpcRangedCombatSystem::Update` ожидает ещё старое здоровье, проверка после следующего `DamageSystem::Update` — новое.

9. Как добавлять поведение, не разрывая цепочку

9.1. Сначала определите место нового факта

Предложение: добавить задачу «проверить аварийный клапан», которая доступна мирному экипажу, требует подхода к устройству и занимает 2 секунды. Не начинайте с нового `case` в `StateSystem`, пока не ответили на четыре вопроса: **что делает цель пригодной, кто может её взять, как подтвердить достижение и что меняется в мире после выполнения**. Для этой задачи потребовался бы компонент цели, например `ValveFaultComponent{faulted,room,interactionPosition}`, авторский параметр длительности и runtime прогресс. ID исполнителя можно хранить через обобщённую заявку либо расширить нынешний `CrewTaskClaimComponent`. Пока текущий `CrewTaskCoordinator` собирает исключительно `RepairableComponent` цели через `IsRepairTargetValid`, новый компонент сам по себе не попадёт в очередь. CrewTaskCoordinator.h:225–276 (../src/Gameplay/AI/CrewTaskCoordinator.h#L225).

Для небольшого расширения **существующего** ремонта, например нового типа генератора, путь короче. Если устройство уже имеет `DamageableComponent`, `RepairableComponent`, Transform и нужный WorkZone, оно попадёт в кандидаты. Нужно решить, какую `importance` оно получит в utility, нужна ли поправка `roleAffinity` в координаторе и что `RepairSystem::CompleteRepair` должен делать с его собственным состоянием после восстановления HP. Иначе `Damageable.health` станет полным, но функциональный флаг нового устройства может остаться `broken=true` и игра увидит противоречие. Добавьте сохранение его авторских полей в `SceneManager` и редактирование в Editor, если настройка должна жить между запусками.

Для **нового вида задачи** сначала опишите состояние данных, затем интерфейс оценки, затем резервирование и лишь после этого исполнение. Оценщик не должен менять мир; координатор должен выпускать ровно одного исполнителя; исполнитель должен быть идемпотентным по отношению к повторным кадрам и безопасным к удалению цели. Если действие занимает время, храните прогресс на цели, если несколько исполнителей могут способствовать одной работе, либо на назначении, если прогресс принадлежит одному исполнителю и должен обнуляться при его уходе. Нынешний ремонт хранит прогресс на `RepairableComponent` цели, поэтому при смене ремонтника работа продолжается. Это осознанное поведение, которое стоит явно выбрать для новой системы.

9.2. Практический маршрут изменения проекта

1. **Данные.** Добавьте минимальный компонент цели и авторские параметры. Разделите сохраняемые настройки и runtime память. Пример разделения уже есть у `CrewMemberComponent`: `role/onDuty/repairSkill` сохраняются, `assignedTask` и счётчик текущего запуска не записываются. SceneManager.cpp:3552–3558 (../src/Scene/SceneManager.cpp#L3552), SceneManager.cpp:4629–4640 (../src/Scene/SceneManager.cpp#L4629).
2. **Валидация цели.** В одном месте проверьте `IsAlive`, `Active`, обязательные компоненты и незавершённость. Нельзя считать «есть Transform» эквивалентом «клапан сломан». Выберите политику при удалении цели: освободить claim, поставить `Idle`, прекратить старый путь.
3. **Оценка.** Задайте численную формулу в понятных единицах и проверьте её на ручных примерах: близкая мелкая авария, дальняя критическая, два NPC разных ролей, порог `idleScore`. Если доступность зависит от маршрута, геометрическую дистанцию можно использовать как дешёвый предварительный фильтр, а достижимость проверять до закрепления назначения.
4. **Резервирование.** Запишите, кто владеет задачей, и решите, удерживается ли назначение при колебаниях score. Для работ вроде ремонта липкость полезна; для тушения быстро растущего пожара может понадобиться прерывание по аварийному приоритету. Не переназначайте каждый кадр без cooldown или гистерезиса.
5. **Исполнитель.** Добавьте состояние и ветку в `StateSystem::UpdateNpc` или отдельную исполнительную систему в конкретной точке `ApplicationClass`. Для перемещения используйте общий `NavigateTo` и `CharacterMovement`, для изменения мира — специализированную систему действия. Границу после потенциального `DestroyEntity` обрабатывайте по ID и результату, не через сохранённый указатель.
6. **Авторинг.** Добавьте поля сохранения и загрузки в SceneManager.cpp (../src/Scene/SceneManager.cpp#L3951), UI создания и редактирования в EditorUI.cpp (../src/Editor/EditorUI.cpp#L6737), диагностику в NPC Studio или Object Debug. Не сохранённый авторский параметр будет возвращаться к значению по умолчанию при перезагрузке сцены.
7. **Проверка.** Пройдите по сценарию «один NPC–одна задача», затем «два NPC–одна задача», удаление цели в пути, ручной Stop посередине, потеря питания двери, pause и повторная загрузка. В каждой проверке фиксируйте ID, состояние, статус, маршрут и фактическое изменение мира.

Если понадобится новый **боевой** тип поведения, например точечный подавляющий огонь, отделите его решение от факта выстрела. Решение должно выставить намерение, таймер и допустимую точку, но не вычитать HP непосредственно. Общий `WeaponSystem::TryFire` уже отвечает за патрон, разброс, физический первый hit, эффекты и очередь `DamageEvent`. Новый режим обязан сохранить проверки фракции, личной видимости либо явно описанного способа подавления, блокировки дула, союзника в линии и разрешения оружия. Если он должен стрелять в точку без визуального контакта, это **новое правило дизайна**, которого текущий `NpcRangedCombatSystem` не допускает. Его нельзя получить простым изменением `brain.state=Attacking`.

9.3. Инварианты, которые стоит проверять в коде ревью

Для ремонта: у пригодной цели есть `RepairableComponent` и конкретный тип (`HullBreach` или `Damageable`); у незавершённой автоматической задачи один claim; `member.assignedTask`, `brain.repairTarget` и claim указывают на согласованные ID; ручной приказ имеет приоритет; до достижения `repairDistance` прогресс не меняется; после уничтожения пробоины нет чтения старых ссылок. Для боя: цель живой активный враг; `targetVisible` соответствует лучу текущего кадра; `visibleAimPoint` читается только при этой видимости; скрытая цель не обновляет `lastKnownPosition`; патрон расходуется только при успешном `TryFire`; попадание адресовано первому Bullet коллайдеру; урон проходит через очередь. Для

движения: Brain хранит цель и маршрут, `CharacterMovement` хранит намерение, Bullet владеет фактической позицией; при неактивности намерение обнулено.

Эти правила полезнее огромного списка «возможных тестов» без причин: каждый инвариант можно привязать к конкретной границе системы. Если он нарушен, ищите **первый проход, после которого данные стали неправильными**, а не последний визуальный симптом. NPC, который не стреляет, может быть исправен как оружие, но не иметь `targetVisible`; NPC, который не чинит, может правильно выбрать цель, но не иметь достижимой рабочей точки.

10. Диагностика по наблюдаемым данным

10.1. Ремонтник не начинает работу

В Object Debug и NPC Studio сначала найдите NPC Entity и ремонтную Entity. Для NPC проверьте `ActiveComponent`, `NPCComponent`, Brain, `isHostile=false`, `CrewMember.onDuty=true`, отсутствие активной `NpcCommand`. Для цели проверьте `Active`, `Repairable`, `!repaired` и либо `HullBreach&&!sealed`, либо `Damageable.health<maxHealth`. Эти условия прямо соответствуют `IsRepairTargetValid`, CrewTaskCoordinator.h:225–243 (../src/Gameplay/AI/CrewTaskCoordinator.h#L225). Затем сравните `brain.repairScore` с `brain.idleScore=0.10`, `CrewTaskCoordinatorSnapshot.openTasks`, `assignmentsThisFrame`, и `CrewTaskClaimComponent.assignee` на цели. Наличие score без claim означает, что локальная utility видит работу, но координатор мог назначить её другому или предпочесть другую пару NPC–задача.

Если claim есть и `brain.state=MoveToRepair`, но персонаж стоит, проверьте `brain.currentRoom`, `navigationTargetRoom`, `doorRouteIndex`, `activeDoor`, `debugDoorPhase`, `blockedReason`, `routeDirty` и `repathTimer`. `No traversable door route` означает проблему графа комнат/питания двери; `WAITING_FOR_OPEN` означает нормальное ожидание `openProgress`; `CROSSING_DIRECT` при нулевом физическом прогрессе указывает на коллизию портала или капсулы. StateSystem.h:2201–2499 (../StateSystem.h#L2201). `NpcNavMeshSystem::GetDebugInfo(npc)` сообщает `compiledIn`, `roomMeshBuilt`, `polygonCount`, путь, динамический обход и статус. NpcNavMeshSystem.h:29–60 (../src/Gameplay/AI/NpcNavMeshSystem.h#L29). При `compiledIn=false` прямой fallback ожидаем, но на сложном корабле он может вести в стену.

Если `brain.state=Repairing`, посмотрите фактическую `Repairable.progress`, `requiredProgress`, `brain.repairProgress`, `repairDuration`, `repairSkill`, `dt`. При `progress=requiredProgress` и статусе ожидания runtime операции исследуйте `HullBreachRepairLinkComponent` и завершение mesh repair. При полном HP оборудования, но `repaired=false`, проверьте, не было ли нового попадания, сбросившего прогресс; если функциональный флаг устройства остаётся сломанным после полного ремонта, расширьте `RepairSystem::CompleteRepair` именно для этого типа.

10.2. Противник не видит или не стреляет

Для восприятия последовательно проверьте `NPCCombatComponent`, `FactionComponent` на обоих участниках, здоровье и Active цели, расстояние до **ближайшего выбранного кандидата**, yaw наблюдателя, `fieldOfViewDegrees`, `seeRange`, Bullet мир и debug sight rays. Если видимый дальний враг игнорируется, посмотрите, не выбран ли более близкий враг за стеной: текущая система не ищет второго после отказа луча. `combat.targetVisible` должен быть истинным именно в нужный кадр, а `visibleAimPoint` — одна из проверенных высот цели. Если луч начинается внутри стены, проверьте кость головы и мировую матрицу модели.

Для стрельбы идите по NpcRangedCombatSystem.h:30–500 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L30) в порядке guard: Active, `PlayerGameTag` отсутствует у NPC, Brain, фракция, здоровье, ручная команда, экипированное оружие, патроны,

перезарядка, `Attacking` или разрешённый movingFire, `targetVisible`, время прицеливания, живой вражеский target, Transform и анимация, правильное имя кости руки, дальность от **дула**, ошибка yaw не более 8°, свободный короткий участок грудь–дуло, отсутствие союзника на линии. `rangedStatus` часто прямо говорит, где остановился проход: `Unarmed`, `Aiming`, `Turning toward target`, `Missing right-hand bone`, `Muzzle blocked by cover`, `Holding fire: ally in firing lane`, `Weapon cooling down`.

При статусе `Fired` и отсутствии потери HP проверьте shot ray: промах всё равно возвращает `true`. Если физический луч задел другое тело первым, `DamageEvent.target` не равен выбранной AI цели. Если событие есть, проверьте следующий шаг `DamageSystem::Update`, `InvulnerableComponent`, `PlayerComponent::godMode` и фактический тип здоровья цели. Для NPC без оружия `AnimationState::Punch` пока нельзя считать доказательством урона: этот код запускает клип, а не `DamageEvent`.

10.3. Короткая таблица симптомов

Наблюдение	Первое поле для проверки	Вероятная граница системы
`repairScore > 0`, назначения нет	`CrewTaskClaimComponent`, `assignedTask`, `onDuty`	Utility → Coordinator
Назначение есть, `progress=0`	`state`, `blockedReason`, дистанция до work zone	Coordinator → State/NavMesh
`progress=1`, пробоина открыта	статус ожидания runtime операции	State → RepairSystem/MeshCutting
NPC идёт на месте	`walkDirection`, фактический Transform, `stuckTimer`	State → Bullet controller
Враг близко, `targetVisible=false`	yaw, FOV, три sight ray, ближайший кандидат	Combat perception → Bullet raycast
`targetVisible=true`, выстрела нет	`rangedStatus`, aim time, кость, дуло, союзник	State/Animation → Ranged combat
`Fired`, HP не изменилось	weapon shot ray, `DamageEvent.target`, следующий кадр	Weapon → Damage queue

11. Что сейчас является пределом реализации

Эта глава различает подтверждённый путь и идеи развития. **Подтверждено кодом:** мирные NPC сами оценивают и получают ремонтные работы, маршрутизируются через комнаты и двери, чинят пробоины и оборудование через общий `RepairSystem`; боевые NPC выбирают враждебную цель по фракциям, проверяют личную видимость, помнят последнее наблюдение, ищут, занимают индивидуальное укрытие, могут стрелять через общий hitscan `WeaponSystem` с Bullet raycast и очередью урона. **Ограничения текущего пути:** групповой `NpcSquadCombatSystem::Update` отключён; безоружная NPC атака в `StateSystem` сейчас анимационная; выбор ближайшего врага предшествует проверке видимости; оценка ремонта использует прямое расстояние, а не фактическую стоимость маршрута; NPC вызов `ApplyRepairProgress` не передаёт `WorldEventLog` и автора действия.

Предложения к развитию, не объявленные готовым функционалом: объединить или документировать расхождение двух ремонтных scoring формул; выбирать лучшую **видимую** угрозу; добавить честное окно melee попадания и `DamageEvent`; подключить стоимость/доступность маршрута в назначение; провести источник NPC ремонта в WorldEventLog; вводить squad слой только после фиксации контрактов индивидуального боя. Каждое такое изменение должно сопровождаться конкретным сценарием, в котором проверяются входные компоненты, выходные поля, физический

результат и поведение при отмене/удалении цели. Так учебник остаётся точной картой движка, а новые идеи не маскируются под уже существующий код.

Построчный разбор систем DX Framework

Эта часть разбирает текущий снимок проекта, а не абстрактный учебный движок. Номера строк относятся к версии кода на момент сборки книги. После правок номера могут сместиться, поэтому главным ориентиром остаются имя файла, имя метода и сама строка.

1 Как читать систему до изменения кода

Читай каждый `Update` в пять проходов:

1. Какие сущности система перебирает.
2. Какие компоненты только читает, а какие изменяет.
3. Какие guard clauses досрочно прекращают обработку.
4. Где принимается решение и где оно исполняется.
5. Какие значения надо показать в инспекторе, чтобы увидеть причину решения.

Не пытайся сразу удержать весь файл в голове. Сначала проследи один конкретный путь: например, «NPC увидел цель, дошёл до края укрытия и сделал один выстрел».

2 ECSRegistry и перебор двух компонентов

Файл `src/ECS/ECSRegistry.h`, метод `ForEach<T1, T2>`.

```
template<typename T1, typename T2, typename Fn>
void ForEach(Fn&& fn)
{
    auto* pool1 = FindPool<T1>();
    auto* pool2 = FindPool<T2>();
    if (!pool1 || !pool2) return;
    for (auto& [entity, comp1] : *pool1)
    {
        if (pool2->Has(entity))
        {
            auto& comp2 = *pool2->Get(entity);
            fn(entity, comp1, comp2);
        }
    }
}
```

Строка	Что происходит	Зачем это здесь	Что сломается при ошибке
`template<typename T1, typename T2, typename Fn>`	Метод получает два типа компонентов и тип функции обратного вызова	Один код работает для любых пар компонентов	Жёстко заданные типы породят отдельный цикл на каждую систему
`void ForEach(Fn&& fn)`	Принимается callable без лишнего копирования	Лямбда может захватывать данные системы	Передача тяжёлой функции по значению создаст копии
`pool1 = FindPool<T1>()`	Ищется существующее хранилище первого типа	Перебирать можно только созданный pool	Создание pool при чтении скроет отсутствие компонентов
`pool2 = FindPool<T2>()`	Ищется хранилище второго типа	Оно будет проверяться для каждой сущности	Без него нельзя получить `comp2`
`if (!pool1 || !pool2) return`	Вся операция прекращается, если одного типа вообще нет в мире	Нет ни одного совпадения	Разыменование `nullptr` приведёт к падению

Строка	Что происходит	Зачем это здесь	Что сломается при ошибке
`for (... : *pool1)`	Перебирается только первый pool	Стоимость зависит от размера первого pool	Если первым выбран очень массовый компонент, цикл будет дороже
`pool2->Has(entity)`	Проверяется наличие второго компонента у той же сущности	Формируется пересечение двух множеств	Без проверки `Get` может вернуть `nullptr`
`comp2 = *pool2->Get(entity)`	Получается ссылка на второй компонент	Лямбда может читать и изменять его	Копия не сохранит изменения в ECS
`fn(entity, comp1, comp2)`	Управление передаётся системе	Registry не знает игровой логики	Неверный порядок аргументов даст путаницу типов

Важная деталь производительности

Этот `ForEach` всегда перебирает `pool1`. Поэтому полезно первым указывать более редкий компонент. Если `TransformComponent` есть у 10 000 сущностей, а `NPCCombatComponent` у 40, вызов `ForEach<NPCCombatComponent, TransformComponent>` выгоднее обратного порядка.

Мини задание

Найди вызов `ForEach<FactionComponent, TransformComponent>` в выборе цели. Ответь: сколько сущностей попадёт во внешний цикл и сколько проверок будет сделано, если Faction есть у 20 сущностей, а Transform у 5000.

3 TransformComponent и мировая матрица

Файл `src/ECS/Components/Components.h`, метод `GetWorldMatrix`.

```
XMMATRIX GetWorldMatrix() const
{
    XMMATRIX scaleMatrix = XMMatrixScaling(scale.x, scale.y, scale.z);
    XMMATRIX rotX = XMMatrixRotationX(rotation.x);
    XMMATRIX rotY = XMMatrixRotationY(rotation.y);
    XMMATRIX rotZ = XMMatrixRotationZ(rotation.z);
    XMMATRIX rotationMatrix = rotX * rotY * rotZ;
    XMMATRIX translationMatrix = XMMatrixTranslation(position.x, position.y, position.z);
    return scaleMatrix * rotationMatrix * translationMatrix;
}
```

Строка	Что происходит	Откуда берутся данные	Проверка
`GetWorldMatrix() const`	Строится матрица из текущего Transform	`position`, `rotation`, `scale` принадлежат компоненту	Метод не должен менять компонент
`XMMatrixScaling`	Создаётся масштаб по трём осям	`scale` по умолчанию `(1,1,1)`	Нулевой масштаб мешает обратной матрице
`XMMatrixRotationX`	Поворот вокруг X	`rotation.x` должен быть в радианах	Градусы дадут неверный огромный угол
`XMMatrixRotationY`	Поворот вокруг вертикальной оси	Для NPC это обычно yaw	Проверь знак и направление модели
`XMMatrixRotationZ`	Поворот вокруг Z	Roll или наклон	Неверная ось наклонит не туда
`rotX * rotY * rotZ`	Три поворота соединяются	Порядок является соглашением проекта	Перестановка изменит ориентацию
`XMMatrixTranslation`	Создаётся перенос в мировую позицию	`position` должна быть мировой	Локальная позиция родителя потребует иерархии

Строка	Что происходит	Откуда берутся данные	Проверка
`scale * rotation * translation`	Возвращается полный локально-мировой конвейер	DirectXMath использует выбранную проектом конвенцию	Другой порядок заставит объект вращаться вокруг иной точки

Как проверить без веры в формулу

Поставь объект в `(5, 0, 0)`, масштаб `(2, 1, 1)`, поворот Y на 90 градусов. Возьми локальную точку `(1, 0, 0)` и выведи её мировую позицию после `XMVector3TransformCoord`. Затем по одному отключай масштаб, поворот и перенос. Так видно вклад каждой матрицы.

4 NpcTacticalDecisionSystem и выбор стиля

Файл `NpcTacticalDecisionSystem.h`, метод `UpdateStyle`.

```
const float dx = threatPosition.x - npcPosition.x;
const float dz = threatPosition.z - npcPosition.z;
const float threatDistance = std::sqrt(dx * dx + dz * dz);

if (tactical.hasThreatDistanceSample && deltaTime > 0.0001f)
{
    tactical.enemyClosingSpeed =
        (tactical.lastThreatDistance - threatDistance) / deltaTime;
}
else
{
    tactical.enemyClosingSpeed = 0.0f;
    tactical.hasThreatDistanceSample = true;
}
tactical.lastThreatDistance = threatDistance;
```

Строка	Состояние до строки	Результат строки	Почему важно
`dx = threat.x - npc.x`	Есть две мировые позиции	Горизонтальная разница по X	Знак показывает сторону, квадрат убирает знак для длины
`dz = threat.z - npc.z`	То же по Z	Горизонтальная разница по Z	Y намеренно исключён из тактики дистанции
`sqrt(dx*dx + dz*dz)`	Есть катеты	Получено расстояние на плоскости	Для вывода скорости нужна настоящая длина, не квадрат
`hasThreatDistanceSample`	Первый кадр ещё не имеет истории	Определяется, можно ли сравнивать измерения	Без истории скорость сближения будет мусором
`deltaTime > 0.0001f`	Кадр мог быть паузой или нулевым шагом	Защита деления	Деление на ноль создаст бесконечность
`last - current`	Известны две дистанции	Положительный результат означает сближение	Обратный порядок поменяет смысл знака
`/ deltaTime`	Есть изменение за кадр	Получаются метры в секунду	Без деления значение зависит от FPS
`enemyClosingSpeed = 0`	Нет достоверной пары измерений	Безопасное начальное значение	NPC не принимает случайное решение в первый кадр
`hasThreatDistanceSample = true`	Первый образец принят	Следующий кадр сможет вычислить скорость	Флаг должен ставиться только после реального измерения
`lastThreatDistance = threatDistance`	Старое значение больше не нужно	Сохранена база для следующего кадра	Если записать раньше расчёта, скорость всегда будет нулём

Далее система уважает ручной выбор:

```
if (tactical.selectedStyle != NpcTacticalStyle::Auto)
{
    tactical.activeStyle = tactical.selectedStyle;
    tactical.tacticCommitTimer = 0.0f;
    tactical.tacticReason = "Стиль принудительно выбран в инспекторе";
    return;
}
```

Строка	Смысл
`selectedStyle != Auto`	Дизайнер хочет детерминированный тест, а не авто выбор
`activeStyle = selectedStyle`	Исполняемый стиль синхронизируется с настройкой
`commitTimer = 0`	Для ручного режима не нужна фиксация авто решения
`tacticReason = ...`	Инспектор объясняет источник решения
`return`	Ни одно авто правило ниже не имеет права перезаписать стиль

Затем вычисляются понятные булевы факты, а уже из них выбирается стиль. Это лучше одного гигантского `if`: каждый факт можно показать в отладчике и протестировать отдельно.

```
const bool enemyIsDangerouslyClose =
    threatDistance < (std::max)(2.5f, preferredRange * 0.35f);
const bool enemyIsCloseForCautiousNpc =
    threatDistance < preferredRange * 0.75f && aggression < 0.65f;
const bool enemyIsPushing = tactical.enemyClosingSpeed > 0.75f;
const bool cautiousUnderFire =
    tactical.underFireTime > 0.0f && aggression < 0.65f;
```

Порядок последующих `if` задаёт приоритет. Опасная близость должна проверяться раньше высокой агрессии, иначе агрессивный NPC продолжит пушить даже в неудобной почти рукопашной дистанции.

5 NpcCoverQuery и доказательство защиты

Файл `NpcCoverQuery.h`, метод `IsProtectedFromTarget`.

```
targetPosition.y += 1.0f;
coverPosition.y += (std::max)(0.2f, protectionHeight);

const XMFLOAT3 direction{
    coverPosition.x - targetPosition.x,
    coverPosition.y - targetPosition.y,
    coverPosition.z - targetPosition.z
};
const float distance = XMVectorGetX(
    XMVector3Length(XMLoadFloat3(&direction)));

if (distance <= 0.01f)
    return false;
```

Строка	Геометрический смысл	Почему не иначе
`target.y += 1.0f`	Источник поднимается примерно на уровень оружия	Луч от пола ошибочно признает низкий бордюр полной защитой
`cover.y += protectionHeight`	Проверяется конкретная высота защищаемого тела	Одна высота не доказывает защиту головы и груди одновременно
`direction = cover - target`	Луч направлен от угрозы к кандидату	Обратный вектор полетит от укрытия прочь от врага
`Length(direction)`	Получается конечная дальность луча	Бесконечный ray может подобрать геометрию за кандидатом

Строка	Геометрический смысл	Почему не иначе
`distance <= 0.01f`	Отклоняется вырожденная проверка	Нормализация и raycast нулевого отрезка ненадёжны

Продолжение:

```
const auto* targetController =
    registry.GetComponent<CharacterController>(targetEntity);
const btCollisionObject* targetBody =
    targetController ? targetController->ghostObj : nullptr;

PhysicsRaycastHit hit{};
PhysicsRaycast::CastRay(
    targetPosition,
    direction,
    world,
    distance,
    targetEntity,
    targetBody,
    hit);

return hit.collisionObject &&
    hit.collisionObject->isStaticOrKinematicObject();
```

Строка	Что происходит	Возможная ошибка
`GetComponent<CharacterController>`	Ищется физическое представление цели	Не все сущности обязаны иметь controller
тернарный оператор	Безопасно выбирается `ghostObj` или `nullptr`	Разыменование отсутствующего controller уронит игру
`PhysicsRaycastHit hit{}`	Результат обнуляется перед вызовом	Старое попадание не должно пережить новый луч
`CastRay`	Проверяется только отрезок до позиции укрытия	Большая дальность даст ложную стену за точкой
`targetEntity`, `targetBody`	Цель исключается из собственного луча	Луч иначе столкнётся со стрелком в самом начале
`hit.collisionObject`	Должно существовать препятствие	Свободная линия означает отсутствие защиты
`isStaticOrKinematicObject()`	Защитой признаётся стабильная геометрия	Живой NPC не должен считаться стеной

Ограничение одной высоты

Этот метод отвечает только на вопрос про одну выбранную высоту. Для стойки нужны минимум лучи к груди и голове. Для приседа можно проверять более низкие точки. Поэтому будущая оценка укрытия должна возвращать не просто `bool`, а профиль: `standingProtected`, `crouchProtected`, `leftPeekValid`, `rightPeekValid`.

6 StateSystem и исполнение движения в укрытие

Файл `StateSystem.h`, ветка `NpcState::TakingCover`.

```
auto* tactical = registry.GetComponent<NpcTacticalComponent>(npc);
auto* combat = registry.GetComponent<NPCCombatComponent>(npc);
if (!tactical || !combat || !combat->hasLastKnownPosition)
{
    brain.state = NpcState::Chasing;
    break;
}
```

Строка	Кто владеет данными	Решение
получение `tactical`	Тактический компонент хранит цель движения и фазу укрытия	Без него состояние не может исполниться
получение `combat`	Боевой компонент хранит контакт	Укрытие оценивается относительно угрозы
guard clause	Проверяет минимальный контракт состояния	Нельзя продолжать с неполными данными
`state = Chasing`	Выбирается восстановительное состояние	Сейчас это политика; защитнику позже может подойти `Searching` или удержание позиции
`break`	Завершается только текущая ветка `switch`	`return` мог бы прекратить обработку всей сущности или системы в зависимости от контекста

Выбор режима перемещения:

```
NPCMovementComponent tacticalMovement = npcMovement;
if (tactical->movingToPeek)
{
    tacticalMovement.speed = (std::max)(
        tacticalMovement.speed,
        tactical->peekMoveSpeed);
    tacticalMovement.allowRunning = false;
}
else
{
    tacticalMovement.allowRunning = npcMovement.allowRunning;
    tacticalMovement.runWhenFartherThan = 0.0f;
}
```

Строка	Смысл	Почему создана копия
`tacticalMovement = npcMovement`	Берутся обычные настройки NPC	Исходный компонент не меняется навсегда
`movingToPeek`	NPC идёт к точке выглядывания, а не к безопасной точке	Эти движения должны выглядеть по-разному
`max(speed, peekMoveSpeed)`	Выглядывание не станет медленнее обычной ходьбы	Резкая анимация требует короткого перемещения
`allowRunning = false`	Выглядывание не переключает locomotion в Run	Позже его заменит отдельный lean или peek clip
ветка `else`	NPC реально бежит к укрытию	Под огнём медленная прогулка неверна
`runWhenFartherThan = 0`	Бег разрешён на любой ненулевой дистанции	Обычный порог дальности здесь слишком поздний

Навигация и прибытие:

```
const NavigationResult result = NavigateTo(
    registry, npc, brain, transform, movement,
    tacticalMovement, tactical->moveTarget,
    room, npcMovement.arrivalDistance,
    NpcState::TakingCover, deltaTime);

if (result == NavigationResult::Arrived)
{
    StopMovement(movement);
    if (tactical->movingToPeek)
    {
        tactical->exposed = true;
        tactical->crouched = false;
        tactical->phaseTimer = (std::max)(
            combat->rangedAimDelay + 1.2f,
            tactical->exposeDuration);
        brain.state = NpcState::Attacking;
    }
    else
    {
        tactical->exposed = false;
        tactical->crouched = tactical->coverUsesCrouch;
        tactical->phaseTimer = (std::max)(0.1f, tactical->hideDuration);
        brain.state = NpcState::InCover;
    }
}
```

Строка	Изменение состояния	Инвариант после строки
`NavigateTo`	Исполнитель пробует приблизиться к `moveTarget`	Решение о точке уже принято другой системой
`Arrived`	Путь закончен	Нельзя оставлять старую скорость
`StopMovement`	Скорость и флаг движения сбрасываются	Анимация не продолжает бег на месте
`movingToPeek`	Различаются край укрытия и защищённая позиция	Одна координата не должна означать обе фазы
`exposed = true`	Тактика знает, что NPC доступен для огня	Получение урона в этой фазе может увеличить `failedPeekCount`
`crouched = false`	Для текущего peek используется стойка	Будущий профиль укрытия сможет выбрать crouch-peek
`phaseTimer = max(...)`	Окно включает прицеливание и короткую очередь	Слишком короткий таймер вернёт NPC до выстрела
`state = Attacking`	Управление передаётся боевой ветке	Сам `TakingCover` не стреляет
защищённая ветка	NPC скрыт и ждёт	`phaseTimer` становится временем ожидания до peek
`state = InCover`	Управление передаётся фазе укрытия	Следующий переход должен быть явным

Если `NavigateTo` возвращает `Blocked`, резервирование сбрасывается, таймер решения обнуляется и поиск должен получить шанс выбрать другую точку. Это пример восстановления после ошибки исполнения, а не нового тактического решения.

7 NpcRangedCombatSystem и точка дула

Файл `src/Gameplay/Combat/NpcRangedCombatSystem.h`.

```
XMMATRIX hand;
if (!mesh->model->TryGetBoneModelTransform(
        *animation,
        weapon->rightHandBone,
        hand))
{
    combat.rangedStatus = "Missing right-hand bone";
    return;
}
```

Строка	Что проверяется	Почему выстрел запрещается
`XMMATRIX hand`	Подготавливается место для матрицы кости	Дуло привязано к текущей анимационной позе
`TryGetBoneModelTransform`	Модель ищет кость, заданную оружием	Имена костей отличаются между моделями
`!result`	Кость не найдена	Вычислять мировое дуло не из чего
`rangedStatus`	Причина сохраняется для инспектора	Иначе NPC молча перестанет стрелять
`return`	Текущая попытка выстрела прекращается	Подстановка нулевой матрицы даст луч из неверного места

Мировая точка дула:

```
XMStoreFloat3(
    &origin,
    XMVector3TransformCoord(
        XMLoadFloat3(&weapon->muzzlePosition),
        WeaponIKSystem::GripMatrix(weapon->worldGrip) *
            hand *
            transform->GetWorldMatrix()));
```

Читаем изнутри наружу:

1. `muzzlePosition` — точка в локальном пространстве оружия.
2. `GripMatrix` учитывает хват и ориентацию оружия.
3. `hand` переносит оружие в текущую позу кости.
4. `GetWorldMatrix` переносит персонажа в мир уровня.
5. `TransformCoord` применяет полный конвейер к точке.
6. `XMStoreFloat3` сохраняет SIMD-вектор в обычный `XMFLOAT3`.

Направление строится только к точке, подтверждённой личным зрением:

```
XMFLOAT3 aimPoint = combat.visibleAimPoint;
XMFLOAT3 direction{
    aimPoint.x - origin.x,
    aimPoint.y - origin.y,
    aimPoint.z - origin.z
};
```

`targetEntity` может оставаться активным после ухода игрока за стену. Поэтому он не является разрешением стрелять. `visibleAimPoint` должен обновляться только при текущем подтверждённом контакте.

8 Проверка chest to muzzle

```
XMFLOAT3 chest = transform->position;
chest.y += 1.45f;

const XMFLOAT3 toMuzzle{
    origin.x - chest.x,
    origin.y - chest.y,
    origin.z - chest.z
};

const float muzzleDistance = XMVectorGetX(
    XMVector3Length(XMLoadFloat3(&toMuzzle)));
```

Строка	Геометрия	Ошибка, которую ловит
`chest = position`	Берётся мировая база персонажа	Нужна стабильная точка внутри тела
`chest.y += 1.45`	Точка поднимается к груди	Луч от ног пересёк бы пол или низ стены
`origin - chest`	Направление идёт к дулу	Проверяем, не высунулся ли ствол сквозь геометрию
`Length`	Получается длина короткого отрезка	Raycast не должен идти дальше дула

После raycast любое препятствие на этом отрезке ставит статус `Muzzle blocked by cover` и запрещает выстрел. Эта проверка не заменяет линию огня: она отвечает только за участок внутри позы NPC и укрытия.

9 Проверка линии огня и союзника

```
PhysicsRaycastHit fireLane{};
PhysicsRaycast::CastRay(
    origin,
    direction,
    world,
    distance,
    npc,
    ownBody,
    fireLane);
```

Аргумент	Точное назначение
`origin`	мировое дуло
`direction`	ненормализованный вектор к видимой точке; обёртка raycast знает длину отдельно
`world`	физический мир
`distance`	конец проверки находится на цели, а не за ней
`npc`	сущность стрелка исключается
`ownBody`	физический ghost стрелка исключается
`fireLane`	первое попадание возвращается сюда

```
const auto* blockingNpc =
    registry.GetComponent<NPCComponent>(fireLane.entity);
const auto* allyFaction =
    registry.GetComponent<FactionComponent>(fireLane.entity);

const bool friendlyInFireLane =
    blockingNpc &&
    blockingNpc->health > 0.0f &&
    allyFaction &&
    GetFactionRelation(
        observerFaction->faction,
        allyFaction->faction) == FactionRelation::Friendly;
```

Булево выражение читается слева направо благодаря short circuit:

1. Первый объект луча должен быть NPC.
2. Он должен быть жив.
3. У него должна быть фракция.
4. Отношение фракций должно быть Friendly.

Только после успешной проверки указателя читаются его поля. Переставлять `blockingNpc->health` перед `blockingNpc` нельзя.

10 WeaponSystem и одна попытка выстрела

Файл `src/Gameplay/Combat/WeaponSystem.h`, метод `TryFire`.

```
if (!world || !registry.IsAlive(shooterEntity) ||
    !registry.HasComponent<ActiveComponent>(shooterEntity) ||
    !WeaponFiringRules::ValidRay(origin, direction))
    return false;
```

Проверка	Что гарантирует следующему коду
`world`	физический raycast можно вызвать
`IsAlive`	entity не уничтожен
`ActiveComponent`	выключенная сущность не стреляет
`ValidRay`	координаты конечны и направление ненулевое

Поиск оружия и перезарядка:

```
const Entity weaponEntity =
    GetEquippedWeaponEntity(registry, shooterEntity);
auto* weapon = registry.GetComponent<WeaponComponent>(weaponEntity);
if (!weapon)
    return false;

if (weapon->ammoInMagazine <= 0)
{
    WeaponFiringRules::StartReload(*weapon);
    return false;
}

if (!WeaponFiringRules::CanFire(*weapon))
    return false;
```

Строка	Состояние оружия	Результат
`GetEquippedWeaponEntity`	Стрелок может иметь инвентарь и несколько предметов	Находится именно экипированное оружие
`GetComponent<WeaponComponent>`	Entity мог быть неверным или предмет не оружие	Получается механическое состояние

Строка	Состояние оружия	Результат
`ammo <= 0`	Магазин пуст	Запускается таймер перезарядки
`return false`	В этот кадр пуля не создаётся	Перезарядка не считается выстрелом
`CanFire`	Проверяет кулдаун, reload и другие правила	Все стрелки используют один контракт

Разброс и фиксация выстрела:

```
XMStoreFloat3(&normalized,
    XMVector3Normalize(XMLoadFloat3(&direction)));

const auto* movement =
    registry.GetComponent<CharacterMovement>(shooterEntity);
const float motionScale = WeaponFiringRules::MotionMultiplier(
    *weapon,
    movement && movement->isMoving);

const float radial = sample(random);
const float angular = sample(random);
const XMFLOAT3 muzzleDirection = normalized;
normalized = WeaponFiringRules::ConeDirection(
    normalized,
    weapon->spreadDegrees * motionScale +
        weapon->currentRecoilDegrees,
    radial,
    angular);

WeaponFiringRules::CommitShot(*weapon, motionScale);
```

Строка	Почему расположена именно здесь
нормализация	Дальность и конус ожидают единичное направление
чтение движения	Разброс зависит от состояния стрелка
`movement && ...`	Отсутствующий компонент безопасно трактуется как неподвижность
две случайные выборки	Одна задаёт радиус, другая угол внутри конуса
сохранение `muzzleDirection`	Muzzle flash должен смотреть по стволу, а не по случайному отклонению пули
`ConeDirection`	Получается фактический луч этого выстрела
`CommitShot`	Списывается патрон, ставится кулдаун и отдача

Сначала фиксируется выстрел, затем делается raycast. Поэтому промах всё равно тратит патрон и создаёт кулдаун — это правильная механика.

Попадание превращается в событие:

```
PhysicsRaycastHit rayHit{};
const bool hit = PhysicsRaycast::CastRay(
    origin,
    normalized,
    world,
    weapon->range,
    shooterEntity,
    shooterObject,
    rayHit);

if (!hit)
    return true;

DamageEvent damageEvent{};
damageEvent.source = weaponEntity;
damageEvent.target = rayHit.entity;
damageEvent.damageAmount = weapon->damage;
damageSystem.AddDamageEvent(damageEvent);
return true;
```

`true` означает «выстрел состоялся», а не «цель получила урон». Это различие нужно для анимации, звука, расхода патронов и тактического поведения.

11 Один кадр боя от восприятия до урона

Полезно читать не отдельные классы, а движение данных:

1. Система восприятия подтверждает личную видимость и обновляет `visibleAimPoint`.
2. Боевая система хранит `targetEntity`, но не считает его вечной видимостью.
3. Тактическое решение выбирает стиль и запрос укрытия.
4. Cover query оценивает кандидатов лучами относительно подтверждённой угрозы.
5. `StateSystem` исполняет `moveTarget` через навигацию.
6. В состоянии атаки `NpcRangedCombatSystem` вычисляет мировое дуло.
7. Проверяются поворот, chest-to-muzzle и дружественная линия огня.
8. `WeaponSystem::TryFire` проверяет оружие, создаёт разброс и физический луч.
9. Первое попадание превращается в `DamageEvent`.
10. `DamageSystem` применяет урон и записывает факт угрозы для следующего тактического решения.

Если NPC не стреляет, проверяй цепочку сверху вниз. Не начинай сразу с `TryFire`: до него мог не дойти вызов.

12 Метод локализации бага

Для каждого перехода храни три строки отладки:

- `decision`: чего NPC хотел;
- `reason`: почему он этого захотел;
- `execution`: что исполнитель реально сделал.

Пример:

```
decision: перейти к укрытию 41
reason: два попадания во время выглядывания
execution: путь не найден, cover 41 отклонено на 4 секунды
```

Это различает ошибку решения и ошибку исполнения. Если решение правильное, но NPC стоит, смотри NavMesh и состояние движения. Если путь есть, но выбрана плохая сторона стены, смотри геометрию cover query и score.

13 Практика по чтению кода

Задание 1 ECS

Нарисуй два множества сущностей: с `NPCCombatComponent` и с `TransformComponent`. Руками отметь пересечение, которое получит лямбда.

Задание 2 матрица дула

В отладке покажи четыре точки разными цветами: локальное дуло после grip, после hand, после world и окончательный `origin`. Если одна стадия прыгает, ты сразу узнаешь виноватую матрицу.

Задание 3 укрытие

Для одного кандидата нарисуй лучи к ногам, груди и голове. В журнале напиши, какой луч встретил статическую геометрию первым.

Задание 4 выстрел

Поставь союзника между двумя врагами. Запиши по кадрам `fireLane.entity`, отношение фракций и `rangedStatus`. Затем убери союзника и сравни цепочку.

Задание 5 объяснение

Не глядя в книгу объясни вслух, почему активный `targetEntity` не даёт права стрелять. Правильный ответ должен упомянуть текущую личную видимость, укрытие и первое физическое попадание луча.

14 Контрольный список

После разбора ты должен уметь:

- отличить выбор сущностей ECS от игровой логики лямбды;
- назвать пространство каждой точки в расчёте дула;
- объяснить, почему `lastKnownPosition` не является всезнанием;
- разделить решение об укрытии, поиск геометрии и исполнение движения;
- прочитать guard clause как контракт следующего участка;
- отличить «выстрел состоялся» от «луч попал»;
- найти, какая система владеет изменяемым полем;
- построить цепочку данных одного кадра;
- добавить отладку причины, а не только результата.

Построчный атлас систем DX Framework

Эта часть дополняет подробные главы короткими рабочими маршрутами остальных подсистем. Для каждого маршрута показано, что является входом, где принимается решение, куда записывается результат и каким наблюдением его проверять. Номера строк относятся к снимку репозитория от 1 ноября 2026 года и нужны как ориентир, а не как вечный API.

1 ApplicationClass и порядок кадра

`ApplicationClass::Frame` является оркестратором. Он не должен содержать внутреннюю математику каждой механики, но обязан задавать порядок стадий.

```
m_inputManager.Update(m_input);
m_events.Clear();
m_jobSystem.ExecuteMainThreadJobs();
m_assetLoadSystem.Update(m_direct3D->GetDevice());
m_sceneManager.UpdateAsyncShipImport(
    m_direct3D->GetDevice(),
    m_direct3D->GetDeviceContext(),
    4.0f);
```

Строка	Что происходит	Почему здесь
обновление input	сырые состояния превращаются во фронты кадра	все потребители должны видеть один snapshot
очистка events	удаляются события прошлого кадра	одноразовое событие не должно повторяться
main thread jobs	применяются подготовленные worker результаты	D3D и мир меняются на разрешённом потоке
asset update	state machine загрузки движется дальше	GPU upload использует device
async ship import	импорт получает ограниченный бюджет	большой ассет не должен заморозить кадр

При добавлении системы сначала выпиши её зависимости. Если она читает результат физики, она не может стоять до fixed step. Если она создаёт движение, оно должно появиться до физического исполнения.

2 Режим Editor и Play

Переключение F5 является фронтом нажатия: `f5Down && !m_wasF5Down`. Без проверки прошлого кадра переход выполнялся бы каждый кадр удержания клавиши.

При выходе из Play код обнуляет `CharacterMovement` и команды Bullet controller. Это не косметика: controller хранит последнее направление и продолжил бы применять его в Editor.

Инвариант режима: Editor может редактировать authoring Transform, Play исполняет runtime логику, а переход явно очищает одноразовые намерения.

3 Время симуляции и время интерфейса

`DebugHud` получает обычный кадровый `dt`, поэтому сообщения исчезают даже на паузе. Игровые системы получают `simulationDeltaTime`, который учитывает pause, step и time scale NPC Studio.

Время	Для чего	Нельзя использовать для
real frame dt	UI загрузка камеры редактора	баланс симуляции на паузе
simulation dt	gameplay AI timers animation	сообщение которое должно жить при pause
fixed physics dt	Bullet integration	UI transition
total world time	журнал последовательность событий	непосредственное движение без delta

4 ECSRegistry построчно

```
Entity CreateEntity()
{
    if (nextEntity == INVALID_ENTITY)
        throw std::overflow_error("ECS entity id space exhausted");
    const Entity entity = nextEntity++;
    m_liveEntities.insert(entity);
    return entity;
}
```

Строка	Смысл
проверка sentinel	запрещает выдать ID зарезервированный как invalid
post increment	возвращает текущий ID и готовит следующий
live set	делает `IsAlive` независимым от наличия компонентов
return	Entity остаётся числовым ключом без поведения

`DestroyEntity` удаляет ID из live set и просит каждый component pool удалить запись. Поэтому внешние системы не должны хранить голый Entity бесконечно без `IsAlive`.

5 ForEach двух компонентов

```
auto* pool1 = FindPool<T1>();
auto* pool2 = FindPool<T2>();
if (!pool1 || !pool2) return;
for (auto& [entity, comp1] : *pool1)
{
    if (pool2->Has(entity))
        fn(entity, comp1, *pool2->Get(entity));
}
```

Перебор идёт по первому pool. Для производительности первым полезно ставить более редкий компонент. Удалять текущий компонент из того же pool внутри callback опасно, если контейнер инвалидирует iterator. Безопасный шаблон: собрать Entity в `toRemove`, удалить после прохода.

6 ComponentManager и владение

Каждый тип компонента имеет отдельный pool. `std::type_index` выбирает pool во время выполнения, а шаблон возвращает правильный статический тип. Registry владеет pools через

`unique_ptr<IComponentPool>`. Система получает ссылку или указатель на компонент, но не становится его владельцем.

Проверка: после `DestroyEntity` `GetComponent<T>` должен вернуть `nullptr` для каждого существовавшего T.

7 TransformSystem

Transform связывает ECS с объектами сцены. Ключевой вопрос каждой строки: локальное это значение или мировое. Позиция, rotation и scale нельзя смешивать с уже собранной world matrix без явного пространства.

Рабочий порядок матрицы проекта нужно проверять по DirectX соглашению и текущему shader contract. Не запоминай `S * R * T` отдельно от того, используются row vectors или column vectors. Проверка проще: возьми точку `(1,0,0)`, применяй операции вручную и сравни с результатом кода.

8 SceneManager создание объекта

Путь spawn состоит из нескольких независимых обязательств.

```
SpawnObjectDesc
    -> CreateEntity
    -> TransformComponent
    -> MeshRendererComponent
    -> optional gameplay components
    -> optional RigidBodyComponent
    -> PhysicsSystem CreateRigidBody
    -> editor name and hierarchy
```

Если видимый объект не имеет физики, проверяй не Renderer, а факт создания `RigidBodyComponent` и вызова `CreateRigidBody`. Если объект существует в Bullet, но не виден, проверяй mesh, material, ActiveComponent и render candidates.

9 Сохранение сцены

`SaveScene` должен сериализовать authoring state, а не адреса runtime ресурсов. Указатели на `btRigidBody`, D3D buffer, model instance и controller не имеют смысла в следующем запуске.

Сохранять	Пересоздавать после load
position rotation scale	world matrix cache
mass shape type размеры	`btCollisionShape` и `btRigidBody`
weapon параметры и ammo если это save game	ray callback и debug traces
faction тактический профиль	текущий pointer на target component
ссылки как Entity с remap	старые числовые ID без remap

10 Загрузка и Entity remap

Старый Entity ID из JSON нельзя считать новым ID registry. Загрузка сначала создаёт новые сущности и таблицу `old -> new`, затем восстанавливает компоненты и только после этого remap ссылок между сущностями.

Пример: дверь хранит две комнаты. Пока комнаты ещё не созданы, нельзя окончательно записать runtime ссылку. Поэтому восстановление ссылок является отдельной фазой.

11 JobSystem

Фоновая задача должна возвращать данные, которые безопасно передать на главный поток.

```
worker queue
    CPU parse mesh computation path preparation
        -> result object
main thread queue
    D3D resource creation ECS mutation Bullet registration
```

Захват ссылки на временный объект в worker lambda создаёт use after free. Захват `this` требует гарантировать жизнь владельца до завершения job или использовать отмену и weak ownership.

12 AssetLoadSystem

Загрузка ассета является state machine, а не одним bool.

1. Запрос создан.
2. Файл читается и разбирается на worker.
3. CPU результат готов.
4. GPU upload выполняется на main thread.
5. Готовый resource публикуется потребителям.
6. Ошибка сохраняется как отдельное конечное состояние.

Потребитель не должен использовать model только потому, что файл уже прочитан. Готовность означает завершение всех стадий, необходимых этому потребителю.

13 InputClass и InputManager

InputClass читает состояние устройства. InputManager сравнивает текущий и прошлый кадр.

```
down     = current
pressed  = current and not previous
released = not current and previous
```

Стрельба automatic weapon может использовать `down`. Переключение режима, перезарядка по кнопке и открытие окна используют `pressed`. Если перепутать, команда повторяется каждый кадр удержания.

14 GameplayInputSystem

Этот слой превращает клавиши в именованные `GameAction`. В результате gameplay не обязан знать VK коды. Контекст `isPlayMode`, UI capture и observer camera решает, разрешено ли действие до того, как оно попадёт в движение или оружие.

Инвариант: ImGui, камера и gameplay не должны одновременно считать мышь своей в одном кадре.

15 CharacterMovementSystem

Система движения читает intent, направление камеры и настройки скорости, затем пишет `CharacterMovement`. Она не должна напрямую наносить урон или выбирать NPC target.

Проверяемые величины:

- ненормализованное input направление;
- нормализованное горизонтальное направление;
- walk или run speed;
- итоговая скорость в единицах мира в секунду;
- displacement или walk direction для controller;
- флаг grounded и запрос jump.

16 CharacterControllerSystem

Controller исполняет движение объёмной капсулой. Его ключевые этапы: reconfigure при изменении размеров, ground probe, step logic, push dynamic body, установка walk direction и запись диагностических флагов.

Если персонаж не движется, сначала проверь intent в `CharacterMovement`, затем controller pointer, затем collision sweep. Если intent уже нулевой, Bullet не является причиной.

17 CharacterFacingSystem

Поворот visual model отделён от вращения Y up capsule. Капсула симметрична вокруг вертикальной оси, поэтому бессмысленно warp физический controller каждый кадр только из-за направления взгляда. Это уменьшает дрожание и сохраняет одного владельца физической позиции.

18 FirstPersonViewSystem

Первое лицо связывает камеру, visual arms и оружие. Оно является presentation слоем: не должно иметь отдельный magazine или собственное решение о попадании. Entity оружия и `WeaponComponent` остаются общим gameplay источником истины.

19 AnimationSystem

Упрощённый рабочий путь анимации:

```
CharacterMovement and action state
    -> animation controller selects clip and pose variant
    -> AnimationSystem advances time
    -> model computes bone matrices
    -> WeaponIK corrects upper body or hands
    -> renderer uploads matrices
```

`deltaTime` продвигает время клипа. Скорость клипа и скорость locomotion должны связываться явным коэффициентом, иначе ноги скользят. IK применяется после базовой позы, потому что он корректирует уже вычисленную цепь костей.

20 TwoBoneIK

Двухкостная IK решает треугольник из корня, сустава и effector. Ей нужны длины двух костей, target и pole direction. Target за пределами суммы длин должен быть ограничен, иначе acos получает значение вне диапазона из-за геометрии или float погрешности.

Проверка: рисуй root, joint, target, pole и окружности достижимости. Без визуализации неверное пространство target выглядит как плохая анимация.

21 RenderCandidateSystem

Система кандидатов отделяет вопрос можно ли рисовать объект от самого draw call. Она фильтрует ActiveComponent, mesh, visibility, frustum и другие условия, а Renderer потребляет подготовленный список.

Если объект не виден, проверяй по порядку: Entity active, mesh/model ready, transform finite, candidate создан, frustum test, material/shader, draw call.

22 Renderer

`Renderer::RenderFrame` организует passes. Обычный путь объекта:

```
Transform
    -> world matrix
camera -> view matrix
D3D -> projection matrix
material textures lights
    -> constant buffers and shader resources
    -> draw indexed
```

Renderer не должен менять здоровье, комнату NPC или ammo. Его вход является снимком состояния, а результатом изображение и presentation statistics.

23 BaseShader и constant buffers

CPU структура и HLSL `cbuffer` обязаны совпадать по порядку полей, размеру и выравниванию 16 байт. `float3` не гарантирует, что следующее поле можно бездумно уложить в оставшиеся четыре байта во всех структурах C++.

При изменении shader contract меняй четыре места: HLSL объявление, C++ buffer struct, создание buffer size и заполнение перед draw. Debug layer DirectX помогает найти несогласованное bind state, но не понимает смысл перепутанных полей.

24 Camera и picking ray

Клик мыши проходит путь screen -> normalized device coordinates -> view space -> world space. Для направления используется inverse view без переноса. После преобразования direction нормализуется.

Проверка: центральный пиксель должен давать направление близкое `camera.GetForward`. Луч из левого края должен отклоняться влево в соответствии с FOV и aspect ratio.

25 Material и texture

Material собирает параметры поверхности, texture является GPU ресурсом. Один material может ссылаться на несколько maps: base color, normal, metallic roughness и другие. Gameplay не должен проверять имя texture, чтобы решить физический материал; для этого нужен отдельный gameplay surface tag.

26 Frustum culling

Frustum состоит из плоскостей камеры. Bounding sphere даёт быстрый грубый тест, AABB точнее для некоторых форм. Culling не удаляет Entity и не отключает симуляцию мира. Он только исключает draw candidate текущей камеры.

27 InteractionSystem

Interaction использует camera или weapon independent ray, находит ближайший допустимый объект, проверяет дистанцию и компонент взаимодействия, затем публикует действие или меняет UI состояния.

Разделяй три факта: объект под прицелом, объект доступен по правилам и кнопка нажата в этом кадре. Только их сочетание выполняет взаимодействие.

28 TriggerZoneSystem

Trigger zone хранит множество сущностей прошлого кадра и сравнивает его с текущим пересечением.

```
current minus previous -> Enter
current intersect previous -> Stay
previous minus current -> Exit
```

Без памяти previous любое пребывание выглядело бы как новый Enter каждый кадр. После удаления Entity устаревший ID надо очищать из множества.

29 EventQueue

Producer вызывает `Emit`, consumers читают события в определённой стадии, а начало следующего кадра очищает очередь. Событие описывает уже случившийся факт. Команда описывает желаемое действие. Смешение этих понятий создаёт циклы, когда система повторно считает собственный запрос фактом.

30 InventorySystem

Inventory хранит Entity предметов, а предмет хранит собственные компоненты. Экипировка не должна копировать `WeaponComponent` в player. Иначе ammo расходуется в одной копии, а UI показывает другую.

Инварианты:

- один предмет не находится одновременно в двух inventories;
- выбранный slot либо пуст, либо ссылается на живую Entity;
- удаление предмета очищает ссылку слота;
- view model не является вторым gameplay оружием.

31 WeaponPickupSystem

Pickup проверяет дистанцию или ray hit, свободный slot, валидность weapon Entity и фронт действия. После успеха меняется ownership и presentation, а не создаётся дубликат оружия без причины.

32 WeaponSystem

`UpdateTimers` один раз за кадр продвигает кулдаун и reload всех оружий. `TryFire` выполняет одну механическую попытку. Адаптер player или NPC решает, когда вызывать попытку.

Это разделение позволяет стрелять в движении, менять тактику во время reload и тестировать оружие без камеры.

33 DamageSystem

DamageEvent является границей между подтверждённым попаданием и изменением gameplay состояния. DamageSystem проверяет target, тип получателя, armour или hull правила, меняет health, создаёт последствия и отладочную запись.

Очередь защищает от изменения component pools прямо внутри raycast обхода и даёт одну точку для god mode, бессмертия, hit reaction и статистики.

34 RepairSystem

Ремонт соединяет input или NPC command, дистанцию, runtime mesh damage, progress и фоновую перестройку геометрии. Сама длительная CPU операция готовится worker, а mutation model, ECS и D3D выполняется на главном потоке.

Нельзя считать ремонт завершённым только по таймеру. Завершение означает, что gameplay damage закрыт, визуальная геометрия обновлена или запланирована, физический shape согласован и reservation освобождён.

35 ParticleSystem

Particle проходит состояния alive time, position, velocity, size, color и lifetime. `Update` уменьшает lifetime и интегрирует движение, renderer собирает только живые particles.

Object pool снижает число allocations, но требует полного reset переиспользуемого particle. Забытое поле из прошлой жизни создаёт редкие визуальные артефакты.

36 ExplosionSystem

Explosion сначала создаёт visual effect, затем перебирает dynamic bodies, считает falloff и применяет impulse. Визуальная частица не должна быть единственным источником урона: gameplay radius и физический impulse должны быть явными параметрами события взрыва.

37 PowerSystem построчно

```
float availableMw = 0.0f;
registry.ForEach<GeneratorComponent>([&](Entity entity, GeneratorComponent& generator)
{
    if (!registry.HasComponent<ActiveComponent>(entity) ||
        !generator.isOnline)
        return;
    availableMw += (std::max)(0.0f, generator.currentOutputMw);
});
```

Строка	Смысл
`availableMw`	бюджет мощности текущего кадра
обход generators	источники независимы от игрока
active и online	объект существует но может не участвовать
clamp снизу	отрицательная генерация не превращается в скрытого consumer
сумма	общий бюджет перед распределением

Затем consumers обнуляются, группируются по priority и получают справедливые доли внутри одного приоритета. `powerRatio` вычисляется только после распределения.

38 SupplyEvenly

Алгоритм water filling делит остаток поровну, насыщенные consumers удаляет и повторяет деление между оставшимися. Это важно: простой один проход оставил бы лишнюю мощность у маленького consumer и не перераспределил её большому.

39 GeneratorSystem и ShipPowerGridSystem

Generator определяет доступную выработку и расход ресурса. PowerSystem распределяет мгновенную мощность. Grid и battery добавляют хранение энергии. Не смешивай MW и MWh: мощность умноженная на часы даёт энергию.

40 RoomSystem

RoomSystem определяет membership объектов в комнатах. Атмосфера, HUD и decompression используют этот результат. Поэтому membership должен обновиться до систем, которые читают `objectsInRoom`.

Для объекта на границе комнаты нужна стабильная политика: приоритет текущей комнаты, epsilon или явный portal transition. Иначе он прыгает между двумя средами каждый кадр.

41 RoomEnvironmentSystem построчно

```
if (deltaTime <= 0.0f)
    return;
deltaTime = (std::min)(deltaTime, 0.10f);
auto& rooms = registry.GetComponents<RoomComponent>();
CharacterExternalVelocitySystem::BeginAtmosphereFrame();
```

Guard clause останавливает симуляцию на pause. Clamp не даёт одному длинному кадру мгновенно опустошить комнату. Pool rooms является рабочим набором. `BeginAtmosphereFrame` очищает накопленные внешние скорости до новых источников suction.

Сначала комнаты расходуют кислород, затем `ApplyBreaches` меняет давление и температуру, после этого `EqualizeConnectedRooms` переносит величины между комнатами.

42 Двухфазный перенос между комнатами

Проект сначала вычисляет массив `RoomTransfer`, потом применяет его. Если менять комнаты прямо во время первого обхода, результат зависит от порядка Entity: третья комната увидит уже изменённое значение второй.

```
phase 1 snapshot differences -> transfers
phase 2 transfers -> all room values
```

Это общий алгоритм для тепла, жидкости, газа, заражения и электрических сетей.

43 DoorSystem

DoorSystem читает command или interaction state, power, lock и progress. Она изменяет openness и collision representation, затем публикует событие изменения. Атмосфера читает door flow multiplier, а NavMesh решает проход.

Одна дверь поэтому имеет несколько представлений: visual leaves, gameplay state, physics blocker, room connection и navigation portal. Их нужно обновлять из одного состояния, а не хранить пять независимых bool.

44 FloodingSystem и PressureSystem

Flooding обновляет объём воды и уровень в compartment. Pressure вычисляет воздействие глубины и целостности корпуса. Физическая сила плавучести использует результат позже.

Проверяй размерности: объём в кубических метрах, уровень в метрах, плотность в килограммах на кубический метр, давление в паскалях или нормализованной игровой величине. Нормализованное поле должно называться так, чтобы его не приняли за SI значение.

45 ShipAlertSystem

Alert system агрегирует факты: пожар, низкий кислород, пробоина, недостаток энергии. Она выбирает наиболее серьёзный уровень, но не должна сама создавать пробоину или отключать генератор. Это projection состояния мира для UI и поведения экипажа.

46 RoomCullingSystem

Room culling использует текущую комнату камеры и связи portal. Это presentation optimization. Невидимая комната продолжает симулировать атмосферу, питание и NPC, если отдельная simulation LOD политика не говорит иначе.

47 EditorUI

EditorUI читает selected Entity, показывает компоненты и пишет authoring поля. Добавление компонента требует три независимых действия: разрешить пункт для подходящей Entity, вызвать `AddComponent` один раз, затем рисовать controls вне `Selectable` блока каждый кадр.

Combo нельзя размещать только внутри события выбора пункта меню: popup закроется, а control не будет существовать на следующих кадрах.

48 PickingSystem

Picking строит camera ray, проверяет scene или Bullet hit и возвращает Entity. Он не должен выбирать NPC target для AI: это editor input query с другими фильтрами и временем жизни.

49 Debug UI как датчик

Хороший inspector показывает не только итоговый state, но и причину последнего перехода, значения timer, owner Entity, источник команды и rejected candidates. Журнал должен быть ring buffer, иначе длинная симуляция бесконечно растит память.

50 Полный вертикальный срез новой системы

Новая механика считается встроенной, когда существует весь путь.

1. Component с authoring и runtime полями.
2. System с явным входом и выходом.
3. Место вызова в кадре.
4. Inspector для настройки.
5. Save и load с version handling.
6. Debug representation и причина отказа.
7. Test scene или автоматический тест.
8. Cleanup runtime ресурсов.
9. Документированный владелец данных.

Если пропустить один пункт, код может компилироваться, но механика останется частично подключённой.

51 Чек лист чтения любой системы

- Найди public entrypoint.
- Выпиши guard clauses.
- Перечисли компоненты чтения.
- Перечисли компоненты записи.
- Отметь внешние ресурсы и их владельцев.
- Определи используемое время.
- Найди повторный вызов и one shot события.
- Найди порядок относительно соседних систем.
- Запиши один инвариант.
- Добавь наблюдаемый proof в debug UI.

Этот чек лист применим и к бою, и к двери, и к shader upload. Различаются данные, но инженерский способ чтения остаётся одним.

Справочник полезных алгоритмов

Этот раздел нужен не для заучивания. Перед новой механикой найди похожую задачу, выпиши входы и выходы, затем реализуй минимальный вариант и обязательно добавь визуальную проверку.

1 Безопасная нормализация

Задача: получить единичное направление без `NaN`.

Вход: произвольный вектор `v`.

Выход: единичный вектор и признак успеха.

```
bool TryNormalize(const XMFLOAT3& value, XMFLOAT3& result)
{
    const XMVECTOR vector = XMLoadFloat3(&value);
    const float lengthSquared = XMVectorGetX(XMVector3LengthSq(vector));
    if (lengthSquared <= 0.000001f)
        return false;

    XMStoreFloat3(&result, XMVector3Normalize(vector));
    return true;
}
```

Сложность: O(1).

Ловушка: проверять длину уже после нормализации.

Применение: лучи зрения, движение, направление ствола, steering.

2 Квадрат расстояния

Задача: быстро сравнить кандидатов или радиус.

```
float DistanceSquared(const XMFLOAT3& a, const XMFLOAT3& b)
{
    const float dx = a.x - b.x;
    const float dy = a.y - b.y;
    const float dz = a.z - b.z;
    return dx * dx + dy * dy + dz * dz;
}
```

Сравнивай с `radius * radius`. Квадратный корень нужен только для настоящего расстояния.

Сложность: O(1) на кандидата, O(n) для полного перебора.

3 Поле зрения через dot

Вход: нормализованный `forward`, направление к цели, полный угол FOV.

```
const float halfFov = XMConvertToRadians(fovDegrees * 0.5f);
const float threshold = std::cos(halfFov);
const bool inside = Dot(forward, normalizedToTarget) >= threshold;
```

Ловушка: использовать полный угол вместо половинного.

Применение: дешёвый фильтр перед raycast. После dot всё равно проверяется препятствие.

4 Кратчайшая разница углов

```
float ShortestAngle(float current, float target)
{
    return std::atan2(
        std::sin(target - current),
        std::cos(target - current));
}
```

Выход: значение от `-pi` до `pi`.

Применение: поворот NPC, турели, оценка ошибки прицеливания.

5 Луч и отрезок

Луч бесконечен при `t >= 0`. Для оружия и зрения обычно нужен отрезок с максимальным расстоянием.

```
struct SegmentRay
{
    XMFLOAT3 origin{};
    XMFLOAT3 direction{}; // Единичный вектор.
    float maxDistance = 0.0f;
};
```

Создание из двух точек:

1. `delta = end - start`;
2. `distance = length(delta)`;
3. отклонить почти нулевую длину;
4. `direction = delta / distance`;
5. `maxDistance = distance`.

Ловушка: передать ненормализованное направление в функцию, которая считает `t` метрами.

6 Ray plane

Формула:

$$t = ((Q - O) \cdot N)/(D \cdot N)$$

Порядок: проверить параллельность, получить `t`, проверить диапазон, вычислить точку.

Сложность: O(1).

Применение: приказ на поверхность, маркер мыши, проекция тактической точки.

7 Ray sphere

Порядок: построить квадратное уравнение, проверить дискриминант, выбрать ближайший неотрицательный корень.

Сложность: O(1).

Применение: грубый broad phase, выбор персонажа, отладочные цели.

Ловушка: вернуть отрицательный корень, лежащий позади начала.

8 Ray AABB slab method

Порядок: для каждой оси найти интервал `t1..t2`, пересечь его с общим `tMin..tMax`, отклонить пустое пересечение.

Сложность: O(1), ровно три оси.

Применение: редактор, broad phase, грубая проверка видимости объекта.

Ловушка: делить на почти нулевую компоненту направления. Параллельная ось проверяется отдельно.

9 Lerp

$$\mathrm{lerp}(a, b, t) = a + (b - a)t$$

Применение: цвет, позиция, числовой параметр.

Ловушка: использовать один и тот же `t` каждый кадр как будто это постоянная скорость. `current = lerp(current, target, 0.1)` зависит от FPS.

10 Экспоненциальное приближение

```
const float alpha = 1.0f - std::exp(-speed * deltaTime);
current += (target - current) * alpha;
```

Применение: камера, плавный поворот, UI, шумные оценки AI.

Плюс: почти одинаковая реакция при разном FPS.

11 Fixed timestep

Физику удобно обновлять фиксированным шагом.

```
accumulator += frameDelta;
while (accumulator >= fixedDelta)
{
    SimulateFixedStep(fixedDelta);
    accumulator -= fixedDelta;
}
```

Ловушка: бесконечно догонять после долгой паузы. Ограничивай `frameDelta` и число шагов.

Применение: физика, детерминированные таймеры, воспроизводимые тесты.

12 Cooldown и one shot trigger

```
cooldown = (std::max)(0.0f, cooldown - deltaTime);
if (request && cooldown <= 0.0f)
{
    ExecuteAction();
    cooldown = cooldownDuration;
}
```

Применение: выстрел, смена укрытия, повторный звук, пересчёт пути.

Ловушка: ставить cooldown каждый кадр, пока запрос активен. Тогда он никогда не достигнет нуля.

13 Конечный автомат

Минимальная модель:

```
Idle -> Searching -> Chasing -> TakingCover -> InCover -> Attacking
```

Для каждого состояния зафиксируй:

- условия входа;
- действия во время состояния;
- условия выхода;
- обязательные компоненты;
- восстановительное состояние при ошибке.

Сложность: O(1) на сущность за кадр, если состояние исполняет ограниченную работу.

Ловушка: менять состояние в нескольких системах без единого владельца перехода.

14 Utility scoring

Каждое действие получает нормализованный score.

$$\text{score} = \text{w_1f_1} + \text{w_2f_2} + \ldots + \text{w_nf_n}$$

Пример выбора укрытия:

```
score = protection * 5
      + rangeFit * 2
      + peekQuality * 3
      - pathCost * 0.2
      - exposureDuringMove * 4
```

Порядок: сначала жёсткие фильтры, затем score. Нельзя компенсировать отсутствующую защиту красивой дистанцией.

Ловушка: факторы разных масштабов. Приводи их примерно к 0..1 или явно учитывай диапазоны весов.

15 Гистерезис

Гистерезис не даёт переключаться между почти равными решениями.

```
if (newScore > currentScore + switchMargin)
    SelectNewCandidate();
```

Можно добавить минимальное время фиксации решения.

Применение: стиль боя, выбор укрытия, оружия, цели.

Ловушка: лечить гистерезисом плохие входные данные. Сначала проверь score и геометрию.

16 A star

A star ищет путь по графу.

$$f(n) = g(n) + h(n)$$

`g` — уже пройденная стоимость, `h` — оценка оставшегося пути.

Структуры: open set, closed set, `cameFrom`, стоимость `g`.

Сложность: зависит от графа и очереди; с приоритетной очередью обычно близка к O(E log V).

Применение: NavMesh или сетка.

Ловушки:

- пересчитывать полный путь каждый кадр;
- считать точку достижимой только потому, что она близко геометрически;
- не инвалидировать путь при смене `moveTarget`;
- не различать `NoPath`, `Blocked`, `Arrived`.

17 Arrival steering

Обычный seek летит к цели на полной скорости и перелетает её. Arrival снижает скорость рядом с целью.

```
const float distance = Length(toTarget);
const float desiredSpeed = maxSpeed *
    (std::min)(1.0f, distance / slowRadius);
const Vec3 desiredVelocity = Normalize(toTarget) * desiredSpeed;
```

Применение: остановка у укрытия и точки ремонта.

Ловушка: `slowRadius` меньше `arrivalDistance` или равен нулю.

18 Separation

Для каждого близкого соседа создаётся отталкивание, обычно сильнее на малой дистанции.

```
separation += away / max(distanceSquared, epsilon)
```

Применение: NPC не слипаются на пути.

Ловушка: запускать полный O(n^2) перебор для тысяч сущностей. Используй spatial hash или соседей навигации.

19 Spatial hash

Мир делится на ячейки. Координата превращается в целочисленный ключ:

```
cellX = floor(position.x / cellSize);
cellZ = floor(position.z / cellSize);
```

Для поиска в радиусе проверяются соседние ячейки, а не все сущности мира.

Средняя сложность: близка к O(1 + k), где k — число локальных кандидатов, при удачном размере ячейки.

Применение: восприятие, separation, поиск ближайших объектов.

20 Случайное направление внутри конуса

Равномерный разброс по площади нельзя получать равномерным углом по двум осям: это искажает плотность.

Практическая схема:

1. две независимые выборки `u` и `v` от 0 до 1;
2. радиальная часть использует `sqrt(u)`;
3. угол равен `2*pi*v`;
4. строится локальное отклонение;
5. оно переносится в базис `right`, `up`, `forward`.

Применение: `WeaponFiringRules::ConeDirection`.

Ловушка: делать новую случайную выборку отдельно для дебаг-луча и урона. Один выстрел должен иметь одно фактическое направление.

21 Семплирование динамических укрытий

Если авторские точки не покрывают все варианты боя, кандидаты могут генерироваться вокруг NPC или препятствий.

Кольцо вокруг NPC

```
for (int i = 0; i < sampleCount; ++i)
{
    const float angle = twoPi * i / sampleCount;
    candidate.x = npc.x + std::cos(angle) * radius;
    candidate.z = npc.z + std::sin(angle) * radius;
}
```

Для каждого кандидата:

1. спроецировать на NavMesh;
2. проверить достижимость;
3. проверить защиту несколькими лучами;
4. найти левый и правый peek;
5. оценить расстояние до угрозы;
6. оценить открытость пути;
7. применить score и hysteresis.

Ловушка: считать точку укрытием только потому, что один низкий луч встретил стену.

22 Профиль укрытия вместо bool

Полезная структура результата:

```
struct CoverProfile
{
    bool standingProtected = false;
    bool crouchingProtected = false;
    bool leftPeekValid = false;
    bool rightPeekValid = false;
    float pathCost = 0.0f;
    float exposureCost = 0.0f;
    float score = -FLT_MAX;
};
```

Так решение не теряет информацию. Анимационный модуль позднее выберет стойку и сторону выглядывания, не повторяя геометрический поиск.

23 Reservoir sampling для случайного кандидата

Если кандидатов много и надо равновероятно выбрать один без массива:

```
int seen = 0;
Entity selected = INVALID_ENTITY;
for (Entity candidate : candidates)
{
    ++seen;
    if (RandomInt(1, seen) == 1)
        selected = candidate;
}
```

Применение: разнообразие патрульных точек или равных укрытий.

24 Ring buffer журнала решений

Храни последние N событий в кольцевом буфере, а не одну быстро меняющуюся строку.

```
timestamp | state before | decision | reason | state after
```

Сложность: O(1) на запись, фиксированная память.

Применение: NPC Logic Studio. Это решает проблему мелькающих статусов, которые невозможно прочесть даже в пошаговом режиме.

25 Приоритетная очередь событий угрозы

События можно оценивать по важности и свежести:

```
priority = damageWeight * damage
         + proximityWeight * proximity
         + recencyWeight * freshness
```

Тихое убийство может не создавать общую тревогу, но локальное событие попадания всё равно разворачивает выжившего NPC к близкой угрозе.

Ловушка: передавать точную текущую позицию врага через событие навсегда. Получатель должен подтвердить контакт собственным зрением.

26 Чек лист выбора алгоритма

Перед реализацией ответь:

1. Какие точные входы?
2. Какой точный выход?
3. В каком пространстве координат данные?
4. Нужен ли точный результат или дешёвый фильтр?
5. Сколько сущностей участвует?
6. Какова сложность на кадр?
7. Как алгоритм ведёт себя на нулевых и граничных значениях?
8. Как визуально доказать его результат?
9. Какая система владеет состоянием?
10. Что записать в журнал причины?

Математика и алгоритмы для игры: второй курс с выводами и расчётами

Первый математический курс дал арифметику векторов и матриц. Теперь цель — научиться превращать игровое правило в алгоритм, заранее оценивать его стоимость и проверять результат числами. Примеры ниже учебные, если явно не назван метод движка. Алгоритм хорош, когда понятны входные данные, единицы, инвариант, крайние случаи и цена выполнения. Формула без этих пяти вещей легко превращается в красивый, но неисправный код.

1. Линейная интерполяция и экспоненциальное сглаживание

Пусть дверь находится в начале хода в позиции 0 м и должна за секунду дойти до 2 м. Линейная интерполяция между `a` и `b` при параметре `t` равна `lerp(a,b,t)=a+(b-a)t`, где `t` ограничен [0,1]. При `t=0` результат 0, при `t=0.25` — 0.5 м, при `t=1` — 2 м. Почему формула устроена так? Разность `b-a` показывает полный путь, умножение на долю показывает пройденную часть, прибавление `a` возвращает нас к началу координат. Для вектора действие выполняется по каждой координате. Если `A=(1;0;2)`, `B=(5;0;6)`, `t=0.25`, получаем `(2;0;3)`.

Частая ошибка — каждый кадр вызывать `position=lerp(position,target,0.1)`. Это не ход длительностью десять кадров. После первого кадра остаётся 90% расстояния, после второго 81%, после n кадров `0.9^n` исходного расстояния. На 120 FPS за секунду шагов больше, поэтому движение приблизится к цели сильнее, чем на 30 FPS. Если нужен путь за фиксированную секунду, храни стартовую позицию и `elapsed/duration`. Если нужно именно плавное приближение, используй коэффициент, зависящий от времени: `alpha=1-exp(-lambda*dt)`, затем `x += (target-x)*alpha`. При `lambda=2/c` и `dt=0.5c` получаем `alpha=1-e^-1≈0.632`, оставшаяся доля 0.368. Два шага по 0.25 с дают ту же общую долю `1-e^-1`, если нет округления и target не менялся. Здесь размерность `lambda*dt` должна быть безразмерной.

В движке подобные операции встречаются при визуальном сглаживании, цветах частиц и повороте. Но угол нельзя бездумно интерполировать как обычное число через границу ±π: от 179° к −179° арифметическая середина 0° идёт длинным путём. Сначала вычисли кратчайшую разницу `atan2(sin(b-a),cos(b-a))`, затем прибавляй её долю. Для полной 3D ориентации вместо смешения коэффициентов матриц применяй интерполяцию кватернионов с проверкой нормировки и выбора короткой дуги.

2. Численные производная и интеграл: скорость из позиции, путь из скорости

Производная отвечает на вопрос «как быстро меняется величина сейчас?». Если позиция в t=1 с равна 2 м, а в t=1.1 с — 2.3 м, средняя скорость на интервале `(2.3-2)/0.1=3 м/с`. Чем меньше интервал, тем ближе оценка к мгновенной скорости, но слишком малый шаг усиливает ошибки `float` и шум измерения. Поэтому для velocity физического тела лучше читать согласованное значение Bullet, чем каждый кадр вычитать два почти одинаковых Transform без учёта телепорта и физического шага.

Интеграл собирает маленькие вклады за время. Если скорость постоянна 3 м/с в течение 0.1 с, путь 0.3 м. В коде это `position += velocity*dt`. Если ускорение 2 м/с^2, скорость за шаг меняется на `acceleration*dt`. Для начальных `v=0`, `x=0`, `dt=0.5`, два полунеявных шага дают: первый `v=1`, `x=0.5`; второй `v=2`, `x=1.5`. Аналитическое положение при постоянном ускорении за 1 с равно `x=0+0·1+0.5·2·1²=1 м`. Разница 0.5 м — численная погрешность крупного шага, а не другая физика. При меньшем фиксированном шаге ошибка падает. Bullet решает больше задач, чем эта простая формула, но понимание интегратора помогает объяснить влияние `dt`.

Проверяй размерности: `a` в м/с², `a*dt` в м/с, `v*dt` в м. Не прибавляй силу в ньютонах сразу к скорости: сначала `a=F/m`. Для массы 10 кг и силы 20 Н ускорение 2 м/с². При импульсе 20 Н·с изменение скорости равно `J/m=2 м/с`. Сила действует в течение времени, импульс уже содержит время. Этим объясняется, почему взрывной толчок лучше задавать импульсом, если он должен произойти мгновенно.

3. Алгоритм маршрута: граф, стоимость и A*

Комнаты корабля можно представить вершинами графа, двери — рёбрами. Пусть стартовая комната A связана с B стоимостью 2 м, B с целью C стоимостью 2 м, а A с C через длинный проход стоимостью 10 м. Минимальный путь — A→B→C стоимостью 4, даже если на экране прямая до C кажется короткой. Если дверь A–B заперта, ребро отсутствует или получает очень высокую стоимость; решение меняется. Для NavMesh один waypoint не является фактическим движением: после планирования капсула ещё должна пройти через физический мир.

Алгоритм Дейкстры хранит лучшую известную цену `g(v)` от старта и выбирает вершину с наименьшей текущей ценой. A* добавляет оценку оставшегося пути `h(v)` и выбирает минимум `f(v)=g(v)+h(v)`. Эвристика `h` не должна завышать реальную остаточную стоимость, если нужна гарантия кратчайшего пути. Для движения по плоскости с ценой не меньше геометрической длины прямая евклидова дистанция до цели подходит. Если за закрытую дверь назначен дополнительный штраф, прямая всё ещё нижняя граница. Если в игре есть телепорт с нулевой ценой, обычная дистанция уже может завышать реальную стоимость и гарантия нарушится.

Пройди выбор вручную. Из A доступны B с `g=2,h=2,f=4` и D с `g=1,h=8,f=9`. Сначала раскрывается B. Если из B в C ребро 2, новая цель получает `g=4,h=0,f=4`. Алгоритм находит маршрут стоимостью 4. Для NPC после двери важно не только построить путь, но и зафиксировать причину его устаревания: дверь изменила состояние, NavMesh перестроен, цель сместилась. Кэшируй путь вместе с revision данных, от которых он зависит. Бесконечный repath каждый кадр может быть дороже самого движения и не решит проблему запертой двери.

4. Выбор работы экипажа как задача оптимизации

Один NPC может выбрать работу по максимальному score, но распределение двух NPC между двумя работами требует учитывать конкуренцию. Пусть инженер A даёт полезность 9 для пробоины X и 8 для Y, инженер B — 7 для X и 1 для Y. Жадный выбор A→X оставит B→Y, сумма 10. Другой вариант A→Y, B→X даёт сумму 15. Значит локально лучший первый выбор не обязательно максимизирует пользу группы. Для маленького экипажа можно перебрать допустимые назначения или использовать алгоритм оптимального паросочетания. Для большой сцены можно оставить детерминированный жадный вариант, если скорость и понятность важнее оптимальности, но это осознанный компромисс.

Формула score должна иметь понятную шкалу. Если severity в [0,1], а расстояние в метрах от 0 до 100, сырое выражение `severity-distance` практически всегда определяется дистанцией. Нормализуй: например `distanceScore=1/(1+d/10м)`. При d=0 он равен 1, при d=10м — 0.5, при d=30м — 0.25. `score=4*severity+2*distanceScore+roleBonus` теперь имеет сравнимые слагаемые. Но score не должен отменять жёсткие запреты: отсутствует путь, нет нужного ресурса, цель уже занята, опасная среда без скафандра. Сначала фильтруй допустимость, затем ранжируй.

Инвариант claim: на эксклюзивную задачу назначен максимум один живой исполнитель. После смерти, ручного приказа, уничтожения цели или истечения срока claim должен освободиться. Если цель удаляется во время ремонта, хранить указатель на компонент через удаление опасно. Сохрани Entity ID, после структурного изменения перечитай Registry. В текущем движке цепь

Utility/Coordinator/StateSystem уже существует; задача оптимального паросочетания здесь предлагается как развитие, а не заявляется готовой функцией.

5. Вероятность, разброс оружия и воспроизводимость

Случайное число из [0,1) может выбирать событие с вероятностью p: `u<p`. При `p=0.2` из 100 независимых испытаний ожидается около 20 успехов, но не ровно 20. Вероятность нуля попаданий за n независимых попыток равна `(1-p)^n`: при p=0.2 и n=5 это `0.8^5≈0.32768`. Нельзя проверять случайную стрельбу тестом «из пяти выстрелов обязаны попасть ровно один раз». Проверяй распределение на большом числе запусков, а конкретный баг воспроизводи с сохранённым seed.

Равномерный выбор угла не всегда даёт равномерную площадь на диске. Если радиус r выбирать просто равномерно от 0 до R, слишком много точек окажется близко к центру относительно площади. Для равномерной площади диска возьми `r=R*sqrt(u)`, угол `ϕ=2πv`, где u и v независимы и равномерны. Тогда `x=r*cosϕ`, `y=r*sinϕ`. Проверка: доля попаданий внутри половины радиуса должна быть площадью маленького круга относительно большого, `(R/2)²/R²=1/4`, а не 1/2. Для конуса направления нужна отдельная геометрия и нормировка; у текущего `WeaponSystem` есть собственное правило разброса, поэтому перед заменой сначала измерь его и не меняй баланс по учебной формуле вслепую.

Для NPC различай наблюдение и случайность. Пусть противник не видит игрока за стеной; случайный разброс не должен выдавать точную скрытую позицию. Он меняет направление луча вокруг уже допустимой точки прицеливания, а не создаёт знание. Запиши seed, shooter Entity, target Entity, исходный aim и фактический ray direction. Тогда один промах можно воспроизвести до пикселя и отличить ошибку геометрии дула от случайного отклонения.

6. Пространственные алгоритмы: AABB и отсев

Ось-ориентированный ограничивающий параллелепипед AABB хранит минимум и максимум по каждой оси. Точка `(x;y;z)` внутри, если `minX≤x≤maxX`, `minY≤y≤maxY`, `minZ≤z≤maxZ`. Для двух AABB пересечение есть, если интервалы перекрываются по всем трём осям. Если хотя бы по одной оси `A.maxX < B.minX`, коробки разделены, и дорогую проверку треугольников можно не делать. Это широкая фаза физики и полезный предварительный отсев видимости/взаимодействия.

Пример: A по X `[0,2]`, B по X `[3,4]`. Разделение уже доказано, Y и Z можно не читать. Если B по X `[1,3]`, X совместима, но ещё нужно проверить Y и Z. Для вращающегося объекта локальный AABB нельзя переносить только двумя диагональными углами; после вращения экстремальная вершина может быть другой. Либо трансформируй все восемь углов, либо используй центр и абсолютные компоненты матрицы для вычисления новой половины размера. Ошибочный world bounds может скрыть видимый mesh на краю камеры или пропустить raycast предварительной фазы.

Для множества объектов наивное сравнение каждого с каждым стоит `n(n-1)/2`. При n=1000 это 499500 пар. Разбиение пространства на клетки, дерево BVH или broadphase Bullet сокращает число реальных кандидатов, особенно если объекты разбросаны. Но структура ускоряет запрос ценой обновления: если каждый объект постоянно движется, её надо поддерживать. Не оптимизируй наугад; сначала измерь число проверяемых пар, стоимость обновления и время кадра.

7. Численная устойчивость и тесты свойств

При вычислениях с `float` не проверяй геометрию через абсолютное `==`. Для длины в метрах выбери допустимую ошибку в метрах; для нормированного dot — безразмерную ошибку. `epsilon=1e-6` не

является универсальным ответом: для корабля километрового масштаба и для миллиметрового muzzle одна и та же абсолютная ошибка имеет разный смысл. Проверь конечность входов через `isfinite`, прежде чем NaN попадёт в Transform, Bullet или shader buffer. Сравнение `NaN < 0` ложно, поэтому обычный `clamp` не всегда спасёт испорченное значение.

Тест свойства проверяет закон, а не копирует реализацию. Для случайных совместимых матриц `transpose(transpose(A))` должно вернуть A в пределах погрешности. Для ненулевого вектора `length(normalize(v))≈1`. Для пары преобразований `p*(M*M^-1)≈p`, если M обратима. Для распределения энергии сумма выданной мощности не превосходит доступную. Для двух комнат без утечек общий запас вещества в физической модели сохраняется. Эти проверки полезны после рефакторинга: они находят ошибку даже если конкретные числа в примере изменились.

Закончи курс мини-проектом: возьми одну пробоину, двух инженеров и две двери. На бумаге вычисли расстояния, score и допустимые назначения; нарисуй граф комнат и лучший маршрут; затем запусти сцену и сравни debug числа. Разница между расчётом и игрой должна иметь имя: физическое препятствие, изменившееся состояние двери, другой `dt`, конкурирующий claim или ошибка формулы. Так математика становится инструментом исследования собственного кода.

8. Луч, плоскость и сфера: вычисляем попадание до Bullet

Луч описывается началом O, единичным направлением D и параметром `t≥0`: `P(t)=O+tD`. Если D нормирован, t измеряется в метрах; если нет, t просто параметр и его нельзя сравнивать с дальностью оружия без пересчёта. Возьмём O=`(0;1;0)`, D=`(0;0;1)`, плоскость стены `z=5`. У луча z-координата `0+t*1=t`, значит t=5 и точка попадания `(0;1;5)`. Для направления `(0;0;2)` тот же контакт имеет t=2.5, но расстояние всё ещё 5 м. Вот почему оружейный `ValidRay` и нормировка направления важны до проверки range.

Плоскость в общем виде задаётся точкой Q и единичной нормалью N: точка P лежит на ней, когда `dot(P-Q,N)=0`. Подставим луч: `dot(O+tD-Q,N)=0`, раскроем линейность, получим `t=dot(Q-O,N)/dot(D,N)`. Знаменатель около нуля означает, что луч почти параллелен плоскости. Если одновременно числитель около нуля, луч лежит в плоскости, и «единственная точка попадания» не определена. Если t отрицателен, пересечение находится позади начала луча. Для оружия дополнительно требуется `t≤range`. Для picking редактора t может ограничиваться ближней и дальней плоскостью камеры.

Сфера с центром C и радиусом r задаётся `|P-C|²=r²`. Подстановка `P=O+tD` даёт квадратное уравнение. При нормированном D обозначь `m=O-C`; тогда `t²+2dot(m,D)t+dot(m,m)-r²=0`. Дискриминант меньше нуля — нет пересечения; равен нулю — касание; положителен — два пересечения, вход и выход. Выбирай наименьший неотрицательный t. Если начало луча внутри сферы, первый корень отрицателен, а второй положителен: это точка выхода. В Bullet тебе не нужно повторять эту математику для каждого объекта, но она помогает понять, почему отладочный луч попал в ближний collider, а не в выбранную NPC цель.

9. Столкновение прямоугольников и разделяющая ось

Для AABB проверка интервалов по X, Y, Z достаточна. Для повернутых коробок оси этих объектов различаются, поэтому простое сравнение мировых min/max может давать ложное пересечение. Теорема разделяющей оси говорит: две выпуклые фигуры не пересекаются, если существует ось, на которой их проекции не пересекаются. Для двумерных прямоугольников кандидаты — нормали к сторонам обоих. Проекция вершины P на единичную ось n — `dot(P,n)`. Для каждого многоугольника найди минимум и максимум этой проекции. Если интервалы разошлись хотя бы на одной оси, столкновения нет. Если не разошлись на всех необходимых осях, фигуры пересекаются.

Пример на оси X: проекция первого прямоугольника `[0,2]`, второго `[3,5]`. Условие `2<3` сразу доказывает отсутствие контакта. На другой оси проверять не нужно. Если интервалы `[0,2]` и `[1,3]` перекрываются, это ещё не доказывает столкновение: по Y фигуры могут быть разделены. Для трёхмерных OBB нужны оси граней обоих объектов и оси попарных векторных произведений их базисных осей; это уже более дорогая проверка. Bullet выполняет специализированную collision работу, но понимание SAT полезно для editor gizmo, UI геометрии и предварительной диагностики collider.

Есть тонкий крайний случай: «касаются» или «пересекаются»? Для интервалов `[0,2]` и `[2,4]` общая точка 2. Если условие разделения строгое `<`, касание считается контактом; если `≤`, нет. В физике обычно нужен допуск и контактный зазор, поэтому символ сравнения — часть игрового правила. На высоких скоростях дискретная проверка положения до и после шага может пропустить тонкую стену: объект окажется по одну сторону на t=0 и по другую на t=dt. Для такого случая используют swept collision/continuous detection или ограничивают шаг, а не просто увеличивают радиус объекта.

10. Управление дверью или камерой: ошибка, скорость и устойчивость

Пусть привод двери должен довести её до позиции 1 м, сейчас она в 0.6 м. Ошибка `e=target-current=0.4 м`. Простейшее пропорциональное управление задаёт скорость `v=Kp*e`. Чтобы результат был в м/с, `Kp` измеряется в 1/с. При `Kp=2/с`, v=0.8 м/с; за `dt=0.1с` дверь сместится на 0.08 м до 0.68 м. Следующая ошибка станет 0.32 м, новая скорость 0.64 м/с. Движение постепенно замедляется и асимптотически приближается к цели. Чтобы не ждать бесконечно, введи допуск, например 1 мм: внутри него установи конечную позицию и закончи переход.

Если применить `current += Kp*e*dt`, устойчивость простого дискретного правила зависит от `Kp*dt`. При значении 1 результат достигается за шаг, при значении 2 прыгает через цель на противоположную сторону, при ещё больших значениях может расходиться. Поэтому изменение dt без проверки может превратить плавную дверь в дрожащую. Для критичного движения используй фиксированный шаг, ограничение скорости или аналитическое сглаживание из главы 1. Для физического тела не записывай Transform поверх Bullet каждый кадр; применяй согласованные силы/импульсы или кинематический контроллер.

PID добавляет интегральный член для накопленной ошибки и дифференциальный член для реакции на скорость её изменения. Но он не является обязательной «улучшенной версией» для каждой двери. В игре простая интерполяция с известной длительностью часто понятнее и легче тестируется. PID полезнее там, где есть инерция и возмущения, например стабилизация корабля. Перед его добавлением измерь реальное поведение, выбери единицы коэффициентов, зажми интегратор при насыщении привода и проверь устойчивость при фиксированном dt. Три буквы без числового эксперимента только скрывают проблему.

Инженерные практики для развития проекта

Эта часть добавляет темы, которые становятся важнее по мере роста движка. Они не создают новую видимую механику за один вечер, но не дают механикам превратиться в неуправляемый монолит.

1 Контракт до класса

Перед новым файлом ответь письменно.

Вопрос	Пример для stamina
Кто владеет данными	`StaminaComponent` на персонаже
Кто меняет значение	`StaminaSystem`
Кто только читает	movement UI tactical decision
Когда обновляется	после intent до выбора фактической скорости
Какая единица	нормализованный диапазон или points
Какие инварианты	`0 <= current <= maximum`
Как сохраняется	authoring maximum да runtime current для save game
Как отлаживается	current drain reason regen delay

Если два класса заявляют право окончательно писать одно поле, контракт ещё не готов.

2 Правило одного источника истины

У оружия magazine живёт в одном `WeaponComponent`. У комнаты pressure живёт в одном `RoomComponent`. У dynamic body физическая позиция после fixed step живёт в Bullet и копируется в Transform. Кэш допустим, если он помечен как derived и имеет понятную процедуру invalidation.

Плохой признак: баг исправляют синхронизацией двух почти одинаковых полей каждый кадр. Обычно лучше удалить одного владельца.

3 RAII и внешние библиотеки

DirectX и Bullet часто возвращают raw pointers, но это не означает отсутствие владельца. Для каждого runtime ресурса в компоненте или manager запиши destroy sequence.

```
Bullet body
    removeRigidBody from world
    delete motion state
    delete body
    delete shape children
    delete shape
    delete backing triangle mesh
D3D resource
    release references held by views
    release view
    release buffer or texture
```

Когда возможно, используй `unique_ptr` с custom deleter или небольшой wrapper. Если component остаётся move only, ECS уже имеет move overload `AddComponent`.

4 Инварианты сильнее комментариев

Комментарий сообщает намерение, но проверка ловит нарушение. Полезные debug assertions:

```
assert(component.maximum >= 0.0f);
assert(component.current >= 0.0f);
assert(component.current <= component.maximum + epsilon);
```

В release код всё равно должен безопасно обрабатывать внешние данные из JSON. Assert помогает разработчику, validation защищает пользователя и save file.

5 Guard clauses

Ранний выход делает основной путь плоским.

```
if (!world)
    return;
if (!registry.IsAlive(entity))
    return;
auto* transform = registry.GetComponent<TransformComponent>(entity);
if (!transform)
    return;

// основной путь начинается здесь
```

Не все ошибки равны. Ожидаемое отсутствие optional component допускает тихий return. Нарушение обязательного инварианта должно оставить diagnostic reason, assert или error log.

6 Именование величин и единиц

Добавляй единицу в имя там, где ошибка опасна: `reloadSeconds`, `outputMw`, `energyMwh`, `angleRadians`, `distanceMeters`. `value`, `speed` и `time` слишком легко смешать.

Величина	Хорошее имя	Типичная ошибка
угол	`yawRadians`	передать degrees в `sin`
длительность	`cooldownSeconds`	уменьшать на milliseconds
мощность	`requiredMw`	считать её запасом энергии
квадрат расстояния	`distanceSquared`	сравнить с обычным radius
нормализованная доля	`powerRatio`	принять за проценты 0..100

7 Координатные пространства как часть типа

Имя `position` не сообщает пространство. Для сложного кода используй `worldPosition`, `localOffset`, `viewDirection`, `modelNormal`. Особенно это важно для muzzle, IK, picking ray и joint frames.

Минимальный комментарий перед формулой должен сообщать переход: local muzzle point умножается на bone matrix, затем на entity world matrix и становится world origin выстрела.

8 Численная устойчивость

Основные правила проекта:

- проверяй длину до нормализации;
- clamp аргумент `acos` в `[-1, 1]`;
- сравнивай squared distance со squared threshold;
- ограничивай большой `deltaTime` для простой симуляции;
- используй epsilon только с понятной единицей;
- проверяй `isfinite` на границе физики и файлов;
- не дели на количество элементов до проверки нуля.

Epsilon `0.0001` не является универсальной магией. Для позиции в метрах и для normalized dot это разные масштабы ошибки.

9 Детерминизм и случайность

Random влияет на spread, wandering, effects и utility tie break. Для воспроизводимого теста используй seed, который сохраняется в debug run.

```
seed + sequence of calls + same inputs -> same decisions
```

Если разные системы используют один глобальный generator, добавление случайной частицы может изменить AI. Лучше иметь отдельные streams или deterministic sampler по Entity и event index.

10 State machine и причина перехода

Храни не только state, но и `lastTransitionReason` для debug. Переход должен выполняться в одном месте или через функцию.

```
void SetState(NpcBrainComponent& brain, NpcState next, std::string reason)
{
    if (brain.state == next)
        return;
    brain.previousState = brain.state;
    brain.state = next;
    brain.lastTransitionReason = std::move(reason);
}
```

Так журнал показывает факт, а не пытается восстановить причину по текущим полям спустя несколько кадров.

11 Decision и execution

Decision выбирает цель и намерение. Execution проверяет возможность и выполняет. Результат execution возвращается как success, running или failure reason.

```
tactical module -> MoveToCover intent
navigation      -> path or no path
movement        -> displacement intent
Bullet          -> actual position
feedback        -> reached blocked stuck
```

Если decision каждый кадр перезаписывает intent до получения feedback, NPC дёргается между решениями.

12 Cooldown hysteresis и commitment

Cooldown ограничивает частоту повторной оценки. Hysteresis требует, чтобы новый вариант был заметно лучше текущего. Commitment запрещает смену на короткой фазе, например резкое выглядывание.

```
switch when
newScore > currentScore + hysteresis
and decisionCooldown <= 0
and not in committed action
```

Эти три механизма решают разные проблемы. Один огромный timer скрывает симптомы, но делает AI медленным.

13 Ring buffer журналов

Журнал решения ограничивается последними N событиями. Поля события: world time, Entity, category, previous state, next state, reason и важные числа.

Не форматируй всю строку заранее, если UI может переключать язык. Храни структурированные поля и локализуй представление.

14 Structured debug snapshot

Inspector не должен читать десятки компонентов несогласованно и придумывать смысл. Система может публиковать компактный snapshot.

```
NpcDebugSnapshot
    state
    target and visibility
    tactical order and reason
    move target and path status
    weapon ammo cooldown reload
    cover candidate rejection counts
```

Snapshot является derived debug data и не участвует в решениях.

15 Тестовая пирамида движка

Уровень	Пример	Что ловит
чистая функция	cone direction limited flow utility score	математику и границы
component system	PowerSystem на маленьком registry	порядок чтения и записи
scenario	два NPC и одна стена	интеграцию нескольких систем
soak	30 минут симуляции	утечки рост журнала редкие застревания
save load	round trip сцены	пропущенные поля и remap
visual regression	shader test scene	presentation изменения

Не каждый тест должен запускать DirectX window. Чистые правила выноси в функции без renderer и world, чтобы они проверялись быстро.

16 Arrange act assert

```
Arrange
    создать registry и минимальные компоненты
Act
    вызвать одну систему с известным dt
Assert
    проверить поля и события
```

В Arrange не копируй целую игровую сцену. Чем меньше обязательных данных, тем лучше тест показывает контракт системы.

17 Golden scenario для NPC

Создай несколько фиксированных сцен.

- прямая видимость без укрытий;
- низкая стена и высокая стена;
- два подходящих укрытия с разной дистанцией;
- цель исчезает за углом;
- empty magazine во время движения;
- friendly в firing lane;
- defender и assault profile;
- входящий near miss без урона;
- повреждение во время exposed phase;
- путь временно недоступен.

Для каждого сценария запиши ожидаемую последовательность состояний, а не только конечную позицию.

18 Property tests для математики

Полезные свойства:

- нормализованный ненулевой vector имеет length около единицы;
- dot normalized vectors находится в `[-1,1]`;
- ray hit fraction находится в `[0,1]`;
- `LimitedFlow` не переносит больше половины разницы за один парный обмен;
- utility score finite и ограничен заданным диапазоном;
- matrix inverse возвращает исходную точку в пределах epsilon;
- power supplied не превышает generation;
- сумма transfer между комнатами сохраняется без утечки до clamp.

19 Save version и migration

Добавь номер schema. При загрузке старой версии применяй migration defaults. Не определяй версию по отсутствию случайного поля, если формат уже развивается.

```
version 3
    add faction = Neutral when absent
version 4
    convert reloadTimeMs to reloadSeconds
version 5
    split coverPoint into cover and peek positions
```

После migration runtime resources всё равно пересоздаются обычным load pipeline.

20 Атомарное сохранение

Безопасная запись:

1. записать временный файл рядом;
2. flush и закрыть;
3. проверить что JSON читается;
4. заменить основной файл;
5. при необходимости сохранить backup.

Падение во время прямой перезаписи не должно уничтожать единственный save.

21 Совместимость компонентов

При добавлении обязательного компонента старые сцены его не имеют. Выбери одну политику: default при load, editor migration tool или система `EnsureComponent`. Не размазывай три разных default по Inspector, load и runtime.

22 Профилирование до оптимизации

Сначала измеряй время систем и количество элементов.

```
Frame
    AI 2.1 ms for 84 NPC
    Physics 3.8 ms for 612 bodies
    Render preparation 1.4 ms for 2300 candidates
    GPU 7.6 ms
```

Среднее скрывает spikes. Записывай median, 95 percentile и maximum. Отдельно считай query counts: raycasts NPC, cover candidates, path rebuilds и draw calls.

23 Пространственное ускорение

Если каждый NPC проверяет каждого кандидата, стоимость растёт квадратично. Spatial hash или grid уменьшает набор соседей.

```
cell = floor(position / cellSize)
query radius -> neighboring cells
exact distance -> final filter
```

Grid только создаёт кандидатов. Точная проверка distance, FOV и raycast всё равно выполняется после.

24 Бюджетирование AI

Не все решения требуют 60 обновлений в секунду.

Слой	Частота пример	Причина
движение controller	каждый simulation frame	плавность и контакт
weapon cooldown	каждый simulation frame	точное время
target candidate scan	5 или 10 Hz	дорогая пространственная оценка
cover rescore	по событию и cooldown	стабильность решения
дальний idle NPC	редкий tick	simulation LOD

Распределяй NPC по фазам, чтобы все не делали дорогой scan в одном кадре.

25 Memory и invalidation

Raw pointer на component действителен только пока pool не изменён способом, который перераспределяет storage, и пока Entity жива. Не сохраняй component pointer между кадрами без контракта контейнера. Сохраняй Entity и повторно получай pointer после `IsAlive`.

Модель, Bullet object и D3D resource имеют отдельные жизни. Смена сцены обязана инвалидировать caches, debug selection и async requests предыдущей сцены.

26 Ошибки фоновых задач

Worker error должен стать состоянием задачи, а не исчезнуть в debug output. Main thread получает success или structured error и может показать asset path, stage и message.

При закрытии сцены long job либо отменяется, либо результат проверяет generation token и не применяется к новому миру.

27 Generation token

```
sceneGeneration = 12
job captures 12
scene reload increments to 13
job completes with 12
main thread rejects stale result
```

Этот простой механизм предотвращает применение старой async геометрии к Entity новой сцены с совпавшим ID.

28 Dependency direction

Желательное направление:

```
Core and math
    <- ECS data
        <- gameplay rules
            <- presentation and editor
```

Renderer может читать gameplay snapshot, но gameplay не должен включать EditorUI для принятия решения. Debug hooks оформляй интерфейсом или данными, а не обратной зависимостью.

29 Модульная тактика NPC

Каждый tactical module реализует общий контракт.

```
CanEvaluate(context)
Score(context)
BuildIntent(context)
Explain(context)
```

Context является read only snapshot восприятия, оружия, расстояния, stamina и укрытий. Module не двигает NPC напрямую. Arbiter выбирает intent, execution layers выполняют.

Новые модули: hold distance, assault, retreat, suppress, flank, reload safety, investigate sound, protect ally. Групповая координация может добавлять context facts, но не должна быть обязательной для одиночного NPC.

30 Capability компоненты

Не раздувай один `NPCComponent` сотней bool. Возможности оформляй компонентами или data profiles.

```
CanUseCoverComponent
CanLeanComponent
StaminaComponent
HearingComponent
SquadMemberComponent
MedicCapability
```

Отсутствие компонента означает отсутствие возможности. System перебирает только нужный pool.

31 Data driven параметры

Порог дистанции, время exposed phase и скорость stamina не должны быть случайными literals в нескольких системах. Собери profile с понятными единицами и validation. Editor меняет profile, runtime читает его.

Баланс параметров не заменяет алгоритм. Если NPC меняет укрытие каждый две секунды из-за отсутствия hysteresis, увеличение таймера лишь маскирует причину.

32 Телеметрия решений

Для каждого дорогого выбора полезны counters.

- candidates total;
- rejected by distance;
- rejected by protection;
- rejected by path;
- rejected reserved;
- chosen score;
- evaluation time microseconds;
- time since last change;
- execution failure reason.

Так видно, что NPC не тупит, а все кандидаты отвергнуты конкретным фильтром.

33 Локализация журналов

Храни event id и параметры, например `CoverRejectedLowProtection` и `protectionHeight=0.82`. Русский и английский UI форматируют их отдельно. Если хранить только готовую английскую строку, перевод быстро станет неполным.

34 Комментарии которые помогают

Хороший комментарий отвечает почему, указывает единицы или контракт внешней библиотеки.

```
// Bullet transform хранит центр капсулы, а ECS позиция находится у ног.
```

Плохой комментарий повторяет синтаксис.

```
// Увеличиваем i на один.
++i;
```

После изменения поведения обновляй комментарий вместе с кодом. Устаревшее объяснение опаснее отсутствующего.

35 Маленький рефакторинг

Безопасная последовательность:

1. зафиксировать scenario и debug output;
2. вынести чистую функцию без изменения результата;
3. сравнить поведение;
4. вынести данные;
5. сравнить снова;
6. только потом менять правило.

Не совмещай перенос архитектуры, новый алгоритм и баланс параметров в одном непроверяемом шаге.

36 Definition of Done

Механика готова, когда:

- работает основной путь;
- отказ имеет понятную причину;
- runtime resources удаляются;
- save load сохраняет нужные данные;
- Inspector показывает важные поля;
- есть минимальный scenario test;
- стоимость измерена;
- комментарии объясняют нетривиальный контракт;
- старые сцены либо совместимы, либо мигрируются;
- место расширения записано в атласе.

37 Рекомендуемый порядок развития проекта

1. Закончить наблюдаемость: единый журнал, counters, debug rays и state reasons.
2. Закрепить golden scenarios NPC и физики.
3. Ввести profiles и единицы для разбросанных параметров.
4. Довести save version и migration.
5. Разделить длинные системы по decision, query и execution.
6. Добавить stamina как учебный вертикальный срез.
7. Добавить lean и crouch как capability и animation action.
8. Добавить hearing и investigate sound.
9. Добавить новые одиночные tactical modules.
10. Только после стабильного одиночного AI подключать squad coordination.

38 Практикум по самостоятельной разработке

Задание 1 Stamina

Самостоятельно опиши component, system order, units, invariants, debug fields и save policy. Код начинай только после схемы.

Задание 2 Physics ownership

Для двери, ящика, NPC и отсека заполни таблицу: тип body, владелец Transform до и после fixed step, group, mask, cleanup.

Задание 3 Scenario test

Создай тест: NPC идёт к новому укрытию, стреляет в движении, начинает reload, доходит, прячется и только потом выглядывает. Запиши ожидаемые состояния по времени.

Задание 4 Save migration

Придумай старую версию `WeaponComponent` без reserve ammo и новую с reserve ammo. Опиши migration и default.

Задание 5 Performance

Добавь counters raycast, cover query и path rebuild. Сравни два NPC и двадцать NPC, не меняя алгоритм.

39 Финальный чек лист разработчика

- Я знаю владельца каждого изменяемого поля.
- Я знаю единицу каждой физической величины.
- Я могу назвать порядок системы в кадре.
- Я отделяю intent от подтверждённого факта.
- Я не сохраняю runtime pointers.
- Я удаляю внешний ресурс до уничтожения его владельца.
- Я могу воспроизвести случайный тест через seed.
- Я вижу причину отказа в журнале.
- Я измеряю прежде чем оптимизировать.
- Я изменяю одно правило за проверяемый шаг.

Эти привычки полезнее любого отдельного паттерна. Они позволяют добавлять новые механики как модули, а не как новые ветви одного бесконечного `Update`.

Редактор, сцена и самостоятельная работа с кодом

Математика и анализ методов полезны, когда после чтения можно открыть свой проект и повторить результат. Эта часть ведёт от создания одного Entity в редакторе до сохранённого эпизода. Здесь разобраны настоящие границы `EditorUI`, `SceneManager` и `ApplicationClass`. Мы не будем считать, что сохранённая сцена автоматически равна сохранению игрового прогресса: это два разных договора.

1. Как объект проходит путь от кнопки редактора до мира

1.1. Начало: шаблон в `EditorUI`

В `src/Editor/EditorUI.cpp` метод `CreateEditorTemplate` находится около строки 1729. Он получает `SceneManager&` и `QuickCreateKind`. Это означает, что UI не владеет отдельным Registry: он просит существующий SceneManager создать или наполнить Entity. `switch (kind)` выбирает профиль — пустой объект, точку взаимодействия, свет, `CrewNpc`, cover point, patrol point, pickup, trigger area и другие. Прочитай одну ветку, например `CrewNpc`, и выпиши каждое добавление компонента. Затем найди `RenderQuickCreateMenu`: его пункты вызывают тот же метод с конкретным kind. Раздели два действия: кнопка меню определяет намерение автора сцены, а фабрика создаёт набор данных.

После создания проверь сущность в Outliner и Inspector. Если `CrewNpc` виден в Hierarchy, это не доказывает, что у него есть модель, Bullet controller, NavMesh доступность и правильно заданная фракция. Quick Create — начальный профиль, который должен пройти проверку обязательных компонентов. Для нового шаблона не копируй случайную ветку и не оставляй несовместимые поля. Составь «паспорт» созданной Entity: Transform, логика, визуал, collision, связи с комнатой, источник питания, save fields. Если у профиля только логика, Inspector должен это явно показать.

1.2. Изменение параметра и его владелец

`RenderComponentManager` в `EditorUI.cpp` собирает интерфейс для компонентов выбранной Entity. Перед добавлением нового поля выясни, кто его меняет в Play. Авторский параметр `maxHealth` редактируется до игры; текущий `health` может изменяться системами каждый кадр. Если дать редактировать live значение и затем сохранить сцену в неожиданном моменте, автор случайно запишет состояние после боя как стартовое. Раздели UI на «настройка» и «текущее состояние». Для расчётного `powerRatio` лучше read-only текст с единицей и источником, чем ползунок, который энергосистема перезапишет через один тик.

Одна величина должна иметь название, единицу, диапазон и default. Например, `repairDurationSeconds=10` означает время при skill=1 и фиксированном requiredProgress; `repairDistanceMeters=1.5` — горизонтальный радиус работы, а не размер collision shape. Если в UI будет просто «Repair 10», следующий разработчик не поймёт, что меняется. Для полей, влияющих на математику, показывай вычисленный результат: при выбранном skill ожидаемое время ремонта; при заданной мощности ожидаемый powerRatio.

1.3. Критерий «объект работает»

Создай в Quick Create одного инженера. Проверь последовательно: он есть в Registry; у него есть нужные компоненты; у него есть визуальная модель или ясно указано, что профиль логический; collider создан и находится у ног модели; он видит доступную работу; маршрут ведёт к рабочей позиции; progress изменяется; завершение отражается в мире. Для оборудования проверь WorkZone, если он задан; текущая ветка пробоины использует другую целевую точку без WorkZone. Любой отказ должен

иметь первый нарушенный контракт. Такой тест занимает меньше времени, чем настройка красивой анимации у NPC, которому ни разу не выдали задачу.

2. `SaveScene` построчно: почему в JSON попадает не всё

2.1. Заголовок файла и список объектов

Около строки 3270 `SceneManager::SaveScene(const std::string& filename)` создаёт `json scene`, задаёт `scene_name`, `version=2` и пустой массив `objects`. Номер версии — контракт с загрузчиком: изменение формата данных должно учитывать старые сцены. Следом создаются `int id=0` и `savedEntities`. Лямбда `saveEntity` проверяет `IsAlive`, отбрасывает `NpcWeaponVisualComponent` и `ShipPartComponent`, исключает повторение Entity через `savedEntities.insert(entity).second` и вызывает `SerializeObject(entity,id++)`. У отбрасывания есть смысл: визуал оружия и импортированные части корабля восстанавливаются из более высокого источника, поэтому их сериализация создала бы дубликаты.

Почему функция не просто обходит все живые Entity? Потому что часть сущностей является runtime или производными от GLB root, а часть логических объектов не имеет mesh. Дальше SaveScene проходит несколько pools: корни кораблей, обычные объекты с mesh, помещения и комнаты без mesh. При чтении каждого цикла спроси: какую категорию объектов он добавляет, какие уже были добавлены и какую категорию может забыть. `savedEntities` защищает от дубля, если одна Entity встречается в нескольких pools. Но объект с новым компонентом, не попавший ни в один из этих обходов, может вообще не сохраниться. При добавлении системы это одна из первых проверок.

2.2. `SerializeObject` и граница authoring/runtime

`SerializeObject` начинается около строки 3428. Он превращает выбранную Entity в JSON. Читая его, для каждого поля подпиши, откуда взялось значение и должно ли оно переживать reload. Позиция модели, стартовые параметры генератора, фракция и роль NPC обычно нужны для новой сцены. Указатели на D3D buffers, `btRigidBody*`, временный результат raycast и очередь job не имеют смысла в файле. Если сериализация пропускает новый компонент, он исчезнет после загрузки даже при успешном сохранении. Если записывает runtime pointer как число, файл будет выглядеть заполненным, но после запуска значение станет недействительным.

Практика: сохрани сцену с двумя объектами с различными Transform. Открой JSON как текст, найди version и обе записи. Измени только один авторский параметр в Editor, сохрани снова и сравни файлы. Если в диффе изменилось множество не связанных runtime полей, формат сцены захватывает лишнее. Если нужный параметр не изменился, проверь Inspector и `SerializeObject` — возможно, UI редактирует копию или не тот компонент.

3. `LoadScene` построчно: пересоздание мира

3.1. Открытие и очистка

Около строки 4301 `LoadScene` принимает путь, D3D device и device context. `std::ifstream file(filename)` открывает файл; если он не открыт, метод возвращает false. Затем JSON считывается, вызывается `Clear()`, создаются таблица `remap` старых ID к новым и набор `restoredEntities`. Если файл синтаксически повреждён и исключение из JSON возникает до `Clear()`, состояние мира может остаться прежним; если после Clear — уже частично очищенным. Для пользовательской игры полезна подготовительная проверка файла до разрушения текущей сцены и понятная ошибка загрузки. Это проектное улучшение, не описание гарантии текущего кода.

`LoadScene` читает блок `world`, если он присутствует, и восстанавливает fog/skybox параметры с fallback на текущие defaults. Для объектов проходит JSON массив. Корень импортированного корабля обрабатывается отдельно: `ImportShipFromGlb` строит структуру из asset, затем возвращаются Transform и параметры power grid/генераторов. Обычный объект создаётся через `DeserializeObject`. После создания любой успешный Entity заносится в `restoredEntities`, а старый `entity_id` связывается с новым в `remap`. Здесь особенно важно отличать сохранённый номер от текущего: Registry выдаёт новые ID, и ссылка `repairTarget=42` не обязана всё ещё указывать на 42.

3.2. Восстановление ссылок и Bullet

После цикла вызывается `RemapEntityReferences(remap,restoredEntities)`. Комментарий в коде объясняет тонкость: remap применяется только к компонентам, наполненным из сохранённых полей. Вспомогательные Entity импортированного GLB уже указывают на новые runtime ID; слепое повторное remap может испортить иерархию, если новый ID случайно совпал со старым. Затем `LoadScene` обходит `RigidBodyComponent`: если `body` отсутствует, вызывает `PhysicsSystem::CreateRigidBody`. Это верный порядок зависимости — физическое тело создаётся после того, как существуют модели и ссылки. В конце разрешаются repair zones и соединения комнат.

Проверь round trip на одном NPC и пробоине. До сохранения запиши ID обоих. В текущем `SaveScene` сохраняются авторские параметры NPC, но не его runtime назначение на ремонт: после загрузки он должен получить новую задачу от AI, а не сохранить старый `repairTarget`. Старые и новые Entity ID могут различаться; всякая действительно сериализованная ссылка должна пройти `remap` на живую цель. Bullet collider должен существовать и отвечать на raycast; если объект виден, но после загрузки можно пройти насквозь, проверь восстановление физики. Повторная загрузка дважды не должна создавать дублированные тела или второй визуал оружия.

3.3. Scene file и сохранение прогресса игры

`SaveScene` хранит авторское устройство уровня и часть настроек мира. Сохранение прохождения требует отдельного договора: здоровье после боя, использованный материал, запас батареи, текущий этап миссии, живые аварии. Не стоит бездумно записывать в authoring сцену все runtime значения: тогда тестовое прохождение изменит исходный уровень. Для save game можно хранить исходную сцену плюс изменения или полный snapshot постоянных компонентов, но в обоих случаях нужен version, Entity remap и восстановление runtime ресурсов. Начни с маленького эпизода: одна дверь, одна пробоина, один NPC, один враг. Сохрани после ремонта и урона, закрой процесс, загрузи и сравни конкретные инварианты.

4. Практикум: один новый компонент проходит весь путь

4.1. Формулируем учебную систему

Пусть у корабельного прибора есть датчик перегрева. Его авторские поля: room ID, порог температуры, название. Runtime поля: текущая температура, статус срабатывания, время последнего перехода. Система читает температуру комнаты, сравнивает с порогом и создаёт событие только при переходе через него. Показать число в Inspector недостаточно: оно должно появиться у созданной Entity, сохраниться, загрузиться и повлиять на игру. Важное математическое замечание: если датчик просто копирует текущую температуру комнаты, `dt` не нужно умножать ни на что. Если он реагирует с задержкой, введи физически или игрово понятную постоянную времени, тогда формула обновления зависит от dt.

4.2. Где менять код

Сначала добавь структуру компонента рядом с другими компонентами проекта и default значения. Затем добавь систему, которую можно вызвать на одной Entity или на `ForEach`. Включи её в проект Visual Studio, если новые `.cpp` не подхватываются автоматически. В `ApplicationClass::Frame` поставь вызов после обновления комнатной температуры и до системы, потребляющей тревогу. В Editor добавь возможность создать датчик и изменять authoring поля; в `SerializeObject` и `DeserializeObject` — сохранение и загрузку этих полей. В debug выводи Entity, room ID, текущее значение, порог и последний переход. Наконец, сделай тестовый мир с двумя датчиками в разных комнатах, чтобы обнаружить ошибочный общий `static` state.

Составь таблицу зависимостей своими словами: источник температуры — RoomSystem; автор порога — Inspector/JSON; вычислитель статуса — SensorSystem; потребитель события — HUD или тревога; сохранение — SceneManager; runtime очистка — при удалении Entity. Если одну строку не можешь заполнить, пока не подключай следующий слой. Пустое место в таблице позже превратится в «необъяснимый» баг.

4.3. Проверка за 15 минут

Создай датчик с порогом 60°C в комнате A и второй с порогом 80°C в комнате B. Сохрани сцену и загрузи её. Установи температуры 65 и 65: первый сработал, второй нет. Подними B до 85: второй сработал один раз. Опусти A до 55: выбери правило возврата, например гистерезис — выключить тревогу ниже 55°C, чтобы показатель 59↔61 не мигал на границе. Удали первый датчик и проверь, что событие/указатель не осталось в UI. Если этот путь пройден без редактирования JSON вручную, ты действительно добавил систему в движок, а не только написал функцию.

5. Завершённый эпизод вместо бесконечного набора механик

5.1. Состояния миссии

Собери один уровень: игрок входит на корабль, узнаёт о неисправной двери, включает генератор, отдаёт приказ инженеру устранить пробоину, отбивается от врага и достигает эвакуации. Переходы миссии определяются фактическими состояниями мира: потребитель двери запитан, пробоина закрыта, необходимый враг побеждён, игрок достиг выходной зоны. HUD сообщает цель и причину, почему следующий этап ещё не доступен. Если пробоина починена раньше генератора, миссия должна иметь явное правило: засчитать сразу, дождаться этапа или разрешить альтернативный путь. Нельзя оставлять это на порядок `if` в одном кадре без дизайна.

Наблюдение от внешнего игрока является проверкой качества. Если он не понял, какой NPC чинит, почему дверь не открывается или откуда пришёл урон, в игре отсутствует важная обратная связь. При этом нужно отличать скрытую игровую информацию от непонятного интерфейса. Враг вправе искать игрока по последнему шуму, но UI не обязан раскрывать точный AI score; достаточно, чтобы реакция выглядела причинной и последовательной.

5.2. Проверяемая сборка и рабочий каталог

`scripts/verify_engine.ps1` собирает Debug/Release, проверяет shader compilation и core contracts. Скрипт выполняется из корня проекта; успешный результат не доказывает, что игру можно перенести. После него собери отдельный каталог с executable, нужными DLL, assets, shaders, scenes и настройками. Запусти из этого каталога после переименования и перемещения. На другой машине или чистом профиле проверь первый запуск, одну миссию, save/load и выход. Если модель исчезла только

вне репозитория, начни с поиска asset path и текущего working directory, а не переписывания vertex shader.

5.3. Что записать после внешнего прогона

Для каждого дефекта укажи последовательность действий, начальную сцену, ожидаемое состояние, фактический результат, кадр/время, затронутые Entity и лог. Ошибка «NPC иногда тупит» не даёт следующего шага; запись «после отключения питания двери D инженер E держит claim на пробоине B, routeStatus=DoorUnpowered, не освобождает его 30 с» сразу указывает на контракт назначения. Исправляй блокирующие проблемы запуска, потери сохранений и невозможности закончить миссию прежде декоративной полировки. Такая дисциплина превращает учебный проект в почти готовую игру.

Карта кода и методов DX_FrameWork

Этот документ нужен, чтобы не переписывать уже существующие методы. Если ты хочешь добавить новую фичу, сначала найди похожий пункт здесь.

Главная архитектура

Проект сейчас держится на пяти слоях:

```
ApplicationClass
    главный кадр, режим Editor/Play, вызов систем

SceneManager
    ECS-сцена, импорт корабля, spawn, save/load, lights, sprites, render helpers

ECSRegistry + Components
    данные entity: Transform, MeshRenderer, Door, Room, RigidBody, HullPanel

Systems
    поведение: PhysicsSystem, DoorSystem, RoomSystem, InteractionSystem, FloodingSystem

Renderer / Shaders / AssimpModelClass
    DirectX, материалы, модели, буферы, render passes
```

Быстрый выбор места для новой фичи

Хочу сделать	Сначала смотреть	Скорее всего использовать
Создать объект в сцене	`SceneManager`	`SpawnObject(SpawnObjectDesc&&)`
Загрузить модель	`SceneManager`	`GetOrCreateModel`
Импортировать корабль из GLB	`SceneManager`	`ImportShipFromGlb`, `ImportShipNodeRecursive`
Добавить комнату	`Components.h`, `SceneManager`	`RoomComponent`, `FindRoomByName`
Связать комнаты	`SceneManager`	`ConnectRooms`, `ConnectRoomsByName`, `ResolveDoorRoomLinks`
Добавить дверь	`Components.h`, `DoorSystem.h`	`DoorComponent`, `InteractionComponent`
Сделать дверь раздвижной	`DoorSystem.h`, naming в Blender	`DoorAnimationMode::Sliding`, `leftLeaf`, `rightLeaf`
Добавить interact	`InteractionSystem.h`, `EventSystem.h`	`InteractionComponent`, `InteractionEvent`
Добавить кислород/атмосферу	`RoomEnvironmentSystem.h`	`RoomComponent::oxygen`, `DoorComponent::GetEffectiveFlowRate`

Хочу сделать	Сначала смотреть	Скорее всего использовать
Добавить физику объекту	`PhysicsSystem`	`RigidBodyComponent`, `PhysicsSystem::CreateRigidBody`
Пересоздать физику после изменения mesh	`PhysicsSystem`	`RefreshRigidBody`
Резать mesh	`MeshCuttingTest`, `AssimpModelClass`	`CutByRayAsync`, `CutCircularHoleForTest`
Выбрать объект мышью в редакторе	`PickingSystem`	`PickAt`
Отрисовать объект	`RenderMesh.h`, `SceneManager`	`RenderMesh`, `RenderAllECS`
Настроить материал	`MaterialClass`	`SetShader`, `SetTexture`, `SetGlass`
Добавить новый шейдер	`ShaderManager`, shader class	`ShaderManager::Load<T>`
Показать UI	`EditorUI.cpp`	`RenderComponentManager`, `RenderDetailsPanel`
Добавить HUD	`GameHud.h`	`GameHud::Render`

ApplicationClass

Файлы:

```
src/Core/ApplicationClass.h
src/Core/ApplicationClass.cpp
```

`ApplicationClass` это координатор всего приложения. Он владеет D3D, камерой, renderer, scene manager, editor UI, gameplay controller, input, events.

`Initialize`

Инициализирует окно/Direct3D/шейдеры/модели/рендер-текстуры/сцену/editor UI. Если нужно добавить глобальную подсистему, чаще всего она создается или связывается здесь.

Используй для:

- создания renderer dependency;
- загрузки базовых моделей;
- загрузки shader через `ShaderManager`;
- настройки callbacks между renderer и editor UI.

Не используй для:

- gameplay-логики кадра;
- временных тестовых изменений, которые должны происходить каждый кадр.

`Frame(float dt)`

Главный кадр. Здесь решается порядок всех систем.

Текущий смысл блоков:

```
0. Input
1. F5/F1/F6 hotkeys
2. Editor input / camera
3. Play interaction and doors
4. Gameplay movement / animation
5. Physics and water
6. Scene systems and lights
7. Renderer
8. Editor UI or debug overlay
9. HUD
10. Present
```

Если новая система должна обновляться каждый кадр, реши, где она должна стоять:

- до физики;
- после физики;
- только в Play;
- только в Editor;
- до рендера;
- после рендера.

Примеры:

```
DoorSystem стоит в Play до gameplay/physics.
RoomSystem и RoomEnvironmentSystem вызываются внутри SceneManager::UpdateAll.
MeshCuttingTest запускается по F6.
```

`SyncCompartmentPositions`

Синхронизирует позиции отсеков подлодки/корабля с root entity. Если root двигается, отсеки получают позицию через локальные смещения.

Используй как пример, когда нужно обновлять дочерние части от root transform.

`UpdatePhysics`

Вызывает физику, давление, затопление и синхронизацию Bullet/Transform. Если объект физический, не двигай только `TransformComponent`, подумай, не надо ли синхронизировать Bullet body.

`ProcessEditorRequests`

Обрабатывает запросы из UI: spawn, import, save/load, file actions. Если editor UI должен попросить движок что-то сделать, лучше добавить request и обработать его здесь, чем напрямую делать тяжелую работу внутри ImGui отрисовки.

`SpawnObjectFromRequest`

Создает объект из `EditorUI::SpawnRequest`. Используй как пример для spawn-логики из UI.

`UpdateCameraControlMode`

Отвечает за режим камеры. Не дублируй управление камерой в gameplay-коде, сначала смотри `CameraController`.

SceneManager

Файлы:

```
src/Scene/SceneManager.h
src/Scene/SceneManager.cpp
```

`SceneManager` это владелец ECS-сцены. Почти все игровые объекты проходят через него.

Импорт корабля

`ImportShipFromGlb`

Главный вход для импорта корабля из GLB.

Что делает:

1. Читает GLB через Assimp.
2. Создает `ShipRoot`.
3. Рекурсивно импортирует node через `ImportShipNodeRecursive`.
4. Если включен batch-путь панелей, собирает `ShipPanelVisualBatch`.
5. Вызывает `ResolveDoorRoomLinks`.
6. Привязывает `HullPanelComponent::room`.

Используй, если нужно импортировать корабль целиком.

Не используй для обычного spawn одного объекта.

`ImportShipNodeRecursive`

Проходит по Assimp node-дереву.

Распознает имена:

```
SHIP_ROOM_*      -> RoomComponent, invisible logic room
SHIP_PANEL_*     -> HullPanelComponent, collision, visual batch
SHIP_DOOR_*      -> DoorComponent + InteractionComponent
COL_* / PHYS_*   -> collision-only
STAIR / BRIDGE   -> static physics helper
```

Ключевая идея:

- `aiNode` хранит transform;
- `aiMesh` хранит геометрию;
- entity получает transform из node world matrix.

Если новая часть корабля должна импортироваться особым образом, добавляй распознавание имени здесь.

`ResolveDoorRoomLinks`

Автоматически связывает дверь с комнатами и створками.

Ожидаемый naming:

```
SHIP_DOOR_HALL__REACTOR
DOOR_LEAF_L_HALL__REACTOR
DOOR_LEAF_R_HALL__REACTOR
SHIP_ROOM_HALL
SHIP_ROOM_REACTOR
```

Что делает:

- парсит имена комнат из имени двери;
- ищет leaf-объекты;
- если leaf найдены, ставит `DoorAnimationMode::Sliding`;
- ищет комнаты;
- если комнаты найдены, записывает `door.roomA`, `door.roomB`;
- вызывает `ConnectRooms`.

Важно:

Створки теперь линкуются до проверки комнат. Это позволяет раздвижной анимации работать даже если имя комнаты временно placeholder, но для oxygen правильные room names все равно нужны.

`FindRoomByName`

Ищет `RoomComponent` по имени. Если передать `HALL`, метод сам приведет к `SHIP_ROOM_HALL`.

Используй всегда вместо ручного перебора комнат, если ищешь room по имени.

`ConnectRooms`

Связывает две комнаты по entity. Добавляет каждую комнату в `connectedRooms` другой комнаты, без дублей.

`ConnectRoomsByName`

То же самое, но принимает имена. Внутри использует `FindRoomByName` и `ConnectRooms`.

Spawn и удаление

`SpawnObject(SpawnObjectDesc&& desc)`

Главный метод создания ECS entity.

Добавляет:

- `TransformComponent`;
- `MeshRendererComponent`, если `addMeshRenderer`;
- `NameComponent`;
- `ActiveComponent`, если `active`;
- `InteractionComponent`, если `addInteraction`;
- `RigidBodyComponent`, если `addRigidBody`;
- `AnimationComponent`, если модель содержит анимации.

Если хочешь создать объект правильно, используй `SpawnObjectDesc`, а не вручную `CreateEntity + AddComponent` в пяти местах.

`SpawnObject(AssimpModelClass* model, float x, float y, float z)`

Короткий overload для простого spawn модели в позиции.

`RemoveObject`

Деактивирует и удаляет entity через ECS. Чистит material, если он не default. Используй для удаления объектов из сцены.

`Clear`

Очищает сцену, sprites, model cache, imported ship models. Используй при загрузке новой сцены.

Обновление и рендер

`UpdateAll`

Делает ECS update, который принадлежит сцене:

- переносит ship parts при движении `ShipRoot`;
- обновляет animation;
- обновляет `RoomSystem`;
- обновляет `RoomEnvironmentSystem`;
- обновляет `PlayerVitalsSystem`.

Если новая логика относится ко всей сцене и не должна жить прямо в `ApplicationClass::Frame`, можно добавить ее сюда.

`RenderAllECS`

Рендерит активные `MeshRendererComponent`:

- собирает lights;
- делает frustum culling;
- сначала рисует непрозрачные объекты;
- потом прозрачные/стекло;
- передает damage decals в PBR;
- поддерживает async mesh path.

Не добавляй новый общий рендер-цикл, пока этот метод можно расширить.

`RenderAllECSExcluding`

Рендерит сцену без одного entity. Используется для reflection pass, чтобы объект не отражал сам себя.

Save / Load

`SaveScene`

Сохраняет world settings, objects, components, materials, lights, sprites.

`SerializeObject`

Собирает json для одного entity. Если добавляешь новый компонент и он должен сохраняться, добавляй запись здесь.

`LoadScene`

Читает json, очищает сцену, восстанавливает объекты, потом remap-ит entity references.

`DeserializeObject`

Создает entity из json. Если добавляешь новый сохраняемый компонент, добавь чтение здесь.

`RemapEntityReferences`

После load старые entity id становятся новыми. Этот метод чинит ссылки внутри компонентов:

- attached lights;
- joints;
- doors left/right leaf;
- rooms;
- panels;
- etc.

Если компонент хранит `Entity`, его нужно remap-ить здесь.

Lights

`SpawnLight`

Создает light entity, transform, light component, optional gizmo.

`RemoveLight`

Удаляет light entity.

`SetLightColor`, `SetLightPosition`, `SetLightEnabled`

Изменяют отдельные свойства света.

`AttachLightToObject`, `DetachLight`

Привязывают свет к entity через offset.

`UpdateLights`

Обновляет attached lights. Если target двигается, свет следует за ним.

`GatherLightsForShader`

Заполняет массивы `XMFLOAT4 outColors[]` и `outPositions[]` для shader uniforms.

ECS

Файлы:

```
src/ECS/ECSRegistry.h
src/ECS/Components/ComponentManager.h
src/ECS/Components/Components.h
src/ECS/ECSTypes.h
```

`ECSRegistry::CreateEntity`

Выдает новый `Entity`. Это просто id.

`DestroyEntity`

Удаляет все компоненты entity из всех component pools.

`AddComponent<T>`

Добавляет компонент entity. Есть copy-версия и move-версия. Move-версия важна для `RigidBodyComponent`, потому что он хранит Bullet pointers и запрещает copy.

`GetComponent<T>`

Возвращает указатель на компонент или `nullptr`.

Паттерн:

```
auto* transform = registry.GetComponent<TransformComponent>(entity);
if (!transform)
    return;
```

`HasComponent<T>`

Проверяет наличие компонента.

`RemoveComponent<T>`

Удаляет компонент.

`GetComponents<T>`

Возвращает pool всех компонентов типа `T`. Используется для ручной итерации.

`ForEach<T>`

Итерирует все компоненты типа `T`:

```
registry.ForEach<DoorComponent>(
    [&](Entity entity, DoorComponent& door)
    {
        // update door
    });
```

Основные компоненты

Файл:

```
src/ECS/Components/Components.h
```

`TransformComponent`

Позиция, поворот, масштаб. `GetWorldMatrix` строит world matrix:

```
scale * rotation * translation
```

Почти все visual/physics/entity logic начинается отсюда.

`MeshRendererComponent`

Хранит:

- `model`;
- `material`;
- `castsShadow`;
- async model path.

Если entity должен рисоваться, у него должен быть этот компонент.

`RigidBodyComponent`

Физика Bullet:

- `body`;
- `shape`;
- mass/dynamic/kinematic;
- shape type;
- collision model;
- triangle mesh;
- convex hulls;
- compound elements.

Компонент move-only. Не копируй его.

Если меняешь mesh/collision, вызывай `PhysicsSystem::RefreshRigidBody`.

`DoorComponent`

Хранит состояние двери:

- `isOpen`;
- `isBroken`;
- `openProgress`;
- `openSpeed`;
- `animationMode`;
- `openHeight`;
- `leftLeaf`;
- `rightLeaf`;
- `roomA`;
- `roomB`;
- `flowRate`.

Методы:

- `TryOpen`;
- `TryClose`;
- `GetEffectiveFlowRate`.

`GetEffectiveFlowRate` важен для кислорода: закрытая дверь дает 0, открытая дает flow по `openProgress`.

`RoomComponent`

Комната:

- `roomName`;
- `objectsInRoom`;
- `connectedRooms`;
- bounds `localMin/localMax`;
- `oxygen`;
- `pressure`;
- `temperature`;
- `leakRate`;
- `isActive`.

`HullPanelComponent`

Панель корпуса:

- имя панели;
- имя комнаты;
- ссылка на room;
- health;
- breach count;
- max breaches;
- external/destroyed flags.

Методы:

- `CanCreateBreach`;
- `Health01`.

`HullBreachComponent`

Gameplay-пробоина:

- panel;
- room;
- local/world position;
- local normal;
- radius;
- leakRate;
- sealed.

`CompartmentComponent`

Старый/параллельный компонент отсека:

- bounds;
- volume;
- water volume/level;
- breaches;
- connected compartments;
- pump rate.

Методы:

- `RecalculateVolume`;
- `HasSizeChanged`;
- `CacheSize`;
- `ContainsPoint`.

`DamageDecalComponent`

Данные для трещин/дефектов в PBR shader. `SyncFromBreaches` строит список cracks по `CompartmentComponent::breaches`.

Systems

DoorSystem

Файл:

```
   src/ECS/System/DoorSystem.h
```

`ProcessEvents`

Читает `InteractionEvent` из `EventQueue`. Если target имеет `DoorComponent`, переключает `isOpen`. Если дверь открылась, создает `DoorOpenedEvent`.

`Update`

Обновляет `openProgress`.

Если `animationMode == Sliding` и есть `leftLeaf/rightLeaf`:

- сохраняет закрытые позиции створок;
- вычисляет slide axis из правой створки минус левая;
- двигает створки в разные стороны;
- root/рама остается на месте.

Иначе:

- использует старый Vertical mode;
- двигает transform root вверх по Y.

InteractionSystem

Файл:

```
InteractionSystem.h
```

`Update`

Строит ray из камеры вперед, raycast-ит Bullet world, игнорирует player, проверяет `InteractionComponent`, сохраняет hovered entity/prompt, по нажатию `C` создает `InteractionEvent`.

Используй это как основу для:

- двери;
- repair tool;
- панели управления;
- предметы;
- станции корабля.

EventQueue

Файл:

```
EventSystem.h
```

`Emit`

Добавляет событие в `std::vector<std::variant<...>>`.

`GetEvents`

Дает const reference на события текущего кадра.

`Clear`

Очищает очередь. Вызов стоит в начале `ApplicationClass::Frame`.

RoomSystem

Файл:

```
RoomSystem.h
```

`Update`

Очищает `objectsInRoom` у всех комнат, потом проверяет transform всех trackable entity и добавляет их в комнату, если позиция внутри room bounds.

Trackable entity:

- player/character controller;
- dynamic rigid body.

RoomEnvironmentSystem

Файл:

```
RoomEnvironmentSystem.h
```

`Update`

Делает две вещи:

1. Уменьшает oxygen в комнате, если в ней есть breathing entity.
2. Выравнивает oxygen между connectedRooms.

`GetDoorFlowMultiplier`

Если между двумя комнатами есть дверь, flow зависит от `DoorComponent::GetEffectiveFlowRate`. Если двери нет, flow = 1.

Это место отвечает за логическую герметичность.

PhysicsSystem

Файлы:

```
src/ECS/System/PhysicsSystem.h
src/ECS/System/PhysicsSystem.cpp
```

Ключевые методы:

- `InitializeWorld`: создает Bullet world.
- `Update`: step simulation, sync transforms.
- `CreateRigidBody`: создает Bullet body из `RigidBodyComponent`.
- `RefreshRigidBody`: пересоздает body/shape после изменения параметров или mesh.
- `CreateBoxShape`, `CreateSphereShape`, `CreateCylinderShape`, `CreateCapsuleShape`: простые формы.
- `CreateConvexHullShape`: convex hull из модели.
- `CreateTriangleMeshShape`: статичный triangle mesh.
- `CreateGImpactMeshShape`: concave mesh для dynamic случаев.
- `CreateCompoundElementShape`: shape для compound child.
- `UpdateCharacterPos`: переносит позицию Bullet character controller обратно в Transform.

Правило:

Если объект имеет Bullet body, не меняй только transform и не жди, что физика сама поймет. Либо двигай Bullet body, либо вызывай refresh/sync.

FloodingSystem

Файлы:

```
src/ECS/System/FloodingSystem.h
src/ECS/System/FloodingSystem.cpp
```

Методы:

- `Update`: обновляет затопление отсеков.
- `CreateBreach`: добавляет пробоину в compartment local space.
- `CreateBreachAtWorldPosition`: переводит world position в local и создает пробоину.
- `SealBreach`: закрывает пробоину.
- `FindCompartmentForObject`: ищет отсек по world position.
- `CalculateFlowRate`: считает поток по пробоине.

Можно использовать как основу для космической разгерметизации, даже если вода потом будет заменена на oxygen/pressure leak.

MeshCuttingTest

Файлы:

```
src/ECS/System/MeshCuttingTest.h
src/ECS/System/MeshCuttingTest.cpp
```

Методы:

- `CutRandomHoleInReactor`: тестовая пробоина.
- `CutByRay`: синхронная резка по ray.
- `CutByRayAsync`: тяжелую часть отправляет в `JobSystem`, а DirectX/Bullet обновляет на main thread.

Не воспринимай как финальную систему разрушения. Это prototype.

JointConnectionSystem

Файлы:

```
src/ECS/System/JointConnectionSystem.h
src/ECS/System/JointConnectionSystem.cpp
```

Методы:

- `Update`: создает/удаляет constraints по компонентам.
- `CreateJoint`: создает физический joint.
- `RemoveJoint`: удаляет constraint.
- `RemoveJointsForEntity`: чистит все joints для entity.

CameraController

Файлы:

```
src/Core/CameraController.h
src/Core/CameraController.cpp
```

Методы:

- `SetInput`: связывает input manager, raw input и HWND.
- `UpdateModeToggle`: переключает camera mode.
- `UpdateFree`: editor/free camera.
- `UpdateFollow`: play camera следует за player.
- `ApplyFreeLook`: вращение мышью.
- `ApplyFreeMove`: WASD/RF перемещение.
- `EnterPlayMode`, `ExitPlayMode`: переключают настройки камеры под Play/Editor.

Если камера ведет себя странно, начинай отсюда.

InputManager / InputClass

Файлы:

```
src/Input/InputManager.h
src/Input/InputManager.cpp
src/Input/InputClass.h
src/Input/InputClass.cpp
```

`InputClass` хранит низкоуровневое состояние клавиш/мыши.

`InputManager` делает поверх этого:

- key pressed edge;
- mouse click;
- drag start/update/end;
- mouse capture helpers.

Используй:

```
inputManager.IsKeyDown(input, 'W')
inputManager.IsKeyPressed(input, 'C')
```

`IsKeyPressed` true только в кадр нажатия.

Персонажи и NPC

Этот раздел описывает актуальную цепочку персонажа. Главное правило: AI создаёт намерение, общий controller исполняет его через Bullet, animation читает результат. Ни utility, ни hostile decision не должны напрямую телепортировать `TransformComponent` во время обычной ходьбы.

`CharacterSpawnSystem`

Файл:

```
src/Gameplay/Player/CharacterSpawnSystem.h
src/Gameplay/Player/CharacterSpawnSystem.cpp
```

`Spawn`

Создаёт runtime Bullet capsule для entity, у которой уже есть `CharacterController` и `TransformComponent`. Повторный вызов сначала уничтожает старый runtime controller. Регистрирует ghost object и character action в `PhysicsWorld`.

Не путать с созданием ECS entity: метод не добавляет модель, NPC brain или movement-компоненты.

`Destroy`

Снимает action и collision object с Bullet world, затем освобождает controller, ghost и capsule. Вызывать перед удалением персонажа/очисткой сцены.

`CharacterControllerSystem`

Файл:

```
src/Gameplay/Player/CharacterControllerSystem.h
```

`Update`

Обрабатывает все `CharacterController`: active-state suspension, ground support, automatic step, external velocity, jump, walk direction и стабильный physics idle. Читает `CharacterMovement`, но не решает игровую цель персонажа.

`IsAutomaticStepActive`

Диагностирует, выполняется ли сейчас автоматическое преодоление порога.

`HasGroundSupport`

Возвращает результат геометрической проверки опоры, используемый как источник grounded.

`IsIdlePhysicsLocked`

Показывает, переведён ли стоящий персонаж в стабильный no-motion режим Bullet.

`ForceReconfigure`

Сбрасывает runtime traversal cache после изменения physics tuning в редакторе. Не пересоздаёт сущность и не заменяет `CharacterSpawnSystem::Spawn`.

`CharacterFacingSystem`

Файл:

```
src/Gameplay/Player/CharacterFacingSystem.h
```

`Update`

Поворачивает визуальный Transform по горизонтальному `walkDirection`. Вызывать после AI/gameplay intent и до анимации/рендера. Не использовать для выбора цели.

`AnimationController`

Файл:

```
src/Animation/AnimationController.h
```

`Update`

Выбирает locomotion-анимацию по `CharacterMovement` для игрока и NPC. AI должен выставлять Idle/Walk/Run intent, а не запускать клип напрямую.

Базовые NPC-данные

Файл:

```
NPCComponents.h
```

`NPCComponent`

Хранит name, description, health/maxHealth и `isHostile`. `isHostile` уже является источником различия экипажа и врага, редактируется в инспекторе и сохраняется как `npc_hostile`.

`NPCMovementComponent`

Хранит walk/run speed, дистанции прибытия и дверей, возможность бега. Это authoring-настройки навигации, а не фактическое движение текущего кадра.

`NpcBrainComponent`

Хранит state, manual command, repair target, комнатный/дверной маршрут, utility score, repath/stuck timers, wander runtime и debug snapshot.

`NpcCommand::IsActive`

Единственная официальная проверка ручного приоритета: любая команда кроме `None` блокирует автоматическое назначение.

`NpcCommand::Clear`

Возвращает command в `None`, очищает entity targets и увеличивает revision.

`NpcCommandSystem`

Файл:

```
NpcCommandSystem.h
```

Методы записывают приоритетный приказ в brain; фактическое выполнение остаётся в `StateSystem`.

Метод	Назначение
`Stop`	постоянная остановка с ручным приоритетом
`Wander`	постоянное блуждание
`MoveToPosition`	одноразовое движение в world point

Метод	Назначение
`MoveToRoom`	одноразовое движение внутрь комнаты
`FollowEntity`	постоянное следование за Entity
`RepairEntity`	одноразовый ручной ремонт
`ResumeAutomatic`	очистить manual command и вернуть utility/crew authority
`ForceRepath`	инвалидировать текущий маршрут, сохранив задачу

`NpcUtilitySystem`

Файл:

```
NpcUtilitySystem.h
```

`Update`

Обходит подходящих NPC и вызывает индивидуальную оценку задач. Должен считать scores для debug даже при manual command, но не менять решение при активном command.

`CalculateRepairScore`

Оценивает пробоину по тяжести и расстоянию, учитывая detection range и reservation.

`CalculateEquipmentRepairScore`

Оценивает повреждённое оборудование. Невалидное, полностью исправное или чужое назначение получает нулевую полезность.

`IsRepairTargetStillValid`

Проверяет общий `RepairableComponent`, состояние breach либо health оборудования.

`CrewTaskCoordinator`

Файл:

```
src/Gameplay/AI/CrewTaskCoordinator.h
```

`Update`

Очищает claims, собирает ремонтные задачи и активный экипаж, сохраняет валидные назначения и закрепляет свободные цели. Hostile, off-duty и NPC с manual command исключаются.

`IsRepairTargetValid`

Публичный контракт валидности repair target: Entity жива, активна, имеет незавершённый `RepairableComponent` и всё ещё повреждена.

`IsClaimedByOther`

Проверяет, закреплена ли цель за другим живым NPC.

`GetSnapshot`

Возвращает telemetry последнего update: available crew, busy crew, open tasks и новые назначения кадра.

`CollectRepairCandidates`

Собирает и сортирует валидные цели. Сортировка нужна для воспроизводимого выбора.

`CleanupInvalidClaims`

Удаляет claims завершённых/удалённых целей или NPC, которые больше не могут продолжать. Синхронизирует `CrewMemberComponent::assignedTask`.

`ReleaseAssignment`

Освобождает claim только если он принадлежит указанному NPC.

`KeepExistingAssignment`

Сохраняет валидную текущую работу и восстанавливает согласованность brain/crew state.

`CalculateAssignmentScore`

Объединяет severity, distance, role affinity и repair skill. Engineer имеет преимущество ремонта, Operations — дополнительное преимущество для энергетического оборудования.

`StateSystem`

Файл:

```
StateSystem.h
```

`Update`

Исполняет manual command или automatic state для каждого NPC с обязательными компонентами. Это consumer решения utility/coordinator и producer `CharacterMovement`.

`HandleManualCommand`

Маршрутизирует Stop, Wander, MoveToPosition, MoveToRoom, FollowEntity и RepairEntity. Постоянные команды не очищаются автоматически; одноразовые завершаются через общую точку.

`UpdateRepairTask`

Унифицирует ремонт пробоины и оборудования через `RepairableComponent`, определяет точку/комнату, ведёт NPC к цели и передаёт прогресс в ремонтный путь. Учитывает `CrewMemberComponent::repairSkill`.

`FindRepairZoneForOwner`

Ищет `WorkZoneComponent` с owner. Содержит fallback по имени/типу оборудования для старых сцен; новый контент должен задавать owner явно.

`NavigateTo`

Общий исполнитель навигации: комнаты, двери, portal crossing, локальный NavMesh waypoint, движение и blocked status. Не копировать эту логику в hostile AI.

`TryBuildDoorRoute`

Строит верхнеуровневый маршрут по графу связанных комнат.

`MoveTowards`

Записывает нормализованное направление, walk/run speed и locomotion state в `CharacterMovement`.

`StopMovement`

Сбрасывает все locomotion-поля вместе. Использовать вместо частичного `isMoving = false`, иначе может остаться Run-анимация или скорость.

`CompleteManualCommand`

Очищает одноразовую команду, маршрут, дверь, status и движение; после этого automatic AI снова получает authority.

`SetBlocked`

Останавливает движение и записывает диагностическую причину, не уничтожая команду/цель.

`UpdateStuckStatus`

Сравнивает реальное изменение Transform с намерением движения. При отсутствии прогресса инвалидирует NavMesh path и записывает snapshot дверного перехода.

`NpcNavMeshSystem`

Файлы:

```
src/Gameplay/AI/NpcNavMeshSystem.h
src/Gameplay/AI/NpcNavMeshSystem.cpp
```

`FindNextWaypoint`

Строит/читает путь Detour внутри комнаты и возвращает следующую точку. `false` означает использовать предусмотренный fallback, а не аварийно завершать приложение.

`FindReachablePoint`

Проецирует желаемую точку на связный остров NavMesh текущего старта; используется для безопасного wander/work-zone target.

`FindDynamicAvoidanceWaypoint`

Строит короткий локальный объезд ближайшего динамического Bullet AABB. Это не полноценный crowd/RVO.

`InvalidatePath`

Сбрасывает путь одного NPC после смены target или обнаружения застревания.

`InvalidateAll`

Сбрасывает room meshes и все paths после изменения геометрии/коллайдеров.

`GetDebugInfo`

Возвращает построенность mesh, fallback status, polygon/waypoint counts, path и debug edges.

`GetDebugDrawSettings`

Возвращает настройки one-frame world visualization NavMesh/path/obstacles.

Дружественный и враждебный NPC

Текущий источник истины — `NPCComponent::isHostile`.

```
isHostile == false → допускается CrewTaskCoordinator и ремонт экипажа
isHostile == true  → исключается из crew assignment; combat AI добавляется отдельно
```

Боевой компонент должен хранить perception/combat runtime (target, ranges, cooldown), но не дублировать флаг hostile. Hostile decision вызывается до facing/controller/animation и переиспользует `NavigateTo`/общий navigation executor.

Renderer

Файлы:

```
Renderer.h
Renderer.cpp
src/ECS/System/RenderMesh.h
src/ECS/System/RenderSystem.h
```

`Renderer::RenderFrame`

Главная точка рендера кадра.

Пассы:

```
shadow map
reflection texture
refraction texture
back buffer main scene
sprites/debug/preview
```

Статистика записывается в `RenderFrameStats`.

`RenderMesh`

Inline helper, который:

1. Берет shader из material.
2. Вызывает `model->Render`.
3. По `ShaderType` выбирает нужный shader class.
4. Передает texture/material/light/fog/matrices.

Если объект белый/невидимый, проверь:

- есть ли material;
- есть ли shader;
- есть ли texture/SRV;
- подходит ли ShaderType.

`RenderSystem::Render`

Простой ECS render loop. Сейчас основная логика богаче в `SceneManager::RenderAllECS`, но этот класс показывает базовый паттерн.

AssimpModelClass

Файлы:

```
src/Render/Model/assimpmodelclass.h
src/Render/Model/assimpmodelclass.cpp
```

Это модель, которая владеет DirectX vertex/index buffers, mesh parts, embedded textures, CPU-копией геометрии и animation data.

Загрузка

`Initialize`

Загружает модель по пути.

`InitializeFromMeshes`

Создает модель из нескольких `aiMesh` одного node. Используется при импорте корабля.

`InitializeFromMesh`

Создает модель из одного `aiMesh`.

`InitializeFromGeometry`

Создает модель из уже готовых CPU vertices/indices. Используется для `ShipPanelVisualBatch`.

Геометрия и bounds

`GetBoundingBox`

Возвращает local AABB модели.

`GetBoundingSphere`

Возвращает center/radius.

`GetVertices`, `GetRawVertices`, `GetRawTriangles`

Дают CPU-копию геометрии. Используются picking/cutting/physics.

Runtime mesh mutation

`ApplyCpuMeshForTest`

Заменяет CPU mesh и пересоздает DirectX buffers.

`CutSphereHoleForTest`

Грубый тест: удаляет треугольники внутри sphere.

`CutCircularHoleForTest`

Более аккуратный prototype: режет круг на поверхности через clipping в tangent space.

Animation

`LoadBones`

Читает bone weights/index из Assimp.

`LoadAnimations`

Читает clips/channels/keyframes.

`UpdateAnimation`

Обновляет `AnimationComponent` по deltaTime.

`SetBoneMatricesFromComponent`

Передает matrices костей в shader buffer.

`ComputeBoneMatrices`

Считает matrices для текущего animation time.

MaterialClass

Файлы:

```
src/Render/Material/MaterialClass.h
src/Render/Material/MaterialClass.cpp
```

Материал хранит shader, textures и параметры для разных shader types.

Частые методы:

- `SetShader`, `GetShader`;
- `SetTexture`, `AdoptTexture`, `SetMainTexture`;
- `SetTexturePath`, `GetTexturePath`;
- `SetGlass`, `IsGlass`, `GetGlassOpacity`;
- `SetClipPlane`, `SetClipEnabled`;
- `SetReflectionMatrix`, `SetReflectionSRV`;
- `SetWaterTranslation`, `SetRefractionSRV`;
- `SetTerrain...`;
- `Bind`, `Unbind`.

Не создавай новый material class под каждую фичу. Расширяй этот, если параметр реально относится к материалу.

ShaderManager / ShaderBaseClass

Файлы:

```
src/Render/Shader/ShaderManager.h
src/Render/Shader/BaseShader.h
```

`ShaderManager::Load<T>`

Создает и инициализирует shader, сохраняет по имени. Если shader уже есть, возвращает существующий.

`Get`, `GetAs`, `GetShaderType`

Дают доступ к загруженным shader.

`ForEach`

Позволяет обновить общий параметр во всех shader, например fog.

`ShaderBaseClass`

Базовый помощник для shader:

- compile;
- input layout;
- sampler;
- constant buffer;
- fog buffer;
- `UpdateConstantBuffer`.

EditorUI

Файлы:

```
src/Editor/EditorUI.h
src/Editor/EditorUI.cpp
```

Главные методы:

- `Initialize`, `Shutdown`, `NewFrame`, `Render`;
- `Update`: основная отрисовка UI;
- `RenderSceneOutliner`: список entity;
- `RenderDetailsPanel`: правая панель деталей;
- `RenderObjectProperties`: transform/name/material;
- `RenderComponentManager`: компоненты entity;
- `RenderSpawnPanel`: spawn/import;
- `RenderFogPanel`: настройки мира;
- `RenderPhysicsSettings`: debug physics;
- `RenderCompartmentEditor`: редактор отсеков;
- `RenderContentBrowser`: файловый браузер;
- `RenderKnifeTool`: UI для clipping/cutting;
- `RenderShaderSelector`: выбор shader.

Правило:

UI должен менять данные/создавать request. Тяжелую работу лучше делать в `ApplicationClass::ProcessEditorRequests`.

PickingSystem

Файлы:

```
src/Config/PickingSystem.h
src/Config/PickingSystem.cpp
```

`PickAt`

Порядок выбора:

1. light spheres;
2. world sprites;
3. ECS mesh objects.

Для mesh сначала проверяет AABB, потом реальные triangles через `RayIntersectsModelTriangles`.

Если объект выбирается неправильно, проверь:

- ActiveComponent;
- MeshRendererComponent;
- TransformComponent;
- bounds модели;
- `EditorSelectionComponent` redirect.

Naming conventions корабля

Комнаты

```
SHIP_ROOM_HALL
SHIP_ROOM_REACTOR
SHIP_ROOM_GUN_ROOM
```

Комната обычно невидимая логическая коробка. Ее bounds идут из mesh bounding box.

Панели корпуса

```
SHIP_PANEL_REACTOR_WALL_B_07
SHIP_PANEL_HALL_FLOOR_01
SHIP_PANEL_GUN_ROOM_CEIL_02
```

`TryParsePanelRoomName` пытается достать room name из имени панели.

Двери

```
SHIP_DOOR_HALL__REACTOR
```

Разделитель между комнатами обязательно двойной:

```
__
```

Створки раздвижной двери

```
DOOR_LEAF_L_HALL__REACTOR
DOOR_LEAF_R_HALL__REACTOR
```

Если leaf не найдены, дверь останется `Vertical`.

Коллизия

```
COL_*
SHIP_COL_*
PHYS_*
COLLISION_*
```

Такие node не рисуются, но могут получить rigid body.

Частые ошибки

"Рама двери поднимается вверх"

Причина:

- `DoorComponent::animationMode` остался `Vertical`;
- leaf-объекты не найдены;
- имена leaf не совпадают с именем двери;
- дверь названа placeholder rooms, а комнаты не совпадают.

Проверка:

```
DoorComponent.animationMode
DoorComponent.leftLeaf
DoorComponent.rightLeaf
```

"Кислород проходит через закрытую дверь"

Проверить:

- связаны ли комнаты напрямую без двери;
- есть ли между roomA/roomB дверь;
- `DoorComponent::GetEffectiveFlowRate`;
- `openProgress`;
- `RoomEnvironmentSystem::GetDoorFlowMultiplier`.

"Объект виден, но по нему нельзя ходить"

Проверить:

- есть ли `RigidBodyComponent`;
- создан ли Bullet `body`;
- правильный `shapeType`;
- `mass = 0` для static;
- collision group/mask;
- не collision-only ли это node.

"Объект выбирается как весь корабль"

Это может быть специально:

```
EditorSelectionComponent.redirectTo = ShipRoot
```

Детали корабля перенаправляют selection на root, чтобы случайно не двигать одну стенку отдельно.

"Mesh изменился, а физика старая"

После изменения геометрии нужен:

```
PhysicsSystem::RefreshRigidBody
```

"Стекло рисуется неправильно"

Проверить:

- `MaterialClass::IsGlass`;
- `AssimpModelClass::MeshPart::isTransparent`;
- второй transparent pass в `SceneManager::RenderAllECS`;
- blend state в PBR path.

Правило перед новым кодом

Перед тем как писать новый метод, задай себе вопросы:

1. Это данные entity? Тогда компонент.
2. Это каждый кадр обрабатывает много entity? Тогда система.
3. Это реакция на действие? Тогда event.
4. Это UI? Тогда `EditorUI`, а тяжелую работу через request.
5. Это spawn/save/load/render/ship import? Тогда сначала `SceneManager`.
6. Это DirectX resource? Тогда ищи похожее в `D3DClass`, `AssimpModelClass`, shader class, `MaterialClass`.

Учебный маршрут по DX Framework и NPC

Цель маршрута — не «прочитать побольше», а научиться объяснять код, предсказывать его поведение и самостоятельно добавлять механики без разрастания одной огромной системы.

1 Правило одного учебного блока

Каждый блок длится от 60 до 90 минут:

1. 20 минут теории.
2. 20 минут чтения текущего кода.
3. 30 минут маленькой самостоятельной реализации.
4. 10 минут теста и записи вывода.

После двух блоков делай перерыв. В конце дня сохраняй не только код, но и короткий отчёт:

```
Что хотел получить
Что предсказал до запуска
Что увидел
Какая строка объясняет результат
Что осталось непонятным
```

2 Интенсив на один свободный день

Это лучший старт перед четырёхнедельным маршрутом.

Блок 1 Векторы и луч

Теория: точка, вектор, `end - start`, длина, нормализация, конечная дальность.

Чтение: `NpcRangedCombatSystem.h`, участок от `visibleAimPoint` до вычисления `distance`.

Практика: вручную посчитать направление для двух точек, затем вывести значение DirectXMath.

Тест: нулевая длина, цель впереди, цель позади, цель выше.

Критерий готовности: можешь без подсказки объяснить каждый аргумент `CastRay`.

Блок 2 Матрица дула

Теория: матрица 4 на 4, сложение, умножение строка на столбец, порядок преобразований.

Чтение: `TransformComponent::GetWorldMatrix` и цепочка `Grip * hand * world`.

Практика: нарисовать промежуточные точки преобразования дула разными цветами.

Тест: повернуть NPC, сменить масштаб, проиграть анимацию, убедиться, что дуло следует за рукой.

Критерий готовности: можешь назвать пространство точки после каждого множителя.

Блок 3 Отладка одного выстрела

Теория: guard clause, первое попадание, дружественная линия огня.

Чтение: `NpcRangedCombatSystem` от chest-to-muzzle до `TryFire`.

Практика: поставить стену и союзника в разные участки луча. До запуска записать ожидаемый статус.

Тест: свободное дуло, заблокированное дуло, союзник в линии, враг за стеной.

Критерий готовности: по журналу видно точную первую причину запрета.

Блок 4 Маленькое самостоятельное изменение

Практика: добавить только новое поле отладки, не новую механику. Например, вывести `yawError`, `muzzleDistance`, `fireLane.entity`.

Проверка: значение сохраняется достаточно долго или попадает в ring buffer, поэтому его можно прочесть.

Что прислать на разбор: изменённый участок, один скрин инспектора, собственное объяснение из пяти предложений.

3 Неделя 1 Математика которую видно

День 1 Точки и векторы

Теория: сложение, вычитание, масштабирование.

Код: расчёты `dx`, `dy`, `dz` в боевых системах.

Задание: функция направления и длины между точками с защитой нулевого случая.

Выходной тест: пять наборов координат, включая одинаковые точки.

День 2 Dot и поле зрения

Теория: скалярное произведение, cosine, половинный FOV.

Код: текущая проверка видимости NPC.

Задание: нарисовать границы FOV и луч к кандидату.

Критерий: цель за стеной проходит FOV, но отклоняется raycast; ты можешь объяснить оба этапа.

День 3 Матрицы вручную

Теория: индексы, сложение ячеек, строка на столбец.

Задание: решить четыре матрицы 2 на 2 на бумаге, затем написать учебную функцию умножения без DirectXMath.

Критерий: все ячейки совпадают с ответом, порядок A B и B A даёт разные результаты.

День 4 Трансформации

Теория: scale, rotation, translation, world и inverse world.

Код: `GetWorldMatrix`.

Задание: преобразовать локальную контрольную точку и вернуть её обратной матрицей.

Критерий: возвращённая точка совпадает с исходной с малой погрешностью.

День 5 Лучи

Теория: параметрическое уравнение, ray против segment, ray-plane и ray-AABB.

Задание: отладочный луч из двух точек и ограниченная длина.

Критерий: препятствие за конечной точкой не считается попаданием.

День 6 Время

Теория: `deltaTime`, cooldown, fixed step, экспоненциальное сглаживание.

Задание: одинаковый таймер при 30, 60 и 144 FPS в тестовой симуляции.

Критерий: расхождение не превышает один шаг симуляции.

День 7 Проверка недели

Без книги объясни: нормализацию, dot, умножение матриц, порядок world transform, создание луча и таймер. Повтори только темы, которые не можешь объяснить на собственном примере.

4 Неделя 2 Архитектура текущего кода

День 8 ECS и владельцы данных

Код: `ECSRegistry::ForEach`, `GetComponent`, компоненты NPC.

Задание: таблица «поле → владелец → кто читает → кто пишет» для `targetEntity`, `lastKnownPosition`, `activeStyle`, `moveTarget`, `ammoInMagazine`.

Критерий: у каждого поля один понятный владелец записи.

День 9 Guard clauses

Код: `WeaponSystem::TryFire`.

Задание: для каждой ранней остановки записать, какой контракт она защищает.

Тест: искусственно вызвать каждую остановку и увидеть отдельную причину.

День 10 Состояния

Код: ветки `TakingCover`, `InCover`, `Attacking`.

Задание: нарисовать переходы и подписать условие каждого ребра.

Критерий: нет перехода «просто потому что так получилось».

День 11 Решение против исполнения

Код: `NpcTacticalDecisionSystem`, `NpcCoverQuery`, `StateSystem`.

Задание: подписать для каждой системы: вход, выход и запрещённые обязанности.

Критерий: query не меняет state, decision не строит путь, state не пересчитывает тактику.

День 12 Журнал решений

Задание: спроектировать ring buffer последних 64 решений с временем, состоянием до и после, причиной и entity кандидата.

Тест: пошаговый режим не теряет короткие промежуточные статусы.

День 13 Одна трасса кадра

Проследи один выстрел от восприятия до `DamageEvent`. Запиши функцию, поле и значение на каждом шаге.

День 14 Рефакторинг без изменения поведения

Выбери один длинный участок и выдели маленькую чистую функцию проверки. До и после сравни одинаковый сценарий и журнал.

Критерий: поведение не изменилось, причина стала читаться легче.

5 Неделя 3 Индивидуальный боевой NPC

День 15 Подтверждение контакта

Теория: perceived data против world truth.

Задание: проверить, что союзник передаёт только сообщение о контакте, а получатель сам подтверждает видимость.

Критерий: NPC не знает точную текущую позицию скрытого врага.

День 16 Выбор цели

Задание: разделить жёсткие фильтры и score цели: жив, враждебен, допустимая комната, расстояние, угроза.

Тест: игрок и NPC одновременно являются кандидатами.

День 17 Профиль укрытия

Задание: на бумаге спроектировать `CoverProfile` для стойки, приседа, левого и правого peek. Код писать только после проверки структуры.

Критерий: низкая стена не считается защитой стоящего NPC, но может защитить сидящего.

День 18 Score укрытия

Задание: жёстко отклонять незащищённые точки, остальные оценивать по защите, пути, дистанции и peek.

Тест: два NPC по одну сторону стены не считают её защитой друг от друга.

День 19 Фазы укрытия

Задание: записать таймлайн `move → hide → peek → aim → burst → return`.

Критерий: таймер экспозиции не истекает до завершения прицеливания.

День 20 Смена укрытия

Задание: смена только по понятным причинам: укрытие потеряло защиту, путь заблокирован, серия опасных peek, враг критически близко.

Критерий: NPC не бегает между укрытиями каждые две секунды без причины.

День 21 Дистанция и отступление

Задание: Defensive удерживает диапазон, Assault получает право сближаться, Balanced использует гистерезис.

Тест: при пуше одного NPC второй не повторяет симметричный пуш автоматически.

6 Неделя 4 Дружественные NPC и расширяемые модули

День 22 Память контакта

Спроектируй данные: кто сообщил, когда, где был подтверждён контакт, уверенность, время устаревания. Точная мировая позиция скрытого врага не читается напрямую.

День 23 Локальная реакция на выстрел

NPC слышит или получает попадание, поворачивается к близкому источнику, но не получает вечную цель. Оставь отдельное правило тишины для будущих тихих убийств.

День 24 Дружественная линия огня

Проверь перестроение союзника, пересечение траекторий и запрет огня. Добавь в журнал entity блокирующего союзника и длительность блокировки.

День 25 Простые роли без группового разума

Добавь роль как параметр индивидуальной тактики: Assault, Defensive, Support. Роль меняет веса и пороги, но не телепатически координирует NPC.

День 26 Контракт тактического модуля

Новый модуль должен возвращать предложение:

```
struct TacticalProposal
{
    TacticalAction action;
    float score;
    const char* reason;
};
```

Арбитр выбирает предложение, а исполнитель выполняет. Это позволяет добавлять Retreat, Suppress, Flank, ReloadInCover как пазлы.

День 27 Полигон

Создай сценарии:

- открытая площадка;
- высокая стена;
- низкая стена;
- двустороннее укрытие;
- узкий проход с союзником;
- две альтернативы пути;
- потеря контакта;
- противник быстро сближается.

Каждый сценарий должен иметь ожидаемый результат до запуска.

День 28 Вертикальный срез

Один NPC должен:

1. обнаружить врага;
2. подтвердить линию зрения;
3. выбрать роль и укрытие;
4. добежать с подходящей скоростью;
5. выбрать стойку и сторону peek;
6. проверить дуло и союзника;
7. сделать ограниченную очередь;
8. перезарядиться за укрытием;
9. сменить позицию только по объяснимой причине;
10. потерять контакт без всезнания.

7 Очерёдность будущих модулей

Добавляй по одному модулю и не смешивай их правила:

Порядок	Модуль	Вход	Предложение
1	Reload safety	патроны, укрытие, угроза	перезарядиться сейчас
2	Retreat	дистанция, здоровье, путь	увеличить дистанцию
3	Suppression	последняя подтверждённая зона, боезапас	подавлять область
4	Flank	профиль укрытия врага, маршруты	обойти сбоку
5	Grenade	группа целей, траектория, союзники	бросить гранату
6	Revive	раненый союзник, риск пути	помочь союзнику
7	Squad coordinator	предложения отдельных NPC	распределить роли

Групповая тактика идёт последней. Сначала одиночный NPC должен быть предсказуемым и объяснимым.

8 Формат мини задания для совместной проверки

Когда присылаешь решение, достаточно пяти частей:

1. Что хотел изменить.
2. Какое условие написал своими словами.
3. Сам изменённый участок кода.
4. Что ожидал до запуска.
5. Что реально произошло и какой статус был в журнале.

Так можно проверить твоё мышление, не отбирая у тебя самостоятельную работу готовым большим куском кода.

9 Критерии окончания маршрута

Маршрут закончен, когда ты можешь:

- вручную построить вектор, матрицу и луч;
- построчно объяснить выбранную систему;
- назвать владельца каждого изменяемого поля;
- отделить восприятие от истинного состояния мира;
- отделить решение от запроса геометрии и исполнения;
- добавить новый тактический модуль через общий контракт;
- доказать решение NPC журналом и трассерами;
- написать тестовый сценарий до запуска;
- найти баг по цепочке данных, а не случайными правками.